
УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ
ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ, ВСЕ

Дата генерации: 15.09.2026, 14:29:09
Файлов исходного кода: 111
Суммарный объём кода: 1.96 MB
Глав/билетов в пособии: 36

ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ

1. 1. Дерево отрезков с операциями на отрезках [id: segment-tree]
2. 2. Двумерная разреженная матрица (Sparse Table)
3. 3. Декартово дерево (Treap) [id: treap]
4. 4. Splay-дерево [id: splay-tree]
5. 5. Двумерная матрица префиксных сумм [id: prefix-sums]
6. 6. Графы. Представление графов, DFS, BFS [id: graph-basics]
7. 7. Графы. Планарные. Покраска [id: graph-planarity]
8. 7. Графы. Планарные. Покраска [id: planarity-euler]
9. 8. Графы. Компоненты связности [id: graph-components]
10. 9. Графы. Топологическая сортировка [id: topological-sort]
11. 10. Графы. Компоненты сильной связности (SCC)
12. 11. Графы. Мосты [id: graph-bridges]
13. 11. Графы. Мосты [id: bridges-code]
14. 12. Графы. Точки сочленения [id: graph-articulation-points]
15. 13. Графы. Эйлеров цикл [id: graph-euler]
16. 13. Графы. Эйлеров цикл [id: euler-path-vs-cycle]
17. Билет 14. Дейкстра [id: dijkstra]
18. Билет 15. Форд-Беллман [id: bellman-ford]
19. Билет 16. Флойд-Уоршелл [id: floyd]
20. 17. Графы. Ускорения поиска [id: pathfinding-accelerations]
21. 17. Графы. Поиск кратчайшего пути. Ускорения [id: shortest-path-accelerations]
22. 18. Графы. Остовное дерево. Краскал [id: mst-kruskal]
23. 18–20. Графы. Остовное дерево: Краскал, Прима, Борувка
24. 19. Графы. Остовное дерево. Прима [id: mst-prim]
25. 20. Графы. Остовное дерево. Борувка [id: mst-kruskal]

- 18. src/data/content_temp.ts
- 19. src/data/content.ts
- 20. src/data/debugRunner.ts
- 21. src/data/glossary.ts
- 22. src/data/graphContent.ts
- 23. src/data/quizzes.ts
- 24. src/data/vizStepBus.ts
- 25. src/data/vizSync.ts

Билеты (учебный контент)

- 26. src/data/tickets/ticket1_segtree.ts
- 27. src/data/tickets/ticket14_20.ts
- 28. src/data/tickets/ticket18_20_mst.ts
- 29. src/data/tickets/ticket21_24.ts
- 30. src/data/tickets/ticket6_13.ts

Интерактивные компоненты и симуляторы

- 31. src/components/ArticulationPointsViz.tsx
- 32. src/components/BellmanFordViz.tsx
- 33. src/components/BfsViz.tsx
- 34. src/components/ChapterImage.tsx
- 35. src/components/ChapterNav.tsx
- 36. src/components/ChapterQuiz.tsx
- 37. src/components/ChapterView.tsx
- 38. src/components/ComplexityViz.tsx
- 39. src/components/DfsBridgesSimulator.tsx
- 40. src/components/DijkstraViz.tsx
- 41. src/components/DPVisualizer.tsx
- 42. src/components/EulerSimulator.tsx
- 43. src/components/FloydViz.tsx
- 44. src/components/GraphComponentsViz.tsx
- 45. src/components/GraphTraversalViz.tsx
- 46. src/components/HeapViz.tsx
- 47. src/components/hints/demoKit.tsx
- 48. src/components/hints/demosExtra.tsx
- 49. src/components/hints/demosGraph.tsx
- 50. src/components/hints/demosStruct.tsx
- 51. src/components/hints/MiniDemo.tsx

86. src/layout/header.html

Утилиты

87. src/utils/cn.ts

88. src/utils/exportHtml.ts

89. src/utils/ticketLinks.ts

Скрипты сборки и экспорта

90. scripts/audit-coverage.mjs

91. scripts/audit-terms.mjs

92. scripts/export/build-guide-pdf.mjs

93. scripts/export/build-pdf.mjs

94. scripts/export/capture-viz-shots.mjs

95. scripts/export/collect-code.mjs

96. scripts/export/common.mjs

97. scripts/export/html-to-blocks.mjs

98. scripts/export/render-pages.mjs

99. scripts/verify/browser-checks.tsx

100. scripts/verify/check-links.mjs

101. scripts/verify/check-reference-code.mjs

102. scripts/verify/dom-env.mjs

103. scripts/verify/live-matrix.test.mjs

104. scripts/verify/probe-compiler.tsx

105. scripts/verify/README.md

106. scripts/verify/run-browser-checks.mjs

107. scripts/verify/run-probe.mjs

CI / автоматизация

108. .github/workflows/deploy-pages.yml

109. .github/workflows/rebuild-pdf.yml

110. .github/workflows/verify.yml

Прочее

111. .ai_skills/universal_guide_authoring/SKILL.md

```
contents: read
pages: write
id-token: write
```

```
concurrency:
  group: pages
  cancel-in-progress: true
```

```
jobs:
  build:
    name: Build static site
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v6

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: 24
          cache: npm

      - name: Install dependencies
        run: npm ci --no-audit --no-fund

      - name: Build for the repository subpath
        env:
          VITE_BASE: /algv0/
        run: npm run build

      - name: Make GitHub Pages fallback
        run: cp dist/index.html dist/404.html

      - name: Upload Pages artifact
        uses: actions/upload-pages-artifact@v5
        with:
          path: ./dist
```

```
+ group: pages
+ cancel-in-progress: true
+
+ jobs:
+   build:
+     name: Build static site
+     runs-on: ubuntu-latest
+     steps:
+       - name: Checkout
+         uses: actions/checkout@v6
+
+       - name: Setup Node.js
+         uses: actions/setup-node@v7
+         with:
+           node-version: 24
+           cache: npm
+
+       - name: Install dependencies
+         run: npm ci --no-audit --no-fund
+
+       - name: Build for the repository subpath
+         env:
+           VITE_BASE: /algv0/
+         run: npm run build
+
+       - name: Make GitHub Pages fallback
+         run: cp dist/index.html dist/404.html
+
+       - name: Upload Pages artifact
+         uses: actions/upload-pages-artifact@v5
+         with:
+           path: ./dist
+
+   deploy:
+     name: Publish to GitHub Pages
+     needs: build
+     runs-on: ubuntu-latest
+     environment:
```

```
      required: false
      default: "150"

concurrency:
  group: rebuild-pdf-${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: write

jobs:
  rebuild-pdf:
    name: код → txt → png → pdf
    runs-on: ubuntu-latest
    # не реагируем на собственный коммит бота
    if: "!contains(github.event.head_commit.message, '[bot]')"

    steps:
      - name: Checkout
        uses: actions/checkout@v6
        with:
          fetch-depth: 0
          persist-credentials: true

      - name: Setup Node.js
        uses: actions/setup-node@v7
        with:
          node-version: "24"
          cache: npm

      - name: Install ImageMagick + DejaVu fonts
        run: |
          sudo apt-get update -qq
          sudo apt-get install -y --no-install-recommends imagemagick
          # На Ubuntu политика ImageMagick иногда запрещает конвертировать текст в изображение
          # разрешаем то, что нужно пайплайну (text -> png)
          for policy in /etc/ImageMagick-6/policy.xml /usr/share/ImageMagick-6/policy.xml; do
            if [ -f "$policy" ]; then
              sed -i 's/<policy domain="text" rights="none"/<policy domain="text" rights="write"/g' $policy
            fi
          done
```

```
      name: full-code-export
      path: |
        public/export/full_code_all_pages.pdf
        public/export/full_code_all_pages.txt
      if-no-files-found: error
      retention-days: 30

- name: Upload artifacts (PNG-страницы)
  uses: actions/upload-artifact@v7
  with:
    name: full-code-pages-png
    path: tmp/export-pages/*.png
    if-no-files-found: error
    retention-days: 7

- name: Commit rebuilt export back to the branch
  run: |
    git config user.name "github-actions[bot]"
    git config user.email "41898282+github-action@users.noreply.github.com"
    git add -- public/export/full_code_all_pages.*
    if git diff --cached --quiet; then
      echo "Экспорт не изменился — коммит не нужен"
      exit 0
    fi
    git commit -m "chore(export): пересборка full-code-export"
    git push origin "HEAD:${GITHUB_REF_NAME}"
  ....

### Patch относительно `main`

```diff
diff --git a/.github/workflows/rebuild-pdf.yml b/.github/workflows/rebuild-pdf.yml
index bdc3c33..ff49c0c 100644
--- a/.github/workflows/rebuild-pdf.yml
+++ b/.github/workflows/rebuild-pdf.yml
@@ -42,15 +42,15 @@ jobs:

 steps:
```

Универсальное пособие • полный исходный код

-----  
### Полное содержимое после изменений

```
````html
<!doctype html>
<html lang="ru" class="dark bg-slate-950 text-slate-100" >
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, >
    <link rel="icon" type="image/svg+xml" href="%BASE_URL" />
    <title> Дискретка для тупых – Интерактивный гид по
  </head>
  <body class="bg-slate-950 text-slate-100 min-h-screen" >
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
````
```

### Patch относительно `main`

```
````diff
diff --git a/index.html b/index.html
index cdea4c7..a44c135 100644
--- a/index.html
+++ b/index.html
@@ -3,6 +3,7 @@
   <head>
     <meta charset="UTF-8" />
     <meta name="viewport" content="width=device-width, >
+    <link rel="icon" type="image/svg+xml" href="%BASE_URL" />
     <title> Дискретка для тупых – Интерактивный гид по
   </head>
   <body class="bg-slate-950 text-slate-100 min-h-screen" >
````
```

## `public/favicon.svg`

### Полное содержимое после изменений



```

import { SimulatorHub } from "../components/SimulatorHub
import { PythonCompiler } from "../components/PythonComp

export default function App() {
 const [activeTab, setActiveTab] = useState<"guide" |
 const [activeChapterId, setActiveChapterId] = useStat
 const [modalVizId, setModalVizId] = useState<string |

 const index = Math.max(
 0,
 chapters.findIndex((c) => c.id === activeChapterId)
);
 const activeChapter = chapters[index] ?? chapters[0];
 const prev = index > 0 ? chapters[index - 1] : undefin
 const next = index < chapters.length - 1 ? chapters[in

 const goTo = useCallback((id: string) => {
 setActiveChapterId(id);
 window.scrollTo({ top: 0, behavior: "smooth" });
 }, []);

 // Стрелки ← → листают темы (как на обучающих сайтах)
 useEffect(() => {
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(ta
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight" && next) goTo(next.id)
 if (e.key === "ArrowLeft" && prev) goTo(prev.id);
 };
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown",
 }, [goTo, next, prev]);

 const progress = useMemo(() => ((index + 1) / chapter

 return (
 <div className="min-h-screen bg-slate-950 text-slate

```

```

 <ChapterView
 key={activeChapter.id}
 chapter={activeChapter}
 prev={prev}
 next={next}
 index={index}
 total={chapters.length}
 onGo={goTo}
 onOpenSimulator={setModalVizId}
 />
 </main>
</div>
</>
) : activeTab === "simulator" ? (
 <main className="px-4 sm:px-6 lg:px-10 py-6 lg:py-8" >
 <SimulatorHub
 onOpen={setModalVizId}
 onGoChapter={({id}) => {
 setActiveTab("guide");
 goTo(id);
 }}
 />
 </main>
) : (
 <PythonCompiler />
)
}

<SimulatorModal vizId={modalVizId} onClose={() => {
 setModalVizId(null);
}} />

<footer className="p-8 text-center text-slate-600" >
 © 2026 Universal Educational Guide. Интерактивное
</footer>
</div>
);
}
....

```

```
`src/components/Navbar.tsx`
```

```
Полное содержимое после изменений
```

```
```tsx
```

```
import React, { useState } from 'react';
import { BookOpen, Cpu, Sparkles, FileDown, FileText, C
import { chapters } from '../data/content';
import { downloadBookHtml } from '../utils/exportHtml';
```

```
interface NavbarProps {
  activeTab: 'guide' | 'simulator' | 'compiler';
  setActiveTab: (tab: 'guide' | 'simulator' | 'compiler
}
```

```
export const Navbar: React.FC<NavbarProps> = ({ activeT
  const [busy, setBusy] = useState(false);
```

```
  const saveBook = async () => {
    setBusy(true);
    try {
      await downloadBookHtml(chapters);
    } finally {
      setBusy(false);
    }
  };
};
```

```
  return (
    <header className="bg-slate-900 border-b border-sla
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg
        ap-3 lg:gap-0">
          <div className="flex items-center gap-3 w-full
            <div className="bg-gradient-to-tr from-indigo
              k-0">
                <Sparkles className="w-6 h-6" />
              </div>
            <div className="min-w-0 flex-1">
```

```

        className={`flex-1 lg:flex-none flex items-
uration-200 whitespace-nowrap ${
            activeTab === 'compiler'
                ? 'bg-emerald-600 text-white shadow-lg
                : 'text-slate-400 hover:text-white'
        }}
    >
    <Terminal className="w-4 h-4" /> Python
</button>
<a
    id="download-pdf-btn"
    href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.pdf"
    target="_blank"
    rel="noreferrer"
    className="flex items-center justify-center
over:bg-emerald-600/20 border border-emerald
    title="Скачать полный PDF сборник всех стра
    >
    <FileDown className="w-4 h-4" /> Скачать PD
</a>
<a
    id="download-txt-btn"
    href={` ${import.meta.env.BASE_URL}export/full
download="full_code_all_pages.txt"
    target="_blank"
    rel="noreferrer"
    className="flex items-center justify-center
    :bg-sky-600/20 border border-sky-500/30 tra
    title="Скачать полный TXT файл со всем кодом
    >
    <FileText className="w-4 h-4" /> TXT
</a>
<button
    id="download-html-btn"
    onClick={saveBook}
    disabled={busy}
    className="flex items-center justify-center

```

```

+         <button
+             id="tab-compiler-btn"
+             onClick={() => setActiveTab('compiler')}
+             className={`flex-1 lg:flex-none flex items
duration-200 whitespace-nowrap ${
+                 activeTab === 'compiler'
+                 ? 'bg-emerald-600 text-white shadow-lg
+                 : 'text-slate-400 hover:text-white'
+             }`}
+         >
+         <Terminal className="w-4 h-4" /> Python
+     </button>
+     <a
+         id="download-pdf-btn"
+         href="/export/full_code_all_pages.pdf"
+         href={`${import.meta.env.BASE_URL}export/full
+         download="full_code_all_pages.pdf"
+         target="_blank"
+         rel="noreferrer"
@@ -73,7 +84,7 @@ export const Navbar: React.FC<NavbarP
    </a>
    <a
        id="download-txt-btn"
        href="/export/full_code_all_pages.txt"
        href={`${import.meta.env.BASE_URL}export/full
        download="full_code_all_pages.txt"
        target="_blank"
        rel="noreferrer"
    ....

```

`src/components/PythonCompiler.tsx`

Полное содержимое после изменений

```

````tsx
import { useCallback, useState } from "react";
import { CheckCircle2, ExternalLink, Info, Loader2, Play

```

```

export function PythonCompiler() {
 const [code, setCode] = useState(STARTER_CODE);
 const [stdin, setStdin] = useState("");
 const [output, setOutput] = useState("Нажмите «Запустить»");
 const [running, setRunning] = useState(false);
 const [runtimeReady, setRuntimeReady] = useState(false);
 const [runtimeMessage, setRuntimeMessage] = useState("");

 const runCode = useCallback(async () => {
 if (running) return;

 setRunning(true);
 setOutput("");
 setRuntimeMessage(runtimeReady ? "Выполняю код..." : "Готово: программа ничего не выполнила");

 const stdout: string[] = [];
 const stderr: string[] = [];
 const inputLines = stdin.split(/\r?\n/);

 try {
 const runtime = await getPythonRuntime();
 setRuntimeReady(true);
 setRuntimeMessage("Python готов — код выполняется");

 runtime.setStdout({ batched: (message) => stdout.push(message) });
 runtime.setStderr({ batched: (message) => stderr.push(message) });
 runtime.setStdin({ stdin: () => (inputLines.length > 0 ? inputLines.shift() : null) });
 await runtime.runPythonAsync(code);

 const text = [...stdout, ...stderr].join("");
 setOutput(text || "Готово: программа ничего не выполнила");
 } catch (error) {
 const captured = [...stdout, ...stderr].join("");
 setOutput(`${captured}${captured && !captured.endsWith('.') ? '.' : ''}`);
 setRuntimeMessage("Не удалось выполнить код — подробности в консоли");
 } finally {
 setRunning(false);
 }
 }, [code, stdin]);
}

```

```

 <span className="text-[11px] text-slate
</div>
<div className="flex items-center gap-2">
 <button
 type="button"
 onClick={reset}
 className="inline-flex items-center gap-2 text-slate-800 transition-colors"
 >
 <RotateCcw className="w-3.5 h-3.5" />
 </button>
 <button
 type="button"
 onClick={() => void runCode()}
 disabled={running}
 className="inline-flex items-center gap-2 text-white text-xs font-bold transition-colors"
 >
 {running ? <Loader2 className="w-3.5 h-3.5" /> : "Запустить"}
 </button>
</div>
</div>

<textarea
 value={code}
 onChange={(event) => setCode(event.target.value)}
 onKeyDown={(event) => {
 if ((event.ctrlKey || event.metaKey) && event.defaultPrevented) {
 void runCode();
 }
 }}
 spellCheck={false}
 aria-label="Код Python"
 className="block w-full min-h-[360px] resize-none focus:ring-2 focus:ring-inset focus:ring-slate-500"
 placeholder="Напишите Python-код..."

```

```
 <div className="flex gap-2 rounded-xl border border-gray-200 p-4">
 <Info className="w-4 h-4 shrink-0 text-indigo-600">
 Ctrl или Cmd + Enter запускает код. Введите код в поле ввода.

 </div>
 <div className="flex gap-2 rounded-xl border border-gray-200 p-4">
 <Info className="w-4 h-4 shrink-0 text-amber-600">
 Первый запуск скачивает примерно несколько килобайт кода.

 </div>
 <div className="flex gap-2 rounded-xl border border-gray-200 p-4">
 <Info className="w-4 h-4 shrink-0 text-emerald-600">
 Код выполняется в браузере и не отправляется на сервер.

 </div>
 </div>
 </div>
 </main>
);
}
....
```

### Patch относительно `main`

```
````diff
diff --git a/src/components/PythonCompiler.tsx b/src/components/PythonCompiler.tsx
new file mode 100644
index 00000000..1113e0f
--- /dev/null
+++ b/src/components/PythonCompiler.tsx
@@ -0,0 +1,205 @@
+import { useCallback, useState } from "react";
+import { CheckCircle2, ExternalLink, Info, Loader2, Plus } from "lucide-react";
+
+const PYODIDE_VERSION = "0.27.7";
+const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/bin/pyodide.js`;
+const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyodide/v${PYODIDE_VERSION}/index.html`;
+
+const STARTER_CODE = `# Первый запуск загрузит Python и установит библиотеки
+name = "алгоритмы"
+
+def main():
+    print("Привет, мир!")
+    # Здесь можно добавить свой код
+
+main()
+
+`;
```



```

+
+ const runCode = useCallback(async () => {
+   if (running) return;
+
+   setRunning(true);
+   setOutput("");
+   setRuntimeMessage(runtimeReady ? "Выполняю код..." :
+
+   const stdout: string[] = [];
+   const stderr: string[] = [];
+   const inputLines = stdin.split(/\r?\n/);
+
+   try {
+     const runtime = await getPythonRuntime();
+     setRuntimeReady(true);
+     setRuntimeMessage("Python готов – код выполняется");
+
+     runtime.setStdout({ batched: (message) => stdout
+     runtime.setStderr({ batched: (message) => stderr
+     runtime.setStdin({ stdin: () => (inputLines.length
+     await runtime.runPythonAsync(code);
+
+     const text = [...stdout, ...stderr].join("");
+     setOutput(text || "Готово: программа ничего не выв
+   } catch (error) {
+     const captured = [...stdout, ...stderr].join("")
+     setOutput(`${captured}${captured && !captured.en
+     setRuntimeMessage("Не удалось выполнить код – пр
+   } finally {
+     setRunning(false);
+   }
+ }, [code, stdin, running, runtimeReady]);
+
+ const reset = () => {
+   setCode(STARTER_CODE);
+   setStdin("");
+   setOutput("Нажмите «Запустить», чтобы увидеть резу.
+   };

```

```

late-800 transition-colors"
+
+       >
+       <RotateCcw className="w-3.5 h-3.5" />
+       </button>
+       <button
+       type="button"
+       onClick={() => void runCode()}
+       disabled={running}
+       className="inline-flex items-center
-60 text-white text-xs font-bold transition-colors"
+       >
+       {running ? <Loader2 className="w-3.5
+       {running ? "Запускаю..." : "Запустить"}
+       </button>
+       </div>
+     </div>
+
+     <textarea
+     value={code}
+     onChange={(event) => setCode(event.target
+     onKeyDown={(event) => {
+       if ((event.ctrlKey || event.metaKey) &
+       event.preventDefault();
+       void runCode();
+     }
+     }}
+     spellCheck={false}
+     aria-label="Код Python"
+     className="block w-full min-h-[360px] re
e-none focus:ring-2 focus:ring-inset focus:ring-emera
+     placeholder="Напишите Python-код..."
+     />
+
+     <div className="border-t border-slate-800
+     <label className="block px-4 pt-3 text-[
+       Ввод для input() • по одной строке на
+     </label>
+     <textarea

```

```
-----
+         </div>
+         <div className="flex gap-2 rounded-xl border
+             <Info className="w-4 h-4 shrink-0 text-eme
+             <span>Код выполняется в браузере и не отпра
+         </div>
+     </div>
+ </div>
+ </main>
+ );
+}
+...
```

`vite.config.ts`

Полное содержимое после изменений

```
````typescript
import path from "path";
import { fileURLToPath } from "url";
import tailwindcss from "@tailwindcss/vite";
import react from "@vitejs/plugin-react";
import { defineConfig } from "vite";
import { viteSingleFile } from "vite-plugin-singlefile"

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// https://vite.dev/config/
export default defineConfig({
 // На GitHub Pages приложение живёт в /algv0/, локальн
 // VITE_BASE можно переопределить, если репозиторий б
 base: process.env.VITE_BASE || "/",
 plugins: [react(), tailwindcss(), viteSingleFile()],
 resolve: {
 alias: {
 "@": path.resolve(__dirname, "src"),
 },
 },
},
```

Универсальное пособие • полный исходный код

-----  
КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 12 | РАЗМЕР: 7  
-----

# AI Agent Custom Instructions

You are the author of a Universal Educational Guide (Универсальное

**\*\*CRITICAL DIRECTIVE\*\***:

Before you start creating new chapters, editing content or using the `view_file` tool to read the skill instructions located at `.ai_skills/universal_guide_authoring/SKILL.md``

This file contains the mandatory rich HTML templates, strictly required to maintain the high quality of this universal guide.

Do NOT generate plain markdown chapters or invent new HTML tags.

-----  
ФАЙЛ 3/111: GEMINI.md

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 16 | РАЗМЕР: 2  
-----

# Gemini App Canvas Instructions

Ты — AI-ассистент, работающий в Gemini App Canvas (или Gemini App Canvas).  
При запросе пользователя создать или сгенерировать новую страницу, ты **\*\*строго\*\*** придерживаешься готовых шаблонов разметки, описанных ниже.

## Правила для малых страниц (Билетов / Глав)

- \*\*Единый стиль:\*\*** Всегда используй Tailwind CSS и HTML-теги.
- \*\*Структура JSON/TS:\*\*** Каждая новая малая страница должна быть описана в формате `{ "id": "...", "content": ... }` для вставки в ``src/data/content.ts``.
- \*\*Строгая структура контента:\*\***
  - Заголовок с цветным бейджем (например, ``bg-indigo-100``).
  - Определение/правило в центре (``border-blue-500`` или ``border-purple-500``).
  - Обязательная сетка аналогий (2 колонки: неформальный язык и формальный язык).

---

ФАЙЛ 6/111: package.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 54 | РАЗМЕР: 2

---

```
{
 "name": "algv0-exam-guide",
 "private": true,
 "version": "0.0.0",
 "type": "module",
 "scripts": {
 "dev": "vite",
 "build": "vite build",
 "preview": "vite preview",
 "audit:content": "node scripts/audit-coverage.mjs",
 "audit:terms": "node scripts/audit-terms.mjs",
 "export:txt": "node scripts/export/collect-code.mjs",
 "export:png": "node scripts/export/render-pages.mjs",
 "export:pdf": "node scripts/export/build-pdf.mjs",
 "export": "npm run export:txt && npm run export:png",
 "export:guide": "node scripts/export/build-guide-pdf.mjs",
 "export:shots": "node scripts/export/capture-viz-shots.mjs",
 "verify:unit": "node scripts/verify/live-matrix.test.mjs",
 "verify:code": "node scripts/verify/check-reference.mjs",
 "verify:links": "node scripts/verify/check-links.mjs",
 "verify:dom": "node scripts/verify/run-browser-check.mjs",
 "verify:probe": "node scripts/verify/run-probe.mjs",
 "verify": "npm run verify:unit && npm run verify:links",
 },
 "dependencies": {
 "clsx": "2.1.1",
 "lucide-react": "^1.21.0",
 "motion": "^12.40.0",
 "pdfkit": "^0.20.1",
 "react": "19.2.6",
 "react-dom": "19.2.6",
 }
}
```

Универсальное пособие • полный исходный код

```

npm run dev # предпросмотр на http://0.0.0.0:5
npm run build # сборка в dist/ (single-file)
````
```

Экспорт кода: код → txt → png → pdf

Пайплайн собирает ****весь**** исходный код репозитория в о

| Команда | Шаг | Результат |
|----------------------|-----------|---|
| `npm run export:txt` | код → txt | `public/export/full_code_all_pages.txt` |
| `npm run export:png` | txt → png | `tmp/export-pages/` |
| `npm run export:pdf` | png → pdf | `public/export/full_code_all_pages.pdf` |
| `npm run export` | всё сразу | txt + png + pdf |

Скрипты лежат в `scripts/export/`. Требуется ImageMagick

```
```bash  
sudo apt-get install -y imagemagick fonts-dejavu-core
````
```

Настройки через переменные окружения:

- * `EXPORT\_DENSITY=200` – DPI растеризации (по умолчанию 300)
- * `EXPORT\_GRAYSCALE=1` – оттенки серого вместо 1-битного

Автоматизация (CI)

`github/workflows/rebuild-pdf.yml` запускается ****на каж**
пересобирает TXT → PNG → PDF, кладёт PNG-страницы и док
и коммитит обновлённые `public/export/full\_code\_all\_pages`
(с меткой `[skip ci]`, чтобы не зациклиться).

Приложение отдаёт эти файлы по ссылкам `/export/full\_code`

Аудит контента

```
```bash
```

## ## Заголовки

- \* Заголовки секций: Используют стандартный тег `<h2>` и
- \* Подзаголовки: Используют тег `<h3>`.

## ## Текст и параграфы

- \* Обычный текст содержится в тегах `<p>`.
- \* Блоки "Шиза" (Аналогии) и другая текстовая инфографика параграфы `<p>`.

## ## Математические формулы

В приложении используется MathJax.

- \* В самом исходном коде страницы (и внутри тегов) формулы.
- \* Таким образом, при извлечении текстового содержимого математически остаются в валидном Markdown-формате латеха (и

## ## Картинки и визуализации (Схемы)

Вместо обычных `<img>` в приложении используются векторы

- \* Они помещены внутрь оберток с классом `<div class="img">`
- \* Все отрисовки находятся внутри векторного тега `<svg>`
- \* Подпись к картинке, если она есть, идет сразу после `<img>`

## ## Списки

- \* Стандартные списки оформлены через `<ul class="list-n">`

## ## Валидный экспорт в Markdown (встроенный)

Экспорт перехватывает основные структурные компоненты HTML

1. Заголовки `<h2>` и `<h3>` заменяются на `##` и `###`
2. Элементы `.analogy-badge` получают выделение маркера
3. Теги `<svg>` заменяются строкой-заглушкой `[ Иллюстрация ]`.
4. Элементы списка `<li>` получают маркер `-`.
5. Результат извлекается в виде текста и чистится от лишнего

Универсальное пособие • полный исходный код

```

* `stack-dfs` → `StackViz.tsx` — **Стек и DFS**
* `queue-bfs` → `QueueViz.tsx` + `BfsViz.tsx` — **Очередь и BFS**
* `heap-beam-search` → `HeapViz.tsx` — **Куча и лучевой поиск**
* `segment-trees` → `SegmentTreeVisualizer.tsx` — **Деревья отрезков**
* `dynamic-programming` → `DPVisualizer.tsx` — **Динамическое программирование**

```

## Уровень 3. Есть 2D, но НЕТ бытовой аналогии («сухари»)

Глава	id	Комментарий
15. Планарность: K5, K3,3 и формула Эйлера	`planarity-euler`	т «на пальцах».
16. Флойд (Флойд–Уоршелл)	`floyd`	Есть матричный алгоритм
17. Алгоритм Джонсона	`johnson-algo`	Перевзвешивание
Обзор: Кратчайшие пути и Графы	`intro`	Вводная страница
Эйлер: Путь ≠ Цикл	`euler-path-vs-cycle`	Ключевая концепция

---

## Уровень 4. Есть аналогия, но НЕТ 2D-визуализации («сухари»)

> Замечание: «5. Двумерная матрица префиксных сумм» пока закрыто

Глава	id	Чего не хватает
3. Декартово дерево (Treap)	`treap`	Нет ни визуализации, ни интерактива
8. Графы. Компоненты связности	`graph-components`	Нет визуализации
24. Классы сложности, сведение задач	`complexity-classes`	Нет визуализации

---

## Сводная таблица по всем главам

#	Глава	Аналогий	2D-виз.	SVG	Картинка
---	---	---	---	---	---



ФАЙЛ 9/111: tsconfig.json

КАТЕГОРИЯ: Конфигурация проекта | СТРОК: 32 | РАЗМЕР: 6

---

```
{
 "compilerOptions": {
 "target": "ES2020",
 "useDefineForClassFields": true,
 "lib": ["ES2020", "DOM", "DOM.Iterable"],
 "module": "ESNext",
 "skipLibCheck": true,
 "types": ["node"],

 /* Bundler mode */
 "moduleResolution": "bundler",
 "allowImportingTsExtensions": true,
 "resolveJsonModule": true,
 "isolatedModules": true,
 "noEmit": true,
 "jsx": "react-jsx",

 /* Path mapping */
 "baseUrl": ".",
 "paths": {
 "@/*": ["src/*"]
 },

 /* Linting */
 "strict": true,
 "noUnusedLocals": true,
 "noUnusedParameters": true,
 "noFallthroughCasesInSwitch": true
 },
 "include": ["src", "vite.config.ts"]
}
```

```
 },
 });
}
```

---

## РАЗДЕЛ: ЯДРО ПРИЛОЖЕНИЯ

---

ФАЙЛ 11/111: src/App.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 218 | РАЗМЕР: 11.3 КБ

```
import { useCallback, useEffect, useMemo, useRef, useState } from "react";
import { PanelLeftClose, PanelLeftOpen } from "lucide-react";
import { chapters } from "../data/content";
import { Navbar } from "../components/Navbar";
import { Sidebar } from "../components/Sidebar";
import { MobileToc } from "../components/MobileToc";
import { ChapterView } from "../components/ChapterView";
import { ChapterNav } from "../components/ChapterNav";
import { SimulatorModal } from "../components/hints/SimulatorModal";
import { LiteGuide } from "../components/LiteGuide";
import { PythonCompiler } from "../components/PythonCompiler";
```

```
/**
```

```
 * «Лёгкий» режим для печати и внешних парсеров: `?lite`
```

```
 * или `?lite=1&topic=mst` (одна тема). Без навигации, без поиска.
```

```
 */
```

```
function readLiteMode(): { lite: boolean; topic?: string } {
 if (typeof window === "undefined") return { lite: false };
 const q = new URLSearchParams(window.location.search);
 const lite = q.has("lite") || q.get("view") === "lite";
 return { lite, topic: q.get("topic") ?? undefined };
}
```

```
export default function App() {
 const liteMode = useMemo(readLiteMode, []);
}
```

```

 setSidebarMode((m) => {
 const next = m === "wide" ? "compact" : m === "co
 window.localStorage.setItem("sidebarMode", next);
 return next;
 });
 }, []);
const sidebarCollapsed = sidebarMode === "hidden";

// Десктопный отступ контента = ширине панели компиля
const contentRef = useRef<HTMLDivElement>(null);
useEffect(() => {
 const apply = () => {
 const el = contentRef.current;
 if (!el) return;
 el.style.paddingRight = compilerOpen && window.inn
 };
 apply();
 window.addEventListener("resize", apply);
 return () => window.removeEventListener("resize", a
}, [compilerOpen, compilerWidth]);

const index = Math.max(
 0,
 chapters.findIndex((c) => c.id === activeChapterId)
);
const activeChapter = chapters[index] ?? chapters[0];
const prev = index > 0 ? chapters[index - 1] : undefi
const next = index < chapters.length - 1 ? chapters[in

const goTo = useCallback((id: string) => {
 setActiveChapterId(id);
 window.scrollTo({ top: 0, behavior: "smooth" });
}, []);

// Открыть компилятор и подогнать визуализацию вверх:
const openCompilerWithViz = useCallback(() => {
 setCompilerOpen(true);
 requestAnimationFrame(() => {

```

```

<div className="flex flex-col lg:flex-row max-w-
 /* Сайдбар только на десктопе — на мобильном
 <div
 id="sidebar-root"
 className={`hidden lg:block shrink-0 border
 -all duration-200 print:hidden ${sidebarCol
 " : "lg:w-56"}`}
 >
 {!sidebarCollapsed && (
 <Sidebar selectedChapterId={activeChapterId}
)}
 </div>
 /* Вкладка-переключатель содержания у левого
 <button
 type="button"
 onClick={toggleSidebar}
 className="hidden lg:flex items-center gap-1
 border-slate-700 bg-slate-900/90 px-1 py-3
 aria-label={sidebarMode === "wide" ? "Сжать
 ание"}
 title={sidebarMode === "wide" ? "Сжать содер
 }
 >
 {sidebarCollapsed ? <PanelLeftOpen className=
 <span className="text-[10px] font-bold [wri
 </button>

 <main id="main-content" className="flex-1 min
 /* Шапка главы с быстрыми переходами */
 <div className="flex items-center justify-b
 <div className="min-w-0">
 <div className="text-[10px] uppercase t
 {activeChapter.category || "Универсал
 </div>
 <h2 className="text-base sm:text-xl fon
 {activeChapter.title}
 </h2>
 </div>
 </div>

```

```
 onHeightChange={setCompilerHeight}
 onClose={() => setCompilerOpen(false)}
 />
))
 </div>
);
}
```

---

ФАЙЛ 12/111: src/hooks/useTermHints.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 129 | РАЗМЕР: 5.1 K

---

```
import { useEffect } from "react";
import { GLOSSARY_LABELS } from "../data/glossary";

/**
 * Ограничители, чтобы текст не рябил, но и чтобы подсказки
 *
 * Считаем не «сколько раз встретился термин», а «сколько
 * написание». Иначе в главе про splay первые же «Splay
 * лимит, и слова «Zig-Zag», «вращений», «AVL-дереве» о
 */
const MAX_PER_SURFACE = 2;
const MAX_PER_TERM = 8;

const SKIP_SELECTOR = "code, pre, script, style, a, tex

/** Числовые константы TreeWalker: глобального `NodeFile
const SHOW_TEXT = 4;
const FILTER_ACCEPT = 1;
const FILTER_REJECT = 2;

const escapeRe = (s: string) => s.replace(/[\.*\+\?\^\$\{\}\(\)]/

/** Одна большая регулярка из всех меток: длинные раньше
function buildRegex(): RegExp {
```

```

 re.lastIndex = 0;
 if (!re.test(text)) continue;
 re.lastIndex = 0;

 const frag = document.createDocumentFragment();
 let last = 0;
 let m: RegExpExecArray | null;

 while ((m = re.exec(text)) !== null) {
 const surface = m[1].toLowerCase();
 const id = labelToId.get(surface);
 if (!id) continue;

 const usedTerm = perTerm.get(id) ?? 0;
 const usedSurface = perSurface.get(surface) ?? 0;
 if (usedTerm >= MAX_PER_TERM || usedSurface >= MAX_PER_SURFACE) continue;
 perTerm.set(id, usedTerm + 1);
 perSurface.set(surface, usedSurface + 1);

 if (m.index > last) frag.appendChild(document.createElement("span"));
 const span = document.createElement("span");
 span.className = "term-hint";
 span.dataset.termId = id;
 span.tabIndex = 0;
 span.setAttribute("role", "button");
 span.setAttribute("aria-label", `Подсказка: ${m[1]}`);
 span.textContent = m[1];
 frag.appendChild(span);
 last = m.index + m[1].length;
 }

 if (last === 0) continue;
 if (last < text.length) frag.appendChild(document.createElement("span"));
 textNode.parentNode?.replaceChild(frag, textNode);
}

/**

```

```
.neon-glow-blue {
 box-shadow: 0 0 15px rgba(59, 130, 246, 0.5);
}
.neon-glow-emerald {
 box-shadow: 0 0 15px rgba(16, 185, 129, 0.5);
}
.neon-glow-rose {
 box-shadow: 0 0 15px rgba(244, 63, 94, 0.5);
}
.neon-glow-yellow {
 box-shadow: 0 0 20px rgba(234, 179, 8, 0.8);
}
.no-scrollbar::-webkit-scrollbar {
 display: none;
}
.no-scrollbar {
 scrollbar-width: none;
}
}

@keyframes pulse-gold {
 0%,
 100% {
 stroke: #eab308;
 stroke-width: 5;
 filter: drop-shadow(0 0 2px rgba(234, 179, 8, 0.5));
 }
 50% {
 stroke: #fef08a;
 stroke-width: 8;
 filter: drop-shadow(0 0 12px rgba(234, 179, 8, 0.9));
 }
}

.animate-pulse-gold {
 animation: pulse-gold 1.5s infinite;
}
```

```
 color: #fff;
 outline: none;
}
@media (hover: none) {
 .term-hint {
 border-bottom-style: solid;
 background: rgba(99, 102, 241, 0.12);
 }
}

/* Ссылка на другой билет («...берите Форда–Беллмана (билет
.ticket-link {
 color: #a5b4fc;
 border-bottom: 1px dotted rgba(165, 180, 252, 0.7);
 text-decoration: none;
 border-radius: 3px;
 padding: 0 1px;
 cursor: pointer;
 transition:
 background-color 0.15s ease,
 color 0.15s ease;
}
.ticket-link:hover,
.ticket-link:focus-visible {
 background: rgba(99, 102, 241, 0.25);
 color: #fff;
 outline: none;
}

/* Контент главы приходит готовым HTML – чиним его под
.chapter-body {
 max-width: 100%;
 overflow-wrap: anywhere;
}

.chapter-body :where(section, div, p, li) {
 min-width: 0;
}
```



```
 line-height: 1.3 !important;
 }
 .chapter-body :where(h3) {
 font-size: 1.05rem !important;
 line-height: 1.35 !important;
 }
 .chapter-body :where(.p-6, .p-8) {
 padding: 0.9rem !important;
 }
 .chapter-body :where(.text-3xl) {
 font-size: 1.4rem !important;
 }
 .chapter-body :where(.text-2xl) {
 font-size: 1.2rem !important;
 }
 .chapter-body :where(.text-xl) {
 font-size: 1.05rem !important;
 }
}
```

```
/* Custom scrollbar styling for dark theme */
::-webkit-scrollbar {
 width: 8px;
 height: 8px;
}
::-webkit-scrollbar-track {
 background: #0f172a;
}
::-webkit-scrollbar-thumb {
 background: #334155;
 border-radius: 4px;
}
::-webkit-scrollbar-thumb:hover {
 background: #475569;
}
```

```
/* Индикатор длительности кадра в пошаговых визуализаци
@keyframes demoProgress {
```

```
main {
 padding: 0 !important;
 max-width: none !important;
}

/* текст темы – чёрным по белому, без теней и плашек
article.chapter-body .prose,
article.chapter-body .prose * {
 color: #111827 !important;
 background-color: transparent !important;
 background-image: none !important;
 box-shadow: none !important;
 text-shadow: none !important;
 border-color: #d1d5db !important;
}

article.chapter-body .prose pre,
article.chapter-body .prose code {
 background-color: #f3f4f6 !important;
 color: #111827 !important;
 font-size: 9pt;
}

article.chapter-body .prose table {
 border-collapse: collapse;
}

article.chapter-body .prose th,
article.chapter-body .prose td {
 border: 1px solid #9ca3af !important;
}

article.chapter-body .prose h2,
article.chapter-body .prose h3,
article.chapter-body .prose h4 {
 page-break-after: avoid;
}
```

Универсальное пособие • полный исходный код

---

ФАЙЛ 14/111: src/main.tsx

КАТЕГОРИЯ: Ядро приложения | СТРОК: 11 | РАЗМЕР: 230 В

---

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "./index.css";
import App from "./App";

createRoot(document.getElementById("root")!).render(
 <StrictMode>
 <App />
 </StrictMode>
);
```

---

ФАЙЛ 15/111: src/types.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 9 | РАЗМЕР: 153 В

---

```
export interface Chapter {
 id: string;
 title: string;
 type: "html" | "markdown";
 content: string;
 description?: string;
 category?: string;
}
```

---

ФАЙЛ 16/111: src/vite-env.d.ts

КАТЕГОРИЯ: Ядро приложения | СТРОК: 19 | РАЗМЕР: 324 В

---

```
/// <reference types="vite/client" />
```

```
<div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
```

```
<div class="space-y-8">
```

```
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
```

```
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 <p class="text-sm font-mono text-slate-300">
] [k] </p>
 </div>
```

```
<p class="text-slate-300 text-sm mb-4">Ячейка
лины
">i. Любой отрезок накрывается двумя
перация идемпотентна: min,
, побитовые И и ИЛИ.</p>
```

```
<div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-xl font-mono text-white">
 <p class="text-sm font-mono text-slate-300">
вет = op из четырёх блоков kx × ky, накрывае
</div>
```

```
<p class="text-slate-300 text-sm mb-4">Двумя
по столбцам. Отсюда и цена: уровней становится
s="font-mono text-emerald-300">log N
```

```
<div class="grid grid-cols-1 sm:grid-cols-3">
 <div class="bg-slate-900 border border-slate-700">
 <p class="text-[10px] uppercase tracking-wider">
```

```

 <div class="absolute -top-3 left-4 border-rose-500 shadow-md">
 Аналогия 2: Торт-пирамидка
 </div>
 <p class="text-slate-300 text-sm mb-4">
 ия, зато красивый, как пирамидка: каждый из
 сех блоков $2^{kx} \times 2^{ky}$), и ответ на запрос
 ой операции нет: крем обратно в торт не запы
 $n(x, x) = x$), и цена $O(1)$ – сто коржей вмес
 <div class="rounded-lg bg-slate-950">
 <div>
 $n \ O(1)$ </div>
 <div>
 = 100 копий матрицы</div>
 </div>
 <p class="text-[12px] text-slate-400">
 шает). Менять элементы тоже нельзя. Сумму ч
 </div>
 </div>
 <p class="text-slate-400 text-sm mt-4">Поря
 ерению) → 2D: построение (уровни <span
 ос (прямоугольник раскладывается на чет
 </div>

<div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <p class="text-slate-300 text-sm mb-4">Эти
 за
 они решают разные задачи.</p>

<div class="bg-slate-800/70 p-5 rounded-lg">
 <p class="text-sm font-bold text-white">
 <p class="text-slate-300 text-sm mb-3">
 цы, у разреженной таблицы – размер, умножен
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border">
 <th class="py-1.5 pr-2 font
```

Почему префиксная тонкая

</div>

<p class="text-slate-300 text-sm mb-4" style="border: 1px solid #000; padding: 5px; display: inline-block;">P[i][j]</p>  
ругой прямоугольник выражается через четыре

</div>

<div class="bg-slate-900 p-6 rounded-lg">

<div class="absolute -top-3 left-4 border-rose-500 shadow-md">

Почему разреженная жирная

</div>

<p class="text-slate-300 text-sm mb-4" style="border: 1px solid #000; padding: 5px; display: inline-block;">ум на его части не вычислишь. Приходится хранить  $\log N \times \log M$  на ка

</div>

</div>

<p class="text-slate-300 text-sm mt-4">Вывод: **лицу** – перекрывающиеся блоки посчитают **лицу** – у минимума нет обратной операции. Стр

</div>

<div class="bg-slate-700/50 p-6 rounded-xl border">

<h3 class="text-xl font-bold text-amber-400">

<div class="bg-rose-950/30 p-5 rounded-lg border">

<ul class="list-disc pl-5 space-y-2 text">

<li><b>Сумма через разреженную таблицу префиксные (билет 5).</b></li>

<li><b>Память.</b>  $N = M = 1000$  даём  $\log N \times \log M$  считайте ячейки, потом пишите код.</li>

<li><b>Один log вместо двух.</b> В  $\log N \times \log M$  st[0][0] st[0][1] st[1][0] st[1][1] st[2][0] st[2][1] st[3][0] st[3][1] st[4][0] st[4][1] st[5][0] st[5][1] st[6][0] st[6][1] st[7][0] st[7][1] st[8][0] st[8][1] st[9][0] st[9][1] st[10][0] st[10][1] st[11][0] st[11][1] st[12][0] st[12][1] st[13][0] st[13][1] st[14][0] st[14][1] st[15][0] st[15][1] st[16][0] st[16][1] st[17][0] st[17][1] st[18][0] st[18][1] st[19][0] st[19][1] st[20][0] st[20][1] st[21][0] st[21][1] st[22][0] st[22][1] st[23][0] st[23][1] st[24][0] st[24][1] st[25][0] st[25][1] st[26][0] st[26][1] st[27][0] st[27][1] st[28][0] st[28][1] st[29][0] st[29][1] st[30][0] st[30][1] st[31][0] st[31][1] st[32][0] st[32][1] st[33][0] st[33][1] st[34][0] st[34][1] st[35][0] st[35][1] st[36][0] st[36][1] st[37][0] st[37][1] st[38][0] st[38][1] st[39][0] st[39][1] st[40][0] st[40][1] st[41][0] st[41][1] st[42][0] st[42][1] st[43][0] st[43][1] st[44][0] st[44][1] st[45][0] st[45][1] st[46][0] st[46][1] st[47][0] st[47][1] st[48][0] st[48][1] st[49][0] st[49][1] st[50][0] st[50][1] st[51][0] st[51][1] st[52][0] st[52][1] st[53][0] st[53][1] st[54][0] st[54][1] st[55][0] st[55][1] st[56][0] st[56][1] st[57][0] st[57][1] st[58][0] st[58][1] st[59][0] st[59][1] st[60][0] st[60][1] st[61][0] st[61][1] st[62][0] st[62][1] st[63][0] st[63][1] st[64][0] st[64][1] st[65][0] st[65][1] st[66][0] st[66][1] st[67][0] st[67][1] st[68][0] st[68][1] st[69][0] st[69][1] st[70][0] st[70][1] st[71][0] st[71][1] st[72][0] st[72][1] st[73][0] st[73][1] st[74][0] st[74][1] st[75][0] st[75][1] st[76][0] st[76][1] st[77][0] st[77][1] st[78][0] st[78][1] st[79][0] st[79][1] st[80][0] st[80][1] st[81][0] st[81][1] st[82][0] st[82][1] st[83][0] st[83][1] st[84][0] st[84][1] st[85][0] st[85][1] st[86][0] st[86][1] st[87][0] st[87][1] st[88][0] st[88][1] st[89][0] st[89][1] st[90][0] st[90][1] st[91][0] st[91][1] st[92][0] st[92][1] st[93][0] st[93][1] st[94][0] st[94][1] st[95][0] st[95][1] st[96][0] st[96][1] st[97][0] st[97][1] st[98][0] st[98][1] st[99][0] st[99][1] st[100][0] st[100][1] st[101][0] st[101][1] st[102][0] st[102][1] st[103][0] st[103][1] st[104][0] st[104][1] st[105][0] st[105][1] st[106][0] st[106][1] st[107][0] st[107][1] st[108][0] st[108][1] st[109][0] st[109][1] st[110][0] st[110][1] st[111][0] st[111][1] st[112][0] st[112][1] st[113][0] st[113][1] st[114][0] st[114][1] st[115][0] st[115][1] st[116][0] st[116][1] st[117][0] st[117][1] st[118][0] st[118][1] st[119][0] st[119][1] st[120][0] st[120][1] st[121][0] st[121][1] st[122][0] st[122][1] st[123][0] st[123][1] st[124][0] st[124][1] st[125][0] st[125][1] st[126][0] st[126][1] st[127][0] st[127][1] st[128][0] st[128][1] st[129][0] st[129][1] st[130][0] st[130][1] st[131][0] st[131][1] st[132][0] st[132][1] st[133][0] st[133][1] st[134][0] st[134][1] st[135][0] st[135][1] st[136][0] st[136][1] st[137][0] st[137][1] st[138][0] st[138][1] st[139][0] st[139][1] st[140][0] st[140][1] st[141][0] st[141][1] st[142][0] st[142][1] st[143][0] st[143][1] st[144][0] st[144][1] st[145][0] st[145][1] st[146][0] st[146][1] st[147][0] st[147][1] st[148][0] st[148][1] st[149][0] st[149][1] st[150][0] st[150][1] st[151][0] st[151][1] st[152][0] st[152][1] st[153][0] st[153][1] st[154][0] st[154][1] st[155][0] st[155][1] st[156][0] st[156][1] st[157][0] st[157][1] st[158][0] st[158][1] st[159][0] st[159][1] st[160][0] st[160][1] st[161][0] st[161][1] st[162][0] st[162][1] st[163][0] st[163][1] st[164][0] st[164][1] st[165][0] st[165][1] st[166][0] st[166][1] st[167][0] st[167][1] st[168][0] st[168][1] st[169][0] st[169][1] st[170][0] st[170][1] st[171][0] st[171][1] st[172][0] st[172][1] st[173][0] st[173][1] st[174][0] st[174][1] st[175][0] st[175][1] st[176][0] st[176][1] st[177][0] st[177][1] st[178][0] st[178][1] st[179][0] st[179][1] st[180][0] st[180][1] st[181][0] st[181][1] st[182][0] st[182][1] st[183][0] st[183][1] st[184][0] st[184][1] st[185][0] st[185][1] st[186][0] st[186][1] st[187][0] st[187][1] st[188][0] st[188][1] st[189][0] st[189][1] st[190][0] st[190][1] st[191][0] st[191][1] st[192][0] st[192][1] st[193][0] st[193][1] st[194][0] st[194][1] st[195][0] st[195][1] st[196][0] st[196][1] st[197][0] st[197][1] st[198][0] st[198][1] st[199][0] st[199][1] st[200][0] st[200][1] st[201][0] st[201][1] st[202][0] st[202][1] st[203][0] st[203][1] st[204][0] st[204][1] st[205][0] st[205][1] st[206][0] st[206][1] st[207][0] st[207][1] st[208][0] st[208][1] st[209][0] st[209][1] st[210][0] st[210][1] st[211][0] st[211][1] st[212][0] st[212][1] st[213][0] st[213][1] st[214][0] st[214][1] st[215][0] st[215][1] st[216][0] st[216][1] st[217][0] st[217][1] st[218][0] st[218][1] st[219][0] st[219][1] st[220][0] st[220][1] st[221][0] st[221][1] st[222][0] st[222][1] st[223][0] st[223][1] st[224][0] st[224][1] st[225][0] st[225][1] st[226][0] st[226][1] st[227][0] st[227][1]

```

 else
 st[r][c][kx][ky] = min(

```

```

// запрос минимума на прямоугольнике (R1,C1)..(R2,C2)
int kx = log2(R2 - R1 + 1);
int ky = log2(C2 - C1 + 1);
int ans1 = min(st[R1][C1][kx][ky], st[R1][C2 - (1 << ky) + 1][C1][kx][ky]);
int ans2 = min(st[R2 - (1 << kx) + 1][C1][kx][ky], st[R2][C1][kx][ky]);
int minimum = min(ans1, ans2);

```

Четыре запроса к четырем углам запроса и вместе с ним образуют минимальный квадрат. Но.

Сколько это памяти в числах

- 100 × 100: уровни 7 × 7 ≈ 49
- 1000 × 1000: уровни 10 × 10 ≈ 100
- Префиксная сумма для той же матрицы

Почему? Потому что для каждого из нескольких сотен на сторону), а запросов очень много.

- [Префиксная сумма](#) – вторая половина этого сравнения:
- [Префиксная сумма](#)

```
<div class="text-center my-6">
 <p class="text-slate-400 text-xs uppercase">
 <p class="my-1 font-bold leading-none">
 И
 </p>
 <p class="text-slate-300 font-mono text">
</div>
```

```
<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Split – таможня
 </div>
 <p class="text-slate-300 text-sm">S
х». Спуск от корня: узел, который вместе с
скрытия – досматривается только одна ве
 </div>
```

```
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Аналогия 2: Merge – стыковка
 </div>
 <p class="text-slate-300 text-sm">M
аг – одно сравнение: чей у у корня больше,
сохраняет порядок x). Итого $O(h)$ сравнений
 </div>
 </div>
```

```
<div class="bg-slate-900/40 rounded-lg bord
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
быстрее $n \cdot \log n$.</p>
 <p class="text-slate-400 text-sm mb-1">
 <p class="text-slate-400 text-sm">• «У6
```



```

<summary class="p-4 font-bold text-slate-700 border-slate-700/50">
 Код: вставка, merge, erase (Скрыто)
</summary>
<div class="p-5 text-sm text-slate-300" style="background-color: #f0f0f0; border-radius: 10px; padding: 10px;">
<pre class="bg-slate-950 p-4 rounded" style="background-color: #2e3436; color: #eeeeec; font-family: monospace; font-size: 0.9em;">
def insert(root, key, pri):
 if root is None:
 return Node(key, pri)
 if key <= root.key: # равные x — в левое
 root.left = insert(root.left, key, pri)
 else:
 root.right = insert(root.right, key, pri)
 return root

for key, pri in pairs_sorted_by_pri_desc: # порядок э
 root = insert(root, key, pri)</pre>
<pre class="bg-slate-950 p-4 rounded" style="background-color: #2e3436; color: #eeeeec; font-family: monospace; font-size: 0.9em;">
def merge(a, b): # все x в a < все x в b
 if not a or not b: return a or b
 if a.pri > b.pri:
 a.right = merge(a.right, b); return a
 b.left = merge(a, b.left); return b

def erase(root, key): # удалить произвольный элемент
 if key < root.key: root.left = erase(root.left, key)
 elif key > root.key: root.right = erase(root.right, key)
 else: root = merge(root.left, root.right)
 return root</pre>
</div>
</details>
</div>
</div>
</section>`,
 },
 {
 id: "splay-tree",
 title: "4. Splay-дерево",
 },

```

по факту: ничего не хранит, дереву разр

```
<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
order border-emerald-500 shadow-md">
```

### Аналогия 1: Стопка бумага

`</div>`  
`<p class="text-slate-300 text-sm">С`  
тировку не ведёшь, но через неделю самое ну  
`erх».</p>`

```

 </div>
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4
border-rose-500 shadow-md">

```

### Аналогия 2: Тропинка через га

`</div>`  
`<p class="text-slate-300 text-sm">И,`  
рно вдвое. Первый проход дорогой, зато опла  
н.`</p>`

</div>  
</div>

Среднее поддерево  $\beta$  (между  $x$  и  $p$ ) – единственное дерево, которое имеет  $x$  и под  $p$ .

```
<pre class="bg-slate-950 p-4 rounded-lg overflow-relaxed">
```

$$\begin{array}{ccc} \begin{array}{cc} / & \backslash \\ x & y \\ / & \backslash \\ \alpha & \beta \end{array} & \begin{array}{c} x \text{ выше} \\ \text{-----} \rightarrow \\ p \text{ выше} \\ \text{<-----} \end{array} & \begin{array}{cc} / & \backslash \\ \alpha & p \\ / & \backslash \\ \beta & y \end{array} \end{array}$$

06х<sub>0</sub>,  
омать дерево поиска поворотами невозможно в  
омент «отстёжки», а врезка «Механика одного

#### class="text-lg font-bold text-white mt-4">

```
<pre class="bg-slate-950 p-4 rounded-lg">
 амбук из 8 узлов 8→7→6→5→4→3→2→1, ищем 1 (ч
наивно (x, потом x): 3 × «два зига» + Zig → высота 8
Zig-Zig (p, потом x): 3 × Zig-Zig + Zig → высота 8
бамбук из 16 узлов: наивно 16 → 16, Zig-Zig 16 → 10<
```

```
 <p class="text-slate-300 text-sm mt-3">
де: <code class="bg-slate-950 px-2 py-0.5 r
s="bg-slate-950 px-2 py-0.5 rounded font-mo
изации не будет.</p>
</div>
```

```
<p class="text-slate-300 text-sm mt-6">Полн
м 1. Спуск 8 узлов, затем три Zig-Zig по
8 до 6 – линия перестала быть линией. Наивно
о перевёрнутой. Важно не врать себе: на кор
ван не на одном запросе, а в сумме. Следующ
по пути. Отсюда амортизация: дорогой за
дом потенциала: потенциал Φ = сумма по узла
ь его; отсюда же теорема о статической опти
ь запросов).</p>
```

```
<div class="bg-slate-900/40 rounded-lg bord
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
га» – просто переворот.</p>
 <p class="text-slate-400 text-sm mb-1">
иза к верху; в симуляторе оно подсвечиваетс
 <p class="text-slate-400 text-sm">• «Sp
о амортизированная.</p>
</div>
```

```
<p class="text-slate-300 text-sm mt-4"><b c
ое <code class="bg-slate-950 px-2 py-0.5 ro
одиночная операция – до <code class="bg-sla
>O(n)</code>; n вставок подряд – амортизиро
-300 border border-slate-800">O(n log n)</c
```

```
<p class="text-slate-300 text-sm mt-4">Все
```

```

 rotate_up(x)
 else: # ZIG-ZAG: НИЖНЯЯ
 rotate_up(x)
 rotate_up(x)
 return x

def find(root, key): # поиск: спуск + сп
 cur, last = root, None
 while cur:
 last = cur
 if key == cur.key: break
 cur = cur.left if key < cur.key else cur.right
 return splay(last) if last else None</pre>
 <p class="text-xs text-slate-400">В
 Одна строка – и вся амортизация.</p>
 </div>
</details>
</div>
</div>
</section>`,
 },
 {
 id: "prefix-sums-2d",
 title: "5. Двумерная матрица префиксных сумм",
 type: "html",
 description: "Сумма подматрицы за O(1): 1D и 2D, дв
 category: "Алгоритмы матрицы",
 content: `
<section id="prefix-sums-2d" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>

 <div class="space-y-8">

 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

```

```

 <td class="py-2 pr-3 font-mono">
 </td>
 </tr>
 </tbody>
</table>
<p class="text-slate-400 text-xs mt-2">
 ышли за границу» – поэтому в 2D почти всегда
</div>

<div class="grid grid-cols-1 sm:grid-cols-3">
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 <p class="text-[11px] text-slate-500
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 <p class="text-[11px] text-slate-500
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-rose-300 t
 <p class="text-[11px] text-slate-500
 </div>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-blue-400 m
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800
 <div class="absolute -top-3 left-4 b
 rder border-emerald-500 shadow-md">
 Аналогия 1: Сетка из вафельных
 </div>
 <p class="text-slate-300 text-sm mb
 о накопилось от угла (0,0) до меня». Хочешь
 у и верни дважды отрезанный угол. Вычитание

```

```

<div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Тонкая: одна матрица
</div>
 <p class="text-slate-300 text-sm mb
одной матрицы (плюс рамка из нулей). Никаки
прямоугольника от угла. Всё остальное полу
</div>
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Не умеет: всё, что нельзя вычес
</div>
 <p class="text-slate-300 text-sm mb
максимума или gcd вычитания не существует:
е запросы – это <a class="ticket-link" data
платит она за это памятью <span class="fon
</div>
</div>

<div class="bg-slate-800/70 p-5 rounded-lg"
 <p class="text-sm font-bold text-white"
 <table class="w-full text-xs text-left"
 <thead>
 <tr class="text-slate-500 borde
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 pr-2 font
 </tr>
 </thead>
 <tbody>
 <tr class="border-b border-slate
 <td class="py-2 pr-3 font-b
 <td class="py-2 pr-3 font-m
 <td class="py-2 pr-3 font-m
 </tr>
 <tr class="border-b border-slate

```

```

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код: 1D (тот же, что выполняет пане
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
 = [3, 1, 4, 1, 5, 9, 2, 6] # исходный ма
n = len(a) # префиксов будет n + 1
P = [0] # P[0] = 0 – пустой преф
for i in range(1, n + 1):
 P.append(P[i - 1] + a[i - 1])
print("n =", n, "| P =", P)</code></pre>
 <p class="text-slate-300 text-sm">П
- часть демонстрации: лента подсвечивает, из
 </div>
</details>

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код: 2D – построение и запрос прямо
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
 = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4

S = [] # рамка из нуле
for rr in range(n + 1):
 S.append([0] + [0] * m)
for i in range(1, n + 1):
 for j in range(1, m + 1):
 S[i][j] = A[i-1][j-1] + S[i-1][j] + S[i][j-1] -

r1, c1, r2, c2 = 1, 1, 2, 2 # прямоугольник
ans = S[r2+1][c2+1] - S[r1][c2+1] - S[r2+1][c1] + S[r1]
print("итого:", ans)</code></pre>

```

```

 },
 {
 id: "graph-planar-colors",
 title: "7. Графы. Планарные. Покраска",
 type: "html",
 description: "Формула Эйлера и теорема о 4 красках.",
 category: "Графы. Теория",
 content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 7
 <h2 class="text-2xl sm:text-3xl font-bold text-blue-400">Графы. Планарные. Покраска</h2>
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-blue-400">Теорема о 4 красках</h3>
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Любой планарный граф можно раскрасить 4 цветами.</p>
 <p class="text-xl font-mono text-white">
 1. Если граф планарный, то он можно вложить в плоскость так, чтобы никакие две
 его ребра не пересекались.
 </p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md">
 Аналогия 1: Карта мира
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Если граф планарный, то его можно вложить в плоскость так, чтобы никакие две
 его ребра не пересекались.
 </p>
 <p class="text-slate-300 text-sm mb-4">
 Границы не пересекаются в одной точке по-прежнему не сливались цветами, Всегда можно всего 4 цвета.
 </p>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "top-sort",

```



```

 </div>
</section>`,
 },
 {
 id: "scc-kosaraju",
 title: "10. Графы. Компоненты сильной связности (SCC)",
 type: "html",
 description: "Разбиение орграфа на макро-вершины. Алгоритм Косарaju",
 category: "Графы. Продвинутое",
 content: `
<section id="scc-kosaraju" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">10. Графы. Компоненты сильной связности (SCC)
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">10.1. Компоненты сильной связности (SCC)
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Сильная связность графа — это свойство, при котором для любых двух вершин u и v графа существует путь из u в v и обратно.
 <p class="text-xl font-mono text-white">Граф, состоящий из нескольких сильно связных компонент, можно преобразовать в ориентированный ациклический граф (DAG), инвертировав направление ребер в каждой компоненте.
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Улицы с односторонним движением
 </div>
 <p class="text-slate-300 text-sm mb-4">Представьте себе город, где все улицы имеют одностороннее движение. Если вы можете добраться из любой точки города в любую другую точку, то город является сильно связным.
 </div>
 <div class="bg-slate-800 p-6 rounded-lg">
 <p class="text-slate-300 text-sm mb-4">В графе с сильной связностью для любых двух вершин u и v существует путь из u в v и обратно. Это означает, что граф можно разбить на несколько компонент, каждая из которых является сильно связной.
 </div>
 </div>
 </div>
 </div>
</section>`,
 },

```

```

 description: "Вершины, удаление которых убивает свя
 category: "Графы. Продвинутые",
 content: `
<section id="graph-articulation" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-rose-400">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-rose-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic">
 <p class="text-xl font-mono text-white">
 теи > 1).</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border
 order border-emerald-500 shadow-md">
 Аналогия 1: Центральный роутер
 </div>
 <p class="text-slate-300 text-sm mb-4">
 правильный роутер - и два сегмента офиса б
 </div>
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-euler",
 title: "13. Графы. Эйлеров цикл",
 type: "html",
 description: "Пройти по всем ребрам ровно 1 раз.",
 category: "Графы",
 content: `
<section id="graph-euler" class="mb-12 scroll-mt-10">

```

```

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-blue-400"
 <div class="bg-slate-900 p-4 rounded-lg mb-1"
 <p class="text-lg text-blue-300 italic"
 <p class="text-xl font-mono text-white"
d-длина.</p>
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия: Копаем туннель под .
 </div>
 <p class="text-slate-300 text-sm mb
ней земли. Но если начать копать одновременно
в 2 РАЗА меньше работы (геометрически: площ
 </div>
 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "mst-kruskal",
 title: "18. Графы. Остовное дерево. Краскал",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-kruskal" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

```

```

 которое торчит из посещенных в непосещенных
 </div>
 <div class="grid grid-cols-1 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 ✦ Аналогия: Заражение вирусом
 </div>
 <p class="text-slate-300 text-sm mb-2">
 ли вирус). Мы стартуем из одного города, и
 Сеть растет только из одного связного куска
 </p>
 </div>
 </div>
</div>
</section>`,
 },
 {
 id: "mst-boruvka",
 title: "20. Графы. Остовное дерево. Борувка",
 type: "html",
 description: "",
 category: "Графы. Остовные",
 content: `
<section id="mst-boruvka" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </h2>

 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-emerald-500">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ается с соседом.</p>
 </p>
 </div>
 </h3>
 <div class="grid grid-cols-1 gap-6">

```

```

 rder border-emerald-500 shadow-md">
 Аналогия 1: Бумеранг (Префикс
 </div>
 <p class="text-slate-300 text-sm mb
идишь "АБРА". Если ты ошибешься при поиске
концовка "АБРА" совпадает с началом "АБРА"
меня перепроверок!</p>
 </div>
</div>
<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounde
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}</pre>
 </div>
 </details>
</div>
</div>
</section>`,
 },
 {
 id: "string-z-func",
 title: "22. Z-функция",
 type: "html",
 description: "",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

```



```

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-blue-400"

 <div class="bg-slate-900 p-4 rounded-lg mb-4"
 <p class="text-lg text-blue-300 italic"
 <div class="text-left font-mono text-sm"
 <p><strong class="text-white">Эйлер
ин раз. Вершины повторять можно.</p>
 <p><strong class="text-white">Эйлер
 <div class="p-3 bg-slate-950 rounded"
 <p class="text-sm text-blue-300"
 <p><strong class="text-white">П
 <p><strong class="text-white">Ц
 </div>
 </div>
 </div>
</div>

<p class="text-slate-300 text-sm">
 Билет 13 (Различие пути и цикла

• <span class="text-green-400 font-
шел-вошёл-вышел).

• <span class="text-yellow-400 font
финиш).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap"
 <div class="bg-slate-800 p-6 rounded-lg bor"
 <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
 Аналогия 1: Путь – из дома в бар,
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Ты вышел из дома (нечётная)
 талый пришёл в бар (вторая нечётная
 Домой вернуться не смог, потому что
 </p>

```

```

 </div>
</section>`,
 },
 {
 id: "bridges-code",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Мосты через DFS Тарьяна: tin/low, усл",
 category: "Графы. Продвинутые",
 content: `<section id="bridges-code" class="mb-20 s
<div class="flex items-center mb-6">
 <span class="bg-emerald-600 text-white px-4 py-
 <h2 class="text-3xl font-bold text-white">Мосты
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-emerald-300 ital
 <div class="text-left font-mono text-sm
 <p><strong class="text-white">tin[v
 <p><strong class="text-white">low[v
 <div class="p-3 bg-slate-950 rounde
 <span class="text-rose-400 font
 <p class="text-xl font-bold tex
 <p class="text-xs text-slate-40
 </div>
 </div>
 </div>
 </div>
 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-slate-300 text-sm">
 Если из сына u и всех его потомков <str
 Единственная ниточка. Порвётся – граф р
 </p>
 </div>
</div>

```



```

<pre class="text-xs font-mono text-
#include <vector>;
#include <algorithm>;

using namespace std;

int n; // число вершин
vector<vector<int>>> adj; // список смежност
vector<bool> visited;
vector<int> tin, low;
int timer;

void dfs(int v, int p = -1) {
 visited[v] = true;
 tin[v] = low[v] = timer++; // устанавливаем

 for (int to : adj[v]) {
 if (to == p) continue; // не идём назад

 if (visited[to]) {
 // обратное ребро: обновляем low
 low[v] = min(low[v], tin[to]);
 } else {
 // ребро дерева DFS
 dfs(to, v); // рекурсивный в
 low[v] = min(low[v], low[to]);

 if (low[to] > tin[v]) {
 // ЭТО МОСТ!
 // bridge_list.push_back({v, to});
 }
 }
 }
}

void find_bridges() {
 timer = 0;
 visited.assign(n, false);
}
```

```

 <p><span class="text-emerald-400 font-
 <p><span class="text-rose-400 font-l
 </div>
</div>

<p class="text-slate-300 text-sm">
 Следствие: Для планарн
ode>.
 Если ребер больше – граф гарантированно
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap
 <div class="bg-slate-800 p-6 rounded-lg bor
 <div class="absolute -top-3 left-4 bg-s
order-green-500 shadow-md">
 K5: Полный граф на 5 вершинах
 </div>
 <p class="text-slate-300 text-sm mb-4">
 У K5: <code class="text-white">V=5,
-white">10 > 9</code> – БАХ, непланарен!
 Все 5 вершин соединены со всеми. На
 Теорема Куратовского: K5 и K3,3 – д
 Если внутри графа спрятано подра
 – плоской раскладки не будет.
 </p>
 </div>

<div class="bg-slate-900 p-6 rounded-lg bor
 <div class="absolute -top-3 left-4 bg-r
der-rose-500 shadow-md">
 K3,3: 3 дома и 3 колодца
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Три дома хотят соединиться с тремя
 Нарисовать без пересечений невозможно
 У него <code class="text-white">V=6
xt-white">9 ≤ 12</code> – проходит, но он в
```

```
<div class="grid grid-cols-1 xl:grid-cols-2" style="border: 1px solid #000; padding: 10px;">
 <div class="bg-slate-800 p-6 rounded-lg" style="border: 1px solid #000;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-emerald-500 shadow-md" style="border: 1px solid #000;">
 Аналогия 1: Расписание экзаменов
```

```
 </div>
 <p class="text-slate-300 text-sm">Время, отведенное на экзамене, — это слот времени. Раскрасить граф = составить расписание. Задача: «берём предметы в каком-то порядке и ставим их в слоты»
```

```
 <div class="bg-slate-900 p-6 rounded-lg" style="border: 1px solid #000;">
 <div class="absolute -top-3 left-4 right-4 bottom-4 border border-fuchsia-500 shadow-md" style="border: 1px solid #000;">
 Аналогия 2: Частоты вышек
```

```
 </div>
 <p class="text-slate-300 text-sm">Свойства: цвет, интерференция = ребро. Диапазон частот вышек = регистров в компиляторе: «живые» одновременно активны
```

```
 </div>
</div>
```

```
<div class="overflow-x-auto my-6" style="border: 1px solid #000; padding: 10px;">
 <table class="w-full text-xs text-left" style="border-collapse: collapse;">
 <thead>
 <tr class="text-slate-500 border-b" style="border-bottom: 1px solid #000;">
 <th class="py-2 pr-3 font-b" style="border: none; text-align: left; padding: 5px 10px 5px 5px;">χ
 <th class="py-2 pr-3 font-b" style="border: none; text-align: left; padding: 5px 10px 5px 5px;">Δ
 <th class="py-2 font-bold" style="border: none; text-align: left; padding: 5px 10px 5px 5px;">$\chi \leq \Delta + 1$
 <th class="py-2 font-bold" style="border: none; text-align: left; padding: 5px 10px 5px 5px;">$\chi \leq \Delta$
 <tr class="border-b border-slate-800" style="border-bottom: 1px solid #000;">
 <td class="py-2 pr-3 font-mono" style="border: none; text-align: left; padding: 5px 10px 5px 5px;">$\chi \leq \Delta + 1$
 <td class="py-2 pr-3 font-mono" style="border: none; text-align: left; padding: 5px 10px 5px 5px;">$\chi \leq \Delta$

```

```

 веркой цветов соседей; проверка двудольности
border border-slate-800">0(V + E)</code>; т
-300 border border-slate-800">NP-трудно</co

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код: жадная раскраска и проверка
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
я раскраска: не больше $\Delta + 1$ цвета
color = [-1] * n # -1 = ещё не покрашен
for v in order: # порядок влияет на чи
 used = set()
 for to in graph[v]:
 if color[to] != -1:
 used.add(color[to]) # цвета уже покрашены
 c = 0
 while c in used: # минимальный свободны
 c += 1
 color[v] = c
chi = max(color) + 1 # χ , который выдал жад

Проверка двудольности = раскраска в 2 цвета (BFS)
from collections import deque
col = [-1] * n
bipartite = True
for start in range(n):
 if col[start] != -1:
 continue
 col[start] = 0
 dq = deque([start])
 while dq:
 v = dq.popleft()
 for to in graph[v]:
 if col[to] == -1:
 col[to] = col[v] ^ 1 # второй цвет

```

```
<p class="text-slate-300 text-sm">
 Важно: Это работает то
 епятся к точкам сочленения. То есть,
 ег - это тупик со степенью 1).
</p>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2 gap-4">
 <div class="bg-slate-800 p-6 rounded-lg border border-slate-500">
 <div class="absolute -top-3 left-4 bg-slate-800 border border-rose-500 shadow-md">
 Аналогия: Главный перекресток
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Представь город, где два района соединены
 её перекроют (удалят вершину) — районы будут
 g>хрупкость системы.
 </p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border border-slate-500">
 <div class="absolute -top-3 left-4 bg-slate-900 border border-emerald-500 shadow-md">
 Телеком: Центральный свитч
 </div>
 <p class="text-slate-300 text-sm mb-4">
 У тебя несколько компьютеров в одной
 абелем (мостом). Сами свитчи — это точки со
 </p>
 </div>
</div>

<details class="bg-slate-900/40 rounded-lg border border-slate-500">
 <summary class="p-4 font-bold text-slate-400">
 order-slate-700/50">
 Душный мод: Код и корень DFS (Скрыто)
 </summary>
```

```

 <ul class="list-disc list-inside text-sm text-rose-400">
 <strong class="text-rose-400">Точки
ong> и показывают физическую уязвимость свя
 <strong class="text-emerald-400">SCC
ависимые "конгломераты".
 Как точка сочленения рушит
Косарайю, свернув каждую SCC в одну супер-в
(сделаем неориентированным), то любая <strong
о и есть критическая SCC! Если удалить эту
. Одно слабое звено изолирует целые касты S

 </div>
</div>
</section>`,
 },
];

```

ФАЙЛ 19/111: src/data/content.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 145 | РАЗМ

```

import { Chapter } from "../types";
import { graphChapters } from "../graphContent";
import { additionalTickets } from "../additionalTickets";
import { tickets14to20 } from "../tickets/ticket14_20";
import { tickets21to24 } from "../tickets/ticket21_24";
import { tickets6to13 } from "../tickets/ticket6_13";
import { ticket1Segtree } from "../tickets/ticket1_segtr
import { tickets18to20Mst } from "../tickets/ticket18_20
import { chapters as newChapters } from "../content_temp

/**
 * Содержание пособия = билеты 1–24 из списка тем.
 *
 * Порядок массивов важен: при одинаковом id побеждает
 * версия главы. `newChapters` (content temp) перетирае

```

```
merged.forEach((c) => {
 if (!toRemove.includes(c.id)) {
 dedup.set(c.id, c);
 }
});

// Самые свежие версии страниц перетирают старые.
if (newChapters) {
 newChapters.forEach((c) => {
 if (!toRemove.includes(c.id)) dedup.set(c.id, c);
 });
}

const finalChapters = Array.from(dedup.values());

finalChapters.sort((a, b) => {
 // Главы без номера билета (введение) стоят ДО билетов
 const numOf = (t: string) => {
 const m = t.match(/(\d+)/);
 return m ? parseInt(m[0], 10) : -1;
 };
 return numOf(a.title) - numOf(b.title);
});

export const chapters: Chapter[] = finalChapters;

/**
 * Билеты, закрытые каждой страницей (для шапки, экспор
 * Страница `mst`, например, закрывает сразу 18, 19 и 20
 */
export const chapterTopics: Record<string, number[]> = {
 "segment-trees": [1],
 "sparse-table": [2],
 treap: [3],
 "splay-tree": [4],
 "prefix-sums-2d": [5],
 "graph-dfs-bfs": [6],
 "planarity-euler-formula": [7],

```

```

const tickets = chapterTopics[chapterId] ?? [];
// Глав без билетов (введение) относим к первому разд
if (!tickets.length) return SECTIONS[0];
const first = Math.min(...tickets);
return SECTIONS.find((sec) => first >= sec.from && fi
}

```

```

export const activeVizIds = [
 "euler-path-vs-cycle", "bridges-code", "planarity-euler",
 "graph-dfs-bfs", "stack-dfs", "queue-bfs", "heap-beam-se",
 "dijkstra", "bellman-ford", "floyd", "aho-corasick",
 "mst", "string-kmp", "string-z-func", "segment-trees", "s",
 "dynamic-programming", "johnson-algo", "graph-articulat",
 "complexity-classes", "treap", "splay-tree", "prefix-sum
];

```

ФАЙЛ 20/111: src/data/debugRunner.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 207 | РАЗМ

```

/**
 * Пошаговый дебаггер поверх Pyodide: CPython выполняет
 * под sys.settrace и на каждой строке записывает фрейм
 * как панель Variables в обычном отладчике.
 */

```

```

export type DebugValue = null | boolean | number | string

```

```

export interface DebugStep {
 /** Номер строки в коде пользователя (1-based). */
 line: number;
 /** Имя функции-фрейма ("<module>" – верхний уровень)
 func: string;
 /** Локальные переменные: имя → герг (усечённый), для
 locals: Record<string, string>;
 /**

```



```

__steps = []
__final = {}
__final_values = {}
__CAP = ${DEBUG_STEP_CAP}
__was_truncated = False

class __TraceLimit(Exception):
 pass

def __safe(v):
 try:
 r = repr(v)
 except Exception:
 r = "<...>"
 return r if len(r) <= 80 else r[:77] + "..."

def __snapshot(v, depth=0):
 """Небольшой JSON-снимок: UI использует его без парсера"""
 if v is None or isinstance(v, (bool, int, str)):
 return v
 if isinstance(v, float):
 return v if v == v and v not in (float("inf"), float("-inf")) else "..."
 if depth >= 4:
 return __safe(v)
 limit = 80 if depth < 2 else 32
 if isinstance(v, (list, tuple)):
 return [__snapshot(x, depth + 1) for x in list(v)]
 if isinstance(v, (set, frozenset)):
 return [__snapshot(x, depth + 1) for x in list(v)]
 if isinstance(v, dict):
 # 8x8 Sparse Table содержит 64+49+25+1 = 139 элементов
 # предел 80 незаметно обрезал уровни 4x4 и 8x8
 dict_limit = 192 if depth == 0 else limit
 return {str(k): __snapshot(x, depth + 1) for k, x in list(v.items())}
 # deque, heap-подобные и другие итерируемые учебные
 if type(v).__name__ == "deque":
 return [__snapshot(x, depth + 1) for x in list(v)]
 return __safe(v)

```

```

 # PEP 709: с 3.12 компрекеншны встроены во фрейм
 # на своей строке каждую итерацию. Склеиваем по
 if __steps and __steps[-1]["line"] == frame.f_lineno:
 __steps[-1]["locals"] = loc
 __steps[-1]["values"] = values
 else:
 __steps.append({"line": frame.f_lineno, "func": frame.f_code.co_name})
 if len(__steps) >= __CAP:
 # Нельзя просто отключить trace: бесконечный цикл
 # тогда продолжит работать вечно. Мягко прерываем
 __was_truncated = True
 raise __TraceLimit()
 # Финальные локалы программы (после последней строки)
 if event == "return" and frame.f_code.co_filename == __FILE__:
 loc, values = __capture(frame)
 __final.update(loc)
 __final_values.update(values)
 return __tracer

__err = ""
try:
 __sys.settrace(__tracer)
 exec(compile(__src, "<user-code>", "exec"), {"__name__": "__main__"})
except __TraceLimit:
 pass
except Exception:
 import traceback as __tb
 __err = __tb.format_exc(limit=3)
finally:
 __sys.settrace(None)

__OUT = __json.dumps({
 "steps": __steps,
 "finalLocals": __final,
 "finalValues": __final_values,
 "error": __err,
 "truncated": __was_truncated,
})

```

---

ФАЙЛ 21/111: src/data/glossary.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 639 | РАЗМ

---

```
import type { DemoKind } from "../components/hints/Mini"
```

```
/**
```

```
 * Глоссарий «википедийных» подсказок.
```

```
 *
```

```
 * Любое слово из `labels`, встреченное в тексте главы,
```

```
 * в подчёркнутый термин: при наведении (или тапе на мо
```

```
 * с объяснением на пальцах и мини-визуализацией.
```

```
 */
```

```
export interface GlossaryEntry {
```

```
 id: string;
```

```
 /** Варианты написания в тексте (регистр не важен). */
```

```
 labels: string[];
```

```
 title: string;
```

```
 /** Объяснение «на пальцах» – 1–2 фразы. */
```

```
 short: string;
```

```
 /** Строгая формулировка / что важно не забыть. */
```

```
 formal?: string;
```

```
 complexity?: string;
```

```
 /** Какая мини-анимация показывается в карточке. */
```

```
 demo?: DemoKind;
```

```
 /** id полного симулятора (см. vizRegistry) – кнопка
```

```
 simulator?: string;
```

```
}
```

```
export const GLOSSARY: GlossaryEntry[] = [
```

```
 {
```

```
 id: "bfs",
```

```
 labels: ["BFS", "обход в ширину", "поиск в ширину",
```

```
 title: "BFS – обход в ширину",
```

```
 short:
```

"Не отсортированный список, а «мешок, из которого  
formal:

"Полное бинарное дерево в массиве: дети узла  $i$  ле

complexity: "вставка/извлечение  $O(\log n)$ , «посмотре

demo: "heap",

simulator: "heap-beam-search",

},

{

id: "stack",

labels: ["стек", "стека", "стеке", "стеком", "LIFO"]

title: "Стек (LIFO)",

short: "Стопка тарелок: кладёшь сверху и берёшь свер

formal: "push / pop / top за  $O(1)$ . На стеке живёт р

demo: "stack",

simulator: "stack-dfs",

},

{

id: "queue",

labels: ["очередь", "очереди", "очередью", "FIFO"],

title: "Очередь (FIFO)",

short: "Очередь в столовой: кто первым встал, тот п

formal: "push\_back / pop\_front за  $O(1)$ . Основа BFS

demo: "queue",

simulator: "queue-bfs",

},

{

id: "dsu",

labels: ["DSU", "СНМ", "система непересекающихся мн

title: "DSU — система непересекающихся множеств",

short:

"«Кто твой староста?» Каждый элемент знает своего

formal:

"find со сжатием путей + union по рангу/размеру. I

complexity: "почти  $O(1)$  — обратная функция Аккерман

demo: "dsu",

simulator: "mst",

},

{

```

 short:
 "«Куда откатиться, если следующая буква не подошла",
 строки.",
 formal: "link(v) = go(link(parent(v)), ch). Считает",
 demo: "suffix-link",
 simulator: "aho-corasick",
},
{
 id: "toposort",
 labels: ["топологическая сортировка", "топсорт", "топ",
 title: "Топологическая сортировка",
 short:
 "Порядок «сначала штаны, потом ботинки»: каждая с",
 циклов.",
 formal: "DFS + вершины в обратном порядке выхода, л",
 complexity: "O(V + E)",
 demo: "toposort",
 simulator: "top-sort",
},
{
 id: "relaxation",
 labels: ["релаксация", "релаксацию", "релаксации",
 title: "Релаксация ребра",
 short:
 "«А через соседа не быстрее?» Если известный путь",
 ,
 formal: "if d[v] > d[u] + w(u,v) → d[v] = d[u] + w(u,v)",
 ан и Флойд.",
 demo: "relax",
 simulator: "dijkstra",
},
{
 id: "prefix-sum",
 labels: ["префиксные суммы", "префиксная сумма", "пре",
 title: "Префиксные суммы",
 short:
 "Заранее посчитанные «сколько всего набежало от н",
 formal: "P[0]=0, P[i]=P[i-1]+a[i-1]. Сумма a[l..r] = P[r] - P[l-1]"

```

```

 complexity: "O(V + E) одним DFS",
 demo: "bridge",
 simulator: "bridges-code",
},
{
 id: "articulation",
 labels: ["точка сочленения", "точки сочленения", "то"],
 title: "Точка сочленения",
 short:
 "Вершина-незаменимый-сотрудник: уволь её — и коллапс",
 formal: "Для не-корня: v — точка сочленения Э ребро",
 demo: "articulation",
 simulator: "graph-articulation",
},
{
 id: "mst",
 labels: [
 "остовное дерево", "остовного дерева", "остовное",
 "минимальное остовное дерево", "MST",
],
 title: "Минимальное остовное дерево (MST)",
 short:
 "Связать все точки проводами так, чтобы суммарная",
 formal: "V-1 ребро, никаких циклов. Краскал (жадно и",
 "но).",
 demo: "mst",
 simulator: "mst",
},
{
 id: "amortized",
 labels: [
 "амортизированная", "амортизированный", "амортизир",
 "амортизированное", "амортизированного", "амортизир",
],
 title: "Амортизированная сложность",
 short:
 "Средняя цена операции «по абонементу»: одна опер",
 formal: "Метод потенциалов: дорогая операция оплачи
```

```

{
 id: "prefix-function",
 labels: [
 "префикс-функция", "префикс-функции", "префикс-фу",
 "префикс", "префикса", "суффиксом", "суффикс", "с",
 "подстроки", "подстроку", "подстрока",
],
 title: "Префикс-функция π (КМП)",
 short:
 "Для каждой позиции запоминаем: какой кусок начался",
 "от кусок.",
 formal:
 " $\pi[i]$ — длина наибольшего собственного префикса s ",
 "возвращая указатель по тексту.",
 complexity: " $O(n + m)$ ",
 demo: "prefix-function",
 simulator: "string-kmp",
},
{
 id: "z-function",
 labels: ["Z-функция", "Z-функции", "Z-функцию", "z-"],
 title: "Z-функция",
 short:
 "Для каждой позиции: сколько символов подряд, начи",
 "аждой букве.",
 formal:
 " $z[i]$ = длина наибольшего общего префикса s и $s[i$ ",
 complexity: " $O(n)$ ",
 demo: "prefix-function",
 simulator: "string-z-func",
},
{
 id: "dp",
 labels: [
 "динамическое программирование", "динамики", "дин",
 "мемоизация", "мемоизацию", "подзадач", "подзадач",
],
 title: "Динамическое программирование",

```

```

 short:
 "На каждом шаге хватаем то, что лучше прямо сейчас"
 formal:
 "Корректность обычно доказывается «леммой о разрезе»"
 demo: "mst",
 simulator: "mst",
},
{
 id: "euler",
 labels: ["эйлеров цикл", "эйлеров путь", "эйлерова теорема"],
 title: "Эйлеров путь и цикл",
 short:
 "Пройти по каждому ребру ровно один раз, не отрываясь"
 formal:
 "Связный граф: эйлеров цикл все степени чётные; эйлеров путь все степени нечётные"
 simulator: "euler-path-vs-cycle",
},
{
 id: "planarity",
 labels: ["планарный", "планарность", "планарного", "планаризация"],
 title: "Планарность",
 short:
 "Граф планарен, если его можно нарисовать на листе бумаги"
 formal:
 "V - E + F = 2 для связного планарного графа. Курсивом выделены вершины, рёбра и области"
 simulator: "planarity-euler-formula",
},
{
 id: "splay",
 labels: [
 "splay", "splay-дерево", "AVL", "AVL-дерево", "всё остальное"
],
 title: "Splay и повороты",
 short:
 "Каждый раз, когда к узлу обратились, его «вытягивают» к корню"
 formal:

```



```

 "После k-й итерации гарантированно верны все кратчайшие пути, которые не содержат отрицательный цикл.",
 complexity: "O(V · E)",
 demo: "relax",
 simulator: "bellman-ford",
 },
 {
 id: "floyd",
 labels: [
 "Флойд", "Флойда", "Флойд-Уоршелл", "Флойда-Уоршелла",
 "промежуточная вершина", "промежуточную вершину",
],
 title: "Флойд-Уоршелл: разрешаем вершину k",
 short:
 "Внешний цикл — это не «шаг», а «какие пересадки сделать, чтобы пересадку в k?",
 formal:
 "D[i][j] = min(D[i][j], D[i][k] + D[k][j]). Порядок пересадки — k.",
 complexity: "O(V³) времени, O(V²) памяти",
 demo: "floyd",
 simulator: "floyd",
 },
 {
 id: "prim",
 labels: ["Прима", "Прим", "Prim"],
 title: "Алгоритм Прима",
 short:
 "Остов растёт как плесень из одной точки: на каждом шаге добавляем ребро, не создавая циклов.",
 formal:
 "Множество посещённых + куча рёбер на границе. Добавляем ребро с минимальным весом, не создавая циклов.",
 complexity: "O(E log V)",
 demo: "mst",
 simulator: "mst",
 },
 {
 id: "boruvka",

```

```
{
 id: "graph",
 labels: [
 "граф", "графа", "графе", "графы", "графов", "граф",
 "вершины", "вершина", "вершин", "ребра с весом",
],
 title: "Граф",
 short:
 "Точки (вершины) и связи между ними (рёбра). Города",
 formal:
 " $G = (V, E)$. Ребро бывает ориентированным (улица с односторонним движением)",
 demo: "graph",
 simulator: "graph-dfs-bfs",
},
{
 id: "coord-compress",
 labels: ["сжатие координат", "сжатия координат", "сжатие координат"],
 title: "Сжатие координат",
 short:
 "Координаты бывают гигантские и дробные, а нам нужно хранить их в виде целых чисел (порядковым номером).",
 formal:
 "sort + unique + binary search: значение → индекс",
 complexity: " $O(n \log n)$ ",
 demo: "coord-compress",
},
{
 id: "treap",
 labels: ["Treap", "декартово дерево", "декартова д-во"],
 title: "Treap = Tree + Heap",
 short:
 "Одно дерево живёт по двум правилам сразу: по ключу и по приоритету. Случайность и держит его сбалансированным.",
 formal:
 "Базис — split(t, x) (разрезать по ключу) и merge(t1, t2) (соединить два дерева через них. Ожидаемая высота $O(\log n)$).",
 complexity: "ожидаемо $O(\log n)$ ",
}
```

```

 demo: "np",
 },
 {
 id: "range",
 labels: ["отрезок", "отрезка", "отрезке", "отрезки"],
 title: "Запрос на отрезке",
 short:
 "Спрашиваем не про один элемент, а про кусок массива

 предподсчёты.",
 formal:
 "Суммы — префиксные суммы $O(1)$. Минимум без изменений.",
 demo: "prefix-sum",
 simulator: "prefix-sums-2d",
 },
 {
 id: "string",
 labels: ["строки", "строке", "строку", "строка", "строкам"],
 title: "Строка как массив символов",
 short:
 "Строковые алгоритмы держатся на трёх вопросах: где

 (Z-функция) и где встречаются слова словаря (бор).",
 formal:
 "Наивный поиск подстроки — $O(N \cdot M)$. Префикс-функция.",
 demo: "prefix-function",
 simulator: "string-kmp",
 },
 {
 id: "rotation",
 labels: ["поворот", "повороты", "поворота", "поворотом"],
 title: "Поворот (rotation) в дереве",
 short:
 "Локальная перевеска двух узлов: ребёнок встаёт на

 дившемся месту. Порядок ключей при этом не ломается.",
 formal:
 "Правый поворот вокруг p: $x = p.left; p.left = x.$

 g и Zig-Zag.",
 complexity: " $O(1)$ ",
 demo: "zig",
 },

```

Универсальное пособие • полный исходный код

```

/** Быстрый доступ по id. */
export const GLOSSARY_BY_ID = new Map(GLOSSARY.map((e) => {

/** Все метки, отсортированные от длинных к коротким (ч
export const GLOSSARY_LABELS: { label: string; id: string }[] =
 e.labels.map((label) => ({ label, id: e.id }))
}).sort((a, b) => b.label.length - a.label.length);

```

ФАЙЛ 22/111: src/data/graphContent.ts

КАТЕГОРИЯ: Контент и данные пособия | СТРОК: 411 | РАЗМЕР: 1.5 КБ

```

import { Chapter } from "../types";

export const graphChapters: Chapter[] = [
 {
 id: "intro",
 title: "Введение. Базовая база: куча, очередь, бинарное
 type: "html",
 category: "Введение",
 content: `
 <section id="intro-content" class="mb-12">
 <div class="flex items-center mb-6 flex-wrap gap-2">
 Базовая база
 <h2 class="text-3xl font-bold text-white">Базовая база</h2>
 </div>

 <div class="bg-slate-900 border border-indigo-500/40 p-6">
 <p class="text-slate-200 text-sm leading-relaxed">
 <b class="text-indigo-300">Базовая база — это
 дерево как массив, куча, очередь и стек, бинарное
 Кодим по минимуму: там, где за нас уже написали.
 </p>
 </div>

 <div class="space-y-8">
```

```
h = []
heapq.heappush(h, 5); heapq.heappush(h, 1); heapq.heappop(h)
h[0] # 1 – минимум виден без всякого обхода
heapq.heappop(h) # 1, куча починилась за $O(\log n)$
```

<p class="text-slate-300 text-sm">Куча – движок, который поддерживает кучу, раз, и список с <span class="font-mono text-sm">heapq</span>.</p>

<div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-r-4 border-b-4 border-t-4">  
<h3 class="text-xl font-bold text-sky-400 mb-4">Очереди и стеки</h3>  
<p class="text-slate-300 text-sm mb-4"><b>Очереди</b> (FIFO) отличаются от стеков (LIFO) тем, что первый вышел) отличаются одной операцией снятия элемента</span>: оба конца работают за  $O(1)$ . Обычный список сдвигает весь хвост и стоит  $O(n)$ .</p>

```
<pre class="bg-slate-950 border border-slate-700 p-4">from collections import deque
q = deque([1, 2])
q.append(3); q.popleft() # 1 – очередь (BFS)
s = [1, 2]; s.append(3); s.pop() # 3 – стек (DFS)</pre>
```

<p class="text-slate-300 text-sm">Вся разница между BFS (волнами), стек – DFS (вглубь).</p>

<div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-r-4 border-b-4 border-t-4">  
<h3 class="text-xl font-bold text-amber-400 mb-4">Бинарный поиск</h3>  
<p class="text-slate-300 text-sm mb-4">Если массив отсортирован, то можно найти элемент за  $O(\log n)$  с помощью бинарного поиска. Если массив не отсортирован, то нужно сортировать его за  $O(n \log n)$ .</p>

```
<pre class="bg-slate-950 border border-slate-700 p-4">from bisect import bisect_left
a = [10, 20, 30, 40, 50]
bisect_left(a, 30) # 2 – индекс первого элемента, не меньше 30</pre>
```

<p class="text-slate-300 text-sm">Без сортировки бинарный поиск не работает.</p>

<div class="bg-slate-700/50 p-6 rounded-xl border-l-4 border-r-4 border-b-4 border-t-4">  
<h3 class="text-xl font-bold text-rose-400 mb-4">Сортировки</h3>  
<p class="text-slate-300 text-sm mb-4">Писать сортировки можно вручную, но лучше использовать встроенные функции.</p>

```

 left(x) = z # z становится левым р
 высоты пересчитываются только у z и x — ост
 </div>
</details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-teal-400 mb-4
 <p class="text-slate-300 text-sm mb-4">Оба обхо
 тару». Вся разница в таре: стек уходит вглуби
 ext-emerald-300">0(V + E): каждая вершн
 <pre class="bg-slate-950 border border-slate-700
def go(graph, start, tank): # tank: список-стек
 seen, tank = {start}, tank([start])
 while tank:
 v = tank.pop() if isinstance(tank, list) else t
 for to in graph[v]:
 if to not in seen:
 seen.add(to); tank.append(to)
 return seen</pre>
 <p class="text-slate-300 text-sm">Дальше эта па
 надстройки над этими восемью строками.</p>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-slate-300 mb-
 <div class="grid grid-cols-1 xl:grid-cols-2 gap
 <div class="bg-slate-800 p-6 rounded-lg bord
 <div class="absolute -top-3 left-4 bg-s
 border-indigo-500 shadow-md">
 Аналогия 1: Схема кинозала
 </div>
 <p class="text-slate-300 text-sm">Дерев
 азателей-«usher'ов» не нужно. Место n, дети
 рху вниз и слева направо.</p>
 </div>
 <div class="bg-slate-800 p-6 rounded-lg bord
 <div class="absolute -top-3 left-4 bg-s

```

```

title: "Билет 14. Дейкстра",
type: "html",
category: "Графы",
content: `<section id="dijkstra-content" class="mb-
<div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 rounded
 <h2 class="text-3xl font-bold text-white">Дейкст
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border
 <h3 class="text-xl font-bold text-emerald-400">Аналогия 1: Позитивное планирование
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">Позитивное планирование — это поиск кратчайшего пути, когда все веса ребер неотрицательны. Это позволяет использовать алгоритм Дейкстры.
 <p class="text-xl font-mono text-white">Пример: Поиск кратчайшего пути от Москвы до Санкт-Петербурга.
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Позитивное планирование
 </div>
 <p class="text-slate-300 text-sm mb-4">Позитивное планирование — это поиск кратчайшего пути, когда все веса ребер неотрицательны. Это позволяет использовать алгоритм Дейкстры.
 (нельзя поехать и заработать). Он всегда работает.
 <div id="slot-chapter-image-dijkstra">
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Ограничения
 </div>
 <p class="text-slate-300 text-sm mb-4">Ограничения — это когда есть дополнительные условия, которые нужно учитывать при поиске кратчайшего пути. Например, если есть ограничение на время в пути или на количество остановок.
 (охождение улицы), Дейкстра сломается, так как

```

```

<div class="grid grid-cols-1 xl:grid-cols-2"
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 ♂ Аналогия 1: Параноик
 </div>
 <p class="text-slate-300 text-sm mb
еряет ВСЕ возможные дороги V-1 раз подряд.
 <div id="slot-chapter-image-bellman
 </div>

 <!-- Аналогия 2 -->
 <div class="bg-slate-900 p-6 rounded-lg"
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Отрицательный цикл
 </div>
 <p class="text-slate-300 text-sm mb
г станет ЕЩЕ короче – значит мы попали во в
 </div>
 </div>

 <div id="slot-bellman-viz" class="mt-8"></div>
</div>
</section>`
},
{
 id: "floyd",
 title: "Билет 16. Флойд-Уоршелл",
 type: "html",
 category: "Графы",
 content: `<section id="floyd-content" class="mb-12"
<div class="flex items-center mb-6">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-3xl font-bold text-white">Флойд

```



```

 },
 {
 id: "aho-corasick",
 title: "Билет 23. Алгоритм Ахо-Корасик",
 type: "html",
 category: "Строки",
 content: `<section id="aho-corasick" class="mb-12 s
<div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-indigo-600 text-white px-4 py-1 round
 <h2 class="text-2xl sm:text-3xl font-bold text-white
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border-l
 <h3 class="text-xl font-bold text-blue-400 mb-4">

 <div class="bg-slate-900 p-4 rounded-lg mb-6 text
 <p class="text-sm text-blue-300 italic mb-2">0с
 <p class="text-lg font-mono text-white">Бор (Tr
 </div>
 <p class="text-slate-300 text-sm">Позволяет найти
 -mono text-amber-300 bg-black/30 px-1 rounded">
 y.</p>
 </div>

<!-- Аналогии -->
<div class="grid grid-cols-1 xl:grid-cols-2 gap-6">
 <div class="bg-slate-800 p-6 rounded-lg border bo
 <div class="absolute -top-3 left-4 bg-slate-700
 emerald-500 shadow-md">
 Аналогия 1: Паутина (Бор)
 </div>
 <p class="text-slate-300 text-sm mb-4">Представ
 мы движемся по веткам. Если нужного продолж
 ("нити страховки") в самый длинный из извест
 </div>

 <div class="bg-slate-900 p-6 rounded-lg border-2

```

```

while (!q.empty()) {
 int v = q.front(); q.pop();
 for (auto const& [ch, child] : nodes[v].trie) {
 // Суффиксная ссылка ребёнка — это переход
 nodes[child].link = nodes[nodes[v].link].go

 // Если суффиксная ссылка сама является сло
 // Иначе наследуем terminal_link
 if (nodes[nodes[child].link].is_terminal) {
 nodes[child].term_link = nodes[child].l
 } else {
 nodes[child].term_link = nodes[nodes[ch
 }

 q.push(child);
 }
}
}
</div>
</div>
</details>
</div>
</section>`
},
{
 id: "alg-map",
 title: "Алгоритмы на карте",
 type: "html",
 category: "Графы",
 content: `<section id="alg-map-content" class="mb-1
<div class="flex items-center mb-6">
 <span class="bg-purple-600 text-white px-4 py-1
 <h2 class="text-3xl font-bold text-white">Алгори
</div>

<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord

```

```

 options: [
 "Да: минимум и сумма считаются одним приёмом",
 "Нет: блоки перекрываются, а вычесть перекрытие",
 "Да, если заранее хранить суммы в нулевом уровне",
],
 correctIndex: 1,
 explanation: "Два (в 2D – четыре) блока накрывают
 обратная операция, но применить её к перекрытию
 ммы, билет 5."
},
{
 question: "Какие операции подходят разреженной та
 options: [
 "Любые ассоциативные",
 "Идемпотентные: min, max, gcd, побитовые И и ИЛИ",
 "Только сумма и произведение"
],
 correctIndex: 1,
 explanation: "Нужно $op(x, x) = x$: тогда дважды на
 приём и уходит в префиксные суммы."
},
{
 question: "Между запросами элементы матрицы меняю
 options: [
 "Разреженную таблицу: она перестраивается за $O(n^2)$ ",
 "Префиксную матрицу: достаточно поправить одну
 "Дерево отрезков: и таблица, и префиксы статичны",
],
 correctIndex: 2,
 explanation: "Обе структуры статичны: изменение э
 ксной. С обновлениями работает дерево отрезков,
 }
},
"prefix-sums-2d": [
 {
 question: "Сколько памяти занимает двумерная преф
 options: [
 "N × M – как исходная матрица, плюс рамка из ну

```

```

 ной операции нет. Для min, max и gcd берут разря
 },
 {
 question: "Изменили один элемент матрицы. Что ста
 options: [
 "Поправится одна клетка за $O(1)$ ",
 "Нужен пересчёт $O(N \cdot M)$: от этой клетки зависит
 "Ничего не станет: таблица остаётся верной"
],
 correctIndex: 1,
 explanation: "Каждая клетка $S[i][j]$ включает в се
 я частые, это задача дерева отрезков (билет 1).
 }
],
"euler-path-vs-cycle": [
 {
 question: "В связном графе ровно 2 вершины с нечёт
 options: [
 "Есть эйлеров цикл (вернись домой!)",
 "Есть эйлеров путь (из одной нечётной в другую)
 "Ни пути, ни цикла нет — это полный провал"
],
 correctIndex: 1,
 explanation: "Если нечётных вершин ровно 2 — это
 ётной, заканчиваем в другой."
 },
 {
 question: "Почему гамильтонов цикл — это NP-полна
 options: [
 "Потому что Гамильтон — болезнь, и лечить её т
 "Потому что для эйлерова цикла есть простой кри
 "Потому что Эйлер был умнее Гамильтона"
],
 correctIndex: 1,
 explanation: "Для эйлерова цикла работает критери
 ого критерия. Это NP-полная задача, которую не
 },
 {

```

```

 "0(2^V) – полный перебор"
],
 correctIndex: 1,
 explanation: "DFS обходит каждую вершину и каждое
}
],
"planarity-euler-formula": [
{
 question: "Планарный граф имеет V=6, E=12. Возможн
 options: [
 "Да, если это двудольный граф",
 "Нет, нарушается неравенство $E \leq 3V - 6$ ",
 "Да, если нет треугольных граней"
],
 correctIndex: 1,
 explanation: " $E \leq 3V - 6 \Rightarrow 12 \leq 12$ – на грани пр
 Так что такой граф непланарен."
 },
 {
 question: "Сколько граней у планарного графа с V=
 options: [
 "3 грани",
 "5 граней",
 "10 граней"
],
 correctIndex: 1,
 explanation: " $F = E - V + 2 = 10 - 7 + 2 = 5$. Не
 },
 {
 question: "Почему K5 непланарен?",
 options: [
 "Потому что $10 > 3*5 - 6 = 9$ – нарушение",
 "Потому что Эйлер был против",
 "Потому что 5 вершин – несчастливое число"
],
 correctIndex: 0,
 explanation: "У K5: V=5, E=10, $3V - 6 = 9$. $10 > 9$
 }
}

```

```

/** Транслировать реальные переменные визуализациям тек
export function emitVizState(
 chapterId: string,
 snapshot: Omit<VizStateEventDetail, "chapterId">
) {
 if (typeof window === "undefined") return;
 const detail: VizStateEventDetail = { chapterId, ...s
 latestByChapter.set(chapterId, detail);
 window.dispatchEvent(new window.CustomEvent<VizStateE
}

/** Вернуть демонстрацию в ручной режим при закрытии/ре
export function clearVizState(chapterId: string) {
 if (typeof window === "undefined") return;
 latestByChapter.delete(chapterId);
 window.dispatchEvent(new window.CustomEvent<VizStateE
}

/** Текущий снимок компилятора для прямой отрисовки мас
export function useVizRuntime(): VizStateEventDetail | n
 const chapterId = useContext(VizChapterContext);
 const [runtime, setRuntime] = useState<VizStateEventD
 chapterId ? latestByChapter.get(chapterId) ?? null
);

 useEffect(() => {
 setRuntime(chapterId ? latestByChapter.get(chapterId
 if (!chapterId) return;
 const handler = (e: Event) => {
 const detail = (e as CustomEvent<VizStateEventDet
 if (detail?.chapterId === chapterId) {
 setRuntime(detail.variables || detail.line !==
 }
 };
 window.addEventListener(CHANNEL, handler);
 return () => window.removeEventListener(CHANNEL, ha
 }, [chapterId]);

```

```

 }, [chapterId, currentStep, maxStep, goToStep, selectStep]
 }
}

```

```

/** Утилиты без небезопасных cast-ов для визуализаторов
export const vizNumber = (value: DebugValue | undefined) => {
 typeof value === "number" && Number.isFinite(value) ? value : null;
}

```

```

export const vizString = (value: DebugValue | undefined) => {
 typeof value === "string" ? value : null;
}

```

```

export const vizArray = (value: DebugValue | undefined) => {
 Array.isArray(value) ? value : null;
}

```

```

export const vizRecord = (value: DebugValue | undefined) => {
 value !== null && typeof value === "object" && !Array.isArray(value)
 ? (value as Record<string, DebugValue>)
 : null;
}

```

```

/* — Матрицы из компилятора: СВОЯ размерность, а не дефолтная

```

```

/**
 * Нормализованная матрица, пришедшая из выполненного Рунима
 *
 * Размерность берётся из самих данных: если пользователь
 * ввёл 5 вершин, визуализация рисует 5x5, а не подставляет его
 * 7x7 (раньше ячейки «не той» размерности превращались в 0)
 *
 * Поддерживаются все учебные способы хранения:
 * list[list] d = [[0, 4], [999, 0]]
 * list[dict] d = [{0: 0, 1: 4}, {1: 0}]
 * dict[dict] d = {0: {0: 0, 1: 4}}
 * dict[(i, j)] d = {(0, 1): 4} — ключ после запятой
 */

```

```

export interface LiveMatrix {
 rows: number;
 cols: number;
 /** Значение ячейки; null — таких данных в коде пользователя нет
 at: (i: number, j: number) => number | null;
}

```

```

 if (n === null) return;
 cells.set(`${i}:${j}`, n);
 rows = Math.max(rows, i + 1);
 cols = Math.max(cols, j + 1);
 };

const asArray = vizArray(value);
if (asArray) {
 asArray.slice(0, MAX_MATRIX_SIDE).forEach((row, i) => {
 const inner = vizArray(row);
 if (inner) {
 inner.slice(0, MAX_MATRIX_SIDE).forEach((cell, j) => {
 return;
 });
 }
 const record = vizRecord(row);
 if (record) {
 for (const [key, cell] of Object.entries(record)) {
 const j = Number(key);
 if (Number.isInteger(j)) put(i, j, cell);
 }
 }
 });
} else {
 const record = vizRecord(value);
 if (!record) return null;
 for (const [key, cell] of Object.entries(record)) {
 const pair = parsePairKey(key);
 if (pair) {
 put(pair[0], pair[1], cell);
 continue;
 }
 const i = Number(key);
 if (!Number.isInteger(i)) continue;
 const inner = vizArray(cell);
 if (inner) {
 inner.slice(0, MAX_MATRIX_SIDE).forEach((v, j) => {
 continue;
 });
 }
 }
}

```



-----  
/\* — Выбор демонстрации внутри страницы (вкладки 1D / 2D)

```
const DEMO_CHANNEL = "algo:viz-demo";
const latestDemoByChapter = new Map<string, string>();

export interface VizDemoEventDetail {
 chapterId: string;
 /** base-запись страницы – demoId пустой; подписанные */
 demoId: string;
}

/** Демонстрация сообщает, какая её вкладка сейчас открыта */
export function emitVizDemo(chapterId: string, demoId: string) {
 if (typeof window === "undefined") return;
 latestDemoByChapter.set(chapterId, demoId);
 window.dispatchEvent(new window.CustomEvent<VizDemoEventDetail>("viz-demo", {
 detail: { chapterId, demoId }
 }));
}

/**
 * Подписка: панель компилятора следует за активной вкладкой
 * Сразу отдаём последнее значение: компилятор часто отстает
 * как пользователь переключился с 1D на 2D.
 */
export function onVizDemo(chapterId: string, cb: (demoId: string) => void) {
 if (typeof window === "undefined") return () => {};
 const latest = latestDemoByChapter.get(chapterId);
 if (latest !== undefined) cb(latest);
 const handler = (e: Event) => {
 const detail = (e as CustomEvent<VizDemoEventDetail>).detail;
 if (detail && detail.chapterId === chapterId) cb(detail.demoId);
 };
 window.addEventListener(DEMO_CHANNEL, handler);
 return () => window.removeEventListener(DEMO_CHANNEL, handler);
}
```

## Универсальное пособие • полный исходный код

```

tree = [0] * (2 * n)
i = 0 # лист: tree[n + i]

v, l, r = 1, 0, n - 1 # вершина и её отрезок
m = (l + r) // 2 # граница детей 2v, 2v+1
print(f"v={v} [{l}..{r}] mid={m}")`,
 variables: [
 { name: "n", role: "длина исходного массива; листы"},
 { name: "i", role: "индекс элемента массива; его левый и правый дети"},
 { name: "v", role: "текущая вершина дерева отрезка"},
 { name: "l, r", role: "границы отрезка, за которыми лежат левый и правый дети"},
 { name: "m", role: "середина отрезка $m = (l + r) // 2$ "},
],
},
treap: {
 vizTitle: "Декартово дерево: build → split → merge",
 stepNote: "Шаг = один спуск в split/merge/erase (перезаписывается целиком.",
 code: `points = [[45, 8], [3, 6], [6, 5], [2, 4], [1, 2]]
n = len(points) # сколько пар (x; y) – столько вершин
mid = 7 # разрез для split: x ≤ mid < y
key = 9 # какую точку удаляем в erase
step = 0 # номер шага – листает кадры дерева
height = 0 # высота построенного дерева
root = None

--- тело алгоритма ---
class Node:
 def __init__(s, x, y):
 s.x = x; s.y = y; s.l = None; s.r = None

def build(pts):
 """Декартово дерево из отсортированных по x точек:
 pts = sorted(pts, key=lambda p: p[0])
 stack = []
 for x, y in pts:
 cur = Node(x, y)
 last = None
```



```

while x.p:
 p = x.p; g = p.p
 step += 1
 if g is None:
 case = "zig" # родители
 rot(x)
 elif (g.l is p) == (p.l is x):
 case = "zig-zig" # x и p -
 rot(p); rot(x)
 else:
 case = "zig-zag" # змейка
 rot(x); rot(x)
 xk, pk, gk = x.k, p.k, (g.k if g else 0)
 print("шаг", step, "->", case, "| x =", xk, "p = ", pk)
 root = x
 print(" корень теперь", x.k, "| обход:", inorder(root))
 return x

```

# 1) Zig: родитель x – уже корень (дерево 40 -> 20)  
 root = N(40); root.l = N(20); root.l.p = root  
 splay(root.l)

# 2) Zig-Zig: бамбук 60 -> 40 -> 20, оба сына левые  
 root = N(60); root.l = N(40); root.l.p = root; root.l.l = N(20)  
 splay(root.l.l)

# 3) Zig-Zag: змейка 60 -> 20 -> 40 (40 – правый сын 20)  
 root = N(60); root.l = N(20); root.l.p = root; root.l.r = N(40)  
 splay(root.l.r),

```

variables: [
 { name: "case", role: "какой случай сработал: \"zig-zig\", \"zig-zag\", \"zig\""},
 { name: "step", role: "номер применения случая – 1, 2, 3"},
 { name: "xk / x", role: "ключ узла, который поднимается"},
 { name: "pk / p", role: "ключ родителя x перед поворотом"},
 { name: "gk / g", role: "ключ деда – нужен в Zig-Zig"},
 { name: "root", role: "корень дерева после расшейки"},
]

```

```

stepNote: "Шаг = заполнение очередного $P[i] = P[i-1]$
code: `a = [3, 1, 4, 1, 5, 9, 2, 6]` # исходный массив
n = len(a) # длина: префиксов будет n
i = 0 # текущая граница префикса

P = [0] # $P[0] = 0$ – пустой префикс
for i in range(1, n + 1):
 P.append(P[i - 1] + a[i - 1])
print("n =", n, "| P =", P)`
variables: [
 { name: "a", role: "исходный массив 1D-демонстрация" },
 { name: "n", role: "длина массива a; префиксов будет n" },
 { name: "P", role: "префиксные суммы: $P[i] = a[0] + \dots + a[i-1]$ " },
 { name: "i", role: "граница префикса: на шаге считаем до i" }
],
},
"dynamic-programming": {
 vizTitle: "Динамическое программирование (мемоизация)",
 stepNote: "Шаг = событие внутри fibМемо(k): вызов, возврат",
 code: `n = 5
memo = {}

def fib(k):
 if k in memo:
 return memo[k]
 if k <= 2:
 return 1
 print(f"считаем fib({k-1}) + fib({k-2})")
 r1 = fib(k - 1)
 r2 = fib(k - 2)
 memo[k] = r1 + r2
 return memo[k]

print("итог:", fib(n))`,
variables: [
 { name: "n", role: "целевой номер числа Фибоначчи" },
 { name: "k", role: "аргумент текущего рекурсивного вызова" },
 { name: "memo", role: "кэш уже посчитанных значений" }
]

```

```

code: `from collections import deque

q = deque(["A"]) # очередь BFS
u = "B" # сосед, найденный из текущ
q.append(u) # enqueue соседа – в конец
front = q[0] # голова очереди: её обрабо
v = q.popleft() # шаг: достали голову и обр
print("front =", front, "| v =", v, "| очередь:", list(
 variables: [
 { name: "front", role: "голова очереди – вершина,
 { name: "v", role: "вершина, извлечённая из очере
 { name: "u", role: "сосед вершины v, которого кла
],
},
"everyday-basics": {
 vizTitle: "Бытовой тренажёр: стек, очередь, куча",
 stepNote: "Шаг = одна бытовая операция: тарелка на
 code: `from collections import deque

st, q = [], deque() # тарелки (стек) и очередь
st.append(" "); top = st.pop()
q.append(" "); q.append(" "); first = q.popleft()

i = 0 # счётчик выполненных опера
for op in ("push", "pop", "enqueue", "dequeue"):
 i += 1
print("тарелка:", top, "| первый из очереди:", first, "
 variables: [
 { name: "top / front", role: "верх стопки (стек)
 { name: "i", role: "индекс в массиве кучи; родите
],
},
"graph-dfs-bfs": {
 vizTitle: "DFS и BFS на графе",
 stepNote: "Шаг = переход из текущей вершины v к неп
 code: `from collections import deque

adj = {"A": ["B", "C"], "B": ["D"], "C": [], "D": []}
```

## Универсальное пособие • полный исходный код

```

v = -1 # вершина, в которой обход начался
stack = [] # фронт обхода (стек для DFS)
i = 0 # номер шага — двигает по какому-то ребру

--- тело алгоритма ---
for start in range(n):
 if comp[start] != -1:
 continue
 comps += 1 # непокрашенная вершина => новая компонента
 c = comps
 stack = [start]
 comp[start] = c
 used[start] = True
 v = start
 while stack:
 v = stack.pop() # для BFS: v = stack.pop(0)
 i += 1
 for to in adj[v]:
 if comp[to] == -1:
 comp[to] = c
 used[to] = True
 stack.append(to)
print("компонент:", comps)
print("разметка comp:", comp)`
variables: [
 { name: "n", role: "число вершин; в демонстрации задано 10" },
 { name: "adj", role: "списки смежности — тот же граф, что в примере" },
 { name: "comp", role: "разметка: номер компоненты", range: "-1 ... comps" },
 { name: "used", role: "булев вариант разметки — для BFS не нужен" },
 { name: "comps", role: "счётчик найденных компонент" },
 { name: "c", role: "номер компоненты, которую крашим сейчас" },
 { name: "v", role: "вершина, где обход находится" },
 { name: "stack", role: "фронт обхода: стек (DFS) или очередь (BFS)" },
 { name: "i / step", role: "номер шага — перематываем кадр" },
],
},
"top-sort": {
```

```
def dfs1(v):
 used[v] = True
 for to in adj[v]:
 if not used[to]:
 dfs1(to)
 order.append(v) # выходим из вершины — она

for v in adj:
 if not used[v]:
 dfs1(v)
 print("DFS из", v, "закончен | order:", order)

транспонирование: каждое ребро разворачиваем
adjT = {}
for v in adj:
 adjT[v] = []
for v in adj:
 for to in adj[v]:
 adjT[to].append(v)
print("обратный граф:", adjT)

фаза 2: идём по order С КОНЦА, DFS по adjT красит одн
comp = {}
c = 0
for i in range(len(order) - 1, -1, -1):
 v = order[i]
 if v in comp:
 continue
 stack = [v]
 comp[v] = c
 while stack:
 u = stack.pop()
 for to in adjT[u]:
 if to not in comp:
 comp[to] = c
 stack.append(to)
 print("SCC", c, "собрана из", v, ":", comp)
```



```
},
 "bridges-code": {
 vizTitle: "Мосты: DFS + tin/low",
 stepNote: "Шаг = строка псевдокода слева; ребро (v,
 code: `tin, low = {"A": 1, "B": 2, "G": 3}, {"A": 1
timer = 3 # как на демо

v, to = "B", "G" # шаг: возврат из G
is_bridge = low[to] > tin[v] # 3 > 2 → мост!
print(f"ребро {v}-{to}: мост? {is_bridge}")`,
 variables: [
 { name: "v", role: "текущая вершина DFS", range:
 { name: "to", role: "сосед; ребро (v, to) проверяе
 { name: "tin[v]", role: "время входа в вершину – m
 { name: "low[v]", role: "минимум tin, куда можно ,
 { name: "timer", role: "глобальный счётчик времени
],
 },
 "euler-path-vs-cycle": {
 vizTitle: "Эйлеров маршрут: алгоритм Иера",
 stepNote: "Шаг = переход по неиспользованному ребру
 идти нельзя.",
 code: `n = 5 # вершин: 0...4 (на
edges = [(0, 1), (0, 2), (1, 3), (2, 4), (1, 2), (3, 4)]
adj = {v: [] for v in range(n)}
for i, (u, v) in enumerate(edges):
 adj[u].append((v, i))
 adj[v].append((u, i))
deg = {v: len(adj[v]) for v in range(n)} # степени ве
odd = [v for v in range(n) if deg[v] % 2 == 1] # нечёт
start = odd[0] if odd else 0 # путь начинаем
used = [False] * len(edges) # какие рёбра :
path = [start] # маршрут: пос.
last = start # где мы сейчас
stack = [start] # стек для отк
v = start
step = 0
```

```

V = n # вершин для формулы
E = sum(len(a) for a in adj) // 2 # рёбер: в списке смежности
F = 2 - V + E # граней: из $V - E + F = 2$

--- тело алгоритма ---
print("V =", V, "| E =", E, "| F =", F, "| $V - E + F = 2$ ")
for v in range(n):
 used = set()
 for u in adj[v]:
 if color[u] != 0:
 used.add(color[u])
 c = 1
 while c in used: # наименьший свободный цвет
 c += 1
 color[v] = c
 chi = max(chi, c)
 i += 1
 print("v =", v, "занято:", sorted(used), "-> цвет", color[v])
print("chi =", chi, "| delta =", delta, "| chi <= delta")
variables: [
 { name: "n", role: "число вершин графа на демонстрации" },
 { name: "adj", role: "списки смежности — тот же граф" },
 { name: "color", role: "массив цветов вершин (0 = не покрашено)" },
 { name: "v", role: "вершина, которую красим сейчас" },
 { name: "u", role: "сосед вершины v, чей цвет уже занят" },
 { name: "delta", role: "максимальная степень $\Delta = \max \deg v$ " },
 { name: "chi", role: "сколько цветов реально понадобится" },
 { name: "i / step", role: "номер шага — перематываем" },
 { name: "V, E, F", role: "для вкладки «Планарность»" },
]
},
dijkstra: {
 vizTitle: "Дейкстра",
 stepNote: "Шаг = старт, извлечение ближайшей вершины",
 code: `import heapq

```

## Универсальное пособие • полный исходный код

```

nodes = ["S", "A", "B", "C", "D", "E", "F"]
n = len(nodes) # 7 вершин => n - 1 = 6 работ
m = len(E) # 9 рёбер
```

```
dist = {v: INF for v in nodes}
dist["S"] = 0 # старт – База
```

```
--- тело алгоритма ---
```

```
last_changed = ""
for i in range(1, n): # итерация i = 1 ... n-1
 last_changed = ""
 for u, v, w in E: # шаг: релаксация ребра (u, v)
 if dist[u] != INF and dist[u] + w < dist[v]:
 dist[v] = dist[u] + w
 last_changed = v
 if last_changed == "":
 print(f"итерация {i}: изменений нет – сошлось для всех вершин")
 break
 print(f"итерация {i}: " + ", ".join(f"{k}={dist[k]}" for k in nodes))
```

```
контрольный (n-й) проход: улучшение здесь означает отрицательный цикл
cycle_edge = None
for u, v, w in E:
 if dist[u] != INF and dist[u] + w < dist[v]:
 cycle_edge = (u, v, w)
 break
```

```
if cycle_edge:
 print("отрицательный цикл: улучшается ребро", cycle_edge)
else:
```

```
 print("отрицательных циклов нет:", {k: dist[k] for k in nodes})
 variables = [
 { name: "n", role: "число вершин графа – ровно n-1"},
 { name: "m", role: "число рёбер, по которым проходят"},
 { name: "i", role: "номер итерации (после i итераций)"},
 { name: "u, v, w", role: "текущее ребро: откуда, куда"},
 { name: "dist[v]", role: "расстояние; если обновилось"},
],
```

```

 на-Форда" },
 { name: "u, v", role: "ребро (u, v) перевзвешивает",
 { name: "d[v]", role: "расстояние Дейкстры в новом графе"
 },
],
 },
 mst: {
 vizTitle: "Остовное дерево: Краскал и DSU",
 stepNote: "Шаг = следующее ребро из отсортированных",
 code: `n = 9 # вершин: A..I — те, что в графе
edges = [(2, "A", "B"), (3, "D", "E"), (4, "B", "C"), (5, "A", "E"),
 (7, "E", "F"), (8, "D", "G"), (9, "C", "F"), (10, "D", "F"),
 (12, "H", "I"), (13, "F", "I")]
edges.sort() # Краскал: рёбра по возрастанию веса
parent = {v: v for v in "ABCDEFGH I"} # DSU: представителю
rank = {v: 0 for v in "ABCDEFGH I"} # высота дерева
mst = [] # рёбра, взятые в остов
total = 0 # вес остова (на этом графе)
i = 0 # номер текущего ребра
u, v, w = None, None, None # концы и вес ребра, которое

--- тело алгоритма ---
def find(x):
 while parent[x] != x:
 parent[x] = parent[parent[x]] # сжатие путей
 x = parent[x]
 return x

for i, (w, a, b) in enumerate(edges):
 u, v = a, b
 ra, rb = find(a), find(b)
 if ra != rb: # разные компоненты
 parent[ra] = rb
 if rank[ra] == rank[rb]:
 rank[rb] += 1
 mst.append((w, a, b))
 total += w
print("вес остова:", total, "| рёбер:", len(mst))

```

```

vizTitle: "Z-функция",
stepNote: "Шаг = вычисление z[i]: внутри Z-блока [l, r]",
code: `s = "abacaba"
n = len(s)
i, l, r = 1, 0, 0 # правый Z-блок [l, r]
z = [None] * n # пустые клетки: значения по умолчанию
z[0] = 0

for i in range(1, n): # шаг: вычисляем z[i]
 if i <= r:
 z[i] = min(r - i + 1, z[i - l]) # из блока
 else:
 z[i] = 0 # свежая клетка: с нуля
 while i + z[i] < n and s[z[i]] == s[i + z[i]]:
 z[i] += 1 # досчёт в лоб
 if i + z[i] - 1 > r:
 l, r = i, i + z[i] - 1
print(z)`
variables: [
 { name: "n", role: "длина строки s", range: "n = len(s)" },
 { name: "i", role: "позиция, для которой считаем z[i]" },
 { name: "l, r", role: "правый крайний Z-блок: отрезок [l, r]" },
 { name: "z[i]", role: "длина наибольшего общего префикса s[i:] и s[l:r+1]" }
],
},
"aho-corasick": {
 vizTitle: "Ахо-Корасик: бор, fail-ссылки и автомат",
 stepNote: "Шаг = один символ текста: state = go(state, c). Если не найдено, то переходят в корень. Если найдено, то переходят с картинкой «водопада».",
 code: `words = ["HERS", "HIS", "SHE", "HE"] # образцы
text = "USHERS" # текст, в котором ищем
n = len(text) # длина текста
state = 0 # текущее состояние автомата
i = 0 # индекс текущего символа
c = "" # сам символ
nodes = 1 # сколько вершин в автомате
queue = [] # очередь BFS, к которой добавляем новые вершины
fail = {} # fail[v] = вершина, в которую переходят, если в автомате нет ребра из v в c`

```

```

 queue.append(u)
print("вершин в боре:", nodes)
print("fail-ссылки:", fail)

def go(v, ch):
 """Переход автомата: сначала по бору, иначе спускаем
 while v != 0 and (v, ch) not in nxt:
 v = fail[v]
 return nxt.get((v, ch), 0)

for i, c in enumerate(text):
 state = go(state, c)
 v = state
 while v != 0: # собираем все образцы
 if v in term:
 matches.append((i, term[v]))
 v = fail[v]
 print(f"i={i} c='{c}' state={state} найдено={matches}")

print("всего вхождений:", len(matches), matches),
variables: [
 { name: "words", role: "набор образцов – по нему",
 },
 { name: "text", role: "сканируемый текст – подста",
 { name: "state", role: "текущее состояние автомата",
 { name: "i / c", role: "индекс и сам символ текста",
 { name: "nxt", role: "переходы бора: nxt[(v, ch)]",
 { name: "fail", role: "fail[v] – вершина наибольшей",
 { name: "term", role: "в каких вершинах заканчивае",
 { name: "queue", role: "очередь BFS, которой натя",
 { name: "matches", role: "найденные вхождения: (ин",
 { name: "nodes", role: "сколько вершин в боре (ко",
],
},
"complexity-classes": {
 vizTitle: "Классы сложности: рост по n и ops",
 stepNote: "Шаг = одна точка (n, ops): визуализация",
 code: `n = 1 # размер входа

```

```

 },
 intro: {
 // мнемодкарточки без шагов алгоритма
 variables: [],
 },

 /* — Дополнительные демонстрации внутри страниц (вкл.)

 "planarity-euler-formula#greedy": {
 vizTitle: "Жадная покраска вершин ($\chi \leq \Delta + 1$)",
 stepNote: "Шаг = вершина v получает наименьший цвет",
 code: `n = 8 # вершин в графе
adj = [[1, 2], [0, 2, 3], [0, 1, 4], [1, 4, 5], [2, 3, 4]]
delta = max(len(a) for a in adj) # максимальная степень
color = [0] * n # 0 = ещё не покрашена
chi = 0 # сколько цветов понадобилось
v = -1 # вершина, которую красим сейчас
i = 0 # номер шага

--- тело алгоритма ---
for v in range(n):
 used = set()
 for u in adj[v]:
 if color[u] != 0:
 used.add(color[u])
 c = 1
 while c in used: # наименьший свободный цвет
 c += 1
 color[v] = c
 chi = max(chi, c)
 i += 1
 print("v =", v, "занято:", sorted(used), "-> цвет", color[v])
print("chi =", chi, "| delta =", delta, "| chi <= delta")
 variables: [
 { name: "color", role: "цвета вершин (0 = не покрашено)", range: "0 ... n" },
 { name: "v", role: "текущая вершина", range: "0 ... n-1" },
 { name: "delta / chi", role: "максимальная степень / количество цветов", range: "1 ... n" },
 { name: "i / step", role: "номер кадра", range: "0 ... n-1" },
]
 }
}
 `

```

```

],
},
"planarity-euler-formula#edges": {
 vizTitle: "Покраска рёбер (Визинг: $\Delta \leq \chi' \leq \Delta + 1$)",
 stepNote: "Шаг = ребро e получает наименьший цвет, <= \Delta",
 code: `n = 8
edges = [(0, 1), (0, 2), (1, 2), (1, 3), (2, 4), (3, 4)]
adj = [[1, 2], [0, 2, 3], [0, 1, 4], [1, 4, 5], [2, 3, 4]]
delta = max(len(a) for a in adj)
ecolor = [0] * len(edges) # цвет каждого ребра
inc = [dict() for _ in range(n)] # inc[v][цвет] = ребро
e = -1 # номер текущего ребра
v = u = -1 # его концы
chi_e = 0

--- тело алгоритма ---
for e, (a, b) in enumerate(edges):
 v, u = a, b
 used = set(inc[a].values()) | set(inc[b].values())
 c = 1
 while c in used:
 c += 1
 ecolor[e] = c
 inc[a][e] = c
 inc[b][e] = c
 chi_e = max(chi_e, c)
 print("ребро", (a, b), "занято:", sorted(used), "-> ", c)
print("chi' =", chi_e, "| delta =", delta, "| delta <= \Delta")
variables: [
 { name: "ecolor", role: "цвета рёбер – рёбра на графе", range: "0..delta" },
 { name: "e", role: "номер текущего ребра", range: "0..len(edges)-1" },
 { name: "v, u", role: "концы текущего ребра – подмножества V", range: "0..n-1" },
 { name: "inc", role: "для каждой вершины: какие цвета имеют инцидентные рёбра", range: "0..delta" },
 { name: "chi_e / delta", role: "χ' – сколько цветов", range: "0..delta+1" }
],
},
"graph-components#dfs": {
 vizTitle: "Компоненты связности: DFS-покраска",

```



```

 },
 "graph-components#dsu": {
 vizTitle: "Компоненты связности: DSU по рёбрам",
 stepNote: "Шаг = одно ребро (u, v): union склеивает",
 code: `n = 10 # вершин в графе
edges = [(0, 1), (1, 2), (2, 3), (0, 3), (1, 3), (4, 5)]
parent = list(range(n)) # DSU: parent[v] = представи
rank = [0] * n # высота дерева – по ней реш
comps = n # сначала каждая вершина – о
e = -1 # номер обрабатываемого ребра
u, v = -1, -1 # концы текущего ребра
step = 0 # номер шага – двигает по ка

--- тело алгоритма ---
def find(x):
 while parent[x] != x:
 parent[x] = parent[parent[x]] # сжатие путей
 x = parent[x]
 return x

def union(a, b):
 global comps
 ra, rb = find(a), find(b)
 if ra == rb:
 return False # ребро внутри
 if rank[ra] < rank[rb]:
 ra, rb = rb, ra
 parent[rb] = ra
 if rank[ra] == rank[rb]:
 rank[ra] += 1
 comps -= 1
 return True

for e, (a, b) in enumerate(edges):
 u, v = a, b
 step = e + 1
 union(a, b)
print("компонент:", comps)

```

```

k = 3 # размер клики, которую ищем

--- тело алгоритма ---
Сведение 3-SAT <=p КЛИКА: вершина = {номер дизъюнкта,
ребро = два литерала из РАЗНЫХ дизъюнктов, не противоречивые
clauses = [(1, 2, -3), (-1, 3, 4), (2, -3, -4)]
V = [(i, l) for i, cl in enumerate(clauses) for l in cl]
E = [(a, b) for a in V for b in V if a[0] < b[0] and a[1] != b[1]]
for step in range(7):
 print("кадр", step, "| вершин:", len(V), "| рёбер:", len(E))
print("формула выполнима <=> в графе есть клика размера k")
variables: [
 { name: "step", role: "номер кадра сведения (0..6)" },
 { name: "cls", role: "класс задачи A, которую сводят к клике" },
 { name: "n", role: "размер входа x; после сведения к клике" },
 { name: "k", role: "размер клики, которую ищет рекурсия" },
],
},
"complexity-classes#verify": {
 vizTitle: "Верификатор 3-SAT: проверка сертификата",
 stepNote: "Шаг = один дизъюнкт: верификатор подставляет значения в формулу",
 code: `clauses = [(1, 2, -3), (-1, 3, 4), (2, -3, -4)]
cert = [1, 0, 1, 0] # сертификат: x1=1, x2=0, x3=1
k = 0 # номер проверяемого дизъюнкта
ok = True # вердикт верификатора

--- тело алгоритма ---
for k, cl in enumerate(clauses):
 hit = False
 for l in cl:
 val = cert[l - 1] if l > 0 else 1 - cert[-l - 1]
 if val == 1:
 hit = True
 ok = ok and hit
 print("дизъюнкт", k + 1, cl, "выполнен" if hit else "не выполнен")
print("сертификат", "принят" if ok else "отвергнут")`,
 variables: [
 { name: "cert", role: "сертификат – назначение переменных" },
],
},

```

```

edges.sort() # Краскал: рёбра по возрастанию веса
parent = {v: v for v in "ABCDEFGHI"} # DSU: представител
rank = {v: 0 for v in "ABCDEFGHI"} # высота дерева
mst = [] # рёбра, взятые в остов
total = 0 # вес остова (на этом графе 51)
i = 0 # номер текущего ребра
u, v, w = None, None, None # концы и вес ребра, которое рассматриваем

--- тело алгоритма ---
def find(x):
 while parent[x] != x:
 parent[x] = parent[parent[x]] # сжатие путей
 x = parent[x]
 return x

for i, (w, a, b) in enumerate(edges):
 u, v = a, b
 ra, rb = find(a), find(b)
 if ra != rb: # разные компоненты
 parent[ra] = rb
 if rank[ra] == rank[rb]:
 rank[rb] += 1
 mst.append((w, a, b))
 total += w
print("вес остова:", total, "| рёбер:", len(mst))
print(mst)`
 variables: [
 { name: "i", role: "номер текущего ребра – перебираем"},
 { name: "u, v, w", role: "концы и вес рассматриваемого ребра"},
 { name: "parent", role: "DSU-массив: цвета компонент"},
 { name: "mst", role: "рёбра, уже взятые в остов", "51"},
 { name: "total", role: "вес остова (51 на этом графе)"},
],
},
"mst#prim": {
 vizTitle: "Остовное дерево: Прим (рост из вершины E)",
 stepNote: "Шаг = из кучи достаётся самое лёгкое ребро",
 code: `import heapq

```

```

vizTitle: "Остовное дерево: Борувка (фазы-«пятiletки»)"
stepNote: "Шаг = одна фаза: каждая компонента ОДНОВЕРХОВА"
code: `n = 9 # вершин: A...I
edges = [(2, "A", "B"), (3, "D", "E"), (4, "B", "C"), (5, "C", "D"),
 (7, "E", "F"), (8, "D", "G"), (9, "C", "F"), (10, "F", "G"),
 (12, "H", "I"), (13, "F", "I")]
comp = {v: v for v in "ABCDEFGH I"} # DSU: корень компоненты
cheapest = {} # корень компоненты -> самое дешёвое ребро
mst = [] # рёбра остова
total = 0 # вес остова (= 51)
phase = 0 # номер фазы («пятiletки») — от 0 до 4
u, v, w = None, None, None # ребро-кандидат

--- тело алгоритма ---
def find(x):
 while comp[x] != x:
 comp[x] = comp[comp[x]]
 x = comp[x]
 return x

while len(mst) < n - 1:
 phase += 1
 cheapest = {}
 for (w, a, b) in edges: # КАЖДАЯ компонента ОДНОВЕРХОВА
 ra, rb = find(a), find(b)
 if ra == rb:
 continue
 u, v = a, b
 if ra not in cheapest or w < cheapest[ra][0]:
 cheapest[ra] = (w, a, b)
 if rb not in cheapest or w < cheapest[rb][0]:
 cheapest[rb] = (w, a, b)
 if not cheapest:
 break # граф несвязен
 for (w, a, b) in cheapest.values():
 ra, rb = find(a), find(b)
 if ra == rb:
 continue # две компоненты

```

```

 { name: "k", role: "уровень: блоки $2^k \times 2^k$ (шаг
 { name: "st2[{r,c,k,k}]", role: "минимум квадратн
 },
},
"sparse-table#2d-query": {
 vizTitle: "Разреженная таблица 2D: запрос",
 stepNote: "Демонстрация кликабельна вручную: выбери
 code: `# st2[{r, c, kx, ky}] из вкладки «построение
r1, c1, r2, c2 = 1, 1, 6, 6

h, w = r2 - r1 + 1, c2 - c1 + 1
kx, ky = h.bit_length() - 1, w.bit_length() - 1
print(f"прямоугольник {h}x{w} кроется блоками уровня ({
 variables: [
 { name: "r1, c1, r2, c2", role: "углы запрашиваем
 { name: "kx, ky", role: "уровень самого крупного
],
},
"prefix-sums-2d#2d": {
 vizTitle: "Префиксные суммы 2D (прямоугольники)",
 stepNote: "Демонстрация кликабельна вручную: наведи
 code: `A = [[1, 2, 3, 4], [5, 6, 7, 8], [9, 1, 2, 3
n, m = 4, 4
i, j = 0, 0
S = [] # строки таблицы: рамка и
for rr in range(n + 1):
 S.append([0] + [None] * m)
S[0] = [0] * (m + 1)
for i in range(1, n + 1):
 for j in range(1, m + 1):
 S[i][j] = A[i - 1][j - 1] + S[i - 1][j] + S[i][j - 1]

r1, c1, r2, c2 = 1, 1, 2, 2
ans = S[r2 + 1][c2 + 1] - S[r1][c2 + 1] - S[r2 + 1][c1]
print("итого:", ans)`
 variables: [
 { name: "i, j", role: "текущая ячейка таблицы S (

```

```

const keep: string[] = [];
for (const line of code.split("\n")) {
 const t = line.trim();
 if (t === "# --- тело алгоритма ---" || /^(for|while)/.test(t)) {
 keep.push(line);
 }
}
// убрать пустые строки с конца
while (keep.length > 0 && keep[keep.length - 1].trim() === "") {
 keep.pop();
}

/**
 * Дефолтное содержимое редактора: шапка + ТОЛЬКО инициализация
 * переменные демо (без всего кода алгоритма).
 */
export function buildInitTemplate(chapterId: string, chapterTitle: string, syncId: string) {
 const sync = getPageSync(chapterId, demoId);

 const head = sync.vizTitle
 ? `# «${chapterTitle}»\n# демо: ${sync.vizTitle}\n#`
 : `# «${chapterTitle}»\n# демо на странице нет – св

 if (!sync.code) return head;
 const init = extractInit(sync.code);
 return `${head}\n${init}\n`;
}

```

## РАЗДЕЛ: БИЛЕТЫ (УЧЕБНЫЙ КОНТЕНТ)

ФАЙЛ 26/111: src/data/tickets/ticket1\_segtree.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 217 | РАЗМЕР: 1.5 КБ

```
import { Chapter } from '../../../types';
```

```

span class="font-mono text-emerald-300">[0, 1
тям <span class="font-mono text-emerald-300
– это одиночные элементы массива. В узле л
чество единиц, И/ИЛИ – любая операция, котор
>).</p>

```

```

<p class="text-slate-300 text-sm mb-4">Отсю
а каждом уровне встречает один из трёх случ
не пересекается – возвращаем нейтральный э
склеиваем. Частичных пересечений на уровне
одного элемента – спуск к листу и пересчёт

```

```

<div class="grid grid-cols-1 sm:grid-cols-3
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
</div>

```

```

<div class="grid grid-cols-1 xl:grid-cols-2
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
 <div class="border border-emerald-500 shadow-md">

```

Аналогия 1: Корпоративная пре

```

 </div>
 <p class="text-slate-300 text-sm">Б
икам департаментов: «внутри всем +5 %». Нач
nt-mono text-emerald-300">lazy[v]) и
свой баланс. Это и есть отложенное обнов
</div>

```

```
<p class="text-slate-500 text-xs mt-3">
 кивают в строгом порядке: сначала присваива
</div>
```

```
<div class="bg-slate-900/40 rounded-lg border
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
за $O(1)$, но строятся один раз и умирают при
ми.</p>
```

```
 <p class="text-slate-400 text-sm mb-1">
идемпотентные запросы (min, gcd, И) по ста
 <p class="text-slate-400 text-sm mb-1">
n ячеек (не $2n!$) – иначе на $n = 6$ поймать п
n ... $2n-1$] и n – степень двойки (или использ
 <p class="text-slate-400 text-sm">•
ть лень во всё дерево – это $O(n)$ и весь смы
</div>
```

```
<p class="text-slate-300 text-sm mt-4"><b c
-2 py-0.5 rounded text-xs font-mono text-em
ся один раз), запрос на отрезке <code class=
-slate-800"> $O(\log n)$ </code>, одиночное обно
-300 border border-slate-800"> $O(\log n)$ </code>
s font-mono text-emerald-300 border border-
xt-xs font-mono text-emerald-300 border bord
ounded text-xs font-mono text-emerald-300 b
```

```
<p class="text-slate-500 text-xs mt-3 leading
 в редакторах и базах, сжатие координат + «
оить», дерево отрезков по дереву (HLD), пер
</p>
```

```
<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
```

```
 Код: построение и запрос суммы (C
</summary>
```

```
<div class="p-5 text-sm text-slate-300
```



```

 y[4 * MAXN]; // «обещание»: прибавит

void push(int v, int l, int r) {
 if (lazy[v] == 0) return;
 int m = (l + r) / 2;
 tree[2 * v] += lazy[v] * (m - l + 1); // п
 lazy[2 * v] += lazy[v];
 tree[2 * v + 1] += lazy[v] * (r - m); // и
 lazy[2 * v + 1] += lazy[v];
 lazy[v] = 0; // о
}

void add(int v, int l, int r, int L, int R, int x) { /
 if (r < L || R < l) return;
 if (L <= l && r <= R) {
 tree[v] += x * (r - l + 1);
 lazy[v] += x; // д
 return;
 }
 push(v, l, r); // о
 int m = (l + r) / 2;
 add(2 * v, l, m, L, R, x);
 add(2 * v + 1, m + 1, r, L, R, x);
 tree[v] = tree[2 * v] + tree[2 * v + 1];
}

```

<p class="text-slate-400 text-xs">Пр  
 г накопиться. Забыл push – получил правильн  
 </div>  
 </details>

<details class="bg-slate-900/40 rounded-lg l  
 <summary class="p-4 font-bold text-slate  
 -b border-slate-700/50">  
 Код: итеративное дерево (2n, снизу)  
 </summary>  
 <div class="p-5 text-sm text-slate-300  
 <pre class="bg-slate-950 p-4 rounde  
 # для удобств

```
</section>
```

```
`
,
 },
];
```

---

ФАЙЛ 27/111: src/data/tickets/ticket14\_20.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТР0К: 1218 | РАЗМ:

---

```
import { Chapter } from '../..types';
```

```
export const tickets14to20: Chapter[] = [
 {
```

```
 id: "dijkstra",
 title: "14. Графы. Поиск кратчайшего пути. Дейкстра",
 type: "html",
 category: "Графы. Пути",
 content: `
```

```
<section id="dijkstra-content" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">
 </div>
```

```
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">

 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 </div>
```

```
<p class="text-slate-300 text-sm mb-4">Дейк
ершины
требование к графу – неотрицательные вес
```

```
ext-emerald-300">w ≥ 0}, зафиксирова
ию расстояния!</p>
</div>
```

```
<div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4
border-rose-500 shadow-md">
```

Ограничения: минус ломает жадно

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4"
lass="font-mono text-emerald-300">s→a = 2</
ono text-emerald-300">b→a = -2. Верн
начала вытащит <span class="font-mono text-
o text-emerald-300">3 - 2 = 1 придёт
Джонсона (билет 17).</p>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl bord
<h3 class="text-xl font-bold text-blue-400 m
```

```
<p class="text-slate-300 text-sm mb-4">Демон
D, Дом 2 E, ТЦ F, Метро G) и 11 одностор
ршины <span class="font-mono text-emerald-300"
(u → v): переменные <span class="font-
>, <span class="font-mono text-emerald-300"
ают картинку напрямую.</p>
```

```
<ol class="list-decimal pl-5 space-y-2 text
 Инициализация. <span class="font-mono
text-emerald-300">∞; в кучу кладём <span class="font-
emerald-300">prev пуст.
```

```
 Извлечь минимум. Достаём пару
"font-mono text-emerald-300">du. Если
xt-emerald-300">done – пара устарела
```

```
 Зафиксировать. <span class="font-mono
меняется.
```

```
</div>
```

```
<div class="bg-slate-800/70 p-5 rounded-lg"
 <p class="text-sm font-bold text-white"
 <table class="w-full text-xs text-left"
 <thead>
 <tr class="text-slate-500 border"
 <th class="py-1.5 pr-2 font"
 <th class="py-1.5 pr-2 font"
 <th class="py-1.5 font-bold"
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-
y-2 pr-2">лучший известный путь $s \rightarrow v$ </td><td class="
 <tr class="border-b border-slate-
y-2 pr-2">предок v на текущем лучшем пути</td><td class="
 <tr class="border-b border-slate-
lass="py-2 pr-2">вершины с окончательным $dist[v]$ </td></tr>
 <tr class="align-top"><td class="text-slate-300"
ие, вершина)</td><td class="py-2">извлечения
 </tbody>
 </table>
```

```
</div>
```

```
</div>
```

```
<div class="bg-slate-700/50 p-6 rounded-xl border"
 <h3 class="text-xl font-bold text-indigo-400">Извлечение из кучи
 <p class="text-slate-300 text-sm mb-4">Инициализация
 извлекается из кучи, $s \rightarrow u$. Отсюда
 $dist[v] = \delta(s, v)$ для всех достижимых v .
 $dist[v] = \infty$ для недостижимых v .
```

```
<details class="bg-slate-900/40 rounded-lg"
 <summary class="p-4 font-bold text-slate-300">Алгоритм Дейкстры
```

```

 </div>
 </details>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border-2 border-slate-800">
 <h3 class="text-xl font-bold text-emerald-400">Дейкстра на графе
 <div class="overflow-x-auto my-4">
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border-b">
 <th class="py-2 pr-3 font-b">
 <th class="py-2 pr-3 font-b">
 <th class="py-2 font-bold">
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-800">
 <td class="text-slate-300">priority_queue (C++)</td><td class="py-2 pr-3 font-mono">
 й выбор</td></tr>
 <tr class="border-b border-slate-800">
 <td class="text-slate-300">ease-key</td><td class="py-2 pr-3 font-mono">
 очереди</td></tr>
 <tr class="border-b border-slate-800">
 <td class="text-slate-300">поиск минимума</td><td class="py-2 pr-3 font-mono">
 </tr>
 <tr class="border-b border-slate-800">
 <td class="text-slate-300">><td class="py-2 pr-3 font-mono">0(V + E)</td></tr>
 <tr class="align-top"><td class="text-slate-300 font-mono">0(E + V · C)</td><td class="py-2">ц
 </tr>
 </tbody>
 </table>
 </div>

<details class="bg-slate-900/40 rounded-lg border-2 border-slate-800">
 <summary class="p-4 font-bold text-slate-300 border-b border-slate-700/50">» Код: Дейкстра на
 <div class="p-5 text-sm text-slate-300"

```

```

 "E": [{"F", 1}],
 "F": [],
 "G": [{"D", 1}, {"E", 2}],
}
dist, prev = dijkstra(g, "A")
print(dist) # A 0, B 3, C 2, D 6, E 7,
print(restore(prev, "A", "F")) # ['A', 'C', 'G', 'E',
 <p class="text-slate-400 text-xs">П
 же вершины, потому что читает ваши <span c
nt-mono">done.</p>
 </div>
</details>

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">> Код: C++ (priority
 <div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded overflow-x-auto te
 t;
#include <queue>;
using namespace std;
const long long INF = 4e18; // не INT_MAX:

int n;
vector<vector<pair<int,int>>>> g;
vector<long long> d;
vector<int> p;

void dijkstra(int s) {
 d.assign(n, INF); p.assign(n, -1); d[s] = 0;
 priority_queue<pair<long long,int>;,
 vector<pair<long long,int>>&
 greater<pair<long long,int>>
 pq.push({0, s});
 while (!pq.empty()) {
 auto [du, u] = pq.top(); pq.pop();
 if (du > d[u]) continue; // устарев
 for (auto [v, w] : g[u]) {

```



```

<div class="grid grid-cols-1 sm:grid-cols-3">
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
 <div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
 </div>
</div>

```

```

<div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-emerald-500 shadow-md">
 Аналогия 1: Патруль Бэтмена на м
 </div>
 <div class="my-3 rounded-lg overflow">
 все ули
овно 1 выходной в году, когда он отдр
-ю ночь Бэтмен делает контрольный проезд: е
о-то района снова уменьшилось – значит, Джо
</div>

```

```

 <div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 border border-rose-500 shadow-md">
 Отрицательный цикл: временная п
 </div>

```



```

 </table>
 <p class="text-slate-400 text-xs mt-3">
бном» порядке: S→A,
порядок рёбер местами – и понадобится до ше
</div>

<div class="bg-slate-800/70 p-5 rounded-lg"
 <p class="text-sm font-bold text-white"
 <p class="text-slate-300 text-sm mb-3">
тава A имеет суммарный вес <span cla
веет» на 2.</p>
 <table class="w-full text-xs text-left"
 <thead>
 <tr class="text-slate-500 borde
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 pr-2 font
 <th class="py-1.5 font-bold
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate
-2 font-mono">0</td><td class="py-2 pr-2 fo
td></tr>
 <tr class="border-b border-slate
-2 font-mono">-2</td><td class="py-2 pr-2 f
/td></tr>
 <tr class="border-b border-slate
-2 font-mono">-6</td><td class="py-2 pr-2 f
</td></tr>
 <tr class="border-b border-slate
"py-2 pr-2 font-mono">-10</td><td class="py
nt-mono">-2</td></tr>
 <tr class="align-top"><td class
12</td><td class="py-2 pr-2 font-mono">-9</
ont-bold">всё ещё уменьшается → отрицательны
 </tbody>

```

```

<div class="overflow-x-auto">
 <table class="w-full text-xs text-l
 <thead>
 <tr class="text-slate-500 b
 <th class="py-1.5 pr-2
 <th class="py-1.5 pr-2
 <th class="py-1.5 pr-2
 <th class="py-1.5 font-l
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-
2">одна вершина → все</td><td class="py-2 p
 <tr class="border-b border-
font-mono">0((V+E) log V)</td><td class="p
 <tr class="border-b border-
ass="py-2 pr-2 text-rose-300">нет</td><td c
 </tr>
 <tr class="border-b border-
ass="py-2 pr-2 text-rose-300">молча ошибётс
emerald-300">обнаруживает (d[i][i] < 0)<
 <tr class="border-b border-
ass="py-2 pr-2">список смежности</td><td cl
 <tr class="align-top"><td c
φ, веса ≥ 0</td><td class="py-2 pr-2">минус
 </tbody>
 </table>
</div>
</div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-emerald-4

 <div class="bg-slate-900/60 p-4 rounded-lg l
 <p class="text-xs font-bold text-amber-:
 :</p>
 <pre class="bg-slate-950 p-3 rounded text-xs font-mono

```

```

if changed_vertex == -1:
 return dist, prev, None # отрицательны

changed_vertex улучшен на n-й итерации:
поднимаемся по prev на n шагов – гарантированно з
y = changed_vertex
for _ in range(n):
 y = prev[y]

cycle = [y]
v = prev[y]
while v != y:
 cycle.append(v)
 v = prev[v]
cycle.append(y)
cycle.reverse()
return dist, prev, cycle

демо-граф страницы (9 рёбер, режим с отрицательным ци
nodes = ["S", "A", "B", "C", "D", "E", "F"]
idx = {name: i for i, name in enumerate(nodes)}
E = [{"S", "A", 4}, {"S", "B", 5}, {"B", "C", -2}, {"A", "C", 1},
 {"C", "D", 3}, {"C", "E", 4}, {"D", "F", 2}, {"E", "F", 1}]
E = [(idx[u], idx[v], w) for u, v, w in E]

dist, prev, cyc = bellman_ford(len(nodes), E + [(idx["D",
print([dist[i] for i in range(len(nodes))]) # [0, 4,
print(cyc) # None –

dist2, _, cyc2 = bellman_ford(len(nodes), E + [(idx["D",
print([nodes[i] for i in cyc2]) # ['D',
print([dist2[i] for i in range(len(nodes))]) # все
 <p class="text-slate-400 text-xs">П
 имая вершина с INF<
 ля float('inf')</sp
 </div>
</details>

```

```

 vector<int> cycle;
 for (int v = y;; v = p[v]) {
 cycle.push_back(v);
 if (v == y && cycle.size() > 1) break;
 }
 reverse(cycle.begin(), cycle.end());
 return cycle;
 }

```

```

int main() {
 int s = 0;
 int x = bellman_ford(s);
 if (x == -1) cout <<< "отрицательных циклов не";
 else {
 cout <<< "цикл: ";
 for (int v : negative_cycle(x)) cout <<< v << " ";
 cout <<< "\n";
 }
}
</code></pre>

```

</div>

</details>

<details class="bg-slate-900/40 rounded-lg">

<summary class="p-4 font-bold text-slate-300">

-b border-slate-700/50">» Оптимизации: ранний выход

<div class="p-5 text-sm text-slate-300">

```

<pre class="bg-slate-950 p-4 rounded overflow-x-auto text-slate-300">
 rt deque

```

```

def spfa(n, adj, s):
 """adj[u] = [(v, w), ...]. Релаксируем только «живые» вершины.
 INF = float("inf")
 dist = [INF] * n
 in_queue = [False] * n
 counter = [0] * n # сколько раз вершина попала в очередь
 dist[s] = 0
 q = deque([s])
 in_queue[s] = True

```

```

отрицательным циклом
 Переполнение INF.
 INT_MAX превращается в отрицательное
 запасом и long long
 Неориентированный граф с мин
 lass="font-mono" style="font-family: monospace;">-3 – это цикл длины
 рицательном цикле. Минусы в неориентированн
 Цикл есть, но не от старта.<
. Нужна проверка всего графа – добав
а, билет 17) и запустите от неё.
 «Ответ $-\infty$ ». Если отрицат
s="font-mono" style="font-family: monospace;">dist[v] = $-\infty$: печатать
ходом в обратном графе.
 SPFA как «быстрый Форд–Беллм
в условии V, E ≤ 1

</div>

<p class="text-slate-300 text-sm mt-4">С
отенциалы <span class="font-mono text-emerg
я Дейкстрой. Флойд–Уоршелл (16) решает все
орядок для DAG – билет 9.</p>
</div>
</div>
</section>`
},
{
 id: "floyd",
 title: "16. Графы. Поиск кратчайшего пути. Флойд",
 type: "html",
 category: "Графы. Пути",
 content: `
<section id="floyd-content" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-r
 </div>

```

```

<div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
</div>
<div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
</div>
<div class="bg-slate-900 border border-
 <p class="text-[10px] uppercase tra
 <p class="font-mono text-emerald-300
</div>
</div>

<div class="grid grid-cols-1 xl:grid-cols-2
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 Аналогия 1: Таблица пересадок в
 </div>
 <p class="text-slate-300 text-sm mb
це только соседние станции. Затем говорите:
 через 1 там, где
плакат окончательный. Порядок «разрешений»
an>.</p>
 </div>

 <div class="bg-slate-900 p-6 rounded-lg
 <div class="absolute -top-3 left-4 l
border-rose-500 shadow-md">
 Аналогия 2: Почему k обязан быть
 </div>
 <p class="text-slate-300 text-sm mb
s="font-mono">k один раз и бо
садку не соберётся. На демо-графе этой стра
ono text-emerald-300">d[2][1] = ∞, х
пользует «поздние» посредники). Проверка пер
 </div>

```

```

 </tbody>
 </table>
 <p class="text-slate-400 text-xs mt-3">
mono text-emerald-300">0 → {0, 4, 7, 2, 6, 1}
"font-mono">0 → 3 → 4 → 6 (2 + 4 + 1)
всего в финальной матрице 11 таких пар. Диагональ
</div>

<div class="bg-slate-800/70 p-5 rounded-lg border">
 <p class="text-sm font-bold text-white">
 <table class="w-full text-xs text-left">
 <thead>
 <tr class="text-slate-500 border">
 <th class="py-1.5 pr-2 font-mono">
 <th class="py-1.5 pr-2 font-mono">
 <th class="py-1.5 font-bold">
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-500">
 <td class="py-2 pr-2 font-mono">конечное</td><td class="py-2 pr-2 font-mono">
 <tr class="border-b border-slate-500">
 <td class="py-2 pr-2 font-mono">INF</td><td class="py-2 pr-2 font-mono">
 <tr class="border-b border-slate-500">
 <td class="py-2 pr-2 font-mono">0</td><td class="py-2 pr-2 font-mono">
 <tr class="align-top"><td class="text-slate-300">< 0</td><td class="py-2 text-rose-400">
делены</td></tr>
 </tbody>
 </table>
 </div>
</div>

<div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-indigo-400">
 <div class="bg-slate-900/40 rounded-lg border">
 <summary class="p-4 font-bold text-slate-300">

```

```

d = [row[:] for row in M]
for a in range(n):
 for b in range(n):
 for c in range(n):
 k, i, j = {"kij": (a, b, c),
 "ijk": (c, a, b),
 "ikj": (c, a, b)}[order]
 if d[i][k] + d[k][j] < d[i][j]:
 d[i][j] = d[i][k] + d[k][j]
return d

good = floyd(M, "kij")
bad = floyd(M, "ijk")
print(good[2][1], bad[2][1]) # 4 и 999 – путь 2→4→6→1
print(sum(1 for i in range(7) for j in range(7) if good[i][j] != bad[i][j]))
4 клетки отличаются; при порядке (i,k,j) их 6

```

В остальных «почти работают» (большинство клеток с



```

print(neg) # [] — от

</div>
</details>

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">> Код: C++ (матрица
 <div class="p-5 text-sm text-slate-300
<pre class="bg-slate-950 p-4 rounded overflow-x-auto te
 t;
#include <algorithm>;
using namespace std;
const long long INF = 4e18;

int n;
vector<vector<long long>>> d(n, vector<long long>(n, INF));

void floyd() {
 for (int i = 0; i < n; ++i) d[i][i] = 0;
 for (int k = 0; k < n; ++k) // k
 for (int i = 0; i < n; ++i) {
 if (d[i][k] == INF) continue; // защи
 for (int j = 0; j < n; ++j) {
 if (d[k][j] == INF) continue;
 d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
 }
 }
}

bool has_negative_cycle() {
 for (int i = 0; i < n; ++i) if (d[i][i] < 0)
 return true;
 return false;
}

long long diameter() { // сама
 long long res = 0;
 for (int i = 0; i < n; ++i)
 for (int j = 0; j < n; ++j)

```

```

 <tr class="border-b border-
ой сети</td><td class="py-2 pr-2 font-mono">
и отсутствии положительных циклов</td></tr>
 <tr class="align-top"><td c
o">if d[i][k]+d[k][j] == d[i][j]: cnt[i][j]
а переполнением</td></tr>
 </tbody>
 </table>
 <p class="text-slate-400 text-xs mt
> при r[i][k]
V³).</p>
 </div>
</details>
</div>

```

```

<div class="bg-slate-700/50 p-6 rounded-xl bord
<h3 class="text-xl font-bold text-amber-400

```

```

<div class="bg-rose-950/30 p-5 rounded-lg b
 <p class="text-sm font-bold text-rose-3
 <ul class="list-disc pl-5 space-y-2 tex
 Порядок циклов. <span cl
ono">(i, j, k) на демо-графе 4 клетки
] = 999 вместо 4).
 Диагональ. <span class="
«починит» её только при наличии цикла, а п
 Переполнение INF. <span c
уходит в минус и «улучшает» всё подряд. Пр
no">long long.
 Параллельные рёбра. Во в
no">d[u][v] стартует с худшего веса.
 Отрицательные циклы. Фор
ых с циклом, ответ
t-mono">d[i][j] = -∞, если существуе
 d[i][k] < ∞</sp
 Недостижимость ≠ 0. В ор
таких пар из 49). Выводить 999 или 0 как «р
 Память. <span class="fon

```

```
</div>
```

```
<p class="text-slate-300 text-sm mb-4">Дейк
в лоб. На экзамене спрашивают, что сдела
строчки перевзвешивания» до промышленных у
```

```
<div class="grid grid-cols-1 xl:grid-cols-2
 <div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4
 <div class="border border-emerald-500 shadow-md">
```

Аналогия 1: Такси с навигатором

```
</div>
```

```
<p class="text-slate-300 text-sm mb
ы одинаково. А* — тот же таксист, но с нави
/span> прибавляется оценка остатка пути по
т туда, где сумма меньше. Навигатор обязан
/span>): занижать можно, завышать нельзя —
0">h = 0 это снова Дейкстра.</p>
```

```
</div>
```

```
<div class="bg-slate-800 p-6 rounded-lg
 <div class="absolute -top-3 left-4
 <div class="border border-emerald-500 shadow-md">
```

Аналогия 2: Две бригады туннеля

```
</div>
```

```
<p class="text-slate-300 text-sm mb
пускают две: от А и от В. Каждая зона роста
я не в момент встречи, а когда сумма
— иначе можно пропустить более дешёвый обх
ирует.</p>
```

```
</div>
```

```
</div>
```

```
<div class="overflow-x-auto my-6">
```

```
<table class="w-full text-xs text-left">
```

```
<thead>
```

```
<tr class="text-slate-500 border
```

```
<th class="py-2 pr-3 font-b
```

```
<th class="py-2 pr-3 font-b
```

```

 <p class="text-xl font-mono text-white">
</div>
<div class="grid grid-cols-1 gap-6 mb-6">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 b
er border-amber-500 shadow-md">
 ⚡ Суть перевзвешивания
 </div>
 <p class="text-slate-300 text-sm mb-3">
 Дейкстра (билет 14) быстрая, но
ленный. Алгоритм Джонсона делает финт: пере
ь теми же.
 </p>
 <ol class="text-sm text-slate-300 sp">
 Добавляем фиктивную вершн
 Запускаем от s Форда-Бел
ажный факт:</i> Джонсон НЕ «удаляет» отрица
кой и прекращает работу, так как при отрица
 Перевзвешиваем: <span class=
тво следует из того, что $h(v) \leq h(u) + w(u,$
 Магия: для любого пу
) – $h(\text{финиш})$, то есть все пути между фик
еняется.
 Запускаем быструю Дейкстр
lass="font-mono text-emerald-300"> $d(u, v) =$

 <p class="text-slate-400 text-xs mt-3">
 еженном графе ($E \approx V$) это $O(V^2 \log V)$ проти
ойд – для маленьких или очень плотных.</p>
 </div>
</div>

```

```

<h3 class="text-lg font-bold text-white mt-3">
<p class="text-slate-300 text-sm mb-3">Если
азад из t (по перевёрнутым рёбрам). Волны р
: в «широком» графе (дорожная сеть) это эконо
огда волны «встретились» в одной вершине; н
огда

```

```

 <p class="text-slate-400 text-sm mb-1">
именимой к отрицательным весам и даёт все п
 <p class="text-slate-400 text-sm mb-1">
in_f + min_b ≥ μ. Остановка по встрече даёт
 <p class="text-slate-400 text-sm mb-1">
ок сломается. Там нужен Dial.</p>
 <p class="text-slate-400 text-sm">•
ктке меньше бесполезных релаксаций.</p>
</div>

<p class="text-slate-300 text-sm mt-4">с
py-0.5 rounded text-xs font-mono text-emera
class="bg-slate-950 px-2 py-0.5 rounded tex
м числом раскрытий, 0-1 BFS <code class="bg
te-800">0(V + E)</code>, Dial <code class="
late-800">0(E + V·C)</code>, Дейкстра с куч
merald-300 border border-slate-800">0(E + V
o text-rose-300 border border-slate-800">0('

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код: Джонсон целиком (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
heapq
INF = float("inf")

def johnson(n, edges):
 # edges: (u, v, w)
 # 1) потенциалы: Форд–Беллман от фиктивной вершины
 h = [0] * (n + 1)
 for s in range(n):
 # из s все рёбра
 # V итераций: V-
 changed = False
 for u, v, w in edges:
 if h[u] + w < h[v]:
 h[v] = h[u] + w
 changed = True

```

```

def bidirectional(g, rg, s, t): # rg – граф с пер
 INF = float("inf")
 df = {s: 0}; db = {t: 0}
 qf = [(0, s)]; qb = [(0, t)]
 mu = INF
 while qf and qb:
 if qf[0][0] + qb[0][0] >= mu: # вот пра
 return mu
 dist, v = heapq.heappop(qf)
 if dist > df.get(v, INF): continue
 for to, w in g[v]:
 nd = dist + w
 if nd < df.get(to, INF):
 df[to] = nd; heapq.heappush(qf, (nd, to)
 if to in db:
 mu = min(mu, nd + db[to]) # встреча:
 dist, v = heapq.heappop(qb)
 if dist > db.get(v, INF): continue
 for to, w in rg[v]:
 nd = dist + w
 if nd < db.get(to, INF):
 db[to] = nd; heapq.heappush(qb, (nd, to)
 if to in df:
 mu = min(mu, nd + df[to])
 return mu

```

# --- A\*: приоритет f = g + h ---

```

def astar(g, s, t, h): # h(v) – допустим
 INF = float("inf")
 gdist = {s: 0}
 heap = [(h(s), 0, s)] # (f, g, v)
 closed = set()
 while heap:
 f, gv, v = heapq.heappop(heap)
 if v == t:
 return gv # ранний выход:
 if v in closed:
 continue

```

```
 <p>• Зачем нужен ранний выход в Дей
 смысла нет.</p>
 <p>• Почему 0-1 BFS работает без ку
 же отсортирована.</p>
 </div>
</details>
</div>
</div>
</section>
 },
];
```

---

ФАЙЛ 28/111: src/data/tickets/ticket18\_20\_mst.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 231 | РАЗМ

---

```
import { Chapter } from '../../types';

/**
 * Билеты 18 · 19 · 20 – одна страница на три алгоритма
 * минимального остовного дерева (MST). Так же, как в с
 * KruskalSimulator, три алгоритма живут на одном графе
 * переключаются вкладками «Краскал / Прима / Борувка».
 *
 * Синхронизация с компилятором: PAGE_SYNC["mst"] + под
 * "mst#kruskal", "mst#prim", "mst#boruvka" (вкладка со
 * через emitVizDemo, код в панели следует за ней).
 */
export const tickets18to20Mst: Chapter[] = [
 {
 id: "mst",
 title: "18–20. Графы. Остовное дерево: Краскал, При
 type: "html",
 description:
 "Три способа построить минимальное остовное дерев
 .",
 },
];
```

```

 <tr class="text-slate-500 border-b" >
 <th class="py-2 pr-3 font-b" >
 <th class="py-2 pr-3 font-b" >
 <th class="py-2 pr-3 font-b" >
 <th class="py-2 pr-3 font-b" >
 <th class="py-2 font-bold">
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slate-500" >
 <td class="py-2 pr-3 font-b" >
 <td class="py-2 pr-3">Рёбра
 <td class="py-2 pr-3 font-m" >
 <td class="py-2 pr-3 font-m" >
 <td class="py-2">рёбра даны
 </tr>
 <tr class="border-b border-slate-500" >
 <td class="py-2 pr-3 font-b" >
 <td class="py-2 pr-3">Пастрё
 <td class="py-2 pr-3 font-m" >
 <td class="py-2 pr-3 font-m" >
 </td>
 <td class="py-2">плотный гр
 </tr>
 <tr class="align-top">
 <td class="py-2 pr-3 font-b" >
 <td class="py-2 pr-3">Все к
 <td class="py-2 pr-3 font-m" >
 <td class="py-2 pr-3 font-m" >
 <td class="py-2">параллельны
 </tr>
 </tbody>
</table>
</div>

<div class="grid grid-cols-1 xl:grid-cols-3" >
 <div class="bg-slate-800 p-6 rounded-lg" >
 <div class="absolute -top-3 left-4" >

```



```

 дом вместо почти-константного find/union.</p>
 <p class="text-slate-400 text-sm mb-1">
 и Краскал построил минимальный остовный <i>
 <p class="text-slate-400 text-sm">•
ин раз, DSU спасает от дубля (find(u) == find(v))
</div>

<p class="text-slate-300 text-sm mt-4">
py-0.5 rounded text-xs font-mono text-emerald-300
бесплатный); Прим с двоичной кучей <code>
rder-slate-800">O(E log V)</code>, на плотн
text-xs font-mono text-emerald-300 border border-1
d text-xs font-mono text-emerald-300 border border-1
ex <code class="bg-slate-950 px-2 py-0.5 rounded">
/p>

<details class="bg-slate-900/40 rounded-lg border border-1
 <summary class="p-4 font-bold text-slate-300">
 -b border-slate-700/50">
 Код 18: Краскал + DSU (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300">
 <pre class="bg-slate-950 p-4 rounded">
 lt;pair<int, pair<int, int>>>>
vector<int> p, rnk;

int find(int v) { return p[v] == v ? v : p[v] = find(p[v]); }
bool unite(int a, int b) {
 a = find(a); b = find(b);
 if (a == b) return false; // уже в одном компоненте
 if (rnk[a] < rnk[b]) swap(a, b);
 p[b] = a;
 if (rnk[a] == rnk[b]) rnk[a]++;
 return true;
}

sort(edges.begin(), edges.end()); // O(E log E)
long long cost = 0; int taken = 0;

```

а – другая задача.</p>

</div>

</details>

<details class="bg-slate-900/40 rounded-lg l

<summary class="p-4 font-bold text-slate

-b border-slate-700/50">

Код 20: Боровка (Скрыто)

</summary>

<div class="p-5 text-sm text-slate-300

<pre class="bg-slate-950 p-4 rounded

= list(range(n))

def find(v):

while parent[v] != v:

parent[v] = parent[parent[v]]

v = parent[v]

return v

comps = n

total = 0

while comps > 1:

best = {}

# компонента

for w, u, v in edges:

cu, cv = find(u), find(v)

if cu == cv:

continue

# уже внутри

if cu not in best or w < best[cu][0]:

best[cu] = (w, u, v)

if cv not in best or w < best[cv][0]:

best[cv] = (w, u, v)

if not best:

break

# граф несвя.

for w, u, v in best.values():

if find(u) == find(v):

continue

# дубль: два

parent[find(u)] = find(v)

comps -= 1

total += w

```
export const tickets21to24: Chapter[] = [
 {
 id: "string-kmp",
 title: "21. Алгоритм Кнута-Морриса-Пратта (КМП)",
 type: "html",
 category: "Строки",
 content: `
<section id="string-kmp" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 сом, оканчивающимся в позиции i.</p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 l
rder border-emerald-500 shadow-md">
 ♂ Аналогия 1: Поиск без отка
 </div>
 <p class="text-slate-300 text-sm mb">
 Мы идём по строке слева направо.
строки СЕЙЧАС закончился под моим пальцем?»
 Представь, ты ищешь в тексте слова
x</code>. Тупой алгоритм начнет искать заголовок
cab</code> суффикс <code>ab</code> – это ро
буквы шаблона». Это поиск за O(N) без возв
 </p>
 </div>
 </div>
 </div>
 </div>

```

```

</details>
<details class="bg-slate-900/40 rounded-lg border-1
 <summary class="p-4 font-bold text-slate-300
 -b border-slate-700/50">
 Код C++ (Скрыто)
 </summary>
 <div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded
vector<int> pi(s.length());
for (int i = 1; i < s.length(); i++) {
 int j = pi[i-1];
 while (j > 0 && s[i] != s[j]) j = pi[j-1];
 if (s[i] == s[j]) j++;
 pi[i] = j;
}</pre>
 </div>
</details>

```

```

<h3 class="text-lg font-bold text-white mt-4">
<div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-xl font-mono text-white">
 p>
 </div>
 <p class="text-slate-300 text-sm mb-4">Префикс
 для шаблона (pattern) один раз, затем идём
 пало — $j += 1$. Не совпало — откатываемся на
 у заново сравнивать его не нужно. Когда j до
 ждения), делаем $j = \pi[j - 1]$.</p>
 <p class="text-slate-300 text-sm mb-4">П
 ждый откат j уменьшает. Значит суммарно отк
 классическая амортизационная оценка, её люб
 <div class="bg-slate-900/40 rounded-lg border-1
 <p class="text-slate-300 text-sm mb-2">
 <p class="text-slate-400 text-sm mb-1">
 льшего собственного префикса, равного суффи
 <p class="text-slate-400 text-sm mb-1">
 ый алгоритм.</p>
 <p class="text-slate-400 text-sm mb-1">

```

```

тот же результат одной префикс-функцией: π от pattern
def kmp_via_concat(text, pattern):
 s = pattern + "#" + text
 pi = prefix_function(s)
 m = len(pattern)
 return [i - 2 * m for i in range(m + 1, len(s)) if pi[i] == m]

```

през границу и вхождения будут ложными.

```

</div>
</details>
</div>
</div>
</section>`
},
{
 id: "string-z-func",
 title: "22. Z-функция — Сдвиг и наложение",
 type: "html",
 category: "Строки",
 content: `
<section id="string-z-func" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-emerald-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">
 <p class="text-xl font-mono text-white">
 ca 0) и её суффиксом, начинающимся с i.</p>
 </div>

 <div class="grid grid-cols-1 xl:grid-cols-2">
 <!-- Аналогия 1 -->
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4">

```

```

 <code>Z[3]</code> (abcd
а дальше s[6]='d' против s[3]='a' — мимо.
 <code>Z[4], Z[5], Z[6]</code>

</div>
</details>
<details class="bg-slate-900/40 rounded-lg
 <summary class="p-4 font-bold text-slate
-b border-slate-700/50">
 Код C++ (Скрыто)
</summary>
<div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded

int l = 0, r = 0;
for (int i = 1; i < n; i++) {
 if (i <= r) z[i] = min(r - i + 1, z[i - 1]);
 while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z
 if (i + z[i] - 1 > r) { l = i; r = i + z[i] - 1; }
}</pre>
 </div>
</details>
</div>
</div>
</section>`
 },
 {
 id: "aho-corasick",
 title: "23. Алгоритм Ахо-Корасик",
 type: "html",
 category: "Строки",
 content: `
<section id="aho-corasick" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 <span class="bg-indigo-600 text-white px-4 py-1 roun
 <h2 class="text-2xl sm:text-3xl font-bold text-white
 </div>

 <div class="space-y-8">

```

```

<pre class="bg-slate-950 p-4 rounded overfl
int get_link(int v) {
 if (t[v].link == -1) {
 if (v == 0 || t[v].p == 0) t[v].link = 0;
 else t[v].link = go(get_link(t[v].p), t[v].pch)
 }
 return t[v].link;
}

```

```

int go(int v, char c) {
 if (t[v].go[c] == -1) {
 if (t[v].next[c] != -1) t[v].go[c] = t[v].next[c];
 else t[v].go[c] = v == 0 ? 0 : go(get_link(v), c);
 }
 return t[v].go[c];
}
</pre>

```

```

 </div>
</details>
</div>
</section>`
 },
 {
 id: "complexity-classes",
 title: "24. Классы сложности, сведение задач",
 type: "html",
 description:
 "P, NP, co-NP, NP-трудные и NP-полные, PSPACE и E.
 ли задача NP-полна.",
 category: "Теория сложности",
 content: `
<section id="complexity-classes" class="mb-12 scroll-mt
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-v
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-purple-400

```

```

 <p class="font-mono text-sm sm:text-bas
 $P \subseteq NP \subseteq PSPACE \subseteq EXP \subseteq NEXP$

 NP-п
доказано)
 </p>
</div>
<div class="overflow-x-auto">
 <table class="w-full text-xs text-left l
 <thead>
 <tr class="text-slate-500 borde
 <th class="py-2 pr-3 font-b
 <th class="py-2 pr-3 font-b
 <th class="py-2 font-bold">
 </tr>
 </thead>
 <tbody class="text-slate-300">
 <tr class="border-b border-slat
py-2 pr-3">детерминированно за $O(n^k)$ </td><td
удольности, максимальный поток</td></tr>
 <tr class="border-b border-slat
py-2 pr-3">сертификат проверяется за полином
ешающая версия), коммивояжёр «есть ли тип $\leq$
 <tr class="border-b border-slat
s="py-2 pr-3">сертификат есть у ответа «НЕТ:
ель)</td></tr>
 <tr class="border-b border-slat
lass="py-2 pr-3">не проще любой задачи из NP
фа, TSP-оптимизация (не решающая версия)</td>
 <tr class="border-b border-slat
ass="py-2 pr-3">NP-трудные И из NP</td><td
сумм</td></tr>
 <tr class="border-b border-slat
"py-2 pr-3">полиномиальная память, время не
й на полиномиальном поле</td></tr>
 <tr class="align-top"><td class
за $2^{(n^k)}$ </td><td class="py-2">обобщённые
 </tbody>
 </table>

```



```

 стной трудной A к B ($A \leq_p B$), а не н
 <p class="text-slate-400 text-sm mb-1">
 <p class="text-slate-400 text-sm mb-1">
 дит по времени на $n = 40$. NP-полнота запрещ
 <p class="text-slate-400 text-sm">•
 ит RSA. «Раз она в NP, значит NP-полная» —
 </div>

```

```

<p class="text-slate-300 text-sm mt-4"><b c
ass="bg-slate-950 px-2 py-0.5 rounded text-
еребор сертификатов <code class="bg-slate-9
^poly(n))</code>; сведение обязано работать
0 border border-slate-800">0(poly(n))</code>

```

```

<details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">

```

Код: верификатор сертификата и пр

```

</summary>
<div class="p-5 text-sm text-slate-300
 <pre class="bg-slate-950 p-4 rounded

```

ьтонов цикл NP-полон: сертификат = порядок

```

def verify_hamilton(n, edges, cert):
 if sorted(cert) != list(range(n)): # каждая
 return False
 allowed = {(u, v) for u, v in edges} | {(v, u) for
 for i in range(n):
 a, b = cert[i], cert[(i + 1) % n] # замык
 if (a, b) not in allowed:
 return False
 return True

```

# 3-SAT  $\leq_p$  КЛИКА: формула с k дизъюнктов  $\rightarrow$  граф из  $3k$  в

```

def reduce_3sat_to_clique(clauses):
 # вершина = (номер дизъюнкта, литерал); ребро, если
 verts = [(ci, lit) for ci, cl in enumerate(clauses)
 edges = []
 for i in range(len(verts)):

```

```
 <p>• Дайте определение NP через верности. </p>
 <p>• Что такое NP-полная задача и к</p>
 <p>• Почему первая NP-полная задача
 <p>• Если $P = NP$, что станет с криптографическими задачами (исчезнет граница approximability) </p>
 <p>• Пример задачи в NP и co-NP, проблема остановки </p>
 <p>• Чем NP-трудная задача отличается от NP-полной </p>
 <p>• Проблема останова </p>
 </div>
</details>
</div>
</div>
</section>`
 }
};
```

---

ФАЙЛ 30/111: src/data/tickets/ticket6\_13.ts

КАТЕГОРИЯ: Билеты (учебный контент) | СТРОК: 576 | РАЗМЕР: 1.5 КБ

---

```
import { Chapter } from "../../types";

export const tickets6to13: Chapter[] = [
 {
 id: "graph-dfs-bfs",
 title: "6. Графы. Представление графов, DFS, BFS",
 type: "html",
 description: "Представление графов и базовые обходы",
 category: "Графы. База",
 content: `
<section id="graph-dfs-bfs" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 </div>
 <div class="flex flex-wrap">
 <div class="flex flex-col flex-grow-1">
 <h3>6. Графы. Представление графов, DFS, BFS</h3>
 <p>Представление графов и базовые обходы</p>
 </div>
 <div class="flex flex-col flex-grow-1">
 <p>Графы — это абстрактная модель объектов и связей между ними. В информатике графы используются для представления структур данных, маршрутов, сетей и т.д. В этом разделе мы рассмотрим различные способы представления графов и алгоритмы для их обхода (DFS, BFS).</p>
 </div>
 </div>
</section>
`
 }
];
```

```
<div class="absolute -top-3 left-4 p-6 rounded-lg border border-indigo-500 shadow-md">
 ♂ DFS (Поиск в глубину) = 0
</div>
<p class="text-slate-300 text-sm mb-4">
 Что делает: Поиск в глубину (DFS) — это алгоритм, который
 да попадает в тупик — делает шаг назад по следу.
</p>
<ul class="text-xs text-indigo-300 list-disc">
 Где используется:
 Топологическая сортировка (линейное время)
 Поиск мостов и точек сочленения
 Компоненты сильной связности
 Детектирование циклов (через visited и parent)

</div>

<!-- Аналогия BFS -->
<div class="bg-slate-900 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 p-6 rounded-lg border border-emerald-500 shadow-md">
 ♂ BFS (Поиск в ширину) = Волна
 </div>
 <p class="text-slate-300 text-sm mb-4">
 Что делает: Распространение волны по соседям. Работает через Очередь (FIFO). Гарантирует минимальное расстояние от источника.
 </p>
 <ul class="text-xs text-emerald-300 list-disc">
 Где используется:
 Кратчайший путь в Невзвешенном графе
 Проверка двудольности графа
 Алгоритм Ахо-Корасик (построение автомата)
 Двунаправленный BFS для поиска кратчайшего пути (экстра).

</div>
</div>
```

```

{
 id: "graph-planar-colors",
 title: "7. Графы. Планарные. Покраска",
 type: "html",
 description: "Планарные графы: формула Эйлера, K5 и K4",
 category: "Графы. Теория",
 content: `
<section id="graph-planar-colors" class="mb-12 scroll-m-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 7
 <h2 class="text-2xl sm:text-3xl font-bold text-slate-900">Графы. Планарные. Покраска
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-700">
 <h3 class="text-xl font-bold text-blue-400">Планарные графы
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">Планарный граф — граф, который можно нарисовать в плоскости так, чтобы никакие две его
 <p class="text-2xl font-mono text-white">
 <p class="text-xs text-slate-400 mt-1">
 </div>

 <div class="space-y-4 text-slate-300 text-s">
 <p>Главные неравенства на число рёбер</p>
 <ul class="list-disc list-inside space-y-2">
 Для простого планарного графа
 <i>Доказательство:</i> Каждая грань
 . Подставляя в формулу Эйлера:

 Для простого двудольного планарного графа
 . Доказательство: В двудольном графе
 2E ≥ 4F ⇒ F ≤ E/2

 <div class="bg-slate-950 p-4 rounded-lg">
 <p class="text-amber-300 font-bold">
 <p class="text-slate-300 text-xs le

```

```


 </div>
 </div>
 </div>
</section>`,
 },
 {
 id: "graph-components",
 title: "8. Графы. Компоненты связности",
 type: "html",
 description:
 "Компоненты связности: обходами DFS/BFS за $O(V + E)$ ",
 category: "Графы. База",
 content: `
<section id="graph-components" class="mb-12 scroll-mt-12">
 <div class="flex items-center mb-6 flex-wrap gap-3">
 Графы
 <h2 class="text-2xl sm:text-3xl font-bold text-emerald-400">8. Графы. Компоненты связности
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-emerald-400">Компоненты связности
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">В графе компоненты связности — это максимальные по включению подграфы, в которых любые две вершины соединены путём.
 <p class="text-xl font-mono text-white">Алгоритмы DFS/BFS позволяют за $O(V + E)$ найти все компоненты связности. Для каждой вершины v можно определить её компоненту связности, присвоив ей метку $comp[v]$.
 </div>
 <p class="text-slate-300 text-sm mb-4">Два компонента связности не пересекаются. Если в графе есть цикл, то он принадлежит ровно одной компоненте связности. Если в графе нет циклов, то он является лесом (объединением деревьев). В этом случае для каждой компоненты связности можно выбрать старшую вершину (корень) и построить дерево.
 <p class="text-slate-300 text-sm mb-4">Алгоритм DFS/BFS для нахождения компонент связности. Для каждой вершины v в графе G выполняем DFS/BFS, начиная с v . При этом мы будем присваивать метку $comp[v]$ каждой вершине v , которую посетим. Счётчик c увеличивается на 1 для каждой новой компоненты связности.
 <p class="text-slate-300 text-sm mb-4">Время $O(V + E)$, память $O(V)$. Через DFS/BFS можно также найти старшую вершину в каждой компоненте связности. Для этого достаточно добавить в DFS/BFS дополнительную метку $parent$ и после каждого добавления спрашивать, является ли текущая вершина v старшей в своей компоненте.
 </div>
 <div class="bg-slate-700/50 p-6 rounded-xl border border-slate-600">
 <h3 class="text-xl font-bold text-emerald-400">Компоненты связности в неориентированном графе
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-emerald-300 italic">В неориентированном графе компоненты связности — это максимальные по включению подграфы, в которых любые две вершины соединены путём.
 <p class="text-xl font-mono text-white">Алгоритмы DFS/BFS позволяют за $O(V + E)$ найти все компоненты связности. Для каждой вершины v можно определить её компоненту связности, присвоив ей метку $comp[v]$.
 </div>
 <p class="text-slate-300 text-sm mb-4">Два компонента связности не пересекаются. Если в графе есть цикл, то он принадлежит ровно одной компоненте связности. Если в графе нет циклов, то он является лесом (объединением деревьев). В этом случае для каждой компоненты связности можно выбрать старшую вершину (корень) и построить дерево.
 <p class="text-slate-300 text-sm mb-4">Алгоритм DFS/BFS для нахождения компонент связности. Для каждой вершины v в графе G выполняем DFS/BFS, начиная с v . При этом мы будем присваивать метку $comp[v]$ каждой вершине v , которую посетим. Счётчик c увеличивается на 1 для каждой новой компоненты связности.
 <p class="text-slate-300 text-sm mb-4">Время $O(V + E)$, память $O(V)$. Через DFS/BFS можно также найти старшую вершину в каждой компоненте связности. Для этого достаточно добавить в DFS/BFS дополнительную метку $parent$ и после каждого добавления спрашивать, является ли текущая вершина v старшей в своей компоненте.
 </div>
 </div>
</section>

```

```

 <tr class="align-top"><td class="text-slate-400 text-xl font-mono text-white py-4 px-3">до кого дойдёт старт</td><td class="py-4 px-3">
 V + E)</td></tr>
 </tbody>
 </table>
 </div>
 <p class="text-slate-400 text-xs mt-3">Классический алгоритм SCC. Обход из s даёт только вершины,
</div>
</div>
</section>`,
 },
 {
 id: "top-sort",
 title: "9. Графы. Топологическая сортировка",
 type: "html",
 description: "Топологическая сортировка ДАГ и детект",
 category: "Графы",
 content: `
<section id="top-sort" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-blue-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-blue-300 italic">
 <p class="text-xl font-mono text-white">
 перед.</p>
 </div>
 <div class="grid grid-cols-1 xl:grid-cols-2">
 <div class="bg-slate-800 p-6 rounded-lg">
 <div class="absolute -top-3 left-4 b
 rder border-emerald-500 shadow-md">
 Аналогия 1: Учеба в вузе
 </div>
 </div>
 </div>
 </div>
 </div>
</section>

```



```

 AG конденсации.
 Переворачиваем все ребра в
 Идем по <code>order</code>:
 тот запуск – образуют очередную SCC!

</div>
</details>
</div>
</div>
</section>`,
 },
 {
 id: "graph-bridges",
 title: "11. Графы. Мосты",
 type: "html",
 description: "Рёбра, удаление которых увеличивает ч",
 category: "Графы. Продвинутые",
 content: `
<section id="graph-bridges" class="mb-12 scroll-mt-10">
 <div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-v">
 </div>
 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord">
 <h3 class="text-xl font-bold text-rose-400 m">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-rose-300 italic m">
 <p class="text-xl font-mono text-white">
 <p class="text-xs text-slate-400 mt-1">
 </div>

 <div class="bg-slate-950 p-4 rounded-lg bor">
 <p class="text-amber-300 font-bold text">
 <p class="text-slate-300 text-xs leadin">
 Проверка <span class="font-mono text-
 тиграфе между u и v может быть 2 параллельны
 азуют цикл длиной 2.


```



```

 id: "graph-articulation",
 title: "12. Графы. Точки сочленения",
 type: "html",
 description: "Вершины, удаление которых увеличивает",
 category: "Графы. Продвинутые",
 content: `
<section id="graph-articulation" class="mb-12 scroll-mt
 <div class="flex items-center mb-6 flex-wrap gap-3"
 <span class="bg-blue-600 text-white px-4 py-1 r
 <h2 class="text-2xl sm:text-3xl font-bold text-
 </div>

 <div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl bord
 <h3 class="text-xl font-bold text-rose-400 m

 <div class="bg-slate-900 p-4 rounded-lg mb-
 <p class="text-lg text-rose-300 italic m
 <div class="text-left font-mono text-sm
 <p><strong class="text-white">Точка
рой увеличивает число компонент связности.<
 <div class="p-3 bg-slate-950 rounde
 <span class="text-rose-400 font
 <p class="text-xl font-bold tex
 <p class="text-xs text-slate-40
 </div>
 </div>
 </div>
 </div>

 <p class="text-slate-300 text-sm">
 Особый случай: Корень дерева DFS
 моего ребёнка в дереве DFS.
 </p>

 <details class="bg-slate-900/40 rounded-lg l
 <summary class="p-4 font-bold text-slate
 -b border-slate-700/50">
 Код поиска точек сочленения (Скры

```

```


<h2 class="text-2xl sm:text-3xl font-bold text-white">
</div>
<div class="space-y-8">
 <div class="bg-slate-700/50 p-6 rounded-xl border">
 <h3 class="text-xl font-bold text-indigo-400">
 <div class="bg-slate-900 p-4 rounded-lg mb-4">
 <p class="text-lg text-indigo-300 italic">
 <p class="text-base sm:text-lg font-monospace">
ности!</p>
 </div>

 <div class="grid grid-cols-1 md:grid-cols-2">
 <!-- Неориентированные графы -->
 <div class="bg-slate-800 p-5 rounded-lg">
 <p class="text-indigo-300 font-bold">
 <ul class="list-disc list-inside space-y-2">
 Эйлеров цикл: связан
-emerald-300">deg[v] % 2 == 0.
 Эйлеров путь: связан
о 2 – путь начинается в одной нечётной и за

 </div>

 <!-- Ориентированные графы -->
 <div class="bg-slate-800 p-5 rounded-lg">
 <p class="text-indigo-300 font-bold">
 <ul class="list-disc list-inside space-y-2">
 Эйлеров цикл: слабос
ald-300">in[v] == out[v].
 Эйлеров путь: слабос
s] = 1 (старт), у одной <span class="
an class="font-mono text-emerald-300">in[v]

 </div>
 </div>

 <h3 class="text-lg font-bold text-white mt-4">

```

```
import React, { useState, useEffect } from "react";
import { useVizStepSync, vizNumber } from '../data/vizS
import { Play, Pause, RotateCcw, Info } from "lucide-re
```

```
interface Node {
 id: number;
 x: number;
 y: number;
 label: string;
}
```

```
interface Edge {
 u: number;
 v: number;
}
```

```
const nodes: Node[] = [
 { id: 0, x: 250, y: 50, label: "0" },
 { id: 1, x: 100, y: 150, label: "1" },
 { id: 2, x: 250, y: 250, label: "2" },
 { id: 3, x: 400, y: 150, label: "3" },
 { id: 4, x: 550, y: 150, label: "4" },
 { id: 5, x: 700, y: 50, label: "5" },
 { id: 6, x: 700, y: 250, label: "6" },
];
```

```
const edges: Edge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 },
 { u: 3, v: 4 }, // 3 and 4 are articulation points {3
 { u: 4, v: 5 },
 { u: 5, v: 6 },
 { u: 6, v: 4 },
```

```
const recordStep = (desc: string, u: number | null) => {
 newSteps.push({
 desc,
 visited: new Set(visited),
 ap: new Set(ap),
 currentNode: u,
 tin: [...currentTin],
 low: [...currentLow],
 });
};
```

recordStep("Начало DFS (Тарьян) для поиска точек со

```
const dfs = (v: number, p: number = -1) => {
 visited.add(v);
 currentTin[v] = currentLow[v] = timer++;
 let children = 0;

 recordStep(`Заходим в вершину ${v}. tin[${v}]=${currentTin[v]}`);

 for (const to of adj[v]) {
 if (to === p) continue;

 if (visited.has(to)) {
 currentLow[v] = Math.min(currentLow[v], currentLow[to]);
 recordStep(
 `Обратное ребро ${v}-${to}. Обновляем low[${v}]`,
 v,
);
 } else {
 children++;
 dfs(to, v);
 currentLow[v] = Math.min(currentLow[v], currentLow[to]);

 recordStep(
 `Возврат в ${v} из ${to}. Обновляем low[${v}]`,
 v,
);
 }
 }
};
```

```

 visited: new Set(),
 ap: new Set(),
 currentNode: null,
 tin: [],
 low: [],
 };

 useEffect(() => {
 let interval: NodeJS.Timeout;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 interval = setInterval(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearInterval(interval);
 }, [isPlaying, currentStepIndex, steps.length]);

 const togglePlay = () => setIsPlaying(!isPlaying);
 const reset = () => {
 setIsPlaying(false);
 setCurrentStepIndex(0);
 };

 return (
 <div className="bg-slate-900 rounded-xl border border-
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Info className="w-5 h-5 text-rose-400" />
 Моделирование поиска точек сочленения
 </h3>

 <div className="flex items-center gap-2 bg-slate-
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 text-white"

```

```

 x2={v.x}
 y2={v.y}
 stroke={isBridge ? "#f43f5e" : "#475568"}
 strokeWidth={isBridge ? "4" : "2"}
 strokeDasharray={isBridge ? "8,4" : "0"}
 />
);
 }}}

```

```

{ /* Nodes */
nodes.map((node) => {
 const isCurrent = currentStep.currentNode === node.id;
 const isVisited = currentStep.visited.has(node.id);
 const isAp = currentStep.ap.has(node.id);

```

```

 const fill = isCurrent
 ? "#3b82f6"
 : isAp
 ? "#e11d48"
 : isVisited
 ? "#10b981"
 : "#1e293b";

```

```

 const stroke = isCurrent
 ? "#60a5fa"
 : isAp
 ? "#f43f5e"
 : isVisited
 ? "#34d399"
 : "#334155";

```

```

 const r = isAp ? 24 : 20;

```

```

 return (
 <g key={node.id}>
 <circle
 cx={node.x}
 cy={node.y}
 r={r}
 fill={fill}

```

```

 low:{" "}
 {currentStep.low[node.id] >= 0
 ? currentStep.low[node.id]
 : "-"}
 </text>
 </>
 })
</g>
);
}}
</svg>
</div>

<div className="bg-slate-950 p-6 border-t lg:bo
 <h4 className="text-sm font-bold text-slate-4
 Статус алгоритма
 </h4>

 <div className="bg-slate-900 border border-sl
 <p className="text-white font-medium min-h-
 {currentStep.desc}
 </p>
 </div>

 <div className="space-y-4 flex-1 overflow-y-a
 <div className="mb-4">
 <h5 className="text-xs text-slate-500 fon
 <div className="flex items-center gap-2 t
 <div className="w-3 h-3 rounded-full bg
 Точка сочленения
 </div>
 <div className="flex items-center gap-2 t
 <div className="w-3 h-3 rounded-full bg
 Посещена
 </div>
 <div className="flex items-center gap-2 t
 <div className="w-3 h-3 rounded-full bg
 Текущая

```

```
 </div>
 </div>
);
 };
};
```

---

ФАЙЛ 32/111: src/components/BellmanFordViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect } from 'react';
import { useVizRuntime, vizNumber, vizRecord, vizString } from './viz';
import { LiveDistTable } from './LiveDistTable';
```

```
interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
 iteration: number;
 checkingEdge: { u: string; v: string; w: number } | null;
 distances: { [key: string]: number };
 previous: { [key: string]: string | null };
 log: string;
 hasNegativeCycle?: boolean;
 activeCodeLine: number;
}
```

```
/** Подсказка для кнопок, когда шаги ведёт компилятор.
const HINT_TEXT = "Шаги сейчас ведёт панель Python: ← →
 и листать вручную.";
```

```
export default function BellmanFordViz() {
 const [hasCycle, setHasCycle] = useState(false);
 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useState(0);
 const runtime = useVizRuntime();
 const [isPlaying, setIsPlaying] = useState(false);
```



```
const codeLines = [
 { line: 1, text: 'function bellman_ford(graph, start'
 { line: 2, text: ' dist = {v: ∞}; dist[start] = 0'
 { line: 3, text: ' for i from 1 to V - 1: // |V| - 1 iterations'
 { line: 4, text: ' for (u, v, weight) in E: // E is an array of edges'
 { line: 5, text: ' if dist[u] + weight < dist[v]:'
 { line: 6, text: ' dist[v] = dist[u] + weight'
 { line: 7, text: ' for (u, v, weight) in E: // Проверка на отрицательный цикл'
 { line: 8, text: ' if dist[u] + weight < dist[v]:'
 { line: 9, text: ' return "ОБНАРУЖЕН ОТРИЦАТЕЛЬНЫЙ ЦИКЛ"'
 { line: 10, text: ' return dist' },
];
```

```
useEffect(() => {
 const simulationSteps: Step[] = [];
 let dist: { [key: string]: number } = { S: 0, A: 999 };
 let prev: { [key: string]: string | null } = { S: null };

 simulationSteps.push({
 iteration: 0, checkingEdge: null,
 distances: { ...dist }, previous: { ...prev },
 log: ' Фура заведена. Начинаем с Базы (S). Расстояние до А: 999'
 activeCodeLine: 2,
 });
```

```
const vCount = nodes.length;
for (let iter = 1; iter < vCount; iter++) {
 let anyUpdate = false;
 for (const edge of edges) {
 const uDist = dist[edge.u];
 const vDist = dist[edge.v];

 let updated = false;
 if (uDist !== 999 && uDist + edge.w < vDist) {
 dist = { ...dist, [edge.v]: uDist + edge.w };
 prev = { ...prev, [edge.v]: edge.u };
 updated = true;
 anyUpdate = true;
 }
 }
}
```

```

 simulationSteps.push({
 iteration: vCount, checkingEdge: null,
 distances: { ...dist }, previous: { ...prev }
 log: ' Проверка завершена. Ни одно ребро не
 activeCodeLine: 10,
 });
 }
}

setSteps(simulationSteps);
setCurrentStepIndex(0);
setIsPlaying(false);
}, [hasCycle]);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length -
 timer = setTimeout(() => setCurrentStepIndex((p)
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
}, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;
if (!baseStep) return <div>Загрузка...</div>;
const vars = runtime?.variables;
const liveI = vizNumber(vars?.i);
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);
const liveW = vizNumber(vars?.w);
const liveDist = vizRecord(vars?.dist);
const compilerLinked = liveI !== null || liveU !== nu
const currentStep: Step = compilerLinked
 ? {
 ...baseStep,
 iteration: liveI ?? 0,
 checkingEdge: liveU && liveV ? { u: liveU, v: l

```



```

 }}}}
 </svg>

 <div className="w-full bg-slate-950/80 p-4 rounded-2xl">
 <p className="font-semibold text-amber-400">
 <p className="min-h-[40px] flex items-center">
 </div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/80 p-4 rounded-2xl">
 <h4 className="text-lg font-bold text-emerald-400">
 <div className="space-y-2 text-sm font-mono text-white">
 {Object.keys(adjList).map(u => {
 <div key={u} className="flex flex-wrap gap-2">

 {adjList[u].map((edge, idx) => {
 const isChk = currentStep.checkingEdge === edge.v
 return (
 <span key={idx} className={`px-2 py-1 rounded-sm ${
 edge.w < 0 ? 'bg-rose-900/80 text-rose-300' : 'bg-emerald-900/80 text-emerald-300'
 }`}>
 {edge.v} ({edge.w})

)
 })}
 </div>
 })}
 </div>
</div>
</div>
</div>

<div className="flex flex-col lg:flex-row gap-6">
 {/* Таблица расстояний */}
 <div className="w-full lg:w-1/2 bg-rose-950/10 p-4 rounded-2xl">
 <div>
 <h4 className="text-lg font-bold text-rose-400">
 <div className="grid grid-cols-4 gap-2 text-sm font-mono text-white">

```

-----  
ФАЙЛ 33/111: src/components/BfsViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

-----  
import { useState, useEffect, useCallback } from 'react';  
import { Play, Pause, RotateCcw, StepForward } from 'lucide-react';

```
const NODES = [
 { id: 0, label: '0', x: 300, y: 50 },
 { id: 1, label: '1', x: 150, y: 150 },
 { id: 2, label: '2', x: 450, y: 150 },
 { id: 3, label: '3', x: 75, y: 250 },
 { id: 4, label: '4', x: 225, y: 250 },
 { id: 5, label: '5', x: 375, y: 250 },
 { id: 6, label: '6', x: 525, y: 250 },
];
```

```
const EDGES = [
 { from: 0, to: 1 }, { from: 0, to: 2 },
 { from: 1, to: 3 }, { from: 1, to: 4 },
 { from: 2, to: 5 }, { from: 2, to: 6 }
];
```

```
const ADJ: { [key: number]: number[] } = {
 0: [1, 2], 1: [3, 4], 2: [5, 6], 3: [], 4: [], 5: [], 6: [],
};
```

```
interface Step {
 activeLine: number;
 queue: number[];
 visited: number[];
 currentNode: number | null;
 logs: string;
}
```

```
const generateBfsSteps = (): Step[] => {
 const steps: Step[] = [];
 const queue: number[] = [];
```

```

const neighbors = ADJ[curr];
if (neighbors.length > 0) {
 for (const n of neighbors) {
 if (!visited.has(n)) {
 steps.push({
 activeLine: 7,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Рассматриваем соседа: ${n}. Он еще не
 });

 visited.add(n);
 queue.push(n);

 steps.push({
 activeLine: 9,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Помечаем ${n} как посещенную и добав.
 });
 }
 }
} else {
 steps.push({
 activeLine: 6,
 queue: [...queue],
 visited: Array.from(visited),
 currentNode: curr,
 logs: `Соседей нет или все уже посещены.`
 });
}
}

steps.push({
 activeLine: 12,

```

```

<div className="flex flex-col md:flex-row items-c
 <div>
 <h3 className="text-2xl font-bold text-white
 Ал
 </h3>
 <p className="text-slate-400 text-sm mt-1">Ви
 </div>
 <div className="flex bg-slate-800 p-1.5 rounded
 <button
 onClick={() => { setIsPlaying(false); setSt
 className="p-2 text-slate-400 hover:text-wh
 title="Сброс"
 >
 <RotateCcw className="w-5 h-5" />
 </button>
 <button
 onClick={() => setIsPlaying(p => !p)}
 className="p-2 text-blue-400 hover:text-bl
 title={isPlaying ? 'Пауза' : 'Воспроизведе
 >
 {isPlaying ? <Pause className="w-5 h-5" />
 </button>
 <button
 onClick={handleNext}
 disabled={stepIdx >= STEPS.length - 1}
 className="p-2 text-emerald-400 hover:text-
 ed:hover:bg-transparent"
 title="Шаг вперед"
 >
 <StepForward className="w-5 h-5" />
 </button>
 </div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-2 g
 {/* Визуал графа */}
 <div className="bg-slate-800 border border-slat
 50px]">

```

```

 }

 return (
 <g key={n.id} className="transition-tran
 {isCurrent && {
 <circle cx={n.x} cy={n.y} r="30" fi
 }}
 <circle
 cx={n.x} cy={n.y}
 r="20"
 fill={fill}
 stroke={stroke}
 strokeWidth="3"
 className="transition-colors durati
 />
 <text
 x={n.x} y={n.y}
 textAnchor="middle"
 dy=".3em"
 className="fill-white font-bold tex
 >
 {n.label}
 </text>
 </g>
);
 }}}}
</svg>
</div>

```

```

{/* Код и Очередь */}
<div className="flex flex-col gap-4">
 <div className="bg-slate-900 border border-sl
 er">
 <div className="bg-slate-800 text-slate-400
 se tracking-wider">
 bfs(graph, start)
 Mar {step
 </div>
 }

```



```
))
 })
 </div>
 <div className="mt-2 text-sm text-amber-300 text-center justify-center">
 "{step.logs}"
 </div>
 </div>
</div>
</div>
</div>
);
}
```

---

ФАЙЛ 34/111: src/components/ChapterImage.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

---

```
import dijkstraImg from '../assets/dijkstra-freakazoid.png';
import bellmanImg from '../assets/bellman-ford-batman.jpg';
import floydImg from '../assets/floyd-network.jpg';
```

```
const imageMap: Record<string, { src: string; alt: string; caption: string; borderColor: string; captionColor: string }> = {
 dijkstra: {
 src: dijkstraImg,
 alt: 'Декстер Дуглас и Фриказоид',
 caption: 'Декстер Дуглас и Фриказоид – волна вирусов',
 borderColor: 'border-blue-500/30',
 captionColor: 'text-blue-400',
 },
 'bellman-ford': {
 src: bellmanImg,
 alt: 'Бэтмен на машине Генри Форда',
 caption: 'Бэтмен на машине Форда – n-1 ночей патрулирует Gotham City',
 borderColor: 'border-rose-500/30',
 captionColor: 'text-rose-400',
 },
 floyd: {
 src: floydImg,
 alt: 'Сеть Флойда',
 caption: 'Сеть Флойда – алгоритм поиска кратчайшего пути',
 borderColor: 'border-teal-500/30',
 captionColor: 'text-teal-400',
 },
};
```

```

total: number;
onGo: (id: string) => void;
compact?: boolean;
}

/**
 * Переходы «предыдущая / следующая тема» — как на Code
 * Компактный вариант живёт в шапке главы, крупный — по
 */
export const ChapterNav: React.FC<Props> = ({ prev, next,
 if (compact) {
 return (
 <div className="flex items-center gap-2 text-xs">
 <button
 disabled={!prev}
 onClick={() => prev && onGo(prev.id)}
 title={prev ? prev.title : "Это первая тема"}
 className="flex items-center gap-1 px-2.5 py-1
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
 >
 <ChevronLeft className="w-4 h-4" />
 Назад
 </button>

 {index + 1} / {total}

 <button
 disabled={!next}
 onClick={() => next && onGo(next.id)}
 title={next ? next.title : "Это последняя тема"}
 className="flex items-center gap-1 px-2.5 py-1
 :bg-slate-700 enabled:hover:text-white disabled:opacity-50"
 >
 Вперёд
 <ChevronRight className="w-4 h-4" />
 </button>
 </div>
);
 }
};

```

ФАЙЛ 36/111: src/components/ChapterQuiz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

-----

```
import React, { useEffect, useState } from "react";
import { ArrowRight, CheckCircle2, RotateCcw, XCircle, } from "lucide-react";
import { quizzes } from "../data/quizzes";
```

```
interface Props {
 chapterId: string;
}
```

```
/**
```

```
 * Блиц по прочитанной теме: 3–5 коротких вопросов с несколькими вариантами ответа.
 *
```

```
 * Вопросы живут в `src/data/quizzes.ts` (по id главы).
 * компонент-викторина существовали, но не были подключены.
 * контент был, а добраться до него было нельзя. Теперь
 * демонстрации: прочитал → посмотрел, как работает → протестировал.
 *
```

```
 * В текстовой версии (`?lite=1`) и в PDF те же вопросы
 * с ответами и разбором – там интерактив не нужен.
 */
```

```
export const ChapterQuiz: React.FC<Props> = ({ chapterId }) => {
 const list = quizzes[chapterId];
 const [idx, setIdx] = useState(0);
 const [picked, setPicked] = useState<number | null>(null);
 const [score, setScore] = useState(0);
 const [finished, setFinished] = useState(false);
```

```
 // другая тема – другой блиц с нуля
```

```
 useEffect(() => {
 setIdx(0);
 setPicked(null);
 setScore(0);
 setFinished(false);
 }, [chapterId]);
```

```

 {finished ? `итог ${score}/${list.length}`
 }
 </div>

 {finished ? (
 <div className="text-center py-3">
 <p className="text-sm text-slate-300 mb-1">
 {score === list.length
 ? "Все верно: тему можно закрывать."
 : score * 2 >= list.length
 ? "Половина и больше верно – стоит пересмотреть."
 : "Стоит вернуться к тексту страницы:"
 }
 </p>
 <p className="text-2xl font-bold text-white">
 {score} / {list.length}
 </p>
 <button
 type="button"
 onClick={restart}
 className="inline-flex items-center gap-1
 font-bold text-indigo-200 hover:bg-indigo-500/20"
 >
 <RotateCcw className="w-3.5 h-3.5" /> Прогресс
 </button>
 </div>
) : (
 <div>
 <p className="text-sm font-semibold text-white">
 <div className="space-y-2">
 {q.options.map((option, i) => {
 const isRight = i === q.correctIndex;
 const chosen = picked === i;
 const tone = !answered
 ? "border-slate-600 bg-slate-800/70 text-white"
 : isRight
 ? "border-emerald-500 bg-emerald-500/50 text-white"
 : chosen
 ? "border-rose-500 bg-rose-500/15 text-white"
 : "border-slate-600 bg-slate-800/70 text-white"
 })}
 </div>
 </p>
 </div>
)}

```

```
 </button>
 </div>
 </>
 })
</div>
</section>
);
};

export default ChapterQuiz;
```

---

ФАЙЛ 37/111: src/components/ChapterView.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import React, { useCallback, useEffect, useRef, useState } from "react";
import { Check, Download, FileText, Loader2, Maximize2, } from "lucide-react";
import type { Chapter } from "../types";
import { useTermHints } from "../hooks/useTermHints";
import { TermPopover, type HintAnchor } from "../hints/TermHint";
import { getViz } from "../vizRegistry";
import { VizChapterContext } from "../data/vizStepBus";
import { VizLiveStatus } from "../VizLiveStatus";
import { ChapterNav } from "../ChapterNav";
import { ChapterQuiz } from "../ChapterQuiz";
import { downloadChapterHtml } from "../utils/exportHtml";
import { useTicketLinks } from "../utils/ticketLinks";
```

```
interface Props {
 chapter: Chapter;
 prev?: Chapter;
 next?: Chapter;
 index: number;
 total: number;
 onGo: (id: string) => void;
 onOpenSimulator: (vizId: string) => void;
```

```

* чтобы на бумагу попал весь текст (доказательства,
* а после печати возвращаем их в исходное состояние.
*/
useEffect(() => {
 const each = () => Array.from(contentRef.current?.q
 const openAll = () => {
 each().forEach((d) => {
 d.dataset.wasOpen = d.open ? "1" : "0";
 d.open = true;
 });
 };
 const restore = () => {
 each().forEach((d) => {
 if (d.dataset.wasOpen === "0") d.open = false;
 delete d.dataset.wasOpen;
 });
 };
 window.addEventListener("beforeprint", openAll);
 window.addEventListener("afterprint", restore);
 return () => {
 window.removeEventListener("beforeprint", openAll);
 window.removeEventListener("afterprint", restore);
 };
}, [chapter.id]);

const saveHtml = useCallback(async () => {
 setSaving("work");
 try {
 await downloadChapterHtml(chapter);
 setSaving("done");
 window.setTimeout(() => setSaving("idle"), 2500);
 } catch {
 setSaving("idle");
 }
}, [chapter]);

const open = useCallback(
 (el: HTMLElement) => {

```

```

};

const viz = getViz(chapter.id);

return (
 <>
 <article className="chapter-body">
 {/* панель загрузки: сохранить именно эту страницу */}
 <div className="flex flex-wrap items-center gap-1.5">
 <button
 onClick={saveHtml}
 disabled={saving === "work"}
 title="Сохранить эту тему одним автономным файлом"
 className="flex items-center gap-1.5 text-x-small border-emerald-500"
 style={{width: 300, hover: {text: "white", border: "emerald-500 1px solid black"}}}
 >
 {saving === "work" ? (
 <Loader2 className="w-3.5 h-3.5 animate-spin" />
) : saving === "done" ? (
 <Check className="w-3.5 h-3.5 text-emerald-500" />
) : (
 <Download className="w-3.5 h-3.5" />
)}
 </button>
 <button
 onClick={() => window.print()}
 title="Распечатать или сохранить эту тему в PDF-формате"
 className="flex items-center gap-1.5 text-x-small border-indigo-500"
 style={{width: 300, hover: {text: "white", border: "indigo-500 1px solid black"}}}
 >
 <Printer className="w-3.5 h-3.5" /> PDF этой страницы, без шапки, содержания и панели комментариев
 </button>
 <a
 href={`?lite=1&topic=${encodeURIComponent(chapter.id)}`}
 target="_blank"
 rel="noreferrer"
 >

```





## Универсальное пособие • полный исходный код

```

const TABS: { id: Tab; label: string; icon: React.ReactElement } = [
 { id: "hierarchy", label: "1 · Иерархия классов", icon: <IconHierarchy /> },
 { id: "reduce", label: "2 · Сведение A $\leq_p$ B", icon: <IconReduce /> },
 { id: "verify", label: "3 · Верификатор (NP)", icon: <IconVerify /> },
 { id: "growth", label: "4 · Рост по n и ops", icon: <IconGrowth /> },
];
```

```
/* ----- Иерархия -----
```

```
const CLASSES = [
 { id: "EXP", label: "EXP", sub: "2(nk)", color: "#0000FF",
 {кванторные формулы}"}],
 { id: "PSPACE", label: "PSPACE", sub: "полиномиальная
 с полной информацией"}],
 { id: "NP", label: "NP", sub: "сертификат проверяется
 AT", "гамильтонов цикл", "раскраска в 3 цвета", "клика"},
 { id: "P", label: "P", sub: "решается за O(nk)", color: "#00FF00",
 {ид", "MST", "2-SAT, двудольность"}],
];
```

```
const NPHARD_NOTE = "NP-трудные – всё, к чему сводится
гамильтонов цикл. Проблема остановки NP-трудна, но в NP."
```

```
/* ----- Сведение -----
```

```
const REDUCE_STEPS = [
 { title: "Дано: задача A, для которой быстрого алгоритма нет
 – то есть B NP-трудная.", boxes: ["A"], active: 0 },
 { title: "Берём вход x задачи A", note: "x – конкретный экземпляр
 "x"]}, active: 1 },
 { title: "Строим f(x) = y за полиномиальное время", note: "y
 шинами из разных дизъюнктов, если литералы не противоречивы",
 ve: 2 },
 { title: "Решаем B на входе y", note: "Гипотетический алгоритм
 – но допустим, что он есть.", boxes: ["A", "x", "f(x)"]}, active: 3 },
 { title: "Переводим ответ обратно: A(x) = B(f(x))", note: "A(x)
 $\Leftrightarrow$ формула выполнима. Полиномиальное время сохранилось",
 5 }],
```

```

const GROWTH_LINES = [
 { id: "n", label: "n", fn: (n: number) => n, color: "#000000" },
 { id: "n log n", label: "n·log2 n", fn: (n: number) => n * Math.log2(n), color: "#000000" },
 { id: "n2", label: "n2", fn: (n: number) => n * n, color: "#000000" },
 { id: "n3", label: "n3", fn: (n: number) => n * n * n, color: "#000000" },
 { id: "2n", label: "2n", fn: (n: number) => Math.pow(2, n), color: "#000000" },
];

export const ComplexityViz: React.FC = () => {
 const chapterId = useContext(VizChapterContext);
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const [tab, setTab] = useState<Tab>("hierarchy");
 const [step, setStep] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [samples, setSamples] = useState<Sample[]>([]);

 useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, tab);
 }, [chapterId, tab]);

 const maxStep = tab === "reduce" ? REDUCE_STEPS.length : 0;
 useEffect(() => {
 setStep(0);
 setPlaying(false);
 }, [tab]);

 useEffect(() => {
 if (!playing) return;
 const t = window.setInterval(() => {
 setStep((s) => {
 if (s >= maxStep) {
 setPlaying(false);
 return s;
 }
 return s + 1;
 });
 }, 1400);
 }, [maxStep, playing]);
}

```

```

 let den = 0;
 for (let i = 0; i < xs.length; i++) {
 num += (xs[i] - mx) * (ys[i] - my);
 den += (xs[i] - mx) ** 2;
 }
 const slope = den === 0 ? 0 : num / den;
 // экспонента на лог-лог растёт неограниченно: сравн
 const half = Math.floor(pts.length / 2);
 const slopeTail = (() => {
 const q = pts.slice(half);
 if (q.length < 2) return slope;
 const lx = q.map((p) => Math.log(p.n));
 const ly = q.map((p) => Math.log(p.ops));
 const ax = lx.reduce((a, b) => a + b, 0) / lx.length;
 const ay = ly.reduce((a, b) => a + b, 0) / ly.length;
 let nn = 0;
 let dd = 0;
 for (let i = 0; i < lx.length; i++) {
 nn += (lx[i] - ax) * (ly[i] - ay);
 dd += (lx[i] - ax) ** 2;
 }
 return dd === 0 ? slope : nn / dd;
 })();
 const cls =
 slopeTail > slope + 0.8 || slopeTail > 4.5
 ? { name: "экспонента / суперполином", color: "#f59e0b",
 : slope <= 1.25
 ? { name: "≈ линейный, $O(n)$ ", color: "#10b981",
 : slope <= 1.8
 ? { name: "≈ $O(n \log n)$ ", color: "#06b6d4", hint: "log"
 : slope <= 2.4
 ? { name: "≈ квадратичный, $O(n^2)$ ", color: "#f59e0b",
 : slope <= 3.4
 ? { name: "≈ кубический, $O(n^3)$ ", color: "#fb9238",
 : { name: "полином высокой степени", color: "#fb9238"
 };
 return { slope, slopeTail, ...cls };
}, [samples]);

```

```

 lg bg-slate-900 border border-slate-700 text-
 /></button>
 <button onClick={() => setPlaying((p) => !p)}
 text-xs font-bold hover:bg-purple-500">
 {playing ? <Pause className="w-4 h-4" /> : }
 </button>
 <button onClick={() => { setPlaying(false); s
 -2 rounded-lg bg-slate-900 border border-sl
 e="w-4 h-4" /></button>
 <span className="text-[11px] text-slate-500 m
</div>
}}

```

```

{ /* ИЕРАРХИЯ
{tab === "hierarchy" && {
 <div className="space-y-4">
 <div className="bg-slate-950 rounded-xl p-3 b
 <svg viewBox="0 0 500 380" className="w-ful
 {CLASSES.map((c) => {
 const on = liveCls ? liveCls.toUpperCase
 return (
 <g key={c.id}>
 <ellipse cx={c.x} cy={c.y} rx={c.rx}
{on ? 3 : 1.5} strokeDasharray={on ? undefi
 <text x={c.x} y={c.y - c.ry + 18} t
 <text x={c.x} y={c.y - c.ry + 32} t
 </g>
);
 })}
 { /* примеры задач – внутри своих овалов *
 {CLASSES.map((c) =>
 c.tasks.map((t, i) => (
 <text key={t} x={c.x} y={c.y + (c.id :
tAnchor="middle" fill="#e2e8f0" fontSize="8
 · {t}
 </text>
))
 })
 }
 }
 }
}

```

```

 }}}
 </div>
</div>
<div className="bg-slate-900/70 border border-
 <p className="text-sm font-bold text-white
 text-purple-400" />{reduceFrame.title}</p>
 <p className="text-xs text-slate-400 leading
</div>
</div>
)}

{/* ----- ВЕРИФИКАТОР -----
{tab === "verify" && (
 <div className="space-y-3">
 <div className="bg-slate-950 rounded-xl p-4 b
 <p className="text-[10px] uppercase tracking
 <div className="flex flex-wrap gap-2 mb-3">
 {CERT.map((v, i) => {
 <span key={i} className={"font-mono tex
 ld-600/50 text-emerald-300" : "bg-slate-900
 x{i + 1} = {liveCert && typeof liveCe

 })}
 {liveCert && <span className="font-mono t
 le-950/40">cert из кода}
 </div>
 <div className="space-y-2">
 {CLAUSES.map((cl, i) => {
 const checked = liveK !== null ? i < li
 const current = liveK !== null ? i ===
 const st = VERIFY_STEPS[i];
 return (
 <div
 key={i}
 className={"flex items-center gap-3
 amber-950/20" : checked ? {st.ok ? "border-
 slate-800 bg-slate-900/40"}}
 >

```

```

e0ps ?? "-"}
```

```

 <button onClick={() => setSamples([])} className="rounded-lg px-2 py-1">очистить график</button>
</div>
<svg viewBox="0 0 500 240" className="w-full h-40">
 {/* оси в лог-лог */}
 <line x1={44} y1={200} x2={486} y2={200} />
 <line x1={44} y1={14} x2={44} y2={200} strokeDash={[]} />
 <text x={265} y={222} textAnchor="middle" font-size={14}>log</text>
 <text x={16} y={110} fill="#64748b" font-size={12}>log</text>
 {GROWTH_LINES.map((g) => {
 const pts: string[] = [];
 for (let i = 0; i <= 20; i++) {
 const n = 1 + i * 0.8;
 const v = g.fn(n);
 const x = 44 + (Math.log(n) / Math.log(2)) * 442;
 const y = 200 - (Math.log(Math.max(1, v)) / Math.log(2)) * 196;
 pts.push(`${x.toFixed(1)},${Math.max(1, v).toFixed(1)}`);
 }
 return <polyline key={g.id} points={pts} strokeDash={[]} stroke-width={2} opacity={0.55} />;
 })}
 {samples.map((s) => {
 const x = 44 + (Math.log(Math.max(1, s.n)) / Math.log(2)) * 442;
 const y = 200 - (Math.log(Math.max(1, s.v)) / Math.log(2)) * 196;
 return <circle key={`${s.n}`} cx={x} cy={y} r={10} stroke-width={2} />;
 })}
 {GROWTH_LINES.map((g, i) => (
 <g key={g.id + "l"}>
 <rect x={330 + (i % 3) * 58} y={16 + i * 8} width={50} height={10} />
 <text x={343 + (i % 3) * 58} y={24 + i * 8} font-size={10}>{g.fn(1.8)}</text>
 </g>
))}
</svg>
{samples.length === 0 && (
 <p className="text-xs text-slate-500 text-center">Пусто. Запусти в компиляторе код с переключателем</p>
)}

```

```

 }}
 </div>
 </div>
);
};

export default ComplexityViz;

```

---

ФАЙЛ 39/111: src/components/DfsBridgesSimulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```

import { useState, useEffect, useRef } from "react";
import { useVizStepSync, vizNumber, vizString } from ".";
import { Undo, Play, Pause, SkipForward, RefreshCw } from "react-";

interface CodeLine {
 line: string;
 highlight: string;
}

const CODE_LINES: CodeLine[] = [
 { line: "void dfs(int v, int p = -1) {", highlight: "normal" },
 { line: " visited[v] = true;", highlight: "normal" },
 { line: " tin[v] = low[v] = timer++;", highlight: "normal" },
 { line: " for (int to : adj[v]) {", highlight: "normal" },
 { line: " if (to == p) continue;", highlight: "normal" },
 { line: " if (visited[to]) {", highlight: "normal" },
 { line: " low[v] = min(low[v], tin[to]);", highlight: "normal" },
 { line: " } else {", highlight: "normal" },
 { line: " dfs(to, v);", highlight: "normal" },
 { line: " low[v] = min(low[v], low[to]);", highlight: "normal" },
 { line: " }", highlight: "normal" },
 { line: " if (low[to] > tin[v]) {", highlight: "normal" },

```

```

 { u: 1, v: 6, key: "1-6" }
];

const DFS_STEPS: DfsStep[] = [
 {
 currentNode: 0, visitedIndices: [0],
 tin: {0:1}, low: {0:1}, stack: [0], bridges: [],
 activeEdge: null, backEdge: null,
 log: "Входим в A. tin[A]=1, low[A]=1.",
 activeCodeLine: [0,1,2]
 },
 {
 currentNode: 1, visitedIndices: [0,1],
 tin: {0:1,1:2}, low: {0:1,1:2}, stack: [0,1], bridges: [],
 activeEdge: [0,1], backEdge: null,
 log: "Идём (A,B). В впервые. tin[B]=2, low[B]=2.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 6, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
 activeEdge: [1,6], backEdge: null,
 log: "Из B идём в G. tin[G]=3, low[G]=3.",
 activeCodeLine: [4,5,7,10,11]
 },
 {
 currentNode: 1, visitedIndices: [0,1,6],
 tin: {0:1,1:2,6:3}, low: {0:1,1:2,6:3}, stack: [0,1,6],
 activeEdge: null, backEdge: null,
 log: "У G нет соседей (кроме B). Возврат. low[G]=3.",
 activeCodeLine: [13,14,15]
 },
 {
 currentNode: 2, visitedIndices: [0,1,6,2],
 tin: {0:1,1:2,6:3,2:4}, low: {0:1,1:2,6:3,2:4}, stack: [0,1,6,2],
 activeEdge: [1,2], backEdge: null,
 log: "Из B идём в C. tin[C]=4, low[C]=4.",
 activeCodeLine: [4,5,7,10,11]
 }
];

```



```

tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
activeEdge: null, backEdge: null,
log: "Возврат F→E. low[E]=min(6,5)=5. (E,F) – не мост,
activeCodeLine: [11,13,16,17]
},
{
currentNode: 3, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
activeEdge: null, backEdge: null,
log: "Возврат E→D. low[D]=min(6,5)=5. (D,E) – не мост,
activeCodeLine: [11,13,16,17]
},
{
currentNode: 2, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:2,6:3,2:4,3:5,4:6,5:7},
activeEdge: null, backEdge: null,
log: "Возврат D→C. low[5] = 5 > tin[2] = 4! (C,D) – мост,
activeCodeLine: [13,14,15,16,17]
},
{
currentNode: 1, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
activeEdge: null, backEdge: null,
log: "Возврат C→B. low[B]=min(low[B],low[C])=1. (B,C) – мост,
activeCodeLine: [11,13,16,17]
},
{
currentNode: 0, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
activeEdge: null, backEdge: null,
log: "Возврат B→A. DFS завершён! Мосты: (B,G) и (C,D). Алгоритм
activeCodeLine: [11,13,16,17,18]
},
{
currentNode: null, visitedIndices: [0,1,6,2,3,4,5],
tin: {0:1,1:2,6:3,2:4,3:5,4:6,5:7}, low: {0:1,1:1,6:3,2:4,3:5,4:6,5:7},
activeEdge: null, backEdge: null,
log: "Готово! Найдены мосты: (B,G) и (C,D). Алгоритм

```

```

return (
 <div className="space-y-4">
 <div className="flex items-center justify-between">
 <h4 className="text-base font-bold text-white">
 <div className="flex items-center gap-1 bg-slate-700 rounded"
 <button onClick={() => { setDfsAuto(false); s
 disabled={dfsIdx === 0}
 aria-label="Шаг назад" title="Шаг назад"
 className="p-1.5 hover:bg-slate-700 rounded"
 <Undo className="h-3.5 w-3.5" />
 </button>
 <button onClick={() => setDfsAuto(!dfsAuto)}
 className={`px-2.5 py-1 rounded text-[10px]
 dfsAuto ? "bg-rose-600 text-white" : "bg-
 }`>
 {dfsAuto ? <><Pause className="h-3 w-3" />
 </button>
 <button onClick={() => { setDfsAuto(false); s
 disabled={dfsIdx === DFS_STEPS.length-1}
 aria-label="Шаг вперёд" title="Шаг вперёд"
 className="p-1.5 hover:bg-slate-700 rounded"
 <SkipForward className="h-3.5 w-3.5" />
 </button>
 <button onClick={() => { setDfsAuto(false); s
 aria-label="Сбросить DFS" title="Сбросить D
 className="p-1.5 hover:bg-slate-700 rounded"
 <RefreshCw className="h-3.5 w-3.5" />
 </button>
 </div>
 </div>

 <div className="grid grid-cols-1 md:grid-cols-5 g
 { /* Graph visualization */ }
 <div className="md:col-span-3 bg-slate-950 roun
 <div className="absolute inset-0 bg-[radial-g
 >
 <svg className="w-full h-[280px] z-10 relativ

```

```

 label}</text>
 {vis && (
 <>
 <rect x={n.x-18} y={n.y-28} width={
"transition-all duration-300" />
 <text x={n.x} y={n.y-20} textAnchor:
} l:{step.low[n.id]||"?"}</text>
 </>
)}
 </g>;
 }}}}
 </svg>
 <div className="flex items-center justify-betw
 Mar {dfsIdx+1}/{DFS_STEPS.length}</sp
 <div className="flex-1 mx-2 bg-slate-950 h-
 <div className="bg-indigo-600 h-full tran
 </div>
 </div>
</div>

{/* Code panel + stack */}
<div className="md:col-span-2 space-y-3">
 {/* Code panel */}
 <div className="bg-slate-950 rounded-lg borde
 <div className="bg-slate-900 px-3 py-1.5 te
 >
 <div className="p-2 overflow-x-auto">
 <pre className="text-[10px] font-mono lea
 {CODE_LINES.map(({cl, i}) => {
 const hl = step.activeCodeLine.includ
 return <div key={i}
 className={`px-1 transition-all dur
 hl ? "bg-indigo-600/20 text-indigo
 }`}>
 <span className="text-slate-600 mr-1
 {cl.line || ' '}
 </div>;

```

```
 </div>
 </div>
);
 }
}
```

---

ФАЙЛ 40/111: src/components/DijkstraViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect } from 'react';
import { useVizRuntime, vizArray, vizNumber, vizRecord,
import { LiveDistTable } from './LiveDistTable';

interface Node { id: string; x: number; y: number; name: string; }
interface Edge { u: string; v: string; w: number; }
interface Step {
 currentNode: string | null;
 checkingEdge: { u: string; v: string } | null;
 distances: { [key: string]: number };
 visited: { [key: string]: boolean };
 previous: { [key: string]: string | null };
 log: string;
 activeCodeLine: number;
}

/** Подсказка для кнопок, когда шаги ведёт компилятор.
const HINT_TEXT = "Шаги сейчас ведёт панель Python: ← →
 и листать вручную.";

export default function DijkstraViz() {
 const nodes: Node[] = [
 { id: 'A', x: 60, y: 150, name: 'Пиццерия (A)' },
 { id: 'B', x: 160, y: 60, name: 'Дом 1 (B)' },
 { id: 'C', x: 160, y: 240, name: 'Парк (C)' },
 { id: 'D', x: 340, y: 60, name: 'Офис (D)' },
 { id: 'E', x: 340, y: 240, name: 'Дом 2 (E)' },
```

```

const [steps, setSteps] = useState<Step[]>([]);
const [currentStepIndex, setCurrentStepIndex] = useSt
const runtime = useVizRuntime();
const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
 const simulationSteps: Step[] = [];
 const dist: Record<string, number> = { A: 0, B: 999 };
 const visited: Record<string, boolean> = { A: false };
 const prev: Record<string, string | null> = { A: null };

 simulationSteps.push({
 currentNode: null, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ' Дейкстра готов к доставке. Начинаем из Пирса',
 activeCodeLine: 2,
 });

 for (let iter = 0; iter < nodes.length; iter++) {
 let minD = 999;
 let u = null;
 for (const n of nodes) {
 if (!visited[n.id] && dist[n.id] < minD) {
 minD = dist[n.id];
 u = n.id;
 }
 }

 if (!u) break;

 visited[u] = true;
 simulationSteps.push({
 currentNode: u, checkingEdge: null,
 distances: { ...dist }, visited: { ...visited },
 log: ` Выбираем вершину ${u} с мин. непосещёнными соседями`,
 activeCodeLine: 5,
 });
 }
});

```

```

 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => {
 setCurrentStepIndex((prev) => prev + 1);
 }, 1500);
 } else if (currentStepIndex >= steps.length - 1) {
 setIsPlaying(false);
 }
 return () => clearTimeout(timer);
 }, [isPlaying, currentStepIndex, steps.length]);

const baseStep = steps[currentStepIndex] || null;

if (!baseStep) return <div className="text-white">3arg</div>;
const vars = runtime?.variables;
const liveU = vizString(vars?.u);
const liveV = vizString(vars?.v);
const liveDist = vizRecord(vars?.dist);
const livePrev = vizRecord(vars?.prev);
const liveDone = vizArray(vars?.done) ?? vizArray(vars?.done);
const compilerLinked = liveU !== null || liveV !== null;
const currentStep: Step = compilerLinked
 ? {
 ...baseStep,
 currentNode: liveU,
 checkingEdge: liveU && liveV ? { u: liveU, v: liveV },
 distances: Object.fromEntries(nodes.map((node) => [node, 0])),
 previous: Object.fromEntries(nodes.map((node) => [node, null])),
 visited: Object.fromEntries(nodes.map((node) => [node, false])),
 log: `Компилятор: u=${liveU ?? "-"}, v=${liveV ?? "-"}`
 }
 : baseStep;

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border">
 {/* Шапка симулятора */}
 <div className="flex flex-wrap items-center justify-center">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-indigo-600 text-white">

```

```

 </div>
 </div>

 <LiveDistTable
 dist={liveDist}
 prev={livePrev}
 done={liveDone}
 demoIds={nodes.map((node) => node.id)}
 title="Ваши расстояния из кода (Дейкстра)"
 />

 <div className="flex flex-col lg:flex-row gap-6 md:gap-10" >
 {/** Граф */}
 <div className="w-full lg:w-2/3 bg-indigo-950/20 p-6 rounded-2xl
 justify-center min-h-[400px]">
 <div className="absolute top-3 left-3 bg-indigo-950/50
 text-white font-bold shadow-sm">
 Шаг {currentStepIndex + 1} из {steps.length}
 </div>

 <svg className="w-full h-auto max-h-[500px]" >
 <defs>
 <marker id="arrow-dh-normal" markerWidth="10"
 <path d="M0,0 L6,3 L0,6 z" fill="#47556b" />
 </marker>
 <marker id="arrow-dh-active" markerWidth="10"
 <path d="M0,0 L6,3 L0,6 z" fill="#f59e00" />
 </marker>
 <marker id="arrow-dh-opt" markerWidth="6"
 <path d="M0,0 L6,3 L0,6 z" fill="#10b981" />
 </marker>
 </defs>
 {/** Рёбра */}
 {edges.map((edge, idx) => {
 const uNode = nodes.find((n) => n.id === edge.u)
 const vNode = nodes.find((n) => n.id === edge.v)
 const isChecking =
 currentStep.checkingEdge === edge.id
 })}
 </svg>
 </div>
 </div>

```

```

const isVisited = currentStep.visited[node.id]
const dist = currentStep.distances[node.id]

return (
 <g key={node.id} className="transition-
 <circle
 cx={node.x} cy={node.y}
 r={isCurrent ? 22 : 18}
 fill={isCurrent ? '#4f46e5' : isVisited ? '#f1c40f' : '#f1c40f'}
 stroke={isCurrent ? '#a5b4fc' : isVisited ? '#f1c40f' : '#f1c40f'}
 strokeWidth={isCurrent ? 4 : 2}
 className="transition-all duration-300"
 />
 <text x={node.x} y={node.y + 5} fill="white"
 {node.id}
 </text>
 <text x={node.x} y={node.y - 28} fill="white"
 "middle" className="bg-indigo-950 px-1 py-0"
 {dist === 999 ? '∞' : `d=${dist}`}
 </text>
 <text x={node.x} y={node.y + 35} fill="white"
 {node.name.split(' ')[0]}
 </text>
 </g>
);
}}}
</svg>
<div className="w-full bg-slate-950/80 p-4 rounded"
 <p className="font-semibold text-amber-400"
 <p className="min-h-[40px] flex items-center"
</div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/
 <h4 className="text-lg font-bold text-emerald
 <div className="space-y-2.5 flex-1 overflow-y
 {Object.keys(adjList).map((u) => {

```



```

<div>
 <h4 className="text-lg font-bold text-rose-900">
 <div className="overflow-x-auto">
 <table className="w-full text-left border-collapse">
 <thead>
 <tr className="border-b border-rose-900">
 <th className="py-2 px-3">Вершина</th>
 <th className="py-2 px-3">Расстояние</th>
 <th className="py-2 px-3">Из (пред. шаг)</th>
 </tr>
 </thead>
 <tbody className="text-sm">
 {nodes.map((n) => {
 const dist = currentStep.distances[n.id]
 const prev = currentStep.previous[n.id]
 const vis = currentStep.visited[n.id]
 const isCurr = currentStep.currentNode === n.id

 return (
 <tr key={n.id} className={`border-b border-rose-900 ${
 isCurr ? 'bg-indigo-950/60 font-weight-bold' : ''
 }`} >
 <td className="py-2.5 px-3 flex items-center">

 {n.name}
 </td>
 <td className="py-2.5 px-3 font-weight-bold">
 {dist === 999 ? '∞' : dist}
 </td>
 <td className="py-2.5 px-3 text-rose-900">
 {prev || '-'}
 </td>
 </tr>
)
 })}
 </tbody>
 </table>
 </div>

```

```

interface DPStep {
 id: number;
 n: number;
 action: 'call' | 'memo_hit' | 'base_case' | 'calc' |
 desc: string;
 codeLine: number;
 memoState: { [key: number]: number };
}

const DP_CODE_LINES = [
 /* 1 */ "function fibMemo(n, memo = {}) {",
 /* 2 */ " if (n in memo) return memo[n];",
 /* 3 */ " if (n <= 2) return 1;",
 /* 4 */ " memo[n] = fibMemo(n - 1, memo) + fibMemo",
 /* 5 */ " return memo[n];",
 /* 6 */ "}"
];

export function DPVisualizer() {
 const [targetN, setTargetN] = useState<number>(5);
 const [steps, setSteps] = useState<DPStep[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 const runtime = useVizRuntime();
 const [isPlaying, setIsPlaying] = useState<boolean>(f
 const [speed] = useState<number>(800);
 const [activeTab, setActiveTab] = useState<'table' |

 useEffect(() => {
 const generatedSteps: DPStep[] = [];
 let stepId = 0;
 const memoMap: { [key: number]: number } = {};

 function simulateFib(n: number): number {
 generatedSteps.push({
 id: stepId++,
 n,
 action: 'call',

```

```

 const res1 = simulateFib(n - 1);
 const res2 = simulateFib(n - 2);
 memoMap[n] = res1 + res2;

 generatedSteps.push({
 id: stepId++,
 n,
 action: 'calc',
 desc: `Сохраняем в кэш: memo[${n}] = ${res1} + ${res2}`,
 codeLine: 5,
 memoState: { ...memoMap }
 });

 return memoMap[n];
 }

 const finalRes = simulateFib(targetN);

 generatedSteps.push({
 id: stepId++,
 n: targetN,
 action: 'done',
 desc: `Расчет завершен! fibMemo[${targetN}] = ${finalRes}`,
 codeLine: 6,
 memoState: { ...memoMap }
 });

 setSteps(generatedSteps);
 setCurrentStepIndex(0);
 setIsPlaying(false);
}, [targetN]);

useEffect(() => {
 let timer: any = null;
 if (isPlaying && currentStepIndex < steps.length - 1) {
 timer = setTimeout(() => {
 setCurrentStepIndex(prev => prev + 1);
 }, speed);
 }
}, [isPlaying, currentStepIndex, steps.length, speed]);

```

Смотри, как таблица DP кэширует результаты

</p>

</div>

<div className="flex flex-wrap items-center gap-

<div className="flex items-center gap-2 bg-sl

<span>N:</span>

<select

value={targetN}

onChange={(e) => setTargetN(Number(e.targ

disabled={isPlaying}

className="bg-transparent text-white font

>

<option value={4}>N = 4</option>

<option value={5}>N = 5</option>

<option value={6}>N = 6</option>

<option value={7}>N = 7</option>

</select>

</div>

<div className="flex items-center bg-slate-95

<button

onClick={() => { setIsPlaying(false); set

className="p-2 text-slate-400 hover:text-v

title="Сбросить"

>

<RotateCcw className="w-4 h-4" />

</button>

<button

onClick={() => { setIsPlaying(false); set

disabled={currentStepIndex === 0}

className="p-2 text-slate-400 hover:text-v

title="Шаг назад"

>

<ChevronLeft className="w-5 h-5" />

</button>

<button

onClick={() => setIsPlaying(!isPlaying)}

```

 : 'bg-slate-900/50 border-slate-800
 }` }
 >
 <span className="text-xs text-slate-400
 <span className={`text-xl md:text-2xl f
 {isCached ? val : '0'}

 {num <= 2 && <span className="text-[9px
 </div>
 });
 }}}}
</div>

{currentStep && (
 <div className="mt-6 pt-4 border-t border-sla
 <div className="text-sm font-medium text-sla
 <span className="inline-block w-2 h-2 rou
 {currentStep.desc}
 </div>
 <div className="text-xs font-mono bg-slate-
 Текущий N: <strong className="text-emeral
 </div>
 </div>
)}
</div>

{/* Tabs */}
<div className="flex items-center gap-2 border-b
 <button
 onClick={() => setActiveTab('table')}
 className={`flex items-center gap-2 px-6 py-3
 activeTab === 'table'
 ? 'border-emerald-500 text-emerald-400 bg
 : 'border-transparent text-slate-400 hove
 }` }
 >
 <Eye className="w-4 h-4" />
 Степ-бай-степ состояние

```

```

 </div>
 <pre className="p-6 font-mono text-sm space-x-4" >
 {DP_CODE_LINES.map((line, idx) => {
 const lineNum = idx + 1;
 const isActive = currentStep?.codeLine === lineNum;
 return (
 <div
 key={lineNum}
 className={`flex items-center px-4 py-2 ${
 isActive
 ? 'bg-emerald-600/20 border-l-4 border-emerald-600'
 : 'text-slate-300 hover:bg-slate-800'
 }`}
 >
 {lineNum}
 {line}
 </div>
);
 })}
 </pre>
 <div className="p-6 bg-slate-900/50 border-l-4 border-emerald-600" >
 {currentStep?.desc}
 </div>
 </div>
)}
</div>

```

```

{/* Progress Bar */}
<div className="mt-8 pt-6 border-t border-slate-800" >
 <div className="flex items-center justify-between" >
 Прогресс выполнения
 Шаг {currentStepIndex + 1} из {steps.length}
 </div>
 <div className="h-2 bg-slate-950 rounded-full overflow-hidden" >
 <div
 className="h-full bg-gradient-to-r from-emerald-500 to-emerald-600"
 style={{ width: `${((currentStepIndex + 1) / steps.length) * 100}%` }}
 </div>
 </div>

```

```

*
* Две части:
* 1. игра «пройди все рёбра» кликами (Эйлер не прощает
* 2. критерий: степени вершин и число нечётных (0 → ц
*
* Синхронизация с Python-компилятором (PAGE_SYNC["eule
* path – маршрут (массив вершин) рисуется поверх графа
* пройдены, deg/odd – степени и нечётные вершины, las
* step – номер шага.
*/
export function EulerSimulator() {
 const chapterId = useContext(VizChapterContext);
 const runtime = useVizRuntime();
 const vars = runtime?.variables;

 const [c1Path, setC1Path] = useState<number[]>([]);
 const [c1VisitedEdges, setC1VisitedEdges] = useState<
 const [c1Msg, setC1Msg] = useState("Кликни на вершину
 const [c1Done, setC1Done] = useState(false);

 useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, "walk");
 }, [chapterId]);

 const resetC1 = () => {
 setC1Path([]); setC1VisitedEdges([]); setC1Msg("Кли
 };

 const clickC1 = (vId: number) => {
 if (c1Done && c1Path.length > 0) return;
 if (c1Path.length === 0) {
 setC1Path([vId]);
 setC1Msg("Старт! Кликай соседние вершины, чтобы п
 return;
 }
 const last = c1Path[c1Path.length - 1];
 const edge = C1_EDGES.find(e => (e.u === last && e.v
 if (!edge) {

```

```

 const val = degRec[String(n.id)] ?? degRec[n.id]
 const num = typeof val === "number" ? val : val.toString()
 return num ?? DEG[n.id];
 })
 : null;
const oddRaw = vizArray(vars.odd);
const odd = oddRaw ? oddRaw.map((x) => (typeof x === "number" ? x : x.toString())) : null;
const last = vizNumber(vars.last) ?? vizNumber(vars.last.toString());
const step = vizNumber(vars.step) ?? vizNumber(vars.step.toString());
if (path.length === 0 && usedFlags.length === 0 &&
 // рёбра маршрута: пары соседних вершин
 const keys: string[] = [];
 for (let i = 0; i + 1 < path.length; i++) keys.push(path[i] + path[i + 1]);
 usedFlags.forEach((f, i) => {
 if (f && C1_EDGES[i]) keys.push(ekey(C1_EDGES[i]));
 });
 return {
 path,
 keys: Array.from(new Set(keys)),
 deg,
 odd,
 last: last !== null && last >= 0 && last < C1_NODES.length ? last : null,
 step,
 complete: keys.length >= C1_EDGES.length,
 };
}, [vars]));

```

```

const path = live && live.path.length ? live.path : c1Path;
const visitedKeys = live ? live.keys : c1VisitedEdges;
const deg = live?.deg ?? DEG;
const odd = live?.odd ?? ODD;
const lastId = live ? live.last : path.length ? path[path.length - 1] : null;
const done = live ? live.complete : c1Done && visitedKeys.length >= C1_EDGES.length;
const msg = live
 ? live.complete
 ? `Компилятор прошёл все ${C1_EDGES.length} рёбра графа. Вершин ${odd.length} → это ${odd.length === 0 ? "цикл" : "маршрут"}.`
 : `Маршрут из кода: ${live.path.length ? live.path : c1Path}`;

```



```

return <g key={i}>
 <line x1={u.x} y1={u.y} x2={v.x} y2={v.y}
 stroke={isLastEdge ? "#a5b4fc" : used ?
 strokeWidth={isLastEdge ? 5 : used ? 4
 className="transition-all duration-300"
 {order !== null && (
 <text x={({u.x + v.x} / 2} y={({u.y + v.y}
 }</text>
)}
 </g>;
}})
{C1_NODES.map(n => {
 const isLast = lastId === n.id;
 const visited = path.includes(n.id);
 const isOdd = odd.includes(n.id);
 return <g key={n.id} className="cursor-point
 {isLast && <circle cx={n.x} cy={n.y} r={11}
 <circle cx={n.x} cy={n.y} r={11}
 fill={isLast ? "#6366f1" : visited ? "#
 stroke={isLast ? "#fff" : visited ? "#6
 strokeWidth={2.5}
 className="transition-all duration-200
 <text x={n.x} y={n.y + 4} textAnchor="mid
 label}</text>
 <text x={n.x} y={n.y - 16} textAnchor="mi
 one">deg {deg[n.id]}{isOdd ? " • нечёт" : "
 </g>;
 })}
</svg>
<div className="absolute top-2 left-2 bg-slate-
 Pe6ëp: {visitedKeys.length}/{C1_EDGES.length}
</div>
</div>

<div className={`p-3 rounded-xl border text-sm ${
 done && visitedKeys.length === C1_EDGES.length
 ? "bg-emerald-950/40 border-emerald-500/30 te
 : c1Done && !live

```

```
 });
}
```

---

ФАЙЛ 43/111: src/components/FloydViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { useState, useEffect } from 'react';
import { liveMatrix, pickLiveVar, useVizRuntime, vizNums
```

```
interface Step {
 k: number; i: number; j: number;
 matrix: (number | null)[][]; updated: boolean;
 log: string; activeCodeLine: number;
}
```

```
/** Подсказка для кнопок, когда шаги ведёт компилятор.
const HINT_TEXT = "Шаги сейчас ведёт панель Python: ← →
ва листать вручную.";
```

```
export default function FloydViz() {
 const [steps, setSteps] = useState<Step[]>([]);
 const [currentStepIndex, setCurrentStepIndex] = useSt
 const runtime = useVizRuntime();
 // k/i/j/d приходят напрямую; номер строки произвольн
 const [isPlaying, setIsPlaying] = useState(false);
```

```
 const nodeNames = ['А 0', 'Б 1', 'В 2', 'Г 3', 'Д 4',
 const nodesSvg = [
 { id: 0, name: 'Аэропорт 0', x: 100, y: 60 },
 { id: 1, name: 'Аэропорт 1', x: 250, y: 60 },
 { id: 2, name: 'Аэропорт 2', x: 400, y: 60 },
 { id: 3, name: 'Аэропорт 3', x: 100, y: 180 },
 { id: 4, name: 'Аэропорт 4', x: 400, y: 180 },
 { id: 5, name: 'Аэропорт 5', x: 175, y: 280 },
 { id: 6, name: 'Аэропорт 6', x: 325, y: 280 },
```

```

k: -1, i: -1, j: -1, matrix: dist.map(r => [...r])
log: ' Инициализация матрицы прямых путей, ∞ (99
activeCodeLine: 2,
});

const N = 7;
for (let k = 0; k < N; k++) {
 simulationSteps.push({
 k, i: -1, j: -1, matrix: dist.map(r => [...r]),
 log: `+ Внешний цикл. Разрешаем пути через пром
 activeCodeLine: 3,
 });

 for (let i = 0; i < N; i++) {
 for (let j = 0; j < N; j++) {
 if (i === j || i === k || j === k) continue;

 const oldVal = dist[i][j];
 const viaVal = dist[i][k] + dist[k][j];
 if (dist[i][k] !== 999 && dist[k][j] !== 999 &
 dist[i][j] = viaVal;
 simulationSteps.push({
 k, i, j, matrix: dist.map(r => [...r]), u
 log: ` Улучшение пути [${i}]->[${j}] чере
 ∞ '.`,
 activeCodeLine: 7,
 });
 }
 }
}

simulationSteps.push({
 k: 7, i: -1, j: -1, matrix: dist.map(r => [...r])
 log: ' Все пары отработаны. Алгоритм Флойда завер
 activeCodeLine: 8,
});

```

```

const sizeMismatch = compilerLinked && liveMat && (liveMat.size !== demoSize)
 ? `Демо-схема графа нарисована для ${demoSize} вершин. При увеличении
 ерности; схема графа остаётся демонстрационной.`
 : null;
const labelOf = (idx: number) => nodeNames[idx]?.split(' ')?.[0];

const currentStep: Step = compilerLinked
 ? {
 ...baseStep,
 k: liveK ?? -1,
 i: liveI ?? -1,
 j: liveJ ?? -1,
 matrix: shownMatrix,
 updated: !!runtime?.changed?.some(({name}) => name === liveName),
 log: `Компилятор: k=${liveK ?? "-"}, i=${liveI ?? "-"}, j=${liveJ ?? "-"},
 name=${liveName}. Подсветка взята из реальных переносов.
 ` : ""}. Подсветка взята из реальных переносов.
 }
 : baseStep;

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border border-slate-700">
 <div className="flex flex-wrap items-center justify-center gap-4">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-white rounded-md">

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-emerald-300">Получить
 </p>
 </div>
 </div>
 <div className="flex items-center gap-2">
 <button onClick={() => { setIsPlaying(false);
 TEXT : "Вернуться к первому шагу"} className="text-sm
 d:cursor-not-allowed">Сброс</button>
 <button onClick={() => setIsPlaying(!isPlaying)} className="text-sm
 нную демонстрацию"> </button>
 </div>
 </div>
 </div>
);

```

```

 markerEnd={marker} strokeDasharray={isTarget
 <circle cx={({uNode.x + vNode.x) / 2}
 isVia ? '#818cf8' : '#64748b'} strokeWidth={
 <text x={({uNode.x + vNode.x) / 2} y={
 8fafc'} fontSize="10" fontWeight="bold" text={
 </g>
 });
 });
}}}
{nodesSvg.map((node) => {
 const isK = currentStep.k === node.id;
 const isI = currentStep.i === node.id;
 const isJ = currentStep.j === node.id;
 let fill = '#1e293b'; let stroke = '#64748b';
 if (isK) { fill = '#4f46e5'; stroke = '#a8a2d8'; }
 else if (isI) { fill = '#b45309'; stroke = '#c9a04d'; }
 else if (isJ) { fill = '#047857'; stroke = '#2e8b57'; }
 return (
 <g key={node.id} className="transition-colors">
 <circle cx={node.x} cy={node.y} r={isTarget
 sJ ? 4 : 2} className="transition-colors duration-200">
 <text x={node.x} y={node.y + 5} fill="white">{node.k}</text>
 <text x={node.x} y={node.y + 32} fill="white">{node.i}</text>
 </g>
);
})}
</svg>

<div className="w-full bg-slate-950/80 p-4 rounded">
 <p className="font-semibold text-amber-400">Список смежности</p>
 <p className="min-h-[40px] flex items-center">
 </div>
</div>

{/* Список смежности */}
<div className="w-full lg:w-1/3 bg-emerald-950/80">
 <h4 className="text-lg font-bold text-emerald-400">Список смежности</h4>

```

```

 {sizeMismatch}
 </p>
 } : null}
 <div className="overflow-x-auto">
 <table className="w-full text-center border">
 <thead><tr className="bg-rose-950/40 text-white">
 <th className="p-2 border-r border-rose-950/40">
 {Array.from({ length: cols }, (_, idx) => {
 <th key={idx} className={`p-2 border-rose-950/40`}>
 {labelOf(idx)} {idx}</th>
 </th>
 </th>
 </thead>
 <tbody className="text-xs font-mono">
 {currentStep.matrix.map((row, i) => {
 <tr key={i} className="border-b border-rose-950/40">
 <td className={`p-2 bg-rose-950/40 text-white`}>
 {currentStep.k === i ? 'text-amber-400' : ''}</td>
 {labelOf(i)} {i}
 </td>
 {row.map((val, j) => {
 const isUpd = currentStep.i === i && currentStep.k !== j
 const isVia = currentStep.k !== -1 && currentStep.j === j);
 let bg = 'bg-slate-900/40 text-slate-400';
 if (isUpd) bg = 'bg-emerald-900/80 text-emerald-400';
 else if (isVia) bg = 'bg-indigo-950/60 text-indigo-400';
 else if (currentStep.i === -1 && currentStep.j === j)
 bg = 'bg-indigo-950/60 text-indigo-400';
 else if (i === j) bg = 'bg-slate-900/40 text-slate-400';
 const shown = val === null ? '.' : val;
 return (<td key={j} className={`p-2 ${bg}`}>
 {shown}
 </td>
 </td>
 </tr>
 </tr>
 </tbody>
 </table>
 </div>
 </div>

```

```

* comps уменьшается.
*
* Синхронизация с Python-компилятором (PAGE_SYNC["graph"])
* - comp / color / used – массив разметки: вершины к
* - parent / p – массив DSU: рисуем множества
* - v – текущая вершина (пульсирует)
* - stack / dq / q – фронт обхода (подсвечивает)
* - comps / c – счётчик компонент (большая)
* - i / e / step – номер шага: двигает по кадру
*/

```

```

type Mode = "dfs" | "dsu";

```

```

interface Node {
 id: number;
 label: string;
 x: number;
 y: number;
}

```

/\*\* Граф из 10 вершин и трёх компонент – ровно как в те

```

const NODES: Node[] = [
 { id: 0, label: "0", x: 90, y: 80 },
 { id: 1, label: "1", x: 175, y: 45 },
 { id: 2, label: "2", x: 205, y: 140 },
 { id: 3, label: "3", x: 105, y: 175 },
 { id: 4, label: "4", x: 330, y: 70 },
 { id: 5, label: "5", x: 420, y: 45 },
 { id: 6, label: "6", x: 380, y: 145 },
 { id: 7, label: "7", x: 130, y: 290 },
 { id: 8, label: "8", x: 230, y: 320 },
 { id: 9, label: "9", x: 330, y: 275 },
];

```

```

const EDGES: [number, number][] = [
 [0, 1], [1, 2], [2, 3], [0, 3], [1, 3], // компонента
 [4, 5], [5, 6], [4, 6], // компонента
 [7, 8], [8, 9], // компонента
];

```

```

 push(start, stack, c, `Новый запуск: c = ${c}, стар
 цвет ${c} и кладём в стек.`);
 while (stack.length) {
 const v = stack[stack.length - 1];
 const next = ADJ[v].find((to) => comp[to] === 0);
 if (next === undefined) {
 stack.pop();
 push(v, stack, c, `Из ${v} идти некуда – возвра
 ;
 continue;
 }
 comp[next] = c;
 stack.push(next);
 push(next, stack, c, `${v} → ${next}: красим в цв
 }, кладём ${next} в стек.`, [v, next]));
 }
 push(null, [], c, `Компонента ${c} готова`, `Стек п
 } вершин. Ищем следующую серую вершину.`);
 }
 push(null, [], c, "Готово: компонент = " + c, `Внешни
 вязности.`);
 return frames;
}

```

```

interface DsuFrame {
 title: string;
 note: string;
 parent: number[];
 comps: number;
 edge: [number, number] | null;
 merged: boolean;
}

```

```

function buildDsuFrames(): DsuFrame[] {
 const frames: DsuFrame[] = [];
 const parent = NODES.map((n) => n.id);
 const find = (v: number): number => (parent[v] === v
 let comps = NODES.length;

```



```

const runtime = useVizRuntime();
const [mode, setMode] = useState<Mode>("dfs");
const [step, setStep] = useState(0);
const [playing, setPlaying] = useState(false);
const timer = useRef<number | null>(null);

const frames = mode === "dfs" ? DFS_FRAMES : DSU_FRAMES;
const maxStep = frames.length - 1;
const frame = frames[Math.min(step, maxStep)] as DfsFrame;

/* — вкладка сообщает компилятору, какой код подстав. */
useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, mode);
}, [chapterId, mode]);

/* — автопроигрывание — */
useEffect(() => {
 if (!playing) return;
 timer.current = window.setInterval(() => {
 setStep((s) => {
 if (s >= maxStep) {
 setPlaying(false);
 return s;
 }
 return s + 1;
 });
 }, 900);
 return () => {
 if (timer.current) window.clearInterval(timer.current);
 };
}, [playing, maxStep]);

const goTo = useCallback((n: number) => setStep(Math.min(n, maxStep)));

/* — переменные компилятора, которые рисуются НАПРЯМУЮ */
const vars = runtime?.variables;
/** comp / color / used из кода пользователя → своя рендеризация */
const liveComp = useMemo(() => {

```

```

const compOf = (id: number): number => {
 if (liveComp && liveComp[id] !== undefined && liveComp[id] !== 0) return liveComp[id];
 if (parentNow) return setOf(parentNow, id);
 return frame.comp?.[id] ?? 0;
};

const frontier = liveFrontier ?? (mode === "dfs" ? {
const cur = liveCur ?? frame.cur;
const comps = liveComps ?? frame.comps;
const edge = frame.edge;
const title = runtime && vars ? `${frame.title} • Контур` : "Контур";

const switchMode = (m: Mode) => {
 setMode(m);
 setStep(0);
 setPlaying(false);
};

return (
 <div className="bg-slate-800/90 p-5 sm:p-6 rounded-2xl">
 <div className="flex flex-wrap items-center justify-between">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-white rounded-md">
 <Network className="w-5 h-5" />
 </div>
 <div>
 <h3 className="text-lg sm:text-xl font-bold">Контур
 <p className="text-xs text-emerald-300">10 минут
 </div>
 </div>
 <div className="flex items-center bg-slate-950 p-2">
 <button
 onClick={() => switchMode("dfs")}
 className={"flex items-center gap-1.5 px-3 py-2 bg-slate-800/90 text-white shadow" : "text-slate-400 hover:bg-slate-700"}
 >
 <Layers className="w-3.5 h-3.5" /> 06ход (D
 </button>

```

```

 { /* поле */ }
 <div className="bg-slate-950 rounded-xl p-3 border"
 <div className="absolute inset-0 bg-[radial-gradient(
 <svg viewBox="0 0 500 370" className="w-full h-full"
 {EDGES.map(([u, v], i) => {
 const a = NODES[u];
 const b = NODES[v];
 const active = edge && ((edge[0] === u && edge[1] === v) || edge[0] === v && edge[1] === u);
 const sameComp = compOf(u) > 0 && compOf(v) > 0 && compOf(u) === compOf(v);
 return {
 <line
 key={i}
 x1={a.x} y1={a.y} x2={b.x} y2={b.y}
 stroke={active ? "#fbbf24" : sameComp ? "#fbbf24" : "#fbbf24"}
 strokeWidth={active ? 4 : sameComp ? 2 : 1}
 strokeDasharray={sameComp ? undefined : [4, 4]}
 className="transition-all duration-300"
 opacity={sameComp || active ? 1 : 0.55}
 />
 };
 })}
 {NODES.map((n) => {
 const c = compOf(n.id);
 const color = c > 0 ? COLORS[(c - 1) % COLORS.length] : "#fbbf24";
 const isCur = cur === n.id;
 const inFrontier = frontier.includes(n.id);
 return {
 <g key={n.id} onClick={() => goTo(step)}
 {isCur && <circle cx={n.x} cy={n.y} r={13} fill={color} />
 {inFrontier && !isCur && <circle cx={n.x} cy={n.y} r={13} fill="none" stroke={color} />
 <circle cx={n.x} cy={n.y} r={13} fill={color} />
 <text x={n.x} y={n.y + 4} textAnchor="middle">{n.label}</text>
 {c > 0 && <text x={n.x} y={n.y - 19} textAnchor="middle">c{c}</text>}
 </g>
 };
 })}
 </div>
 </div>
 </div>

```

```

 <span className="text-[10px] uppercase tracking
 {vars ? {
 ["v", "comps", "c", "i", "e", "comp", "parent
 vars[name] === undefined ? null : {
 <span key={name} className="text-[11px] fo
 py-1">
 {name} = {JSON.stringify(vars[name]).sl

 }
 }
 } : {

 открой панель Python – переменные <code cla
 erald-400">parent</code>, <code className="
 comps</code> покрасят этот граф сами

 })
</div>
</div>
);
};

```

```
export default GraphComponentsViz;
```

ФАЙЛ 45/111: src/components/GraphTraversalViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import { useState, useEffect, useMemo } from 'react';
import { useVizStepSync, vizArray, vizString } from '...
import { Play, Pause, SkipForward, Undo, RefreshCw } from

```

```

type Node = { id: string; label: string; x: number; y:
type Edge = { u: string; v: string };

```

```
const nodes: Node[] = [
```

```

function dfs(v: string) {
 vis.add(v);
 stack.push(v);
 steps.push({ type: 'visit', node: v, desc: `Зашли в узел ${v}`, visitedNodes: Array.from(vis) });

 for (const u of adj[v]) {
 if (!vis.has(u)) {
 steps.push({ type: 'process', node: v, desc: `Переходим к узлу ${u}`, visitedNodes: Array.from(vis) });
 dfs(u);
 }
 }

 stack.pop();
 steps.push({ type: 'backtrack', node: v, desc: `Выходим из узла ${v}`, visitedNodes: Array.from(vis) });
}

dfs(start);
return steps;
}

function generateBFSSteps(start: string) {
 const steps: Action[] = [];
 const vis = new Set<string>();
 const q: string[] = [];

 q.push(start);
 vis.add(start);
 steps.push({ type: 'visit', node: start, desc: `Начинаем с узла ${start}`, visitedNodes: Array.from(vis) });

 while (q.length > 0) {
 const v = q[0];
 steps.push({ type: 'process', node: v, desc: `Обрабатываем узел ${v}`, visitedNodes: Array.from(vis) });
 q.shift();
 }
}

```

```

useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
}, [mode]);

useEffect(() => {
 if (autoPlay) {
 const t = setInterval(() => {
 setStepIdx(p => {
 if (p >= steps.length - 1) {
 setAutoPlay(false);
 return p;
 }
 return p + 1;
 });
 }, 1200);
 return () => clearInterval(t);
 }
}, [autoPlay, steps.length]);

const step = steps[stepIdx] || steps[0];

const isVisited = (nodeId: string) => step.visitedNodes.includes(nodeId);
const isCurrent = (nodeId: string) => step.node === nodeId;

return (
 <div className="space-y-4">
 <div className="flex bg-slate-900 border border-slate-800 p-4">
 <button onClick={() => setMode("dfs")}
 className={`px-4 py-2 text-xs font-medium text-slate-400 hover:text-slate-300`} `>
 DFS (В глубину)
 </button>
 <button onClick={() => setMode("bfs")}
 className={`px-4 py-2 text-xs font-medium text-slate-400 hover:text-slate-300`} `>
 BFS (В ширину / Волна)
 </button>
 </div>
 </div>
);

```



```
 <RefreshCw strokeWidth={3} size={20} />
 </button>
 </div>

 { /* Description Log */ }
 <div className="flex-1 bg-slate-900/50 p-4 rounded-md shadow-sm
ow-hidden">
 <div className="text-[10px] text-slate-400">
 {steps.length}</div>
 <div className="text-sm text-slate-400">
 {step.desc}
 </div>
 </div>
 </div>
</div>
);
}
```

---

ФАЙЛ 46/111: src/components/HearViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useState, useCallback } from 'react';
import { Heart } from 'lucide-react';
import { useVizRuntime, vizArray, vizNumber } from '../hooks';
```

```
interface Patient {
 id: string;
 name: string;
 emoji: string;
 symptom: string;
 priority: number; // 1 to 99 (Severity)
 animState: 'in' | 'idle' | 'winner' | 'out';
}
```

```
// Max-heap helpers for Patients
```



```

 const posInLevel = index - (Math.pow(2, level) - 1);
 const count = Math.pow(2, level);
 const x = ((posInLevel + 0.5) / count) * 100;
 const y = 10 + level * 24;
 return { x, y };
}

// Priority presets for realistic ER scenarios
const PATIENT_PRESETS = [
 { name: 'Иван (Инфаркт)', symptom: ' Боли в сердце', emoji: '👤🩺' },
 { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '👤🩺' },
 { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '👤🩺' },
 { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '👤🩺' },
 { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '👤🩺' },
 { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '👤🩺' },
];

const INITIAL_PATIENTS: Patient[] = (() => {
 let arr: Patient[] = [];
 const start = [
 { name: 'Иван (Инфаркт)', symptom: ' Боли в сердце', emoji: '👤🩺' },
 { name: 'Маша (Перелом)', symptom: ' Открытый перелом', emoji: '👤🩺' },
 { name: 'Семён (Температура)', symptom: ' Жар 39.5', emoji: '👤🩺' },
 { name: 'Кот Барсик (Укус)', symptom: ' Укусил шмеля', emoji: '👤🩺' },
 { name: 'Оля (Порез)', symptom: ' Глубокий порез', emoji: '👤🩺' },
 { name: 'Пётр (Кашель)', symptom: ' Простуда', emoji: '👤🩺' },
];
 start.forEach((p, i) => {
 arr = heapInsert(arr, { id: `init${i}`, ...p, animS: 0 });
 });
 return arr;
})();

let uid = 300;

export const HeapViz: React.FC = () => {
 const [heap, setHeap] = useState<Patient[]>([]);
 const runtime = useVizRuntime();

```

```

 const finalSymptom = preset.symptom;

 const newPatient: Patient = {
 id: `p${uid++}`,
 name: finalName,
 emoji: finalEmoji,
 symptom: finalSymptom,
 priority,
 animState: 'in',
 };

 const nextHeap = heapInsert(heap, newPatient).map(x => {
 // Mark specifically this new patient as 'in' in the heap
 const target = nextHeap.find(x => x.priority === priority);
 const marked = nextHeap.map(x => x.id === (target?.id));

 setHeap(marked);
 addLog(` Поступил пациент: ${finalName} с тяжестью ${finalSymptom}`);

 setTimeout(() => {
 setHeap(nextHeap);
 setBusy(false);
 }, 500);
 }, [busy, heap]);

 const handleRandomInsert = () => {
 const preset = PATIENT_PRESETS[Math.floor(Math.random() * PATIENT_PRESETS.length)];
 const randomShift = Math.floor(Math.random() * 10);
 const prio = Math.max(10, Math.min(99, preset.priority + randomShift));
 insertPatient(preset.name, prio);
 };

 const handleCustomInsert = (e: React.FormEvent) => {
 e.preventDefault();
 if (!customName.trim()) return;
 insertPatient(customName, customPriority);
 setCustomName('');
 };

```

```

 setBusy(false);
 }, 600);
 }, 800);

 }, 350);
 }, 650);
}, [busy, heap]);

const reset = () => {
 setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 0 })));
 setLog([' База данных скорой помощи перезагружена.']);
 setHealing(null);
 setTimeout(() => setHeap(INITIAL_PATIENTS.map(x => ({ ...x, animStat: 0 }))), 1000);
};

// --- Render binary tree lines ---
const edges = visualHeap.flatMap((_, idx) => {
 const res: React.ReactElement[] = [];
 const p = nodePos(idx);
 const l = 2 * idx + 1;
 const r = 2 * idx + 2;
 if (l < visualHeap.length) {
 const lp = nodePos(l);
 res.push(
 <line key={`el${idx}`}
 x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
 x2={` ${lp.x}%`} y2={` ${lp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
 }
 if (r < visualHeap.length) {
 const rp = nodePos(r);
 res.push(
 <line key={`er${idx}`}
 x1={` ${p.x}%`} y1={` ${p.y + 4}%`}
 x2={` ${rp.x}%`} y2={` ${rp.y - 4}%`}
 stroke="#475569" strokeWidth="2"
 />
);
 }
});

```

```

<form onSubmit={handleCustomInsert} className="
 <input
 type="text"
 required
 value={customName}
 onChange={e => setCustomName(e.target.value)}
 placeholder="Имя пациента"
 className="flex-1 bg-slate-800 border border-emerald-500 transition-colors placeholder:
 />
 <div className="flex flex-col items-center min-w-[300px]">
 Приоритет
 <input
 type="range"
 min="10"
 max="99"
 value={customPriority}
 onChange={e => setCustomPriority(Number(e.target.value))}
 className="w-20 accent-emerald-500 cursor-pointer"
 />
 </div>
 <button
 type="submit"
 disabled={busy || !customName.trim() || heap.length === 0}
 className="bg-teal-600 hover:bg-teal-500 disabled:bg-slate-800 text-white px-4 rounded-xl shadow-lg active:scale-95 transition-all"
 >
 Везти
 </button>
</form>

{/* Забрать самого тяжелого */}
<button
 onClick={extractMax}
 disabled={busy || heap.length === 0}
 className="flex-1 min-w-[170px] bg-gradient-to-r from-teal-600 to-teal-500 disabled:cursor-not-allowed text-white font-mono transition-all cursor-pointer flex items-center justify-center"
 >
 Забрать
 </button>

```

```

 ${patient.animState === 'in' ? 'anim
 ${patient.animState === 'winner' ?
 ${patient.animState === 'out' ? 'an
 ${isCompilerNode ? 'ring-4 ring-amb
 ${isRoot ? 'bg-rose-950/80 border-r
 `}
 style={{
 left: `${pos.x}%`,
 top: `${pos.y}%`,
 width: isRoot ? 60 : 52,
 height: isRoot ? 60 : 52,
 }}
 >
 <span className="text-2xl leading-non
 <span className="text-[10px] font-mon
 {patient.priority}%

 {/* Tooltip */}
 <div className="absolute bottom-full
 <div className="bg-slate-950 border
0 shadow-xl">
 <div className="font-bold text-wh
 <div className="text-emerald-400"
 </div>
 </div>
 </div>
);
 }}}}
</div>

```

```

 <div className="w-full h-3 bg-gradient-to-r f
 </div>

```

```

{/* ПРАВАЯ КОЛОНКА: ОПЕРАЦИОННЫЙ СТОЛ */}
<div className="lg:col-span-4 bg-slate-900 round
 <div className="absolute top-4 left-4 text-[1
 Операционная койка

```

```
 <div className="bg-slate-900 border border-slate-800" data-bbox="173 101 1000 121">
 <div className="text-[10px] font-mono text-slate-400" data-bbox="206 126 1000 146">
 {log.map((entry, idx) => (
 <div data-bbox="240 173 1000 261">
 key={idx}
 className={`text-xs font-mono flex items-center justify-between gap-2`} data-bbox="273 221 1000 241">
 >
 {idx === 0 ?
 {entry}
 : {entry}
 </div>
 </div>
 </div>
))}
 </div>
 </div>
);
};
```

---

ФАЙЛ 47/111: src/components/hints/demoKit.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 10

---

```
import React, { createContext, useContext, useEffect, use
```

```
/** Общие примитивы для мини-визуализаций подсказок. */
```

```
export const W = 260;
```

```
export const H = 130;
```

```
/**
```

```
 * Общий множитель скорости. Кадры были слишком короткими
```

```
 * заметить, что именно переехало, и анимация читалась
```

```
 */
```

```
export const SPEED = 2.1;
```

```
/** Длительность переезда узлов. Должна быть заметно меньше
```

```
export const MOVE_MS = 800;
```

```
export const EASE = "cubic-bezier(.34,.01,.2,1)";
```

```
export const MOVE = `all ${MOVE_MS}ms ${EASE}`;
```

```

 <div className="text-[10px] text-slate-400 text-center"
 {caption}
 </div>
)}
 <div className="text-[9px] text-slate-600 text-center"
 {paused ? "пауза – курсор на карточке" : "наведение"}
 </div>
</div>
);
};

```

```

export const Node: React.FC<{
 x: number;
 y: number;
 r?: number;
 fill: string;
 label?: string | number;
 stroke?: string;
}> = ({ x, y, r = 13, fill, label, stroke }) => (
 <g style={{ transition: MOVE }}>
 <circle cx={x} cy={y} r={r} fill={fill} stroke={stroke} />
 {label !== undefined && (
 <text x={x} y={y} textAnchor="middle" dominantBaseline="middle">
 {label}
 </text>
)}
 </g>
);

```

```

export const Edge: React.FC<{
 a: [number, number];
 b: [number, number];
 color?: string;
 width?: number;
 dashed?: boolean;
}> = ({ a, b, color = C.edge, width = 2, dashed }) => (
 <line
 x1={a[0]}
 y1={a[1]}
 x2={b[0]}
 y2={b[1]}
 stroke={color}
 stroke-width={width}
 stroke-dasharray={dashed ? [4, 4]} : []
 />
);

```

```

 <Edge a={pos.k} b={pos.j} color={viaOpen ? C.done
 <text x={130} y={112} textAnchor="middle" fontSize=
 9
 </text>
 <text x={72} y={54} textAnchor="middle" fontSize=
 3
 </text>
 <text x={188} y={54} textAnchor="middle" fontSize=
 4
 </text>

 <Node x={pos.i[0]} y={pos.i[1]} label="i" fill={C
 <Node x={pos.k[0]} y={pos.k[1]} label="k" fill={v
 <Node x={pos.j[0]} y={pos.j[1]} label="j" fill={r
 </Svg>
);
};

```

/\*\* Компоненты связности: несколько «островов», которые

```

export const ComponentsDemo: React.FC = () => {
 const comps: [string, string][][] = [
 ["A", "B"], ["B", "C"],
 ["D", "E"],
 ["F", "G"], ["G", "H"], ["F", "H"],
];
 const pos: Record<string, [number, number]> = {
 A: [26, 40], B: [62, 92], C: [26, 92],
 D: [120, 40], E: [120, 92],
 F: [190, 34], G: [232, 78], H: [178, 96],
 };
 const colors = [C.active, C.warn, C.done];
 const step = useLoop(comps.length + 1, 900);
 const compOf = (v: string) => comps.findIndex((e) =>

 return (
 <Svg caption={step === 0 ? "Граф распался на острова"
 {comps.flatMap((edges, ci) =>

```



```

 fontSize={10}
 fontWeight="700"
 fill={C.warn}
 >
 {w}
 </text>
)}
 </g>
);
}}}
{Object.entries(pos).map(([k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k} fill={C.idl
)})}
</Svg>
);
};

```

/\*\* Сжатие координат: разрежённые числа → плотные индексы

```

export const CoordCompressDemo: React.FC = () => {
 const raw = [1000, 5, 74000, 5, 300];
 const sorted = [5, 300, 1000, 74000];
 const step = useLoop(3, 1200);
 const boxW = 46;
 const startX = (260 - raw.length * (boxW + 4)) / 2;

 return (
 <Svg
 caption={
 step === 0
 ? "Исходные координаты – огромные и с дырами"
 : step === 1
 ? "Сортируем уникальные значения"
 : "Каждое число заменяем его номером → плотн
 }
 >
 {raw.map((v, i) => {
 <g key={i}>
 <rect x={startX + i * (boxW + 4)} y={20} width

```

```

const step = useLoop(frames.length, ms);
const f = frames[step];
const color = (id: string) =>
 id === "x" ? "#6366f1" : id === "p" ? "#0ea5e9" : id === "r" ? "#f08080" : "#f08080";

return (
 <Svg caption={captions[step]}>
 {f.edges.map(([a, b], i) => (
 <line
 key={i}
 x1={f.pos[a][0]}
 y1={f.pos[a][1]}
 x2={f.pos[b][0]}
 y2={f.pos[b][1]}
 stroke={C.edge}
 strokeWidth={1.8}
 style={{ transition: MOVE }}
 />
))}
 {Object.entries(f.pos).map(([id, [x, y]]) => (
 <g key={id} style={{ transform: `translate(${x}, ${y})` }}>
 <circle r={13} fill={color(id)} stroke={C.idle} />
 <text textAnchor="middle" dominantBaseline="middle">
 {id}
 </text>
 </g>
))}
 </Svg>
);
};

```

```

/** Zig – один поворот, когда родитель уже корень. */
export const ZigDemo: React.FC = () => (
 <SplayFrames
 captions={["p – корень, x под ним", "Один поворот: x поворачивается к p"]}
 frames={[
 { pos: { p: [130, 32], x: [78, 92] }, edges: [{"p": "x"}] },
 { pos: { x: [130, 32], p: [182, 92] }, edges: [{"x": "p"}] }
]}
 />
);

```

```

 A: [32, 65], B: [92, 30], C: [92, 100], D: [152, 65],
};
```

```
const GRAPH_EDGES: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "D"], ["C", "D"], ["D",
];
```

```
export const BfsDemo: React.FC = () => {
 const order = ["A"], ["B", "C"], ["D"], ["E", "F"]];
 const step = useLoop(order.length + 1, 850);
 const visited = new Set(order.slice(0, step).flat());
 const wave = step > 0 && step <= order.length ? order
 return (
 <Svg caption={`Уровень ${Math.max(0, Math.min(step,
 {GRAPH_EDGES.map(([u, v], i) => {
 <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_POS[v]}
)}}
 {Object.entries(GRAPH_POS).map(([k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k}
 fill={wave.includes(k) ? C.active : visited.h
 stroke={wave.includes(k) ? "#a5b4fc" : undefi
)}}
)}}
 </Svg>
);
};
```

```
export const DfsDemo: React.FC = () => {
 const path = ["A", "B", "D", "E", "F", "C"];
 const step = useLoop(path.length + 1, 750);
 const visited = path.slice(0, step);
 const head = visited[visited.length - 1];
 return (
 <Svg caption={head ? `Идём вглубь: ${visited.join("
 {GRAPH_EDGES.map(([u, v], i) => {
 const iu = visited.indexOf(u);
 const iv = visited.indexOf(v);
 const onPath = iu >= 0 && iv >= 0 && Math.abs(i
 return <Edge key={i} a={GRAPH_POS[u]} b={GRAPH_P
)}}
)}}
)}
};
```

```

 <text x={224} y={74} textAnchor="middle" fontSize=
 </Svg>
);
};

export const RelaxDemo: React.FC = () => {
 const step = useLoop(3, 1100);
 const dV = [12, 12, 7][step];
 const improved = step === 2;
 return (
 <Svg caption={improved ? "12 > 3 + 4 → пишем 7. Путь"
 <Edge a={[60, 70]} b={[200, 70]} color={improved
 <text x={130} y={54} textAnchor="middle" fontSize=
 <Node x={60} y={70} r={20} fill={C.active} label=
 <Node x={200} y={70} r={20} fill={improved ? C.do
 <text x={60} y={106} textAnchor="middle" fontSize=
 <text x={200} y={106} textAnchor="middle" fontSize=
 <text x={130} y={22} textAnchor="middle" fontSize=
 </Svg>
);
};

export const BridgeDemo: React.FC = () => {
 const step = useLoop(2, 1300);
 const pos: Record<string, [number, number]> = {
 A: [40, 40], B: [40, 95], C: [95, 68], D: [170, 68]
 };
 const edges: [string, string][] = [
 ["A", "B"], ["A", "C"], ["B", "C"], ["C", "D"], ["D
];
 const cut = step === 1;
 return (
 <Svg caption={cut ? "Убрали C-D → граф развалился н
 {edges.map(([u, v], i) => {
 const isBridge = u === "C" && v === "D";
 if (isBridge && cut) return null;
 return <Edge key={i} a={pos[u]} b={pos[v]} colo
 })}
)}
);
};

```

```

const step = useLoop(chosen.length + 1, 800);
const taken = new Set(chosen.slice(0, step));
return (
 <Svg caption={`Жадно берём самые лёгкие рёбра без ц
 {edges.map((e, i) => {
 const on = taken.has(`${e.u}-${e.v}`);
 return (
 <g key={i}>
 <Edge a={pos[e.u]} b={pos[e.v]} color={on ?
 <text x={({pos[e.u][0] + pos[e.v][0]} / 2} y
 textAnchor="middle" fontSize={9} fontWeigh
 </g>
);
 })}
 {Object.entries(pos).map(([k, [x, y]]) => (<Node
 </Svg>
);
};

export const SccDemo: React.FC = () => {
 const pos: Record<string, [number, number]> = {
 A: [45, 35], B: [110, 35], C: [77, 95], D: [185, 45]
 };
 const edges: [string, string][] = [["A", "B"], ["B",
 const step = useLoop(2, 1400);
 const scc = ["A", "B", "C"];
 const inScc = (u: string, v: string) => step === 1 &&
 return (
 <Svg caption={step === 1 ? "{A,B,C} – цикл: из кажд
 {edges.map(([u, v], i) => {
 <Edge key={i} a={pos[u]} b={pos[v]} color={inSc
 })}
 {Object.entries(pos).map(([k, [x, y]]) => {
 <Node key={k} x={x} y={y} label={k} fill={step
 })}
 </Svg>
);
};
};

```

```

 fill={i === items.length - 1 ? (popping ? C
 <text x={130} y={111 - i * 26} textAnchor="mi
 </g>
)))
 <text x={130} y={16} textAnchor="middle" fontSize=
</Svg>
);
};

```

```

export const QueueDemo: React.FC = () => {
 const states = [["A", "B"], ["A", "B", "C"], ["B", "C
 const step = useLoop(states.length, 800);
 const items = states[step];
 return (
 <Svg caption="pop_front слева, push_back справа – к
 <defs>
 <marker id="mdArrow" markerWidth="6" markerHeigh
 <path d="M0,0 L6,3 L0,6 Z" fill={C.done} />
 </marker>
 </defs>
 <text x={12} y={44} fontSize={9} fill={C.dim}>ВЫХ
 <text x={206} y={44} fontSize={9} fill={C.dim}>ВХ
 {items.map((v, i) => (
 <g key={v} style={{ transition: "all .3s" }}>
 <rect x={30 + i * 54} y={56} width={46} heigh
 <text x={53 + i * 54} y={71} textAnchor="midd
 >
 </g>
)))
 <line x1={24} y1={71} x2={10} y2={71} stroke={C.d
 </Svg>
);
};

```

```

export const DsuDemo: React.FC = () => {
 const parents = [[0, 1, 2, 3], [0, 0, 2, 3], [0, 0, 2
 const step = useLoop(parents.length, 950);
 const p = parents[step];

```



```

 о префиксные суммы P">
 <text x={4} y={18} fontSize={8} fill={C.dim}>a</text>
 {a.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 39} y={24} width={34} height={12}
 fill={show && i >= l && i <= r ? C.active : C.dim} />
 <text x={37 + i * 39} y={36} textAnchor="middle">{v}</text>
 </g>
))}
 <text x={4} y={72} fontSize={8} fill={C.dim}>P</text>
 {pref.map((v, i) => (
 <g key={i}>
 <rect x={20 + i * 33} y={78} width={29} height={12}
 fill={show && i === l ? C.bad : show && i === r ? C.dim : C.idle}
 stroke={C.idleStroke} style={{ transition: "stroke 0.2s" }} />
 <text x={34 + i * 33} y={90} textAnchor="middle">{v}</text>
 </g>
))}
 </Svg>
);
};

```

```

export const SparseTableDemo: React.FC = () => {
 const step = useLoop(3, 1000);
 const len = [1, 2, 4][step];
 const count = 8 - len + 1;
 return (
 <Svg caption={`Уровень j=${step}: готовые ответы для`}>
 {Array.from({ length: 8 }).map((_, i) => (
 <g key={i}>
 <rect x={12 + i * 30} y={18} width={26} height={12}
 fill={C.dim} />
 <text x={25 + i * 30} y={29} textAnchor="middle">{a[i]}</text>
 </g>
))}
 {Array.from({ length: count }).map((_, i) => (
 <rect key={i} x={12 + i * 30} y={52 + (i % 4) * 12}

```



```
export const PrefixFunctionDemo: React.FC = () => {
 const s = "abacaba";
 const pi = [0, 0, 1, 0, 1, 2, 3];
 const step = useLoop(s.length + 1, 700);
 const i = Math.max(0, step - 1);
 const len = step > 0 ? pi[i] : 0;
 return (
 <Svg caption={step > 0 ? `π[${i}] = ${len}: префикс`
 {s.split("").map((ch, idx) => (
 <g key={idx}>
 <rect x={18 + idx * 33} y={26} width={28} height={20}
 fill={step > 0 && (idx < len || (idx > i - 1)) ? C.fill : C.idle}
 style={{ transition: "fill .25s" }} />
 <text x={32 + idx * 33} y={40} textAnchor="middle">{ch}</text>
 <text x={32 + idx * 33} y={70} textAnchor="middle">{idx}</text>
 </g>
))}
 <text x={4} y={40} fontSize={8} fill={C.dim}>s</text>
 <text x={4} y={70} fontSize={8} fill={C.dim}>π</text>
 <text x={130} y={106} textAnchor="middle" fontSize={12}>
 π[i] – длина самого длинного «префикс = суффикс»
 </text>
 </Svg>
);
};
```

```
export const DpDemo: React.FC = () => {
 const rows = 3, cols = 5;
 const step = useLoop(rows * cols + 1, 320);
 return (
 <Svg caption={`Заполняем таблицу подзадач: готово $
 {Array.from({ length: rows }).map((_, r) =>
 Array.from({ length: cols }).map((_, c) => {
 const idx = r * cols + c;
 const filled = idx < step;

```

```
| "bfs" | "dfs" | "heap" | "stack" | "queue" | "dsu"
| "suffix-link" | "toposort" | "relax" | "prefix-sum"
| "segment-tree" | "bridge" | "articulation" | "mst"
| "prefix-function" | "dp" | "floyd" | "components" |
| "zig" | "zig-zig" | "zig-zag";
```

```
const REGISTRY: Record<DemoKind, React.FC> = {
 bfs: BfsDemo,
 dfs: DfsDemo,
 heap: HeapDemo,
 stack: StackDemo,
 queue: QueueDemo,
 dsu: DsuDemo,
 trie: TrieDemo,
 "trie-bfs": TrieBfsDemo,
 "suffix-link": SuffixLinkDemo,
 toposort: ToposortDemo,
 relax: RelaxDemo,
 "prefix-sum": PrefixSumDemo,
 "sparse-table": SparseTableDemo,
 "segment-tree": SegmentTreeDemo,
 bridge: BridgeDemo,
 articulation: ArticulationDemo,
 mst: MstDemo,
 amortized: AmortizedDemo,
 np: NpDemo,
 scc: SccDemo,
 "prefix-function": PrefixFunctionDemo,
 dp: DpDemo,
 floyd: FloydDemo,
 components: ComponentsDemo,
 graph: GraphBasicsDemo,
 "coord-compress": CoordCompressDemo,
 zig: ZigDemo,
 "zig-zig": ZigZigDemo,
 "zig-zag": ZigZagDemo,
};
```

```

/** Модальное окно с полноценным симулятором – вызывает
export const SimulatorModal: React.FC<Props> = ({ vizId
 useEffect(() => {
 if (!vizId) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 document.body.style.overflow = "hidden";
 return () => {
 window.removeEventListener("keydown", onKey);
 document.body.style.overflow = "";
 };
 }, [vizId, onClose]);

 const viz = getViz(vizId ?? undefined);
 if (!vizId || !viz) return null;
 const Component = viz.Component;

 return createPortal(
 <div className="fixed inset-0 z-[110] flex items-en
 <div className="absolute inset-0 bg-slate-950/80
 <div className="relative w-full sm:max-w-5xl max-l
 ded-2xl shadow-2xl flex flex-col">
 <div className="flex items-center gap-3 px-4 py
 <div className="min-w-0 flex-1">
 <h3 className="font-bold text-white text-sm
 {viz.hint && <p className="text-[11px] text
 <VizLiveStatus demoTitle={viz.title} classN
 </div>
 <button
 onClick={onClose}
 className="p-2 rounded-lg bg-slate-800 hove
 aria-label="Заккрыть симулятор"
 >
 <X className="w-5 h-5" />
 </button>
 </div>
 </div>
```

```
const CARD_W = 320;
const GAP = 10;
```

```
export const TermPopover: React.FC<Props> = ({ anchor, onClose }) => {
 const cardRef = useRef<HTMLDivElement>(null);
 const [pos, setPos] = useState<{ top: number; left: number }>(null);

 useEffect(() => {
 if (!anchor) {
 setPos(null);
 return;
 }
 const vw = window.innerWidth;
 const vh = window.innerHeight;
 const width = Math.min(CARD_W, vw - 16);
 const height = cardRef.current?.offsetHeight ?? 300;

 let left = anchor.rect.left + anchor.rect.width / 2;
 left = Math.max(8, Math.min(left, vw - width - 8));

 const below = anchor.rect.top < height + GAP + 8;
 const top = below ? anchor.rect.bottom + GAP : anchor.rect.top - height - GAP;

 setPos({ top: Math.max(8, Math.min(top, vh - height - GAP)), left }, [anchor]);

 useEffect(() => {
 if (!anchor) return;
 const onKey = (e: KeyboardEvent) => {
 if (e.key === "Escape") onClose();
 };
 window.addEventListener("keydown", onKey);
 window.addEventListener("scroll", onKey, true);
 return () => {
 window.removeEventListener("keydown", onKey);
 window.removeEventListener("scroll", onKey, true);
 };
 }, [anchor, onClose]);
 }, [anchor, onClose]);
};
```

```
 {entry.formal && {
 <p className="text-[11.5px] leading-relaxed" >
 {entry.formal}
 </p>
 }}

 {entry.complexity && {
 <div className="text-[11px] font-mono text-slate-500" >
 Сложность: {entry.complexity}
 </div>
 }}

 {viz && simulatorId && {
 <button
 onClick={() => onOpenSimulator(simulatorId)}
 className="w-full flex items-center justify-center text-slate-500 rounded-lg py-2 transition-colors"
 >
 <Play className="w-3.5 h-3.5" /> Показать
 </button>
 }}
 </div>
 </div>,
 document.body
);
};
```

---

ФАЙЛ 54/111: src/components/JohnsonViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import { useState } from 'react';
import { RotateCcw, SkipForward, Undo } from 'lucide-react';
import { useVizStepSync, vizString } from '../data/vizSteps';
```

```

const getEdgeColorClass = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "opacity-0";
 if (step === 1) return "stroke-amber-500 stroke-width-2px";
 if (step === 2) return "stroke-pink-500 stroke-width-2px";
 return "stroke-slate-700 stroke-dasharray-4 4 stroke-width-2px";
 }

 if (e.id === 'AB') {
 if (step === 4) return "stroke-amber-400 stroke-width-2px";
 if (step > 4) return "stroke-emerald-500";
 }
 if (e.id === 'BC') {
 if (step === 5) return "stroke-amber-400 stroke-width-2px";
 if (step > 5) return "stroke-emerald-500";
 }
 if (e.id === 'CA') {
 if (step === 6) return "stroke-amber-400 stroke-width-2px";
 if (step > 6) return "stroke-emerald-500";
 }

 if (step >= 4) return "stroke-slate-600"; // отсюда и далее
 return "stroke-slate-500";
};

const getMarkerUrl = (e: any) => {
 if (e.u === 'S') {
 if (step === 0 || step === 7) return "";
 if (step === 1) return "url(#arrow-amber)";
 if (step === 2) return "url(#arrow-pink)";
 return "url(#arrow-slate-dim)";
 }
 if (e.id === 'AB' && step === 4) return "url(#arrow-amber)";
 if (e.id === 'AB' && step > 4) return "url(#arrow-emerald)";
 if (e.id === 'BC' && step === 5) return "url(#arrow-amber)";
 if (e.id === 'BC' && step > 5) return "url(#arrow-emerald)";
 if (e.id === 'CA' && step === 6) return "url(#arrow-amber)";
 if (e.id === 'CA' && step > 6) return "url(#arrow-emerald)";

```

```

 if (e.id === 'CA' && step > 6) return true;
 return false;
 };

 const getDesc = () => {
 switch (step) {
 case 0: return (
 <div>
 Исходный граф. Имеется ребро
 Если мы запустим быстрый алгоритм Д
 </div>
);
 case 1: return (
 <div>
 Шаг 1: Точка отсчета.

 Добавляем фиктивную вершину S. Соед
ми.
 Это наша база для расчета потенциал
 </div>
);
 case 2: return (
 <div>
 Шаг 2: Ищем потенциалы $h(v)$.
 Запускаем медленный, но мощный алгор
ных вершин:
 <span className="text-pink-300 ml-1
 </div>
);
 case 3: return (
 <div>
 Шаг 3: Магическая формула.
 Всем ребрам графа назначаем новый в
 <div className="text-center font-mo
 Это повышает или понижает вес ребра
 </div>
);
 case 4: return (
 <div>

```

```

<div className="flex flex-col md:flex-row gap-4" style={
 <div className="bg-[#0f172a] flex-1 min-h-[300px] p-4">
 <svg width="400" height="300" class="w-full h-full">
 <defs>
 <marker id="arrow-slate" viewBox="0 0 10 5" orient="vertical"
start-reverse">
 <path d="M 0 0 L 10 5 L 10 0" style="stroke: slategray; stroke-width: 2px;"/>
 </marker>
 <marker id="arrow-slate-dim" viewBox="0 0 10 5" orient="vertical"
uto-start-reverse">
 <path d="M 0 0 L 10 5 L 10 0" style="stroke: slategray; stroke-width: 1px;"/>
 </marker>
 <marker id="arrow-amber" viewBox="0 0 10 5" orient="vertical"
start-reverse">
 <path d="M 0 0 L 10 5 L 10 0" style="stroke: #f1c40f; stroke-width: 2px;"/>
 </marker>
 <marker id="arrow-pink" viewBox="0 0 10 5" orient="vertical"
tart-reverse">
 <path d="M 0 0 L 10 5 L 10 0" style="stroke: #f08080; stroke-width: 2px;"/>
 </marker>
 <marker id="arrow-emerald" viewBox="0 0 10 5" orient="vertical"
o-start-reverse">
 <path d="M 0 0 L 10 5 L 10 0" style="stroke: #27ae60; stroke-width: 2px;"/>
 </marker>
 </defs>

 {/* Edges */}
 {edges.map((e, i) => {
 const u = nodes.find(n => n.id === e.u);
 const v = nodes.find(n => n.id === e.v);

 const isS = e.u === 'S';
 if (isS && (step === 0 || step === 1)) {
 const midX = (u.x + v.x) / 2;
 const midY = (u.y + v.y) / 2;

```





```

 Play,
 Pause,
 RotateCcw,
 Layers,
} from "lucide-react";

interface GNode {
 id: number;
 x: number;
 y: number;
 label: string;
}

interface GEdge {
 u: number;
 v: number;
}

const initialNodes: GNode[] = [
 { id: 0, x: 150, y: 100, label: "0" },
 { id: 1, x: 300, y: 100, label: "1" },
 { id: 2, x: 225, y: 220, label: "2" }, // 0->1->2->0
 { id: 3, x: 450, y: 160, label: "3" },
 { id: 4, x: 600, y: 160, label: "4" }, // 3 <-> 4 (Se
];

const initialEdges: GEdge[] = [
 { u: 0, v: 1 },
 { u: 1, v: 2 },
 { u: 2, v: 0 },
 { u: 2, v: 3 }, // Connection edge
 { u: 3, v: 4 },
 { u: 4, v: 3 },
];

interface AlgoStep {
 phase: "init" | "dfs1" | "transpose" | "dfs2" | "fini
 desc: string;

```

```

 phase,
 desc,
 visited: new Set(visited),
 currentNode,
 order: [...order],
 transposed,
 sccMap: { ...sccMap },
 currentSccId,
 activeEdges: transposed
 ? initialEdges.map((e) => ({ u: e.v, v: e.u }
 : [...initialEdges],
));
 });
};

recordStep(
 "init",
 "Инициализация алгоритма Косарайю. Начинаем первый",
 null,
 false,
 -1,
);

// Adj list
const adj: number[][] = Array.from({ length: 5 }, () => []);
initialEdges.forEach((e) => adj[e.u].push(e.v));

// DFS 1: post-order list
const dfs1 = (u: number) => {
 visited.add(u);
 recordStep(
 "dfs1",
 `DFS 1: зашли в вершину ${u}, помечаем посещение`,
 u,
 false,
 -1,
);
 for (const to of adj[u]) {

```

```

let sccId = 0;

const dfs2 = (u: number, currentScc: number) => {
 visited.add(u);
 sccMap[u] = currentScc;
 recordStep(
 "dfs2",
 `DFS 2: заходим в ${u} на транспонированном графе`,
 u,
 true,
 currentScc,
);

 for (const to of adjT[u]) {
 if (!visited.has(to)) {
 dfs2(to, currentScc);
 }
 }
};

// Run DFS2 using the order (reversed from back to front)
for (let i = order.length - 1; i >= 0; i--) {
 const u = order[i];
 if (!visited.has(u)) {
 sccId++;
 recordStep(
 "dfs2",
 `DFS 2: берем следующую непосещенную из стека`,
 u,
 true,
 sccId,
);
 dfs2(u, sccId);
 } else {
 recordStep(
 "dfs2",
 `DFS 2: вершина ${u} из стека уже покрашена в ${sccMap[u]}`,
 u,
 false,
 sccMap[u],
);
 }
}

```

```

 }
 return () => clearInterval(timer);
 }, [isPlaying, currentStepIdx, steps.length]);

const togglePlay = () => setIsPlaying(!isPlaying);
const reset = () => {
 setIsPlaying(false);
 setCurrentStepIdx(0);
};

const getSccColor = (id: number) => {
 if (id === 1) return "fill-emerald-600 stroke-emera
 if (id === 2) return "fill-indigo-600 stroke-indigo
 return "fill-slate-800 stroke-slate-500";
};

return (
 <div className="bg-slate-900 rounded-xl border bord
 <div className="bg-slate-800 p-4 border-b border-
 <h3 className="font-bold text-white flex items-
 <Layers className="w-5 h-5 text-emerald-400"
 Визуализатор Алгоритма Косарайю (Поиск SCC)
 </h3>

 <div className="flex items-center gap-2 bg-slate-
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-
 d-500 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

```

```

 <path d="M 0 1 L 10 5 L 0 9 z" fill="#1f77b4" stroke="#1f77b4" stroke-width="1" />
 </marker>
 <marker
 id="arrow-transposed"
 viewBox="0 0 10 10"
 refX="22"
 refY="5"
 markerWidth="6"
 markerHeight="6"
 orient="auto-start-reverse"
 >
 <path d="M 0 1 L 10 5 L 0 9 z" fill="#666666" stroke="#666666" stroke-width="1" />
 </marker>
 </defs>

```

```

 { /* Edges */ }
 {initialEdges.map((edge, idx) => {
 const u = initialNodes.find((n) => n.id === edge.u);
 const v = initialNodes.find((n) => n.id === edge.v);

 // If transposed, draw reversed coordinates
 const x1 = step.transposed ? v.x : u.x;
 const y1 = step.transposed ? v.y : u.y;
 const x2 = step.transposed ? u.x : v.x;
 const y2 = step.transposed ? u.y : v.y;

 const isSccEdge =
 step.sccMap[edge.u] &&
 step.sccMap[edge.v] &&
 step.sccMap[edge.u] === step.sccMap[edge.v];
 let stroke = "#475569";
 let markerId = "arrow";

 if (step.transposed) {
 stroke = isSccEdge ? "#818cf8" : "#4f46e5";
 markerId = "arrow-transposed";
 } else {
 if (

```

```

 if (sccId) {
 classes = getSccColor(sccId);
 } else if (isCurrent) {
 classes = "fill-amber-600 stroke-amber-600";
 } else if (isVisited && step.phase === "done") {
 classes = "fill-emerald-700 stroke-emerald-700";
 }

 return (
 <g key={node.id}>
 <circle
 cx={node.x}
 cy={node.y}
 r="20"
 className={`transition-all duration-300`}
 strokeWidth="3"
 />
 <text
 x={node.x}
 y={node.y}
 dy=".3em"
 textAnchor="middle"
 fill="white"
 fontSize="13px"
 fontWeight="bold"
 className="select-none"
 >
 {node.label}
 </text>

 {sccId && (
 <text
 x={node.x}
 y={node.y - 28}
 textAnchor="middle"
 fill="#10b981"
 fontSize="10px"
 fontWeight="bold"
 >
 {sccId}
 </text>
)}
 </g>
);
}

```

```

 <span className="text-slate-600 text-
) : {
 [...step.order].reverse().map(({val, i
 <div
 key={idx}
 className="bg-indigo-950 border b
ex items-center gap-1"
 >
 {val}
 </div>
))
 })
</div>
</div>

{/* List of actions log */}
<div>
 <span className="text-[10px] text-slate-5
 ЛОГ ОПЕРАЦИЙ:

 <div className="space-y-1.5 max-h-[150px]
 {steps
 .slice(0, currentStepIdx + 1)
 .reverse()
 .slice(0, 5)
 .map(({s, idx}) => {
 <div
 key={idx}
 className={`text-[11px] p-1.5 roun
 >
 {s.desc}
 </div>
 })}
 </div>
 </div>
</div>

<div className="mt-4 pt-3 border-t border-sla

```



```
interface Vertex {
 id: number;
 label: string;
 x: number;
 y: number;
 color: string; // 'none', 'red', 'blue', 'purple', 'm
}

interface Edge {
 id: string;
 u: number;
 v: number;
 weight: number;
 status: 'pending' | 'inspecting' | 'included' | 'reje
 color: string;
}

interface StepLog {
 title: string;
 description: string;
 type: 'info' | 'success' | 'warning' | 'error';
}

// 9 Vertices layout
const initialVertices: Vertex[] = [
 { id: 0, label: 'A', x: 20, y: 20, color: 'none' },
 { id: 1, label: 'B', x: 50, y: 13, color: 'none' },
 { id: 2, label: 'C', x: 80, y: 20, color: 'none' },
 { id: 3, label: 'D', x: 18, y: 50, color: 'none' },
 { id: 4, label: 'E', x: 50, y: 50, color: 'none' },
 { id: 5, label: 'F', x: 82, y: 50, color: 'none' },
 { id: 6, label: 'G', x: 20, y: 80, color: 'none' },
 { id: 7, label: 'H', x: 50, y: 87, color: 'none' },
 { id: 8, label: 'I', x: 80, y: 80, color: 'none' },
];

// 12 Edges connecting the 9 vertices
const initialEdges: Edge[] = [
```

```

 description: 'Оно соединяет две бесцветные верши
 type: 'info'
 }, ...prev]);
},
// Step 2: Include A-B
() => {
 setVertices(prev => prev.map(v => v.id === 0 || v
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро A-B в осто́ве',
 description: 'Вершины A and B окрашены в красны
 type: 'success'
 }, ...prev]));
},
// Step 3: Inspect D-E (3)
() => {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Шаг 2: Берем второе ребро D-E (вес 3)',
 description: 'Оно соединяет две другие бесцветн
 type: 'info'
 }, ...prev]));
},
// Step 4: Include D-E
() => {
 setVertices(prev => prev.map(v => v.id === 3 || v
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро D-E в осто́ве',
 description: 'Вершины D и E окрашены в синий цв
 type: 'success'
 }, ...prev]));
},
// Step 5: Inspect B-C (4)
() => {
 setEdges(prev => prev.map(e => e.id === '1-2' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Берем ребро B-C (вес 4)',

```

```

 title: 'Шаг 5: Берем ребро В-Е (вес 6)',
 description: 'Н0: это ребро соединяет вершины В
 type: 'warning'
 }, ...prev]));
},
// Step 10: Reject В-Е
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Мы выбрасываем ребро В-Е. Иначе п
 type: 'error'
 }, ...prev]));
},
// Step 11: Inspect E-F (7)
() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Шаг 6: Берем ребро E-F (вес 7)',
 description: 'Оно соединяет фиолетовую вершину
 type: 'info'
 }, ...prev]));
},
// Step 12: Include E-F
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро E-F в осто́ве',
 description: 'Вершина F окрашена в фиолетовый ц
 type: 'success'
 }, ...prev]));
},
// Step 13: Inspect D-G (8)
() => {
 setEdges(prev => prev.map(e => e.id === '3-6' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Берем ребро D-G (вес 8)',

```

```

 type: 'info'
 }, ...prev]);
},
// Step 18: Include G-H
() => {
 setVertices(prev => prev.map(v => v.id === 7 ? {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Успех: Ребро G-H в осто́ве',
 description: 'Вершина H перекрашена в фиолетовый',
 type: 'success'
 }, ...prev]));
},
// Step 19: Inspect E-H (11) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Берем ребро E-H (вес 11)',
 description: 'Оба конца E и H уже фиолетовые. С',
 type: 'warning'
 }, ...prev]));
},
// Step 20: Reject E-H
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Отказ: Цикл обнаружен!',
 description: 'Выбрасываем ребро E-H.',
 type: 'error'
 }, ...prev]));
},
// Step 21: Inspect H-I (12)
() => {
 setEdges(prev => prev.map(e => e.id === '7-8' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Берем ребро H-I (вес 12)',
 description: 'Соединяет фиолетовую H и последнюю',
 type: 'info'
 }, ...prev]));
},

```

```

// Step 3: Include D-E, add D's edges to heap
() => {
 setVertices(prev => prev.map(v => v.id === 3 ? {
 setEdges(prev => prev.map(e => e.id === '3-4' ? {
 ..e, status: 'in_heap' } : e));
 setHeapQueue(['A-D (5)', 'B-E (6)', 'E-F (7)', 'D-E (8)']);
 setLogs(prev => [{
 title: 'Успех: Плесень захватила вершину D',
 description: 'В кучу добавлены новые пути из D: A-D (5), B-E (6), E-F (7), D-E (8)',
 type: 'success'
 }, ...prev]));
},
// Step 4: Pop A-D (5)
() => {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 setLogs(prev => [{
 title: 'Шаг 3: Куча выдает ребро A-D (вес 5)',
 description: 'Плесень расширяется в сторону вершины D',
 type: 'info'
 }, ...prev]));
},
// Step 5: Include A-D, add A's edges to heap
() => {
 setVertices(prev => prev.map(v => v.id === 0 ? {
 setEdges(prev => prev.map(e => e.id === '0-3' ? {
 ..e, status: 'in_heap' } : e));
 setHeapQueue(['A-B (2)', 'B-E (6)', 'E-F (7)', 'D-E (8)']);
 setLogs(prev => [{
 title: 'Успех: Плесень поглотила вершину A',
 description: 'Новое ребро A-B(2) добавлено в кучу',
 type: 'success'
 }, ...prev]));
},
// Step 6: Pop A-B (2)
() => {
 setEdges(prev => prev.map(e => e.id === '0-1' ? {
 setLogs(prev => [{
 title: 'Шаг 4: Куча выдает ребро A-B (вес 2)',

```

```

() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setLogs(prev => [{
 title: 'Шаг 6: Куча выдает ребро В-Е (вес 6)',
 description: 'ВНИМАНИЕ! Плесень проверяет верши
 type: 'warning'
 }, ...prev]));
},
// Step 11: Reject B-E
() => {
 setEdges(prev => prev.map(e => e.id === '1-4' ? {
 setHeapQueue(['E-F (7)', 'D-G (8)', 'C-F (9)', 'E
 setLogs(prev => [{
 title: 'Отказ: Игнорируем внутреннее ребро',
 description: 'Ребро В-Е отброшено. Плесень не р
 type: 'error'
 }, ...prev]));
},
// Step 12: Pop E-F (7)
() => {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 setLogs(prev => [{
 title: 'Шаг 7: Куча выдает ребро E-F (вес 7)',
 description: 'Плесень из центра Токио (Е) тянет
 type: 'info'
 }, ...prev]));
},
// Step 13: Include E-F, add F's edges
() => {
 setVertices(prev => prev.map(v => v.id === 5 ? {
 setEdges(prev => prev.map(e => e.id === '4-5' ? {
 eap' } : e));
 setHeapQueue(['D-G (8)', 'C-F (9 - цикл)', 'E-H (
 setLogs(prev => [{
 title: 'Успех: Вершина F захвачена',
 description: 'Ребро F-I(13) добавлено в кучу. Р
 type: 'success'
 }, ...prev]));
},

```

```

 type: 'error'
 }, ...prev]);
},
// Step 18: Pop G-H (10)
() => {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 setLogs(prev => [{
 title: 'Шаг 10: Куча выдает ребро G-H (вес 10)',
 description: 'Плесень ползет к вершине H.',
 type: 'info'
 }, ...prev]));
},
// Step 19: Include G-H, add H's edges
() => {
 setVertices(prev => prev.map(v => v.id === 7 ? {
 setEdges(prev => prev.map(e => e.id === '6-7' ? {
 eap' } : e));
 setHeapQueue(['E-H (11 - цикл)', 'H-I (12)', 'F-I
 setLogs(prev => [{
 title: 'Успех: Вершина H захвачена',
 description: 'В кучу добавлено H-I(12).',
 type: 'success'
 }, ...prev]));
},
// Step 20: Pop E-H (11) - Cycle!
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setLogs(prev => [{
 title: 'Шаг 11: Куча выдает E-H (вес 11)',
 description: 'Обе вершины E и H уже в плесени.'
 type: 'warning'
 }, ...prev]));
},
// Step 21: Reject E-H
() => {
 setEdges(prev => prev.map(e => e.id === '4-7' ? {
 setHeapQueue(['H-I (12)', 'F-I (13)']));
 setLogs(prev => [{

```

```

// Node 5(F) -> E-F(7)
// Node 6(G) -> D-G(8)
// Node 7(H) -> G-H(10)
// Node 8(I) -> H-I(12)
setEdges(prev => prev.map(e =>
 ['0-1', '3-4', '1-2', '4-5', '3-6', '6-7', '7-8'
]));
setLogs(prev => [{
 title: 'Первая пятилетка: Каждая деревня выбирает
description: 'Каждая из 9 деревень одновременно
 А выбирает A-B(2), D выбирает D-E(3), а G вы
 !',
 type: 'info'
}, ...prev]);
},
// Step 2: Concurrently merge into Kolkhozes (Components)
() => {
 // Concurrently build chosen roads. Merge into:
 // Component 1: {A, B, C} (colored red)
 // Component 2: {D, E, F, G, H, I} (colored gold/olive)
 setVertices(prev => prev.map(v =>
 [0, 1, 2].includes(v.id) ? { ...v, color: 'red' } : v
));
 setEdges(prev => prev.map(e => {
 if (['0-1', '1-2'].includes(e.id)) {
 return { ...e, status: 'included', color: 'red' }
 }
 if (['3-4', '4-5', '3-6', '6-7', '7-8'].includes(e.id)) {
 return { ...e, status: 'included', color: 'blue' }
 }
 return e;
 }));
 setLogs(prev => [{
 title: 'Слияние: Деревни объединились в колхозы
description: 'Все выбранные дороги построены ра
 оз {D, E, F, G, H, I}.',
 type: 'success'
 }, ...prev]);
}

```



```

const currentSteps =
 activeAlgo === 'kruskal' ? kruskalSteps :
 activeAlgo === 'prim' ? primSteps :
 boruvkaSteps;

const handleNextStep = () => {
 if (stepIndex < currentSteps.length) {
 currentSteps[stepIndex]();
 setStepIndex(stepIndex + 1);
 }
};

const introLog = {algo: 'kruskal' | 'prim' | 'boruvka'
 if (algo === 'kruskal') {
 return [{
 title: 'Введение в раскраску (Краскал)',
 description: 'Представь, что все 9 вершин графа
 type: 'info',
 }
]
}
 if (algo === 'prim') {
 return [{
 title: 'Введение в Плесень (Прима)',
 description: 'Логика «Плесени»: выбираем старто
 ь (Heap) и захватываем соседей!',
 type: 'info',
 }
]
}
 return [{
 title: 'Введение в Коллективизацию (Борувка)',
 description: 'Логика «Борувки»: Каждая из 9 дерев
 ы объединиться в колхоз.',
 type: 'info',
 }
];

const handleReset = {algo: 'kruskal' | 'prim' | 'boru

```

```

 return null;
 }, []));

useVizStepSync(stepIndex, goToStep, currentSteps.length);

/** A..I ↔ id 0..8 */
const idOf = (name: string | number | null | undefined): number | null {
 if (name === null || name === undefined) return null;
 if (typeof name === 'number') return Number.isInteger(name) ? name : null;
 const c = name.trim().toUpperCase().replace(/[^A-I]/g, '');
 return c ? c.charCodeAt(0) - 65 : null;
};

/** Разбор одного ребра из кода: (2, 'A', 'B') / [2, 'A', 'B']
const parseEdge = (item: unknown): { u: number; v: number; w: number | null } | null {
 if (item === null || item === undefined) return null;
 if (typeof item === 'string') {
 const nums = item.match(/-?\d+(?:\.\d+)?/g);
 const letters = item.match(/[A-Ia-i]/g);
 let u: number | null = null;
 let v: number | null = null;
 if (letters && letters.length >= 2) { u = idOf(letters[0]); v = idOf(letters[1]); }
 else if (nums && nums.length >= 2) { u = idOf(nums[0]); v = idOf(nums[1]); }
 if (u === null || v === null) return null;
 return { u, v, w: nums && nums.length ? Number(nums[2]) : null };
 }
 if (Array.isArray(item)) {
 const letters = item.filter((x) => typeof x === 'string');
 const nums = item.filter((x) => typeof x === 'number');
 if (letters.length >= 2) {
 const u = idOf(letters[0]); const v = idOf(letters[1]);
 if (u === null || v === null) return null;
 return { u, v, w: nums.length ? nums[2] : null };
 }
 if (nums.length >= 2) {
 const u = idOf(nums[nums.length - 2]); const v = idOf(nums[nums.length - 1]);
 if (u === null || v === null) return null;
 return { u, v, w: nums.length > 2 ? nums[0] : null };
 }
 }
 return null;
};

```

```

 if (id > 8) return;
 feed(id, val);
 });
}
/**
 * Ключ множества вершины: если код дал DSU-массив
 * идём по цепочке до корня – иначе вершины одной к
 * разные цвета. Для произвольных меток («comp1») б
 */
const compOfVertex = new Map<number, string>();
rawComp.forEach((label, id) => {
 if (parentIdx.size === 0) {
 compOfVertex.set(id, label);
 return;
 }
 let cur = id;
 const seen = new Set<number>([id]);
 for (let hop = 0; hop < 16; hop++) {
 const nxt = parentIdx.get(cur);
 if (nxt === undefined || nxt === cur || seen.has(nxt)) {
 seen.add(nxt);
 cur = nxt;
 }
 compOfVertex.set(id, String.fromCharCode(65 + cur));
 });
const takenRaw = vizArray(vars.mst) ?? vizArray(vars.mst);
const takenEdges = (takenRaw ?? []).map(parseEdge);
const takenVertices = (vizArray(vars.taken) ?? vizArray(vars.taken))
 .map((x) => idOf(typeof x === 'string' ? x : vizNumber(x)))
 .filter((x): x is number => x !== null);
const curU = idOf(vizString(vars.u) ?? vizNumber(vars.u));
const curV = idOf(vizString(vars.v) ?? vizNumber(vars.v));
const curW = vizNumber(vars.w);
const heapRaw = vizArray(vars.heap) ?? vizArray(vars.heap);
const heapEdges = (heapRaw ?? []).map(parseEdge);
const cheapestRaw = vizRecord(vars.cheapest) ?? vizRecord(vars.cheapest);
const cheapestEdges = cheapestRaw ? Object.values(cheapestRaw)
 .filter(e => e !== null) : [];

```

```

const getColorClass = (color: string, id: number) =>
 switch (color) {
 case 'red': return 'bg-red-500 border-red-300 tex
 case 'blue': return 'bg-blue-500 border-blue-300
 case 'purple': return 'bg-purple-600 border-purple
 case 'mold': return 'bg-emerald-500 border-emerald
 case 'gold': return 'bg-amber-400 border-amber-200
 default:
 if (activeAlgo === 'prim' && id === 4 && stepIn
 return 'bg-slate-700 border-emerald-500 text-
 }
 return 'bg-slate-700 border-slate-500 text-slate
 }
};

const getEdgeStroke = (status: string, color: string)
 if (status === 'inspecting') return '#f59e0b';
 if (status === 'rejected') return '#ef4444';
 if (status === 'in_heap') return '#0284c7';
 if (status === 'included') {
 if (color === 'red') return '#ef4444';
 if (color === 'blue') return '#3b82f6';
 if (color === 'purple') return '#a855f7';
 if (color === 'mold') return '#10b981';
 }
 return '#475569';
};

return (
 <div className="space-y-12">

 {/* 3-in-1 MST Simulator Box */}
 <div className="bg-slate-900/90 border border-slate-900"
 <div className="flex flex-col lg:flex-row justify-between"
 <div>
 <div className="flex items-center gap-2 mb-2"


```

```

 <button
 onClick={() => handleReset('boruvka')}
 className={
 "flex items-center gap-1.5 px-3 py-2
+
 (activeAlgo === 'boruvka'
 ? 'bg-rose-600 text-white shadow-md
 : 'text-slate-400 hover:text-slate-1
)
 >
 <Users className="w-3.5 h-3.5" />
 Борувка (Билет 20)
 </button>
 </div>

 <button
 onClick={() => handleReset(activeAlgo)}
 className="flex items-center justify-center gap-2
-200 font-semibold text-xs rounded-xl border
 >
 <RotateCcw className="w-3.5 h-3.5" />
 Сбросить
 </button>

 <button
 onClick={handleNextStep}
 disabled={stepIndex >= currentSteps.length}
 className={
 "flex items-center justify-center gap-2
initial cursor-pointer " +
 (stepIndex >= currentSteps.length
 ? "bg-slate-800 text-slate-500 border
 : activeAlgo === 'kruskal'
 ? "bg-indigo-600 hover:bg-indigo-500
 : activeAlgo === 'prim'
 ? "bg-emerald-600 hover:bg-emerald-500
 : "bg-rose-600 hover:bg-rose-500 text
)
 >
 Следующий шаг
 </button>

```

```


 })
 {live.cheapestEdges.length > 0 && {

 cheapest: {live.cheapestEdges.length}

 })
 {live.total !== null && {

 вес остова = {live.total}

 })
 </>
) : (

 открой панель Python и запусти код вкладки
 'ка'>»: переменные
 {live.cheapestEdges}

 <code className="font-mono text-purple" style="font-family: monospace; color: #800080;">
 e className="font-mono text-purple-300">u</code>
 <code className="font-mono text-purple-300" style="font-family: monospace; color: #800080;">
 mono text-purple-300">w</code> перекрасят э

)
 </div>

<div className="grid grid-cols-1 lg:grid-cols-3" style="font-family: monospace;">
 {/* Graph Canvas */}
 <div className="lg:col-span-2 bg-slate-950 rounded-full relative min-h-[480px] overflow-hidden">

 {/* Legend */}
 <div className="absolute top-4 left-4 bg-slate-900 p-4 text-white" style="font-family: monospace;">
 -center gap-3 text-xs">
 <div className="flex items-center gap-1.5">
 <div className="w-3 h-3 rounded-full bg-slate-900"></div>
 {activeAlgo === 'kruskal' ? (
 <div className="flex items-center gap-1.5">

```

```

 { /* SVG Edges */
 <svg className="absolute inset-0 w-full h-full"
 spectRatio="none">
 {effEdges.map(edge => {
 const u = effVertices[edge.u];
 const v = effVertices[edge.v];
 const strokeColor = getEdgeStroke(edge);
 const isInspecting = edge.status === 'inspect';
 const isRejected = edge.status === 'rejected';
 const isInHeap = edge.status === 'in_heap';

 return (
 <g key={edge.id}>
 <line
 x1={u.x + "%"}
 y1={u.y + "%"}
 x2={v.x + "%"}
 y2={v.y + "%"}
 stroke={strokeColor}
 strokeWidth={isInspecting ? 4.5 : 1}
 strokeDasharray={isRejected ? '4,4' : ''}
 className={"transition-all duration-200"}
 />
 <text
 x={({u.x + v.x} / 2 + "%")}
 y={({u.y + v.y} / 2 + "%")}
 fill={isInspecting ? '#f59e0b' :
 : '#94a3b8'}
 fontSize="4.5"
 fontFamily="monospace"
 fontWeight="bold"
 textAnchor="middle"
 dominantBaseline="middle"
 className="select-none filter drop-shadow"
 dy="-1.5"
 >
 {edge.weight}
 </text>
 </g>
);
 })}
 </svg>
);
}

```

```

<div className="bg-slate-950 rounded-2xl border-2 border-b
 <div className="p-4 bg-slate-900 border-b border-r border-t
 <h4 className="font-bold text-slate-200 text-sm">
 Ход выполнения
 </h4>
 </div>
</div>

```

```

{activeAlgo === 'prim' && {
 <div className="p-3 bg-slate-900/60 border-2 border-b border-r border-t
 <p className="text-[11px] font-bold text-slate-200">
 ✂ Приоритетная очередь (Min-Heap):
 </p>
 <div className="flex flex-wrap gap-1.5">
 {heapQueue.length === 0 ? {
 Очередь пуста
 } : {
 heapQueue.map((item, idx) => {

 {item}

 })
 })
 </div>
 </div>
}}

```

```

<div className="flex-1 p-4 overflow-y-auto">
 {logs.map((log, idx) => {
 <div
 key={idx}
 className={
 "p-4 rounded-xl border transition-all duration-200"
 (log.type === 'success'
 ? 'bg-emerald-950/40 border-emerald-500'
 : log.type === 'warning'
 ? 'bg-amber-950/40 border-amber-500'
 : log.type === 'error'
 ? 'bg-red-950/40 border-red-500'
 : 'bg-slate-900/40 border-slate-500')
 >
 {log.message}
 </div>
)}
)}

```



```

 == 'prim' ? "bg-emerald-600 text-white" : "
 >
 Прима (Билет 19)
 </button>
 <button
 onClick={() => setActiveCheatTab('boruvka
 className={"px-3 py-1.5 rounded text-xs f
 == 'boruvka' ? "bg-rose-600 text-white" : "
 >
 Борувка (Билет 20)
 </button>
</div>
</div>

{activeCheatTab === 'kruskal' && (
 <div className="grid grid-cols-1 lg:grid-cols
 <div className="lg:col-span-1 space-y-4">
 <div className="bg-slate-950 p-4 rounded-
 <p className="text-slate-400 text-xs fo
 <Clock className="w-3.5 h-3.5 text-in
 </p>
 <p className="text-2xl font-black text-
 <p className="text-slate-400 text-xs mt
 </div>
 <div className="bg-slate-950 p-4 rounded-
 <p className="text-slate-400 text-xs fo
 <Award className="w-3.5 h-3.5 text-in
 </p>
 <p className="text-2xl font-black text-
 <p className="text-slate-400 text-xs mt
 </div>
 <div className="bg-slate-950 p-4 rounded-
 <p className="text-slate-400 text-xs fo
ожении:</p>
 <p className="text-xs text-slate-300 le
ди и красим вершины в один цвет, бракуя цик
 </div>
 </div>
 </div>

```

```

 </p>
 <p className="text-2xl font-black text-white">
 <p className="text-slate-400 text-xs mt-2">
 </div>
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-medium">
 <Award className="w-3.5 h-3.5 text-emerald-400">
 </p>
 <p className="text-2xl font-black text-white">
 <p className="text-slate-400 text-xs mt-2">
 </div>
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-medium">
 в предложении:
 <p className="text-xs text-slate-300 leading-5">
 не чистые вершины через самое дешевое доступное ребро.
 </div>
 </div>
 <div className="lg:col-span-2">
 <div className="bg-slate-950 p-4 rounded-3xl">
 <p className="text-slate-400 text-xs font-medium">
 <Code className="w-3.5 h-3.5 text-emerald-400">
 </p>
 <pre className="text-xs text-emerald-400 font-mono">
 80 flex-1">

```

```
{`import heapq
```

```

def prim(graph, start):
 visited = [False] * len(graph)
 heap = [(0, start, -1)] # (weight, node, parent)
 mst = []

 while heap:
 w, u, prev = heapq.heappop(heap)
 if visited[u]: continue
 visited[u] = True
 if prev != -1: mst.append((prev, u, w))

```

ФАЙЛ 57/111: `src/components/LiteGuide.tsx`  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```

padding: 0 .25rem; }
.lite-wrap pre { background: #f3f4f6; border: 1px solid #d1d3d4; padding: 1.45rem; }
.lite-wrap table { border-collapse: collapse; width: 100%; }
.lite-wrap th, .lite-wrap td { border: 1px solid #d1d3d4; padding: .5rem; }
.lite-wrap th { background: #f3f4f6; }
.lite-wrap details { margin: .6rem 0; border: 1px solid #d1d3d4; padding: .5rem; }
.lite-wrap summary { cursor: default; font-weight: 700; }
.lite-toc { font-size: .92rem; }
.lite-toc ol { margin: .4rem 0 0 1.2rem; }
@media print {
 .lite-wrap { max-width: none; padding: 0; }
 h2 { page-break-before: always; }
 h2:first-of-type { page-break-before: avoid; }
 pre, table, details { page-break-inside: avoid; }
}
;

```

```

/** Раскрываем все спойлеры и вычищаем служебную разметку */
function toPlainTextHtml(html: string): string {
 return html
 .replace(/<details(?:[>]*\bopen\b)/g, "<details open")
 // кнопки/иконки внутри текста не нужны в текстовой версии
 .replace(/<button[\s\S]*?<\s*\/button>/gi, "")
 .replace(/<svg[\s\S]*?<\s*\/svg>/gi, "")
 // интерактивные подсказки терминов оставляем как есть
 .replace(/<sdata-term-id="[^"]*">/g, "")
 .replace(/<class="term-hint"[^"]*">/g, "")
 // ссылки на другие билеты должны вести в текстовую версию
 .replace(/<href="\?topic=/g, 'href="\?lite=1&topic=')
 .replace(/<class="ticket-link"/g, "");
}

```

```

interface Props {
 /** id одной темы; undefined – все темы подряд. */
 topicId?: string;
}

```

```

<article key={chapter.id} id={chapter.id}>
 <h2>{chapter.title}</h2>
 <p className="lite-topics">
 {topics.length > 0 && <>Билет{topics.leng
 {chapter.category ? <>Раздел: {chapter.ca
 id страницы: <code>{chapter.id}</code>
 </p>

 <div
 dangerouslySetInnerHTML={{
 __html: linkifyTicketsHtml(toPlainTextH
 }}
 />

 {viz && (
 <div className="lite-viz">
 <h4>Визуализация: {viz.title}</h4>
 {viz.hint && <p>{viz.hint}</p>}
 {syncKeys.map((k) => {
 const sync = PAGE_SYNC[k];
 const demo = k.includes("#") ? k.spli
 return (
 <div key={k}>
 <p>
 {demo ? `Вкладка «${demo}»`
 {sync.vizTitle ? ` – ${sync.viz
 {sync.stepNote ? <> Шаг демонстр
 </p>
 {sync.variables.length > 0 && (

 {sync.variables.map((v) => (
 <li key={v.name}>
 <code>{v.name}</code> – {
 {v.range ? ` (${v.range})

)}}

)}
)}
)}
)}
)}
)}
)}

```

```
export default LiteGuide;
```

---

ФАЙЛ 58/111: src/components/LiveDistTable.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import type { DebugValue } from "../data/debugRunner";

interface LiveDistTableProps {
 /** dist из кода пользователя: имя вершины → расстояние */
 dist: Record<string, DebugValue> | null;
 /** prev/p – предок на кратчайшем пути (необязательно) */
 prev?: Record<string, DebugValue> | null;
 /** done/visited – зафиксированные вершины (необязательно) */
 done?: DebugValue[] | null;
 /** Идентификаторы вершин, нарисованных на схеме демо */
 demoIds: string[];
 /** Подпись таблицы (по умолчанию «Ваши расстояния»). */
 title?: string;
}

const fmt = (value: DebugValue | undefined): string => {
 if (value === undefined || value === null) return "–";
 if (typeof value === "number") {
 if (value === Number.POSITIVE_INFINITY) return "∞";
 if (value === Number.NEGATIVE_INFINITY) return "−∞";
 return String(value);
 }
 if (typeof value === "string") {
 const t = value.trim().toLowerCase();
 if (t === "inf" || t === "infinity") return "∞";
 if (t === "-inf" || t === "-infinity") return "−∞";
 return value;
 }
 return String(value);
}
```

```

<div className="mt-2.5 flex flex-wrap gap-1.5">
 {shown.map(({key}) => {
 const parent = prev ? fmt(prev[key]) : null;
 const fixed = doneSet.has(key);
 return (
 <span
 key={key}
 className={`inline-flex items-baseline gap-1
 ${fixed
 ? "border-emerald-500/40 bg-emerald-500/10"
 : "border-slate-700 bg-slate-900/70 text-emerald-400"}
 `}
 title={fixed ? "вершина зафиксирована (done)" : ""}
 >
 <b className={fixed ? "text-emerald-300" : ""}>
 =
 {fmt(dist[key])}
 {parent && parent !== "-" ?
 {fixed ?

 } : null}

);
 })}
 {keys.length > shown.length ? (

 +{keys.length - shown.length} ещё

) : null}
</div>
</div>
);
}

```

```

 highlightColor: 'text-emerald-400',
 rest: '}'. Ищет пути от всех ко всем.',
 complexity: 'O(V³)',
 border: 'border-emerald-500/30 hover:border-emerald-500/30',
 titleColor: 'text-emerald-400',
 badge: 'text-emerald-400/70 bg-emerald-950/40',
 },
];

export default function MnemonicCards() {
 return (
 <div className="grid grid-cols-1 md:grid-cols-3 gap-4">
 {cards.map((c, i) => (
 <div key={i} className={`bg-slate-800 rounded-2xl p-4`}>
 <div className="h-48 overflow-hidden">
 <img src={c.img} alt={c.alt}
 className="w-full h-full object-cover grid"
 />
 </div>
 <div className="p-4">
 <h3 className={`text-lg font-bold mb-1 ${c.titleColor}`}>{c.title}</h3>
 <p className="text-slate-400 text-sm">{c.desc} {c.complexity}
 </p>
 <div className={`mt-3 text-xs font-mono px-2`}>{c.rest}</div>
 </div>
))}
 </div>
);
}

```

ФАЙЛ 60/111: src/components/MobileToc.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useEffect, useState } from "react";
```



```

 aria-label="Открыть содержание"
 >

 <List className="w-4 h-4" />

 Содержание • {index + 1} из {total}

 </button>
 </div>

 {open && (
 <div className="lg:hidden fixed inset-0 z-[90]">
 <div className="absolute inset-0 bg-slate-950">
 <div className="absolute inset-y-0 left-0 w-[100px] h-[100px] bg-white te-[tocIn_.18s_ease-out]">
 <div className="flex items-center justify-between">

 Содержание
 <button
 onClick={() => setOpen(false)}
 className="p-2 -mr-2 text-slate-400 hover:text-white"
 aria-label="Заккрыть содержание"
 >
 <X className="w-5 h-5" />
 </button>
 </div>
 </div>
 <div className="flex-1 min-h-0">
 <Sidebar
 selectedChapterId={selectedChapterId}
 setSelectedChapterId={setSelectedChapterId}
 onNavigate={() => setOpen(false)}
 />
 </div>
 </div>
 </div>
 </div>
)}
 </div>
</div>

```



```

 title="Полный PDF-дамп исходного кода
 >
 <FileDown className="w-3.5 h-3.5" />

<a
 id="download-txt-btn"
 href={` ${import.meta.env.BASE_URL}exp
 download="full_code_all_pages.txt"
 target="_blank"
 rel="noreferrer"
 className="flex items-center gap-2 px
/20 border border-sky-500/30 transition-all
 title="Скачать полный TXT файл со все
 >
 <FileText className="w-3.5 h-3.5" />

<button
 id="download-html-btn"
 onClick={saveBook}
 disabled={busy}
 className="flex items-center gap-2 w-
te hover:bg-amber-600/20 border border-amber
ointer"
 title="Скачать всё пособие одним авто
 >
 {busy ? <Loader2 className="w-3.5 h-3
 </button>
</div>
</details>
</div>
</div>
</div>
</header>
);
};

```

```

edges: [number, number][];
adj: number[][];
pos: { x: number; y: number }[];
names: string[];
}

const PALETTE = ["#6366f1", "#10b981", "#f43f5e", "#f59e0b"];
const colorOf = (c: number) => (c >= 1 && c <= PALETTE.length) ? PALETTE[c - 1] : "#ccc";

function makeGraph(key: GraphKey): Graph {
 if (key === "bip") {
 const edges: [number, number][] = [[0, 3], [0, 4], [1, 3], [1, 4], [2, 3], [2, 4], [3, 4]];
 const pos = [
 { x: 80, y: 60 }, { x: 80, y: 150 }, { x: 80, y: 240 },
 { x: 380, y: 60 }, { x: 380, y: 150 }, { x: 380, y: 240 }
];
 const g: Graph = { key, label: "Двудольный (6 вершин, 6 ребер)", n: 6, edges };
 g.adj = buildAdj(g);
 return g;
 }
 const edges: [number, number][] = [[0, 1], [0, 2], [1, 2], [1, 3], [1, 4], [1, 5], [2, 3], [2, 4], [2, 5], [3, 4], [3, 5], [4, 5]];
 const pos = [
 { x: 70, y: 60 }, { x: 175, y: 40 }, { x: 150, y: 160 },
 { x: 250, y: 185 }, { x: 365, y: 110 }, { x: 350, y: 240 }
];
 const g: Graph = { key, label: "Основной (8 вершин, 12 ребер)", n: 6, edges };
 g.adj = buildAdj(g);
 return g;
}

function buildAdj(g: Graph): number[][] {
 const adj: number[][] = g.edges.reduce((acc: number[][], [u, v]) => {
 acc[u].push(v);
 acc[v].push(u);
 return acc;
 }, Array.from({ length: g.n }, () => [] as number[]));
 adj.forEach((a) => a.sort((x, y) => x - y));
}

```

```

 color: [...color], v, u: null, e: null, used,
 });
}
const chi = Math.max(...color);
frames.push({
 title: `Готово: жадная покраска использовала ${chi}
 note: `χ_жадн = ${chi}. Истинное хроматическое число
 , а теорема Брукса ($\chi \leq \Delta$, если граф не полный и
 color: [...color], v: null, u: null, e: null,
});
return frames;
}

```

/\*\* Проверка двудольности BFS-ом: 2 краски    нет нечётных

```

function bipartiteFrames(g: Graph): Frame[] {
 const frames: Frame[] = [];
 const color = new Array(g.n).fill(0);
 const queue: number[] = [];
 let conflict: [number, number] | null = null;
 frames.push({
 title: "Старт: пробуем покрасить в 2 цвета",
 note: "BFS красит вершину в цвет, противоположный р
 двудольный ($\chi > 2$).",
 color: [...color], v: null, u: null, e: null,
 });
 for (let start = 0; start < g.n && !conflict; start++)
 if (color[start] !== 0) continue;
 color[start] = 1;
 queue.push(start);
 frames.push({
 title: `Запускаем BFS из ${g.names[start]}: цвет
 note: `Вершина ${g.names[start]} ещё не покрашена
 и 2.`
 color: [...color], v: start, u: null, e: null,
 });
 while (queue.length && !conflict) {
 const v = queue.shift() as number;
 for (const u of g.adj[v]) {

```

```

const delta = Math.max(...g.adj.map((a) => a.length))
frames.push({
 title: "Старт: рёбра не покрашены",
 note: `Правильная рёберная покраска: у каждой верши
 бер одной вершины различны), а по теореме Визинга
 color: new Array(g.n).fill(0), ecolord: [...ecolor],
});
g.edges.forEach(([a, b], i) => {
 const usedA = Array.from(inc[a].values());
 const usedB = Array.from(inc[b].values());
 const used = Array.from(new Set([...usedA, ...usedB]
 let c = 1;
 while (used.includes(c)) c += 1;
 ecolord[i] = c;
 inc[a].set(i, c);
 inc[b].set(i, c);
 frames.push({
 title: `Рёбро (${g.names[a]}, ${g.names[b]}): цве
 note: `В вершине ${g.names[a]} заняты цвета ${us
 y) => x - y).join(", ") || "-"}}. Наименьший св
 да.`
 color: new Array(g.n).fill(0), ecolord: [...ecolor
 });
});
const chiE = Math.max(...ecolor);
frames.push({
 title: `Готово:  $\chi' = \chi_E$  при  $\Delta = \Delta$ `,
 note:
 g.key === "bip"
 ? `Двудольный граф: $\chi' = \Delta = \Delta$ – это теор
 сс 1 по Визингу.`
 : ` $\chi' = \chi_E$. Теорема Визинга: $\Delta \leq \chi' \leq \Delta + 1$
 $\chi' = \Delta + 1$ – классом 2 (например, K3 и любой
 color: new Array(g.n).fill(0), ecolord: [...ecolor],
});
return frames;
}

```

```

 if (s >= maxStep) {
 setPlaying(false);
 return s;
 }
 return s + 1;
 });
}, 1300);
return () => window.clearInterval(t);
}, [playing, maxStep]);

/* живые переменные компилятора */
const liveStep = vars ? vizNumber(vars.step) ?? vizNumber(0) : null;
useEffect(() => {
 if (liveStep === null || !Number.isFinite(liveStep))
 setStep(Math.max(0, Math.min(maxStep, liveStep)));
}, [liveStep, maxStep]);

const liveColor = vars ? vizArray(vars.color) ?? vizArray(0) : null;
const liveEcolor = vars ? vizArray(vars.ecolor) ?? vizArray(0) : null;
const liveV = vars ? vizNumber(vars.v) : null;
const liveU = vars ? vizNumber(vars.u) : null;
const liveE = vars ? vizNumber(vars.e) : null;
const liveChi = vars ? vizNumber(vars.chi) ?? vizNumber(0) : null;
const linked = Boolean(liveColor || liveEcolor || (liveV !== null || liveU !== null || liveE !== null || liveChi !== null));

const frame = frames[Math.min(step, maxStep)] ?? ({ t: 0, color: 0, ecol: 0 });
const nodeColor = (id: number): number => {
 if (liveColor && typeof liveColor[id] === "number")
 return frame.color?.[id] ?? 0;
};
const edgeColor = (i: number): number => {
 if (liveEcolor && typeof liveEcolor[i] === "number")
 return frame.ecolor?.[i] ?? 0;
};
const curV = liveV !== null && Number.isFinite(liveV) ? liveV : 0;
const curU = liveU !== null && Number.isFinite(liveU) ? liveU : 0;
const curE = liveE !== null && Number.isFinite(liveE) ? liveE : 0;
const delta = Math.max(...graph.adj.map((a) => a.length));

```

</div>

```
<div className="flex flex-wrap items-center gap-2"
 <button onClick={() => { setStep(0); setPlaying(
 rounded-lg bg-slate-900 border border-slate-
 <button onClick={() => go(-1)} disabled={step =
 00 border border-slate-700 text-slate-300 hove
 <button onClick={() => setPlaying((p) => !p)} c
 ext-xs font-bold hover:bg-indigo-500">
 {playing ? <Pause className="w-4 h-4" /> : <P
 </button>
 <button onClick={() => go(1)} disabled={step >=
 late-900 border border-slate-700 text-slate-3
 on>
 <span className="text-[11px] text-slate-500 ml-
 <span className="ml-auto flex flex-wrap items-c
 <span className="bg-slate-950 border border-s
 {tab === "edges"
 ? <span className="bg-slate-950 border bord
 rs}
 : <span className="bg-slate-950 border bord
 }}

</div>
```

```
<div className="grid grid-cols-1 lg:grid-cols-5 g
 <div className="lg:col-span-3 bg-slate-950 roun
 <svg viewBox="0 0 480 300" className="w-full l
 {graph.edges.map(([a, b], i) => {
 const ec = edgeColor(i);
 const isCur = i === curE;
 const isBad = Boolean(frame.bad && ((fram
));
 return (
 <g key={i}>
 <line
 x1={graph.pos[a].x} y1={graph.pos[a
 stroke={isBad ? "#f43f5e" : ec > 0
```



```

 <p className="text-[10px] uppercase tracking
 <div className="flex flex-wrap gap-1.5 font
 {(tab === "edges" ? graph.edges.map((_, i)
 <span key={i} className="px-1.5 py-0.5
 ? colorOf(c) : "#475569", background: c > 0
 {tab === "edges" ? `e${i}` : graph.na

)))
 </div>
</div>
<div className="bg-slate-900/70 border border
 {tab === "greedy" && <>Жадная покраска: <b
 : <b className="text-slate-200"> $\chi \leq \Delta$,
 в $\chi \leq 5$ (доказывается через $E \leq 3V - 6$ и це
 ром).</>
 {tab === "bipartite" && <>Двудольный граф
 /DFS за $O(V + E)$. Распознавание 3-раскрашив
 {tab === "edges" && <>Рёберная покраска: <b
 Δ (Кёниг). Задача « $\chi' = \Delta$?» NP-полна даже
 </div>
</div>
</div>
<div className="mt-3 flex flex-wrap items-center
 <span className="text-[10px] uppercase tracking
 /> из компилятора:
 {vars && linked ? (
 ["v", "u", "e", "color", "ecolor", "chi"].map
 vars[name] === undefined ? null : (
 <span key={name} className="text-[11px] f
 y-1">
 {name} = {JSON.stringify(vars[name]).sl

)
)
) : (

 открой панель Python: массив <code classNam

```

---

ФАЙЛ 63/111: src/components/PlanarityDemo.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
export function PlanarityDemo() {
 return (
 <div className="space-y-4">
 <h4 className="text-base font-bold text-white">K5
 <p className="text-xs text-slate-400">Просто смотрим
 <div className="grid grid-cols-1 md:grid-cols-2 gap-4">
 { /* K5 */ }
 <div className="bg-slate-950 rounded-xl p-4 border border-slate-800">
 <div className="absolute top-2 left-2 bg-slate-900/30 w-auto z-20">K5 – НЕПЛАНАРЕН</div>
 <svg className="w-full h-full absolute inset-0">
 {(() => {
 const pts = [
 {id:0,x:160,y:40},
 {id:1,x:270,y:100},
 {id:2,x:230,y:230},
 {id:3,x:90,y:230},
 {id:4,x:50,y:100}
];
 const edges = [
 [0,1],[0,2],[0,3],[0,4],
 [1,2],[1,3],[1,4],
 [2,3],[2,4],
 [3,4]
];
 function findPt(id:number) { return pts.find(p => p.id === id); }
 function intersect(a:number,b:number,c:number,d:number) {
 if (a===c||a===d||b===c||b===d) return true;
 const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
 function ccw(ax:number,ay:number,bx:number,by:number) {
 return (bx-ax)*(by-ay)-(by-ay)*(bx-ax);
 }
 const o1=ccw(p1.x,p1.y,p2.x,p2.y),o2=ccw(p1.x,p1.y,p3.x,p3.y),o3=ccw(p1.x,p1.y,p4.x,p4.y),o4=ccw(p2.x,p2.y,p3.x,p3.y),o5=ccw(p2.x,p2.y,p4.x,p4.y),o6=ccw(p3.x,p3.y,p4.x,p4.y);
 return (o1>0&&o2>0&&o3>0&&o4>0&&o5>0&&o6>0);
 }
 })()}
 </svg>
 </div>
 </div>
 </div>
);
}
```

```

<div className="bg-slate-950 rounded-xl p-4 border"
 <div className="absolute top-2 left-2 bg-slate-900/30 w-auto z-20">K3,3 – НЕПЛАНАРЕН</div>
 <svg className="w-full h-full absolute inset-0"
 {(() => {
 const pts = [
 {id:0,x:60,y:60},{id:1,x:160,y:60},{id:2,x:260,y:60},
 {id:3,x:60,y:220},{id:4,x:160,y:220},{id:5,x:260,y:220},
];
 const edges = [
 [0,3],[0,4],[0,5],
 [1,3],[1,4],[1,5],
 [2,3],[2,4],[2,5]
];
 function findPt(id:number) { return pts.find(p => p.id === id); }
 function intersect(a:number,b:number,c:number,d:number) {
 if (a===c||a===d||b===c||b===d) return true;
 const p1=findPt(a),p2=findPt(b),p3=findPt(c),p4=findPt(d);
 if (!p1||!p2||!p3||!p4) return false;
 function ccw(ax:number,ay:number,bx:number,by:number,cx:number,cy:number) {
 return (cy-ay)*(bx-ax)>(by-ay)*(cx-ax);
 }
 return ccw(p1.x,p1.y,p3.x,p3.y,p4.x,p4.y)<0
 && ccw(p1.x,p1.y,p2.x,p2.y,p3.x,p3.y)<0;
 }
 const crosses:number[]=[];
 for(let i=0;i<edges.length;i++) for(let j=i+1;j<edges.length;j++) {
 if (intersect(edges[i][0],edges[i][1],edges[j][0],edges[j][1])) {
 if (!crosses.includes(i)) crosses.push(i);
 if (!crosses.includes(j)) crosses.push(j);
 }
 }
 const lines = []; const circles = [];
 for(let i=0;i<edges.length;i++) {
 const p1=findPt(edges[i][0]),p2=findPt(edges[i][1]);
 const xing=crosses.includes(i);
 lines.push(<line key={`l${i}`} x1={p1.x} x2={p2.x} y1={p1.y} y2={p2.y}
 stroke={xing?"#f43f5e":"#4f46e5"} stroke-width={2}/>);
 }
 })}
 </svg>
</div>

```

```

 ChevronLeft,
 ChevronRight,
 ChevronUp,
 CircleStop,
 Copy,
 Eye,
 EyeOff,
 HelpCircle,
 Link2,
 Loader2,
 Pause,
 Pencil,
 Play,
 Plus,
 RotateCcw,
 SkipBack,
 SkipForward,
 Terminal,
 Unlink,
 Variable,
 X,
} from "lucide-react";
import { buildInitTemplate, buildSyncTemplate, getPageS
import { clearVizState, emitVizState, onVizDemo } from
import {
 baseVarName,
 buildDebugRunner,
 changedVars,
 DEBUG_STEP_CAP,
 orderVars,
 type DebugResult,
} from "../data/debugRunner";
import { Tooltip } from "./Tooltip";

const PYODIDE_VERSION = "0.27.7";
const PYODIDE_MODULE_URL = `https://cdn.jsdelivr.net/pyo
const PYODIDE_INDEX_URL = `https://cdn.jsdelivr.net/pyo
const PLAYBACK_SPEED_KEY = "pythonCompilerPlaybackSpeed

```

```

declare global {
 interface Window {
 __algv0PythonRuntime?: PythonRuntimeProvider;
 }
}

function getPythonRuntime(): Promise<PyodideRuntime> {
 const injected = typeof window !== "undefined" ? window : undefined;
 if (injected) return typeof injected === "function" ? injected : undefined;
 if (!runtimePromise) {
 runtimePromise = import(/* @vite-ignore */ PYODIDE_MODULE_PATH)
 .then((module as PyodideModule).loadPyodide({ indexURL: PYODIDE_MODULE_PATH }));
 }
 return runtimePromise;
}

function errorText(error: unknown) {
 if (error instanceof Error) return error.message;
 return String(error);
}

/**
 * Предел вывода в панель «Результат».
 *
 * Трасса ограничена по ЧИСЛУ шагов (DEBUG_STEP_CAP), но не по
 * напечатанной строки: print("x" * 10_000_000) или print("x" * 10_000_000)
 * — это один шаг, после которого весь текст уезжал в <div>.
 * Обрезаем по границе строки и честно пишем, сколько э
 */
const OUTPUT_CHAR_CAP = 20_000;

function clipOutput(text: string): string {
 if (text.length <= OUTPUT_CHAR_CAP) return text;
 const cut = text.slice(0, OUTPUT_CHAR_CAP);
 const lastBreak = cut.lastIndexOf("\n");
 const head = lastBreak > OUTPUT_CHAR_CAP / 2 ? cut.slice(0, lastBreak) : cut;
 return `${head}\n... вывод обрезан: показаны первые ${head.length} символов`;
}

```

```

 items: [
 {
 label: "for i in range(n)",
 tip: "Вложенные циклы по i и j – как обход матрицы",
 code: "n, m = 3, 4\nfor i in range(n):",
 },
 {
 label: "while ...",
 tip: "Классический while-счётчик – как n-1 итерация",
 code: "i = 1\nwhile i < 5:\n print(f\"итерация {i}\")",
 },
 {
 label: "for i, x in enumerate(...)",
 tip: "Индекс и элемент одновременно: i – позиция, x – элемент",
 code: "for i, x in enumerate(['a', 'b', 'c']):",
 },
],
},
{
 group: "Структуры",
 items: [
 {
 label: "Стек (список)",
 tip: "LIFO, как рекурсия DFS: append = push, pop = pull",
 code: "st = []\nst.append('A') # push\nst.pop() # pull",
 },
 {
 label: "Очередь (deque)",
 tip: "FIFO для BFS: popleft() за O(1). list.pop(0) за O(n)",
 code: "from collections import deque\nq = deque()\nq.append('A')\nq.popleft()",
 },
 {
 label: "Куча (heapq)",
 tip: "Мин-куча: (вес, вершина) – как очередь приоритетов",
 code: "import heapq\nh = []\nheapq.heappush(h, (1, 'A'))\nheapq.heappop(h)",
 },
],
}

```

```

const syncVarBases = useMemo(
 () =>
 [...new Set(
 sync.variables.flatMap((v) =>
 v.name
 .split(/\s*(?:,|\s\/)\s*/)
 .map(baseVarName)
 .filter((name) => /^[A-Za-z_][A-Za-z0-9_]*$/
)
)],
 [sync]
);
const syncVarSet = useMemo(() => new Set(syncVarBases
/** Полный эталон: глаз сразу вставляет его в обычный
const template = useMemo(() => buildSyncTemplate(chap
/** Дефолт редактора: только инициализация демо; она
const initTemplate = useMemo(() => buildInitTemplate(

const [code, setCode] = useState(() => {
 const saved = savedEditors.get(chapterId);
 if (!saved) return initTemplate;
 if (saved.code === saved.base || saved.code.trim()
 return saved.code;
});
/** «Глаз» – переключатель: эталонный код демо ↔ иници
const [refOn, setRefOn] = useState(() => {
 const saved = savedEditors.get(chapterId);
 return !!saved && saved.code.trim() !== "" && saved
});
const [stdin, setStdin] = useState("");
const [output, setOutput] = useState("Код выполнится
const [running, setRunning] = useState(false);
const [runtimeReady, setRuntimeReady] = useState(false);
const [runtimeMessage, setRuntimeMessage] = useState(
/** Принудительный повтор (глаз / Ctrl+Enter), даже е
const [traceRequest, setTraceRequest] = useState(0);
const [copied, setCopied] = useState(false);
const [stripTab, setStripTab] = useState<"vars" | "hi

```

```

useEffect(() => {
 savedEditors.set(chapterId, { code, base: initTemplate,
}, [code, initTemplate, chapterId]);

// демонстрация сообщает, какая вкладка открыта – код
useEffect(() => setDemoId(null), [chapterId]);
useEffect(() => onVizDemo(chapterId, (d) => setDemoId(d)), [chapterId]);
useEffect(() => () => clearVizState(chapterId), [chapterId]);

// Смена страницы (или вкладки демо): нетронутый редактор требует
// инициализацию; свой код не затираем – предлагаем сменить
useEffect(() => {
 const prev = prevTplRef.current;
 if (initTemplate === prev.base && template === prev.full) {
 prevTplRef.current = { base: initTemplate, full: template };
 setPlaying(false);
 setEditHold(null);
 setDebug(null); // трасса ссылается на строки старого кода
 setRefOn(false);
 if (code === prev.base || code === prev.full || code === prev.full) {
 setCode(initTemplate);
 setDesyncedFrom(null);
 } else {
 setDesyncedFrom(chapterId);
 }
 }
 // eslint-disable-next-line react-hooks/exhaustive-deps
}, [initTemplate, template]);

const resync = useCallback(() => {
 prevTplRef.current = { base: initTemplate, full: template };
 setCode(initTemplate);
 setRefOn(false);
 setDesyncedFrom(null);
 setStripTab("vars");
 setPlaying(false);
 setEditHold(null);
 setDebug(null);

```



```

const variables = { ...afterValues };
const locals = { ...afterLocals };
for (const [name, value] of Object.entries(beforeValues)) {
 const scalar = value === null || typeof value !== 'object';
 if (scalar && !assigned.has(name) && name in afterValues) {
 variables[name] = value;
 if (step?.locals[name] !== undefined) locals[name] = step?.locals[name];
 }
}
const changed = changedVars(step?.locals, locals);
return {
 line: step?.line,
 func: step?.func,
 locals,
 variables,
 changed: Object.keys(changed).filter((name) => changed[name]),
};
}, []);

```

```

const publishDebug = useCallback(
 (result: DebugResult, idx: number, source: string) => {
 if (!aliveRef.current) return; // панель закрыта
 if (!hasViz || result.steps.length === 0) return;
 const snapshot = snapshotAt(result, idx, source);
 emitVizState(chapterId, {
 line: snapshot.line,
 func: snapshot.func,
 variables: snapshot.variables,
 changed: snapshot.changed,
 });
 },
 [chapterId, hasViz, snapshotAt]
);

```

```

// Единственная точка публикации: любое перемещение т
// визуализации согласованный снимок, без side-effect
useEffect(() => {
 if (debug) publishDebug(debug.result, debug.idx, de

```

```

 return;
 }
 const t = window.setTimeout(() => stepBy(1), 650 / playbackSpeed);
 return () => window.clearTimeout(t);
}, [playing, debug, stepBy, playbackSpeed]);

const debugRun = useCallback(async (autoPlay = false) => {
 if (!running) return;

 const source = code;
 const input = stdin;
 const request = traceRequest;
 setRunning(true);
 setPlaying(false);
 setEditHold(null);
 setDebug(null);
 clearVizState(chapterId);
 setOutputOpen(false);

 // Пустой редактор: Pyodide не гоняем, панель в режиме ожидания
 // связь с визуализацией снимаем — демо показывает, что не работает
 if (source.trim() === "") {
 setDebug({ result: { steps: [], error: "", truncated: false } });
 setOutput("В редакторе пусто — выполнять нечего.\n\nПустой код: связь с визуализацией снята.\n\n«глаз».");
 setRuntimeMessage("Пустой код: связь с визуализацией снята.");
 setOutputOpen(true);
 setRunning(false);
 return;
 }

 setRuntimeMessage(runtimeReady ? "Обновляю трассу..." : "Запускаю трассу...");

 const stdout: string[] = [];
 const stderr: string[] = [];
 const inputLines = input.split(/\r?\n/);

 try {

```

```

 });
 if (result.error) setOutputOpen(true);
 setRuntimeMessage(
 result.truncated
 ? `Выполнение остановлено после ${DEBUG_STEPS} шагов`
 : "Трасса готова автоматически. ← → — шаги,
);
 }
} catch (error) {
 if (!aliveRef.current) return;
 const message = errorText(error);
 setDebug({ result: { steps: [], error: message, truncated: true },
 clearVizState(chapterId);
 setOutputOpen(true);
 setOutput(`Ошибка:\n${message}`);
 setRuntimeMessage("Не удалось выполнить код — произошла ошибка");
} finally {
 setRunning(false);
}
}, [chapterId, code, stdin, traceRequest, running, runtimeReady]);

// Код выполняется сам: debounce даёт спокойно допечать код
// изменился во время загрузки Pyodide, после завершения загрузки
useEffect(() => {
 if (running || editHold === code) return;
 if (debug?.src === code && debug.input === stdin && debug.truncated) {
 const timer = window.setTimeout(() => void debugRunCode());
 return () => window.clearTimeout(timer);
 }
}, [code, stdin, traceRequest, running, runtimeReady]);

// Стрелки ← → в режиме отладки листают ШАГИ (а не строки кода)
useEffect(() => {
 if (!debug || debug.result.steps.length === 0) return;
 const onKey = (e: KeyboardEvent) => {
 const target = e.target as HTMLElement | null;
 if (target && /^(INPUT|TEXTAREA|SELECT)$/.test(target.tagName)) return;
 if (e.metaKey || e.ctrlKey || e.altKey) return;
 if (e.key === "ArrowRight") {
 setDebugIndex(debug.result.steps.length - 1);
 }
 };
 window.addEventListener("keydown", onKey);
 return () => window.removeEventListener("keydown", onKey);
}, [debug]);

```

```

 if (curDebugLine === undefined || !listRef.current)
 listRef.current
 .querySelector(`[data-line="${curDebugLine}"]`)
 ?.scrollIntoView({ block: "nearest" });
 }, [curDebugLine]);

 // производные данные шага: что изменилось, в каком п
 // Строка, которую назначает текущий шаг, подсвечивае
 // локали берём со следующего шага трассы (для послед
 // трейсером на возврате из фрейма). Значит на строке
 const shownLocals = useMemo(() => {
 if (!debug || !curDebug?.step) return {};
 return snapshotAt(debug.result, debug.idx, debug.sr
 }, [debug, curDebug, snapshotAt]);
 const debugChanged = curDebug?.step ? changedVars(cu
 const debugOrdered = curDebug?.step ? orderVars(shown

 /** Одна и та же кнопка запуска всегда видна: из реда
 * проигрывает её, в отладчике переключает play/pause
 const runOrTogglePlayback = useCallback(() => {
 if (running) return;
 if (!debug || debug.result.steps.length === 0) {
 void debugRun(true);
 return;
 }
 if (playing) {
 setPlaying(false);
 return;
 }
 if (debug.idx >= debug.result.steps.length - 1) step
 setPlaying(true);
 }, [debug, debugRun, playing, running, stepTo]);

 /** Вставка сниппета из шпаргалки в позицию курсора.
 const insertSnippet = useCallback(({snippet: string}) => {
 const ta = editorRef.current;
 setCode((current) => {
 if (!ta) return current + (current.endsWith("\n")

```

```

 const next = Math.round(startH + startY -
 onHeightChange(Math.max(minHeight, Math.m
 });
 const onUp = () => {
 el.removeEventListener("pointermove", onM
 el.removeEventListener("pointerup", onUp
 el.removeEventListener("pointercancel", on
 };
 el.addEventListener("pointermove", onMove a
 el.addEventListener("pointerup", onUp as Ev
 el.addEventListener("pointercancel", onUp a
 }}
>
 <div className="absolute left-1/2 top-1/2 h-1
 digo-400 transition-colors" />
 </div>
 })

 { /* Левый край меняет ширину на десктопе. */ }
 { onWidthChange && (
 <div
 role="separator"
 aria-orientation="vertical"
 aria-label="Потяните, чтобы изменить ширину п
 title="Потяните, чтобы изменить ширину панели
 className="hidden lg:block absolute top-0 bot
 onPointerDown={(e) => {
 const el = e.currentTarget;
 el.setPointerCapture(e.pointerId);
 const startX = e.clientX;
 const startW = width ?? 560;
 const onMove = (ev: PointerEvent) => {
 const next = Math.round(startW + (startX
 onWidthChange(Math.max(380, Math.min(next
);
 const onUp = () => {
 el.removeEventListener("pointermove", onM
 el.removeEventListener("pointerup", onUp

```

```

</Tooltip>
<Tooltip side="bottom" content="Скопировать код"
 <button
 type="button"
 onClick={() => void copyCode()}
 className="p-1.5 rounded-lg text-slate-400"
 aria-label="Скопировать код"
 >
 {copied ? <Check className="w-4 h-4 text-emerald-500" /> : <CopyIcon />}
 </button>
</Tooltip>
<Tooltip side="bottom" content="Вернуть комментарий"
 <button
 type="button"
 onClick={resync}
 className="p-1.5 rounded-lg text-slate-400"
 aria-label="Сбросить к шаблону страницы"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
</Tooltip>
<Tooltip side="bottom" content="Скрыть панель"
 <button
 type="button"
 onClick={onClose}
 className="p-1.5 rounded-lg text-slate-400"
 aria-label="Заккрыть компилятор"
 >
 <X className="w-4 h-4" />
 </button>
</Tooltip>
<Tooltip
 side="bottom"
 content={
 refOn
 ? "Убрать эталонный код: редактор станет
 собственному встроенному демо. Кнопка ↺ вернет
 : "Вставить полный эталонный код этого

```

```

ld-400" />}
{running
<select
 aria-label="Скорость автопрохода"
 value={playbackSpeed}
 onChange={(event) => setPlaybackSpeed(Num
 className="cursor-pointer rounded bg-slate
>
 {PLAYBACK_SPEEDS.map((speed) => (
 <option key={speed} value={speed}>{speed
)))}
</select>
</label>
</div>
</div>

{/* — Мини-вкладки: переменные страницы / шпаргал
<div className="shrink-0 border-b border-slate-800
 <div className="flex items-center gap-1 px-2 pt
 <button
 type="button"
 onClick={() => setStripTab("vars")}
 className={`inline-flex items-center gap-1.5
 stripTab === "vars" ? "bg-slate-800 text-
 }`}
 >
 <Variable className="w-3.5 h-3.5" /> Перемен
 </button>
 <button
 type="button"
 onClick={() => setStripTab("hints")}
 className={`inline-flex items-center gap-1.5
 stripTab === "hints" ? "bg-slate-800 text
 }`}
 >
 <HelpCircle className="w-3.5 h-3.5" /> Подс
 </button>
 <div className="ml-auto pr-1">

```

```

text-slate-300 max-w-full">
 {chapterTitle}
 </div>
 {desyncedFrom !== null && (
 <Tooltip
 side="bottom"
 content="В редакторе ваш код, а страница
т заменён). "
 >
 <button
 type="button"
 onClick={resync}
 className="flex items-center gap-1.5
0/40 text-amber-400 hover:bg-amber-500/20 t
 >
 <Unlink className="w-3.5 h-3.5 shrink
Страница сменилась – подставить её пер
 </button>
 </Tooltip>
)}
 {hasViz && (
 <div className="flex flex-wrap items-cent
 {sync.variables.map((v) => (
 <Tooltip
 key={v.name}
 side="bottom"
 content={

 <span className="block font-bol
 <code className="font-mono te

 {v.role
 {v.range && <span className="bl

 }
 >

```



```

 >
 <Plus className="w-3 h-3 text-slate-600"
 {s.label}
 </button>
 </Tooltip>
)})
</div>
</div>
)})
<p className="text-[10px] text-slate-600">Начало
</div>
})
</div>

```

```

{/* — Редактор (всегда на месте) + трасса + ввод */}
<div className="flex-1 min-h-0 flex flex-col">
 {/* Заголовок редактора: сразу видно, что выполняется */}
 <div className="shrink-0 flex items-center gap-1">
 <Pencil className="w-3.5 h-3.5 text-emerald-400" />

 Редактор • выполняется этот текст

 {state && (
 <span
 className="inline-flex items-center gap-1"
 font-weight="bold"
 text="amber-300"
 shrink="0"
 title="Трасса ниже построена по предыдущему состоянию"
 >
 <Loader2 className="w-2.5 h-2.5 animate-spin" />

)}

 {code.trim() === "" ? "пусто — ничего не выведено" : ""}

 </div>

 <textarea
 ref={editorRef}

```

```

 <Tooltip content="В начало трассы">
 <button type="button" aria-label="В начал
 }} disabled={!curDebug?.step} className="p-1
 transition-colors">
 <SkipBack className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip content="Шаг назад (←) – автопро
 <button type="button" aria-label="Шаг на
 bled={!curDebug?.prev} className="p-1 round
 tion-colors">
 <ChevronLeft className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip content="Шаг вперёд (→) – автопро
 <button type="button" aria-label="Шаг в
 abled={!curDebug?.step || debug.idx >= debu
 e hover:bg-slate-800 disabled:opacity-40 tr
 <ChevronRight className="w-4 h-4" />
 </button>
 </Tooltip>
 <Tooltip content="В конец трассы">
 <button type="button" aria-label="В кон
 result.steps.length - 1); }} disabled={!cur
 d text-slate-400 hover:text-white hover:bg-
 <SkipForward className="w-4 h-4" />
 </button>
 </Tooltip>
 <span className="ml-1.5 text-[10px] text-
 {curDebug?.step ? (
 <>
 шаг {debug.idx + 1}/{debug.result.s
 {curDebug.step.func !== "<module>" &
 </>
) : (
 "шагов нет – исправьте ошибку и начини
)}


```

```

 aria-label="Остановить прогон трассы"
 onClick={stopDebug}
 className="inline-flex shrink-0 items-center gap-1
ext-[10px] font-bold text-rose-300 transition-colors
 >
 <CircleStop className="w-3.5 h-3.5" />
 стоп
 <kbd className="rounded border border-gray-200
 </button>
 </Tooltip>
</div>

{traceOpen && (
 <>
 {/* листинг с подсветкой текущей строки */}
 <div ref={listRef} className="flex-1 min-h-[200px]
 {debug.src.split("\n").map((line, i) => {
 const ln = i + 1;
 const active = ln === curDebug?.step?.line;
 return (
 <div
 key={i}
 data-line={ln}
 className={`flex whitespace-pre px-3 py-2
 active ? "bg-emerald-500/15 text-emerald-300" :
 }`}
 >
 <span
 className={`w-7 shrink-0 select-none text-right
 active ? "text-emerald-300" : "text-gray-400"
 }`}
 >
 {active ? "►" : ln}

 {line || " "}
 </div>
);
 })}
)}
);
}}}
```

```

 }}}
 </div>
 </div>
 </>
})
</div>
})
{debug && !showTrace && debug.result.error ? {
 <pre className="shrink-0 max-h-28 overflow-auto py-2 font-mono text-[11px] leading-snug text-gray-800">
 {debug.result.error}
 </pre>
} : null}
<div className="shrink-0 border-t border-slate-800">
 <label htmlFor="python-stdin" className="text-sm font-medium">
 stdin для input()
 </label>
 <input
 id="python-stdin"
 value={stdin}
 onChange={(event) => {
 setEditHold(null);
 setStdin(event.target.value);
 }}
 spellCheck={false}
 placeholder="строки ввода через ↵"
 className="w-full bg-transparent font-mono text-sm"
 />
</div>
</div>

{/* — Вывод ————— */}
<div className="shrink-0 border-t border-slate-800">
 <button
 type="button"
 onClick={() => setOutputOpen((o) => !o)}
 className="flex items-center justify-between gap-2 text-sm font-medium text-white"
 >
 Вывод
 {outputOpen ? "⌵" : "⌴"}
 </button>
 <pre
 className="shrink-0 max-h-40 overflow-auto font-mono text-sm"
 style={outputOpen ? {} : {display: "none"}}
 >
 {output}
 </pre>
</div>

```

```

 color: string;
 bubbleText: string;
 animState: 'in' | 'idle' | 'eating' | 'out';
}

const PRESETS = [
 { name: 'Студент Вася', emoji: '🍷', color: 'from-blue',
 bubbleText: 'голода после матана!' },
 { name: 'Кодер Семён', emoji: '🍲', color: 'from-violet',
 bubbleText: 'ищу ИИ за порцию борща!' },
 { name: 'Кот Борис', emoji: '🐈', color: 'from-amber',
 bubbleText: 'есть сметаны, плиз.' },
 { name: 'Бабушка Люба', emoji: '👵', color: 'from-rose',
 bubbleText: 'есть сметаны, плиз.' },
 { name: 'Инопланетянин', emoji: '👽', color: 'from-emerald',
 bubbleText: 'есть сметаны, плиз.' },
 { name: 'Бизнесмен', emoji: '💼', color: 'from-slate',
 bubbleText: 'есть сметаны, плиз.' },
];

let counter = 3;

export const QueueViz: React.FC = () => {
 const [queue, setQueue] = useState<Customer[]>([
 { id: 'q1', name: 'Студент Вася', emoji: '🍷', color: 'from-blue',
 bubbleText: 'голода после матана!', animState: 'idle' },
 { id: 'q2', name: 'Кодер Семён', emoji: '🍲', color: 'from-violet',
 bubbleText: 'ищу ИИ за порцию борща!', animState: 'idle' },
 { id: 'q3', name: 'Кот Борис', emoji: '🐈', color: 'from-amber',
 bubbleText: 'есть сметаны, плиз.', animState: 'idle' },
]);

 const runtime = useVizRuntime();
 const liveQ = vizArray(runtime?.variables?.q);
 const liveV = vizString(runtime?.variables?.v);
 const displayQueue: Customer[] = liveQ
 ? liveQ.map((value, index) => ({
 id: `python-${index}`,
 name: value.name,
 emoji: value.emoji,
 color: value.color,
 bubbleText: value.bubbleText,
 animState: value.animState
 }))
 : [];

```

```

 setQueue(prev => prev.map(c => c.id === newGuest.id ?
 }, 450);
 }, [queue, isServing]);

```

```

// DEQUEUE: Выдать суп первому (голова / Head, FIFO)
const dequeue = useCallback(() => {
 if (isServing) return;
 if (queue.length === 0) {
 setShake(true);
 addLog('⚠ DEQUEUE: В очереди никого нет! Повар скучает');
 setTimeout(() => setShake(false), 400);
 return;
 }

```

```

 setIsServing(true);
 const headGuest = queue[0];

```

```

 addLog(`  DEQUEUE: Повар наливает суп первому – ${headGuest.name}`);

```

```

 // Step 1: Head guest gets eating animation
 setQueue(prev => prev.map((c, idx) => idx === 0 ? {

```

```

 setTimeout(() => {
 // Step 2: Guest leaves with happy face, queue shake
 setQueue(prev => prev.map((c, idx) => idx === 0 ? {

```

```

 setServedHistory(prev => [`${headGuest.emoji} ${headGuest.name} поел`]);

```

```

 setTimeout(() => {
 setQueue(prev => prev.slice(1));
 setIsServing(false);
 addLog(`  ${headGuest.name} поел и ушел сытым. 🍴`);
 }, 400);
 }, 900);
 }, [queue, isServing]);

```

```

 }, [queue, isServing]);

```

```

const reset = () => {
 counter = 3;

```

```

 iver:scale-95 transition-all cursor-pointer
 >
 Встать в очередь (ENQUEUE)
 </button>

 <button
 onClick={dequeue}
 disabled={isServing || queue.length === 0}
 className="flex-1 min-w-[150px] bg-gradient-to-r
 opacity-40 disabled:cursor-not-allowed text
 :scale-95 transition-all cursor-pointer fle
 >
 Налить суп первому (DEQUEUE)
 </button>

 <button
 onClick={reset}
 className="px-5 py-3.5 rounded-2xl bg-slate-800
 r-pointer border border-slate-700"
 >
 Сброс
 </button>
</div>

{/* ИНТЕРАКТИВНАЯ КУХНЯ */}
<div className="grid grid-cols-1 md:grid-cols-12

 {/* ЛЕВАЯ КОЛОНКА: ОЧЕРЕДЬ ЗА СУПОМ */}
 <div className={`md:col-span-9 bg-slate-900 rounded
 -[300px] overflow-hidden ${shakeEmpty ? 'anim
 <div className="absolute top-4 left-4 text-[1.2em]
 Очередь голодных гостей (Буфер FIFO / BFS)
 {liveQ && <span className="ml-2 text-emerald-500
 </div>

 <div className="flex-1 flex flex-col justify-between
 <div className="flex items-end justify-between

```





```

 className="w-full py-1.5 px-3 rounded-
0 flex items-center gap-2 text-[11px] text-
 >
 {item}
 </div>
)))
 </div>
})
</div>

{/* Пол */}
<div className="w-full h-3 bg-gradient-to-r f
</div>
</div>

{/* ЛОГ ДЕЙСТВИЙ */}
<div className="bg-slate-900 border border-slate-
 <div className="text-[10px] font-mono text-slate
 {log.map((entry, idx) => {
 <div
 key={idx}
 className={`text-xs font-mono flex items-ce
 >
 {idx === 0 ? <span className="text-indigo-5
 {entry}
 </div>
 })}
 </div>
</div>
);
};

```

```

const vars = runtime?.variables;

useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, "aho");
}, [chapterId]);

const liveWordsRaw = vars ? vizArray(vars.words) ?? v
const liveWordsSig = useMemo(() => (liveWordsRaw ? JS
const liveState = vars ? vizNumber(vars.state) ?? viz
const liveStep = vars ? vizNumber(vars.step) ?? vizNum
const liveQueue = vars ? vizArray(vars.queue) : null;
const liveNodes = vars ? vizNumber(vars.nodes) : null
const liveFail = vars ? vizRecord(vars.fail) : null;

// набор образцов из кода перестраивает бор целиком
useEffect(() => {
 if (!liveWordsSig) return;
 const parsed = (JSON.parse(liveWordsSig) as unknown
 .map((w) => (typeof w === "string" ? w : typeof w
 .filter((w) => Boolean(w))
 .map((w) => w.toUpperCase().replace(/[^A-ZА-Я]/g,
 .filter((w) => w.length > 0)
 .slice(0, 8);
 if (!parsed.length) return;
 if (parsed.join("|") === words.join("|")) return;
 setWords(parsed);
 buildInitialTrie(parsed);
 // eslint-disable-next-line react-hooks/exhaustive-
}, [liveWordsSig]);

// состояние автомата и шаг BFS – подсвечиваем вершин
useEffect(() => {
 if (liveState !== null && liveState >= 0 && nodes[l
 if (liveStep !== null && liveStep >= 0) {
 // simulationStep – грубый статус: 0 = бор готов,
 const total = Object.keys(nodes).length;
 const failReady = liveFail ? Object.keys(liveFail
 setSimulationStep(liveStep <= 0 ? 0 : failReady ?

```

```

 if (newNodes[curr].children[c] === undefined) {
 count++;
 newNodes[curr].children[c] = count;
 newNodes[count] = {
 id: count,
 char: c,
 parent: curr,
 children: {},
 fail: 0,
 term_link: 0,
 is_terminal: false,
 words: [],
 depth: newNodes[curr].depth + 1
 };
 }
 curr = newNodes[curr].children[c];
 }
 if (!newNodes[curr].words.includes(word)) {
 newNodes[curr].words.push(word);
 newNodes[curr].is_terminal = true;
 }
});

let currentX = 0;
const paddingX = 70;
const paddingY = 80;

const layoutDFS = (u: number, depth: number) => {
 const childKeys = Object.values(newNodes[u].children);
 newNodes[u].y = 40 + depth * paddingY;

 if (childKeys.length === 0) {
 newNodes[u].x = 40 + currentX * paddingX;
 currentX++;
 } else {
 let leftX = -1;
 let rightX = -1;
 childKeys.forEach(v => {

```

```

const removeWord = (wordToRemove: string) => {
 if (words.length <= 1) {
 setSpeechText('Оставь хотя бы одно слово, иначе мы не сможем продолжить!');
 return;
 }
 const updated = words.filter((w) => w !== wordToRemove);
 setWords(updated);
 buildInitialTrie(updated);
};

// Запуск BFS
const startBFS = () => {
 const rootChildren = Object.values(nodes[0].children);
 const queue = [...rootChildren];
 const updatedNodes = { ...nodes };
 rootChildren.forEach((childId) => {
 updatedNodes[childId].fail = 0;
 });

 setNodes(updatedNodes);
 setBfsQueue(queue);
 setSimulationStep(1);
 setCurrentNode(null);
 setSpeechText(
 'Начинаем обход в ширину (BFS)! Все вершины на главном уровне суффикса просто нет. Жми «Следующий шаг»!'
);
};

// Один шаг BFS
const stepBFS = () => {
 if (bfsQueue.length === 0) {
 setSimulationStep(2);
 setCurrentNode(null);
 setSpeechText(
 'Ура! Очередь пуста! Мы успешно построили полную таблицу переходов!'
);
 }
};

```

```

 setSpeechText(logs);
 };

 return (
 <div className="bg-slate-800/90 rounded-2xl p-6 border" >
 {/* Шапка виджета */}
 <div className="flex flex-wrap items-center justify-between" >
 <div className="flex items-center space-x-3" >

 <div>
 <h4 className="text-2xl font-bold text-white" >
 Интерактивный конструктор: Говорящий Эхо-
 </h4>
 <p className="text-sm text-slate-400" >
 Построение дерева и расчет суффиксных ссы
 </p>
 </div>
 </div>
 <div className="flex items-center gap-2" >
 <button
 onClick={() => buildInitialTrie(words)}
 className="flex items-center gap-1 bg-slate-700 text-white
 transition-colors"
 title="Сбросить автомат"
 >
 <RotateCcw className="w-4 h-4" /> Сбросить
 </button>
 </div>
 </div>

 {/* Верхняя панель: Говорящий лосось и диалоговое
 <div className="grid grid-cols-1 md:grid-cols-3 gap-4" >
 {/* Аватар Лосося (Сеня) с анимацией моргания и
 <div className="flex flex-col items-center justify-center" >
 <div className="w-40 h-40 relative flex items-center justify-center" >
 full border border-blue-500/30 shadow-inner
 {/* SVG Лосося */}
 <svg

```

```

 <path d="M 165 105 Q 155 120 165 125 Q
) : (
 <path d="M 165 110 Q 155 115 168 118" s
)}

 {/* Улыбка / жабры */}
 <path d="M 130 80 Q 125 100 130 120" stro
 <path d="M 120 85 Q 115 100 120 115" stro
</svg>

 {/* Декоративные пузыри */}
 <div className="absolute -top-1 right-4 tex
 <div className="absolute top-8 right-1 text
</div>
 <span className="mt-3 bg-rose-600/30 border b
 Эхо-Лосось Сеня

</div>

{/* Пузырь с текстом (Баббл) */}
<div className="md:col-span-2 bg-slate-800 bord
 tween min-h-[140px]">
 {/* Треугольник баббла */}
 <div className="absolute -left-3 top-1/2 -tra
 slate-800 border-b-[12px] border-b-transpar

 <div className="flex items-center space-x-2 t
 <Volume2 className={`w-4 h-4 ${isTalking ?
 Прямое вещание из аквариума:
 </div>

 <p className="text-slate-200 text-base leadin
 {speechText}
 </p>

 <div className="mt-4 pt-3 border-t border-sla
 <div className="text-xs text-slate-400 flex
 <Info className="w-3.5 h-3.5 text-blue-40

```

```

 { /* SVG Canvas for Automaton Tree */ }
 <div className="mb-8 bg-slate-900 border border-slate-800" >
 <h5 className="text-white font-bold mb-3 flex items-center" >
 Дерево и Суффиксные (fail) ссылки
 </h5>

 {(() => {
 let maxX = 0;
 let maxY = 0;
 Object.values(nodes).forEach(n => {
 if (n.x && n.x > maxX) maxX = n.x;
 if (n.y && n.y > maxY) maxY = n.y;
 });
 const svgWidth = Math.max(maxX + 80, 400);
 const svgHeight = Math.max(maxY + 60, 200);

 return (
 <svg viewBox={`0 0 ${svgWidth} ${svgHeight}`} >
 <defs>
 <marker id="arrowhead" markerWidth="6" markerHeight="6" >
 <polygone points="0 0, 6 3, 0 6" fill="white" />
 </marker>
 <marker id="fail-arrow" markerWidth="6" markerHeight="6" >
 <polygone points="0 0, 6 3, 0 6" fill="white" />
 </marker>
 </defs>

 { /* Draw Edges */ }
 {Object.values(nodes).map(node => {
 if (node.id === 0) return null;
 const parent = nodes[node.parent];
 if (!parent.x || !parent.y || !node.x || !node.y) return null;

 const isCurrent = currentNode === node.id;

 return (
 <g key={`edge-${node.id}`} >
 <line

```

```

- 16}` }
 fill="none"
 stroke={isCurrent ? '#f43f5e' : 'rgb(0,0,0)'}
 strokeWidth={isCurrent ? 2 : 1.5}
 strokeDasharray="4 4"
 markerEnd="url(#fail-arrow)"
 />
);
}}

{/* Draw Nodes */}
{Object.values(nodes).map(node => {
 if (!node.x || !node.y) return null;
 const isRoot = node.id === 0;
 const isCurrent = currentNode === node.id;
 const isInQueue = bfsQueue.includes(node.id);

 let fill = '#1e293b';
 let stroke = '#475569';

 if (isCurrent) {
 fill = '#2563eb';
 stroke = '#60a5fa';
 } else if (isInQueue) {
 fill = '#b45309';
 stroke = '#fbbf24';
 } else if (node.is_terminal) {
 fill = '#059669';
 stroke = '#34d399';
 }

 return (
 <g key={`node-${node.id}`}>
 <circle
 cx={node.x}
 cy={node.y}
 r="16"
 fill={fill}

```



```

<div className="flex flex-wrap gap-2 mb-4">
 {words.map((w) => {
 <span
 key={w}
 className="bg-slate-800 border border-
er gap-1 group"
 >
 {w}
 <button
 onClick={() => removeWord(w)}
 className="text-slate-500 hover:tex
 title="Удалить"
 >
 ×
 </button>

)}}
</div>
</div>

```

```

<form onSubmit={handleAddWord} className="fle
 <input
 type="text"
 value={newWord}
 onChange={(e) => setNewWord(e.target.valu
 placeholder="НОВОЕ СЛОВО..."
 maxLength={8}
 className="bg-slate-800 border border-sla
 outline-none focus:border-blue-500"
 />
 <button
 type="submit"
 className="bg-slate-700 hover:bg-slate-60
 ition-colors"
 >
 <Plus className="w-3.5 h-3.5" /> Добавить
 </button>
</form>

```

```

 ? 'bg-slate-800/80'
 : 'hover:bg-slate-800/40'
 }` }
>
<td className="py-2.5 px-3 flex items-center" >
 <span
 className={`w-2 h-2 rounded-full
 ${node.id === 0
 ? 'bg-purple-500'
 : isCurrent
 ? 'bg-blue-400 animate-pulse'
 : isInQueue
 ? 'bg-amber-400'
 : 'bg-slate-600'}
 }` }
 >
 #{node.id}
 {node.id === 0 &&
 </td>
<td className="py-2.5 px-3">

 {node.char}

</td>
<td className="py-2.5 px-3 text-slate-500" >
 {node.id === 0 ? '-' : `#${node.id}</td>
<td className="py-2.5 px-3" >
 {node.id === 0 ? (

) : (

 #{node.fail}

)}
</td>
<td className="py-2.5 px-3" >
 {node.id === 0 || node.term line

```

```

 {compilerLinked ? (
 <
 {liveWordsSig && words = {words.join(", ")}}
 {liveNodes !== null && вершин в боре = {liveNodes}}
 {liveState !== null && state = {liveState}}
 {liveStep !== null && шаг BFS = {liveStep}}
 {liveQueue && queue = {JSON.stringify(liveQueue).slice(0, 100)}}
 </>
) : (

 открой панель Python: массив <code className="text-mono text-cyan-300">state</code>/<code className="text-mono text-cyan-300">step</code>

)}
 </div>
 </div>
);
};

```

ФАЙЛ 67/111: src/components/SegmentTreeVisualizer.tsx  
 КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState, useEffect, useCallback, useMemo } from 'react';
import { Play, Pause, RotateCcw, ChevronLeft, ChevronRight } from 'lucide-react';
import { useVizRuntime, vizNumber } from '../data/vizStore';

```

```

// ===== TYPES =====
interface Step {
 id: number;
 desc: string;
}

```

```

 " build(2v, l, mid)", // 6
 " build(2v+1, mid+1, r)", // 7
 " tree[v] = tree[2v]+tree[2v+1]", // 8
 "}", // 9
};
void CODE;

function go(v: number, l: number, r: number) {
 if (l === r) {
 steps.push({ id: id++, desc: `build(${v}, ${l}, ${r}`,
 , highlightNodes: [v], arrayHighlight: [l, r] });
 tree[v] = arr[l];
 steps.push({ id: id++, desc: `tree[${v}] = ${arr[l]}`,
 l, r] });
 return;
 }
 const m = (l + r) >> 1;
 steps.push({ id: id++, desc: `build(${v}, ${l}, ${r}`,
 { ...tree }, highlightNodes: [v], arrayHighlight: [l, r] });
 go(2 * v, l, m);
 steps.push({ id: id++, desc: `Вернулись в build(${v}, ${l}, ${r}`,
 lightNodes: [v], arrayHighlight: [l, r] });
 go(2 * v + 1, m + 1, r);
 tree[v] = (tree[2 * v] ?? 0) + (tree[2 * v + 1] ?? 0);
 steps.push({ id: id++, desc: `Объединяем: tree[${v}] =`,
 tate: { ...tree }, highlightNodes: [v], mergeChildren: [l, r] });
}
go(1, 0, n - 1);
steps.push({ id: id++, desc: `Дерево построено! Корень:`,
 Highlight: [0, n - 1] });
return steps;
}

```

// 2. Top-Down Query steps

```

function genQueryTopDownSteps(arr: number[], qL: number, qR: number) {
 const n = arr.length;
 const tree = buildTreeFull(arr);
 const steps: Step[] = [];

```

```

let id = 0;
let l = qL + n;
let r = qR + n + 1; // half-open [l, r)
let res = 0;

steps.push({ id: id++, desc: `Старт: l = qL + n = ${qL + n}, codeLine: 3, treeState: tree, highlightNodes: [l] });

while (l < r) {
 const highlights: number[] = [];
 if (l & 1) {
 res += tree[l] ?? 0;
 steps.push({ id: id++, desc: `l=${l} нечётный → l`, codeLine: 5, treeState: tree, highlightNodes: [l], arrayHighlight: [l] });
 highlights.push(l);
 l++;
 }
 if (r & 1) {
 r--;
 res += tree[r] ?? 0;
 steps.push({ id: id++, desc: `r=${r + 1} нечётный`, codeLine: 6, treeState: tree, highlightNodes: [r], arrayHighlight: [r] });
 highlights.push(r);
 }
 l >>= 1;
 r >>= 1;
 if (l < r) {
 steps.push({ id: id++, desc: `Поднимаемся: l = ${qL + n}, codeLine: 7, treeState: tree, highlightNodes: [l], arrayHighlight: [qL, qR], result: res });
 }
}
steps.push({ id: id++, desc: `Запрос sum([${qL}..${qR}], codeLine: 8, treeState: tree, highlightNodes: [1], arrayHighlight: [qL, qR], result: res });
return steps;
}

// ===== TREE RENDERING HELPER =====
interface VNode { v: number; l: number; r: number; val: number; }

```

```

" // -----",
" if (r & 1) res += tree[--r];",
" l >>= 1; r >>= 1;",
" return res; }"
];

// ===== PLAYER COMPONENT =====
function Player({ steps, color }: { steps: Step[]; color: string }) {
 const [idx, setIdx] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [speed, setSpeed] = useState(700);

 useEffect(() => { setIdx(0); setPlaying(false); }, [steps]);

 useEffect(() => {
 if (!playing || idx >= steps.length - 1) { if (idx < steps.length - 1) {
 const t = setTimeout(() => setIdx(i => i + 1), speed);
 return () => clearTimeout(t);
 }
 }, [playing, idx, steps, speed]);

 const step = steps[idx] || steps[0];
 if (!step) return null;

 const clr = { indigo: { btn: 'bg-indigo-600 hover:bg-indigo-500 border-indigo-500 bg-indigo-600', childRing: 'ring-indigo-900' }, emerald: { btn: 'bg-emerald-600 hover:bg-emerald-500 border-emerald-500 bg-emerald-600', childRing: 'ring-emerald-900' }, amber: { btn: 'bg-amber-600 hover:bg-amber-500 border-amber-500 bg-amber-600', childRing: 'ring-rose-900' } };
 const [color] = clr[color];

 return { step, idx, setIdx, playing, setPlaying, speed };
}

// ===== SIMULATION PANEL =====
function SimPanel({ title, icon, steps, code, color, arr }: { title: string; icon: string; steps: Step[]; code: string; color: string; arr: number[] }) {
 const p = Player({ steps, color });

```

```

<div className={`rounded-2xl border overflow-hidden
 ? 'border-emerald-500/30' : 'border-amber-500/30'}
 {/* Header */}
 <div className={`px-4 py-2.5 flex items-center justify-between
 -emerald-950/50' : 'bg-amber-950/50'} border-b border-emerald-500/30`>
 <h4 className="font-bold text-sm text-white flex-1">

 </div>

```

```

 {/* Array strip */}
 <div className="px-3 py-2 bg-slate-950 border-b border-emerald-500/30"
 {arr.map((v, i) => {
 const inRange = step.arrayHighlight && i >= step.startIndex;
 return (
 <div key={i} className={`flex flex-col items-center justify-between
 go' ? 'bg-indigo-950 border-indigo-500' : 'bg-emerald-950 border-emerald-500'}
 -translate-y-0.5` : 'bg-slate-900 border-slate-500/30'`>

 </div>
);
 })}
 </div>

```

```

 {/* Tree */}
 <div className="flex-1 p-2 overflow-x-auto bg-slate-950"
 <div className="scale-[0.85] origin-top min-w-max"
 </div>

```

```

 {/* Code */}
 <pre className="px-3 py-2 text-[9px] md:text-[10px] font-mono"
 x">
 {code.map((line, i) => {
 const ln = i + 1;
 const active = step.codeLine === ln;
 return (
 <div key={ln} className={`px-2 py-0.5 rounded
 2 border-indigo-500 text-indigo-200' : 'text-slate-500'`>

```

```

 <option value={1200}>Медл.</option>
 <option value={700}>Норма</option>
 <option value={350}>Быстро</option>
 <option value={120}>Турбо</option>
 </select>
 </div>

 { /* Progress bar */ }
 <div className="h-1 bg-slate-950"><div className=
 + 1) / total) * 100 : 0}%` }} /></div>
 </div>
);
}

// =====
// MAIN EXPORTED COMPONENT
// =====
export function SegmentTreeVisualizer() {
 const runtime = useVizRuntime();
 const liveI = vizNumber(runtime?.variables?.i);
 const [arr, setArr] = useState<number[]>([5, 8, 3, 12]);
 const [customInput, setCustomInput] = useState('5, 8,');
 const [qL, setQL] = useState(1);
 const [qR, setQR] = useState(4);
 const [view, setView] = useState<'build' | 'queries'>

 // Clamp query range when array changes
 useEffect(() => {
 setQL(prev => Math.min(prev, arr.length - 1));
 setQR(prev => Math.min(prev, arr.length - 1));
 }, [arr]);

 const handleApply = (e: React.FormEvent) => {
 e.preventDefault();
 const parsed = customInput.split(',').map(s => parseInt(s));
 if (parsed.length >= 2 && parsed.length <= 16) {
 setArr(parsed);
 }
 }
}

```



```

 </div>
 <div className="flex flex-wrap gap-1.5 pt-1" >
 {arr.map(({v, i}) => (
 {v}
 {i}

 </div>
 </div>
 </form>

```

```

 {/* Query range */}
 <div className="bg-slate-950 p-4 rounded-xl border border-slate-800" >
 <label className="text-[10px] font-bold uppercase" > Query range
 <div className="flex items-center gap-2" >
 <div className="flex items-center gap-1" >
 Range
 <input
 type="number"
 min={0}
 max={arr.length - 1}
 value={qL}
 onChange={e => setQL(Math.min(Number(e.target.value), arr.length - 1))}
 className="w-14 bg-slate-900 border border-slate-800 focus:border-indigo-500"
 />
 </div>
 <div className="flex items-center gap-1" >
 Range
 <input
 type="number"
 min={0}
 max={arr.length - 1}
 value={qR}
 onChange={e => setQR(Math.min(Number(e.target.value), arr.length - 1))}
 className="w-14 bg-slate-900 border border-slate-800 focus:border-indigo-500"
 />
 </div>
 </div>
 </div>

```

```

 arr={arr}
 />
 })

 {view === 'queries' && (
 <div className="grid grid-cols-1 xl:grid-cols-2"
 <SimPanel
 key={`qtd-${arr.join('-')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Сверху`
 icon="↑"
 steps={queryTDSteps}
 code={QUERY_TD_CODE}
 color="emerald"
 arr={arr}
 />
 <SimPanel
 key={`qbu-${arr.join('-')}-${qL}-${qR}`}
 title={`Запрос sum([${qL}..${qR}]) – Снизу`
 icon="↓"
 steps={queryBUSteps}
 code={QUERY_BU_CODE}
 color="amber"
 arr={arr}
 />
 </div>
)}

 {view === 'theory' && (
 <div className="space-y-5">
 {/* Why it splits this way */}
 <div className="bg-slate-950 p-5 rounded-xl b
 <h4 className="text-base font-bold text-whi
 во разбилось именно так для N={arr.length}?
 <p className="text-sm text-slate-300 leadin
 Каждый узел берёт <code className="bg-sla
 к получает <code className="bg-slate-900 px
 -slate-900 px-1 rounded font-mono text-emerg
 евая часть забирает на 1 больше (округление

```



```

 <Compass className="w-4 h-4 text-indigo-400" />
 </h3>
 {!compact && (
 <div className="relative">
 <Search className="w-4 h-4 text-slate-500 absolute left-2 top-1/2 -tran
 <input
 value={query}
 onChange={(e) => setQuery(e.target.value)}
 placeholder="Поиск по темам.."
 className="w-full bg-slate-800/70 border border-bo
 e-500 focus:outline-none focus:border-indigo
 />
 {query && (
 <button
 onClick={() => setQuery("")}
 className="absolute right-2 top-1/2 -tran
 aria-label="Очистить поиск"
 >
 <X className="w-3.5 h-3.5" />
 </button>
)}
 </div>
)}
</div>

<div className="flex-1 overflow-y-auto px-3 pb-4">
 {groups.length === 0 && <p className="text-sm text-slate-500">
 Нет результатов
 }

 {groups.map(({ section, items }) => (
 <div key={section.id}>
 <div className="flex items-baseline gap-1.5">

 {section.from > 0 && (

 {section.from}

)}
 </div>
 <div className="space-y-1">
 {items.map((chapter) => {

```

```

 />
 })
 </button>
);
 }}}
</div>
</div>
)))
</div>

<div className={`shrink-0 m-3 mt-0 bg-gradient-to
 x] text-slate-300 space-y-1.5 hidden lg:block $
 <div className="font-bold text-indigo-400 flex .
 <Sparkles className="w-3.5 h-3.5" /> Как поль
 </div>
 <p className="leading-relaxed">
 <b className="text-slate-200">Порядок чтения
 номерам билетов 1 → 24, каждая опирается на п
 Следующая тема» продолжают этот же порядок.
 </p>
 <p className="leading-relaxed">
 Подчёркнутые термины в тексте раскрываются по
 симулятора.
 </p>
 <p className="text-slate-400 italic">Стрелки ←
 </div>
</aside>
);
};

```

ФАЙЛ 69/111: src/components/Simulator.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```

import React, { useState } from 'react';
import { Layers, AlignLeft, Award, Plus, ArrowRight, Ar

```

```

// Heap state
const [heap, setHeap] = useState<Hypothesis[]>([
 { id: '1', text: 'Кандидат: "Привет, я искусственный"', },
 { id: '2', text: 'Кандидат: "Привет, я испек вкусный"', },
 { id: '3', text: 'Кандидат: "Привет, я испанский ин"', },
]);
const [newCustomText, setNewCustomText] = useState('');
const [newCustomProb, setNewCustomProb] = useState(0.1);
const [heapMessage, setHeapMessage] = useState<string>('');

// Stack handlers
const addPlate = () => {
 const presets = [
 { name: 'Сковорода от омлета', icon: '🥞' },
 { name: 'Кастрюля от супа', icon: '🍲' },
 { name: 'Чашка от кофе', icon: '☕' },
 { name: 'Миска с остатками салата', icon: '🥗' },
 { name: 'Тарелка от тако', icon: '🌮' },
];
 const randomPreset = presets[Math.floor(Math.random() * presets.length)];
 const newPlate = {
 id: Date.now().toString(),
 name: randomPreset.name,
 icon: randomPreset.icon
 };
 setStack(prev => [...prev, newPlate]);
 setPopMessage(`Положили поверх стопки: ${newPlate.name} 🍽️`);
};

const popPlate = () => {
 if (stack.length === 0) {
 setPopMessage("⚠️ Стопка пуста! Все тарелки вымыты");
 return;
 }
 const topPlate = stack[stack.length - 1];
 setStack(prev => prev.slice(0, prev.length - 1));
 setPopMessage(`Вымыта верхняя тарелка (LIFO): ${topPlate.name} 🍽️`);
};

```

```

const resetQueue = () => {
 setQueue([
 { id: '1', name: 'Студент Вася', icon: ' ' },
 { id: '2', name: 'Голодный кодер', icon: ' ' },
 { id: '3', name: 'Кот Борис', icon: ' ' },
]);
 setDequeueMessage(" Очередь восстановлена.");
};

// Heap handlers
const addHypothesis = (e: React.FormEvent) => {
 e.preventDefault();
 if (!newCustomText.trim()) return;
 const item: Hypothesis = {
 id: Date.now().toString(),
 text: `Кандидат: "${newCustomText}"`,
 probability: Number(newCustomProb)
 };
 setHeap(prev => [...prev, item]);
 setNewCustomText('');
 setHeapMessage(` Добавлена гипотеза с вероятностью`);
};

const popHeap = () => {
 if (heap.length === 0) {
 setHeapMessage("⚠ Куча пуста! Нет кандидатов для");
 return;
 }
 // Find highest probability
 let maxIdx = 0;
 for (let i = 1; i < heap.length; i++) {
 if (heap[i].probability > heap[maxIdx].probability) {
 maxIdx = i;
 }
 }
 const best = heap[maxIdx];
 setHeap(prev => prev.filter((_, idx) => idx !== maxIdx));
 setHeapMessage(` Выбран самый "тяжелый" кандидат:

```

```

 <div className="text-left">
 <div className="font-bold text-sm text-wh
 <div className="text-xs opacity-80">LIFO
 </div>
 </div>
 <span className="text-xs bg-blue-500/20 text-l
 DFS

 </button>

 <button
 onClick={() => setActiveTab('queue')}
 className={`flex items-center justify-between
 activeTab === 'queue'
 ? 'bg-indigo-600/20 border-indigo-500 tex
 : 'bg-slate-800/60 border-slate-700/60 te
 }`
 >
 <div className="flex items-center gap-3">
 <AlignLeft className={`w-5 h-5 ${activeTab :
 <div className="text-left">
 <div className="font-bold text-sm text-wh
 <div className="text-xs opacity-80">FIFO
 </div>
 </div>
 </div>
 <span className="text-xs bg-indigo-500/20 tex
 BFS

 </button>

 <button
 onClick={() => setActiveTab('heap')}
 className={`flex items-center justify-between
 activeTab === 'heap'
 ? 'bg-emerald-600/20 border-emerald-500 t
 : 'bg-slate-800/60 border-slate-700/60 te
 }`
 >

```



```

border-blue-500/40 px-4 py-2.5 rounded-xl
>
 Помыть верхнюю
</button>
<button
 onClick={resetStack}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-
tion-colors cursor-pointer"
>
 <RotateCcw className="w-5 h-5" />
</button>
</div>
</div>

{popMessage && {
 <div className="bg-blue-950/60 border borde
imate-fade-in">
 <span className="inline-block w-2 h-2 roun
 {popMessage}
 </div>
}}

{/* Visualization */}
<div className="bg-slate-950/60 p-6 rounded-2
>
 {stack.length === 0 ? {
 <div className="text-center py-12 text-sl
 <div className="text-5xl mb-3"> </div>
 <p className="text-base font-bold">Стон
 <p className="text-xs mt-1">Добавьте гр
 </div>
) : {
 <div className="w-full max-w-sm flex flex
 <div className="absolute top-0 text-xs
full flex items-center gap-1">
 ВЕРШИНА СТЕКА (TOP)
 <ArrowDown className="w-3.5 h-3.5 ani

```

```

 { /* QUEUE TAB */ }
 { activeTab === 'queue' && {
 <div className="space-y-6">
 <div className="bg-slate-800/80 p-5 rounded-x
 tify-between gap-4">
 <div>
 <h3 className="text-lg font-bold text-whi
 Симуляция очереди за супом
 <span className="text-xs font-mono bg-in
 /span>
 </h3>
 <p className="text-slate-400 text-xs mt-1
 Кто первым пришел, тот первым получил с
 </p>
 </div>
 <div className="flex flex-wrap items-center
 <button
 onClick={addPerson}
 className="flex-1 md:flex-none flex ite
 -2.5 rounded-xl text-sm font-bold shadow-lg
 >
 <Plus className="w-4 h-4" /> Встать в о
 </button>
 <button
 onClick={servePerson}
 className="flex-1 md:flex-none flex ite
 er border-indigo-500/40 px-4 py-2.5 rounded
 >
 Налить суп первому
 </button>
 <button
 onClick={resetQueue}
 title="Сбросить"
 className="p-2.5 bg-slate-800 hover:bg-
 tion-colors cursor-pointer"
 >
 <RotateCcw className="w-5 h-5" />
 </button>

```

```

 isFirst
 ? 'bg-gradient-to-b from-indigo-500 to-slate-900'
 : 'bg-slate-900/90 border-slate-900'
 }` }
 >
 {isFirst && {
 <span className="absolute -top-10 left-1/2 transform translate-x-50%
se tracking-wider shadow">
 Первый (Head)

 }}
 {

 {person.name}

 {isFirst ? 'bg-indigo-500/30 text-white' : 'bg-slate-900/30 text-slate-500'}

 </div>
);
}}

<div className="flex flex-col items-center justify-center gap-6 min-w-[120px] h-[120px]">
 <Plus className="w-6 h-6 mb-1" />

 </div>
</div>
)}
<div className="mt-6 text-xs text-slate-500">
 ПЕРВЫМ ПРИШЕЛ → ПЕРВЫМ ПОЛУЧИЛ (FIFO) • ИЛИ
</div>
</div>
</div>
})

```

```

<div className="md:col-span-6">
 <label className="block text-xs font-bold">
 <input
 type="text"
 value={newCustomText}
 onChange={e => setNewCustomText(e.target.value)}
 placeholder="например: 'Привет, я лучший!'"
 className="w-full bg-slate-900 border border-emerald-500 transition-colors"
 />
 </div>
<div className="md:col-span-3">
 <label className="block text-xs font-bold">
 Вероятность:
 </label>
 <input
 type="range"
 min="0.01"
 max="0.99"
 step="0.01"
 value={newCustomProb}
 onChange={e => setNewCustomProb(Number(e.target.value))}
 className="w-full accent-emerald-500 cursor-pointer"
 />
</div>
<div className="md:col-span-3">
 <button
 type="submit"
 disabled={!newCustomText.trim()}
 className="w-full flex items-center justify-center text-emerald-500/40 disabled:opacity-50 disabled:cursor-not-allowed"
 >
 <Plus className="w-4 h-4" /> Добавить в список
 </button>
</div>
</form>

```

```

 <span className={`flex items-center
 isTop ? 'bg-emerald-500 text-
 }`}>
 #{index + 1}

 <div>
 <div className={`font-bold te
 {item.text}
 </div>
 {isTop && (
 <span className="text-[10px
-0.5 rounded inline-block mt-1">
 Вершина Кучи (Root) • Буд

)}
 </div>
 </div>

 <div className="flex items-center
 /* Custom progress bar */
 <div className="hidden sm:block
 <div
 className={`h-full rounded-
 style={{ width: `${percent}%
 ></div>
 </div>
 <span className={`font-mono font
 isTop ? 'text-emerald-400 tex
 }`}>
 {percent}%

 </div>
</div>
);
 }}}}
</div>
)}
<div className="mt-6 text-xs text-slate-500

```

```
 }, [chapterId, tab]));
```

```
return (
```

```
 <div className="bg-slate-800/90 p-6 rounded-2xl border" >
```

```
 <div className="flex flex-wrap items-center justify-center" >
```

```
 <div className="flex items-center gap-3" >
```

```
 <div className="p-2.5 bg-indigo-600 text-white" >
```

```

```

```
 </div>
```

```
 <div>
```

```
 <h3 className="text-2xl font-bold text-white" >
```

```
 <p className="text-sm text-indigo-300" >Один
```

```
 </div>
```

```
 </div>
```

```
 </div>
```

```
 <div className="flex flex-wrap gap-2 mb-5" >
```

```
 {[
```

```
 { id: "cases" as Tab, label: "1 · Механика: Z"
```

```
 { id: "big" as Tab, label: "2 · Большое дерево"
```

```
 { id: "play" as Tab, label: "3 · Собери сам"
```

```
]).map((t) => {
```

```
 <button
```

```
 key={t.id}
```

```
 onClick={() => setTab(t.id)}
```

```
 className={`px-4 py-2 rounded-lg text-xs font-medium
```

```
 ${tab === t.id ? "bg-indigo-600 text-white" : ""}
```

```
 `}
```

```
 >
```

```
 {t.label}
```

```
 </button>
```

```
])
```

```
 </div>
```

```
 {tab === "cases" && <SplayRotationsViz />}
```

```
 {tab === "big" && <SplayWalkViz />}
```

```
 {tab === "play" && <SplayRotationSandbox />}
```

```

t ? { key: t.key, left: clone(t.left), right: clone(t
/** Заготовки: линия (zig-zig), змейка (zig-zag) и прос
export const PRESETS: { name: string; hintCase: string;
{
 name: "Линия влево",
 hintCase: "Zig-Zig",
 target: 1,
 build: () => {
 const n1 = leaf(1);
 const n2: TNode = { key: 2, left: n1, right: leaf
 const n4: TNode = { key: 4, left: n2, right: leaf
 return { key: 6, left: n4, right: leaf(7) };
 },
},
{
 name: "Змейка",
 hintCase: "Zig-Zag",
 target: 3,
 build: () => {
 const n3 = leaf(3);
 const n2: TNode = { key: 2, left: leaf(1), right:
 const n4: TNode = { key: 4, left: n2, right: leaf
 return { key: 6, left: n4, right: leaf(7) };
 },
},
{
 name: "Один уровень",
 hintCase: "Zig",
 target: 2,
 build: () => ({ key: 4, left: { key: 2, left: leaf(
},
{
 name: "Смешанный",
 hintCase: "Zig-Zag, затем Zig-Zig",
 target: 5,
 build: () => {
 const n5 = leaf(5);

```

```

 if (gg.left === p) gg.left = x;
 else gg.right = x;
 return root;
}

/** Какой поворот сделал бы настоящий splay из текущего
export function canonicalNext(root: TNode, key: number)
 const path = findPath(root, key);
 if (path.length < 2) return null;
 const x = path[path.length - 1];
 const p = path[path.length - 2];
 const g = path.length >= 3 ? path[path.length - 3] : null;

 if (!g) return { node: x.key, kind: "Zig" };

 const xLeft = p.left === x;
 const pLeft = g.left === p;
 if (xLeft === pLeft) return { node: p.key, kind: "Zig" };
 return { node: x.key, kind: "Zig-Zag (сначала нижняя поворот)" };
}

/* ----- раскладка -----

interface Placed {
 key: number;
 x: number;
 y: number;
 depth: number;
}

function layout(root: TNode | null): { nodes: Placed[];
 const nodes: Placed[] = [];
 const edges: [number, number][] = [];
 let order = 0;

 const walk = (n: TNode | null, depth: number) => {
 if (!n) return;
 walk(n.left, depth + 1);

```



```

 setMoves([]);
 setShowAnswer(false);
 },
 [presetIdx]
);

const { nodes, edges, width, height } = useMemo(() =>
const path = useMemo(() => new Set(findPath(tree, target)));
const expected = useMemo(() => (solved ? null : canonicalNext(tree, target)));

const clickNode = (key: number) => {
 if (solved) return;
 const p = findPath(tree, key);
 if (p.length < 2) return; // корень крутить не с кем

 const want = canonicalNext(tree, target);
 setHistory((h) => [...h, clone(tree) as TNode]);
 setMoves((m) => [...m, { node: key, correct: want?.node === key }]);
 setTree((t) => rotateUp(clone(t) as TNode, key));
};

const undo = () => {
 if (history.length === 0) return;
 setTree(history[history.length - 1]);
 setHistory((h) => h.slice(0, -1));
 setMoves((m) => m.slice(0, -1));
};

const wrong = moves.filter((m) => !m.correct).length;
const minimal = useMemo(() => findPath(preset.build(), target));

return (
 <div className="w-full bg-slate-950 rounded-2xl border-slate-700">
 <div className="flex items-start justify-between p-4">
 <div>
 <h4 className="text-sm sm:text-base font-bold">
 <p className="text-[12px] text-slate-400 lead">
 Клик по узлу = один поворот с его родителем

```

```

 const onPath = path.has(a) && path.has(b);
 return {
 <line
 key={i}
 x1={na.x}
 y1={na.y}
 x2={nb.x}
 y2={nb.y}
 stroke={onPath ? "#6366f1" : "#475569"}
 strokeWidth={onPath ? 3 : 1.8}
 style={{ transition: "all 700ms cubic-bezier(0.4, 0, 0.2, 1)" }}
 />
);
 }
}

{nodes.map((n) => {
 const isTarget = n.key === target;
 const clickable = !solved && n.depth > 0;
 return {
 <g
 key={n.key}
 onClick={() => clickable && clickNode(n)}
 style={{
 transform: `translate(${n.x}px, ${n.y}px)`,
 transition: "transform 700ms cubic-bezier(0.4, 0, 0.2, 1)",
 cursor: clickable ? "pointer" : "default",
 }}
 >
 {isTarget && {
 <circle r={22} fill="none" stroke="#6366f1" strokeWidth={2} />
 }}
 <circle
 r={16}
 fill={isTarget ? "#6366f1" : path.has(a) && path.has(b) ? "#a5b4fc" : "#475569"}
 stroke={isTarget ? "#a5b4fc" : "#475569"}
 strokeWidth={2}
 />
 <text textAnchor="middle" dominantBaseline="middle" />
 </g>
 };
})}

```

```


 </div>

 {showAnswer && !solved && expected && {
 <div className="mt-3 text-[12px] bg-amber-950/40" >
 Сейчас splay крутил бы узел{" "}
 {expected}
 </div>
 }}

 {solved && {
 <div
 className={`mt-3 rounded-lg border px-3 py-2.5`}
 wrong === 0
 ? "bg-emerald-950/40 border-emerald-700/60"
 : "bg-rose-950/30 border-rose-800/60 text-rose-50"
 >
 <div className="flex items-center gap-2 font-mono" >
 {wrong === 0 ? <CheckCircle2 className="w-4 h-4" /> : <X className="w-4 h-4" />}
 {wrong === 0
 ? `Узел ${target} в корне за ${moves.length} ходов`
 : `Узел ${target} в корне за ${moves.length} ходов`}
 </div>
 {wrong > 0 && {
 <ul className="list-disc pl-5 space-y-0.5">
 {moves.map(
 (m, i) =>
 !m.correct && {
 <li key={i}>
 ход {i + 1}: крутили {m.expected}

 }
)
)}

 }}
 >
 {wrong === 0 && moves.length > minimal && {

```

```

edges: [NodeId, NodeId][];
kind: "start" | "aim" | "result";
title: string;
text: string;
/** Пара, которую крутим на этом шаге: кадр-«прицел».
focus?: [NodeId, NodeId];
/** Кто переехал на этом кадре – рисуем призрак и стрелку.
moved?: NodeId[];
ms: number;
}

interface Case {
 key: "zig" | "zig-zig" | "zig-zag";
 label: string;
 when: string;
 rotations: string;
 /** Номера узлов: роль → число. */
 keys: Partial<Record<NodeId, number>>;
 /** Обход слева направо – одинаковый на всех кадрах.
inorder: string[];
 /** Что означают поддеревья A..D. */
 ranges: string;
 frames: Frame[];
}

const VB = { w: 360, h: 285 };
const AIM_MS = 2600;
const RESULT_MS = 3400;
const MOVE = "transform 1200ms cubic-bezier(.34,.01,.2,

/* ----- Zig -----
const ZIG_BEFORE = {
 pos: { p: [195, 50], x: [105, 130], C: [280, 130], A:
 edges: [["p", "x"], ["p", "C"], ["x", "A"], ["x", "B"]
} as unknown as Pick<Frame, "pos" | "edges">;

const ZIG: Case = {
 key: "zig",

```

```
edges: [{"g", "p"}, {"g", "D"}, {"p", "x"}, {"p", "C"}
] as unknown as Pick<Frame, "pos" | "edges">;
```

```
const ZZ_S1 = {
 pos: { p: [165, 40], x: [85, 112], C: [212, 112], g:
 edges: [{"p", "x"}, {"p", "g"}, {"x", "A"}, {"x", "B"}
] as unknown as Pick<Frame, "pos" | "edges">;
```

```
const ZIGZIG: Case = {
 key: "zig-zig",
 label: "Zig-Zig",
 when: "x и p – дети с ОДНОЙ стороны (оба слева или оба справа)",
 rotations: "2 поворота, первым – верхний",
 keys: { x: 20, p: 40, g: 60 },
 inorder: ["A", "20", "B", "40", "C", "60", "D"],
 ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
 frames: [
 {
 ...ZZ_S0,
 kind: "start",
 title: "Было: прямая линия 60 → 40 → 20",
 text: "Три узла выстроились в «бамбук»: каждый следующий узел
зглаживает.",
 ms: RESULT_MS,
 },
 {
 ...ZZ_S0,
 kind: "aim",
 title: "Прицел 1: ВЕРХНЕЕ ребро 60 – 40",
 text: "Главная хитрость этого случая: первым крутит
focus: ["g", "p"],
 ms: AIM_MS,
 },
 {
 ...ZZ_S1,
 kind: "result",
 title: "Поворот 1: 40 поднялось над 60",
 text: "40 встало в корень, 60 опустилось к нему почти
наравне с 20"
```

```

key: "zig-zag",
label: "Zig-Zag",
when: "x и p – дети с РАЗНЫХ сторон (змейка)",
rotations: "2 поворота, первым – нижний",
keys: { x: 40, p: 20, g: 60 },
inorder: ["A", "20", "B", "40", "C", "60", "D"],
ranges: "A < 20 · B: 20–40 · C: 40–60 · D > 60",
frames: [
 {
 ...ZG_S0,
 kind: "start",
 title: "Было: 60 → влево на 20 → вправо на 40",
 text: "Узлы идут «змейкой»: сначала шаг влево, по",
 ms: RESULT_MS,
 },
 {
 ...ZG_S0,
 kind: "aim",
 title: "Прицел 1: НИЖНЕЕ ребро 20 – 40",
 text: "Дед (60) стоит на месте и в этом повороте",
 focus: ["p", "x"],
 ms: AIM_MS,
 },
 {
 ...ZG_S1,
 kind: "result",
 title: "Поворот 1: змейка выпрямилась",
 text: "40 заняло место 20 и забрало его к себе ле",
 moved: ["x", "p", "C"],
 ms: RESULT_MS,
 },
 {
 ...ZG_S1,
 kind: "aim",
 title: "Прицел 2: ребро 60 – 40",
 text: "Остался одиночный поворот вокруг деда – и",
 focus: ["g", "x"],
 }
]

```

```

<svg viewBox={`0 0 ${VB.w} ${VB.h}`} className="w-f
 <defs>
 <marker id="splay-arrow" viewBox="0 0 10 10" re
 <path d="M0,0 L10,5 L0,10 z" fill="#a5b4fc" />
 </marker>
 </defs>

```

*{/\* призраки: где узел стоял до поворота \*/}*

```

{ghosts.map((id) => {
 const from = prev[id]!;
 const to = pos[id]!;
 const r = isSubtree(id) ? 15 : 20;
 const dx = to[0] - from[0];
 const dy = to[1] - from[1];
 const len = Math.hypot(dx, dy) || 1;
 const pad = r + 8;
 return (
 <g key={`ghost-${id}`} opacity={0.55}>
 <circle cx={from[0]} cy={from[1]} r={r} fill="#a5b4fc" />
 {len > pad * 2 + 6 && (
 <line
 x1={from[0] + (dx / len) * pad}
 y1={from[1] + (dy / len) * pad}
 x2={to[0] - (dx / len) * pad}
 y2={to[1] - (dy / len) * pad}
 stroke="#a5b4fc"
 strokeWidth={1.5}
 strokeDasharray="4 3"
 markerEnd="url(#splay-arrow)"
 />
)}
 </g>
);
})}

```

```

{edges.map(([a, b], i) => {
 const pa = pos[a];
 const pb = pos[b];

```

```

 fontSize={sub ? 12 : 15}
 fontWeight="700"
 fill={sub ? "#94a3b8" : "#fff"}
 >
 {sub ? id : keys[id]}
 </text>
 {!sub && (
 <text textAnchor="middle" y={-r - 7} font
 {id}
 </text>
)}
 </g>
);
}}
</svg>
);
};

```

```

/** Имя случая из кода пользователя → вкладка демонстра
const CASE_BY_NAME: Record<string, number> = {
 zig: 0, "zig-zig": 1, zigzig: 1, "zig zig": 1, "zig-z
};

```

```

export const SplayRotationsViz: React.FC = () => {
 const [caseIdx, setCaseIdx] = useState(0);
 const [frame, setFrame] = useState(0);
 const [playing, setPlaying] = useState(false);
 const [slow, setSlow] = useState(false);

```

```

/* — связь с Python-компилятором: case переключает в
const runtime = useVizRuntime();
const vars = runtime?.variables;
const liveCase = vars ? vizString(vars.case) ?? vizSt
const liveStep = vars ? vizNumber(vars.step) ?? vizNum
const liveX = vars ? vizNumber(vars.xk) ?? vizNumber(
const liveP = vars ? vizNumber(vars.pk) ?? vizNumber(
const liveG = vars ? vizNumber(vars.gk) ?? vizNumber(
const lastSig = useRef<string | null>(null);

```



```

return (
 <div className="w-full bg-slate-950 rounded-2xl border-2 border-slate-800">
 <div className="flex gap-2 mb-4 overflow-x-auto no-scrollbar">
 {CASES.map((c, i) => (
 <button
 key={c.key}
 onClick={() => setCaseIdx(i)}
 className={`px-3 sm:px-4 py-2 rounded-lg text-sm transition-colors
 i === caseIdx ? "bg-indigo-600 text-white" : "bg-slate-800 text-slate-300"
 `}
 >
 {c.label}
 </button>
))}
 </div>

 <div className="mb-3 text-xs sm:text-[13px] text-slate-400">
 Корреляция
 ({active.rotation}°)
 </div>

 {/* шаг + пояснение стоят НАД картинкой: сначала шаг, потом пояснение */}
 <div className="mb-3 rounded-xl border border-slate-800">
 <div className="flex items-center gap-2 mb-1 flex-wrap">
 <span
 className={`text-[10px] font-bold uppercase transition-colors
 current.kind === "aim"
 ? "bg-amber-500/20 text-amber-300"
 : current.kind === "start"
 ? "bg-slate-700 text-slate-300"
 : "bg-indigo-500/20 text-indigo-300"
 `}
 >
 {current.kind === "aim" ? "прицел" : current.kind === "start" ? "начало" : "конец"}

 Шаг {idx + 1} из {total}. {current.title}
 </div>

```

```
</div>
```

```
<div className="flex flex-wrap items-center gap-2"
```

```
 <button
```

```
 onClick={() => {
```

```
 setPlaying(false);
```

```
 setFrame((f) => (f - 1 + total) % total);
```

```
 }}
```

```
 disabled={idx === 0}
```

```
 className="px-3 py-2 rounded-lg bg-slate-800 l
```

```
 d: hover: text-slate-300 text-xs font-bold tr
```

```
 >
```

```
 ← Назад
```

```
</button>
```

```
<button
```

```
 onClick={() => {
```

```
 setPlaying(false);
```

```
 setFrame((f) => (f + 1) % total);
```

```
 }}
```

```
 className="px-4 py-2 rounded-lg bg-indigo-600
```

```
 w-indigo-900/40"
```

```
>
```

```
 {last ? "⏮ Сначала" : "Дальше →"}
```

```
</button>
```

```
<button
```

```
 onClick={() => setPlaying((p) => !p)}
```

```
 className="flex items-center gap-1.5 px-3 py-1
```

```
 ext-xs font-bold transition-colors"
```

```
>
```

```
 {playing ? <Pause className="w-3.5 h-3.5" />
```

```
 {playing ? "Стоп" : "Авто"}
```

```
</button>
```

```
<button
```

```
 onClick={() => setSlow((s) => !s)}
```

```
 title="Автопрокрутка ещё медленнее"
```

```
 className={`flex items-center gap-1.5 px-3 py-1
```

```
 slow
```

```
 ? "bg-emerald-600/20 border-emerald-500 tr
```

```

 })
 {liveX !== null && <span className="text-[1
py-1">x = {liveX}}
 {liveP !== null && <span className="text-[1
py-1">p = {liveP}}
 {liveG !== null && <span className="text-[1
py-1">g = {liveG || "нет (Zig)"}}
 {liveStep !== null && <span className="text
-2 py-1">step = {liveStep}}
 </>
) : (

 запусти код из панели Python: переменная <с
случай,
 а <code className="font-mono text-indigo-300

)}
</div>

<p className="mt-2 text-[11px] text-slate-500">
 Листай кнопкой «Дальше» в своём темпе – кадры н
</p>
</div>
);
};

```

ФАЙЛ 73/111: src/components/SplayWalkViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

```
import React, { useEffect, useMemo, useRef, useState } from 'react';
```

```
/**
```

```
 * Splay на БОЛЬШОМ дереве – по урокам treap-прохода.
```

```
 *
```

```
 * Главный принцип: НИКАКИХ «хоп». Каждый одиночный пов
```

```

 if (!cur.left) {
 cur.left = { key, left: null, right: null, parent: r };
 return r;
 }
 cur = cur.left;
 } else {
 if (!cur.right) {
 cur.right = { key, left: null, right: null, parent: r };
 return r;
 }
 cur = cur.right;
 }
}
}
}

```

/\*\* Поднять child над его родителем (один атомарный поворот)

```

export function rotateUp(child: SN): void {
 const p = child.parent;
 if (!p) return;
 const g = p.parent;
 if (p.left === child) {
 p.left = child.right;
 if (child.right) child.right.parent = p;
 child.right = p;
 p.parent = child;
 } else {
 p.right = child.left;
 if (child.left) child.left.parent = p;
 child.left = p;
 p.parent = child;
 }
 child.parent = g;
 if (!g) return;
 if (g.left === p) g.left = child;
 else g.right = child;
}

```

/\*\* Рёбра в виде "p→c" — для diff между кадрами. \*/

```

 rotCount: number;
}

/** Полный пошаговый прогон splay(key): спуск → (прицел
export function splaySteps(
 rootIn: SN | null,
 key: number,
 isInsert = false,
): { steps: SplayStep[]; found: boolean } {
 let root: SN | null = cloneWithParents(rootIn);
 const steps: SplayStep[] = [];
 if (!root)
 return { steps: [{ kind: "done", root: null, note:

// --- спуск ---
let cur: SN | null = root;
const pathKeys: number[] = [];
let found = false;
let last: SN = root;
while (cur) {
 last = cur;
 pathKeys.push(cur.key);
 if (key === cur.key) {
 found = true;
 break;
 }
 cur = key < cur.key ? cur.left : cur.right;
}
const x = found ? cur : last;
steps.push({
 kind: "search",
 root: cloneWithParents(root)!,
 pathKeys: [...pathKeys],
 note: isInsert
 ? `Вставили ${key} обычным BST-спуском. Теперь по
 : found
 ? `Спуск по ключу ${key}: путь ${pathKeys.join(
 : `Ключа ${key} нет. Спуск унёлся в ${last.key}

```

```

const diff = diffEdges(before, collectEdges(root));
steps.push({
 kind: "rot",
 root: cloneWithParents(root)!,
 caseLabel: label,
 note: `Поворот: ${child.key} встал над ${p.key}.`,
 rotCount,
});
};

while (target.parent) {
 const p = target.parent;
 const g = p.parent;
 if (!g) {
 emitRotation(
 target,
 "Zig",
 [p.key, target.key],
 `Zig: у ${target.key} нет деда — хватит одного
);
 continue;
 }
 const pIsLeft = g.left === p;
 const xIsLeft = p.left === target;
 if (pIsLeft === xIsLeft) {
 const side = pIsLeft ? "оба слева" : "оба справа"
 emitRotation(
 p,
 "Zig-Zig",
 [g.key, p.key],
 `Zig-Zig (${side}: линия ${g.key}—${p.key}—${ta
 ${g.key}, а ${target.key} пока не трогаем.`
);
 steps.push({
 kind: "aim",
 root: cloneWithParents(root)!,
 aimEdge: [target.parent.key, target.key],
 caseLabel: "Zig-Zig",

```

```

 rotCount,
 });
 return { steps, found };
}

/* ----- служебное -----

const depthOf = (t: SN | null, key: number): number =>
 let d = 0;
 let cur = t;
 while (cur) {
 if (key === cur.key) return d;
 cur = key < cur.key ? cur.left : cur.right;
 d += 1;
 }
 return -1;
};

const heightOf = (t: SN | null): number => (t ? 1 + Math.max(
const inorderKeys = (t: SN | null): number[] => (t ? [
 ...inorderKeys(t.left),
 t.key,
 ...inorderKeys(t.right)
] : []);

export const INITIAL_KEYS = [50, 40, 35, 32, 30, 28, 12];

const buildInitial = (): SN => {
 let t: SN | null = null;
 for (const k of INITIAL_KEYS) t = bstInsert(t, k);
 return t!;
};

const CODE_LINES = [
 { line: 1, text: "while x has parent:" },
 { line: 2, text: " p = parent(x); g = parent(p)" },
 { line: 3, text: " if g is None: # Z" },
 { line: 4, text: " rotate(x over p)" },
 { line: 5, text: " elif p,g,x на одной линии: # " },
 { line: 6, text: " rotate(p over g) # B" },
 { line: 7, text: " rotate(x over p)" },
 { line: 8, text: " else: # Z" },
 { line: 9, text: " rotate(x over p) # H" }
];

```

```

/>
<text x="48" y="130" textAnchor="middle" fill="black">
<circle cx="80" cy="126" r="13" fill="#0f172a" stroke="black" stroke-width="2">
<text x="80" y="130" textAnchor="middle" fill="black">
<text x="98" y="92" fill={isCut ? "#f59e0b" : "black"}>
 {isCut ? "β отстёрнута" : "β при x"}
</text>
<text x="16" y="160" fill="#64748b" font-size="12">
</g>
<text x="163" y="96" fill="#10b981" font-size="24">
<g transform="translate(186,6)">
 <text y="10" fill="#94a3b8" font-size="11" font-weight="bold">
 <line x1="70" y1="46" x2="45" y2="86" stroke="black" stroke-width="2">
 <line x1="70" y1="46" x2="95" y2="86" stroke="black" stroke-width="2">
 <line x1="45" y1="86" x2="27" y2="126" stroke="black" stroke-width="2">
 <line
 x1="45" y1="86" x2="63" y2="126"
 stroke={isRot ? "#10b981" : "#64748b"}
 stroke-width={isRot ? 3 : 2}
 />
<circle cx="70" cy="46" r="15" fill="#059669" stroke="black" stroke-width="2">
<text x="70" y="50" textAnchor="middle" fill="black">
<circle cx="45" cy="86" r="15" fill="#1e293b" stroke="black" stroke-width="2">
<text x="45" y="90" textAnchor="middle" fill="black">
<circle cx="27" cy="126" r="13" fill="#0f172a" stroke="black" stroke-width="2">
<text x="27" y="130" textAnchor="middle" fill="black">
<circle
 cx="63" cy="126" r="13"
 fill={isRot ? "#78350f" : "#0f172a"}
 stroke={isRot ? "#10b981" : "#64748b"}
 stroke-width={isRot ? 2.5 : 1.5}
/>
<text x="63" y="130" textAnchor="middle" fill="black">
<circle cx="95" cy="126" r="13" fill="#0f172a" stroke="black" stroke-width="2">
<text x="95" y="130" textAnchor="middle" fill="black">
<text x="112" y="92" fill={isRot ? "#10b981" : "black"}>
 {isRot ? "↓ β теперь при p" : "β при p"}
</text>

```



```

const baseRef = useRef<SN>(initial);
const timer = useRef<ReturnType<typeof setTimeout> | null>(null);

const runKey = (key: number, isInsert = false) => {
 const { steps: st } = splaySteps(baseRef.current, key);
 setSteps(st);
 setIdx(0);
 setPlaying(true);
};

useEffect(() => {
 if (!playing || !steps) return;
 if (idx >= steps.length - 1) {
 setPlaying(false);
 baseRef.current = steps[steps.length - 1].root ?? null;
 setTreeVers((v) => v + 1);
 return;
 }
 timer.current = setTimeout(() => setIdx((v) => v + 1), 100);
 return () => {
 if (timer.current) clearTimeout(timer.current);
 };
}, [playing, idx, steps]);

const step = steps ? steps[Math.min(idx, steps.length - 1)] : null;
const viewRoot = step ? step.root : baseRef.current;
const pos = useMemo(() => layout(viewRoot), [viewRoot]);
const nodes = useMemo(() => [...pos.entries()].map(([key, node]) => node), [pos]);
const edges = useMemo(() => {
 const es: { a: number; b: number }[] = [];
 const walk = (n: SN | null) => {
 if (!n) return;
 if (n.left) es.push({ a: n.key, b: n.left.key });
 if (n.right) es.push({ a: n.key, b: n.right.key });
 walk(n.left);
 walk(n.right);
 };
 walk(viewRoot);
}, [viewRoot]);

```

```

 if ((step.kind === "aim" || step.kind === "cut") &&
 const e = step.aimEdge ?? step.cutEdge!;
 if (e[0] === k) return "#1e293b";
 if (e[1] === k) return "#78350f";
 }
 if (step.kind === "rot" && step.pathKeys?.includes(
 return "#0f172a";
 });
const strokeColor = (k: number): string => {
 if (!step) return "#64748b";
 if (step.kind === "search" && step.pathKeys?.includes(
 if ((step.kind === "aim" || step.kind === "cut") &&
 const e = step.aimEdge ?? step.cutEdge!;
 if (e.includes(k)) return "#f59e0b";
 }
 if (step.pathKeys?.includes(k)) return "#38bdf8";
 return "#64748b";
};

return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border
 {/ * Wanka */}
 <div className="flex flex-wrap items-center justify
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-indigo-600 text-white

 </div>
 <div>
 <h3 className="text-2xl font-bold text-white
 <p className="text-sm text-indigo-300">Клики
 </div>
 </div>
 <div className="flex flex-wrap items-center gap
 <input
 value={input}
 onChange={(e) => setInput(e.target.value)}
 onKeyDown={(e) => {
 if (e.key === "Enter") {

```

```

 Перемешать
 </button>
 </div>
 </div>

 { /* Плеер */
 {steps && {
 <div className="flex flex-wrap items-center gap-2" >
 <button onClick={() => { setIdx(0); setPlaying(true); }} >
 В начало
 </button>
 <button onClick={() => { setPlaying(false); setIdx(0); }} >
 Пауза
 </button>
 <button onClick={() => setPlaying(!playing)} >
 {playing ? "▶ Пауза" : "▶ Пуск"}
 </button>
 <button onClick={() => { setPlaying(false); setIdx(0); }} >
 В конец
 </button>

 Шаг {idx + 1} из {steps.length} • поворотов

 {step?.caseLabel && {

 {step.caseLabel}

 }}
 </div>
 }}
}

{ /* Дерево */
<div className="bg-indigo-950/20 rounded-xl p-4 b

```

```

 <g
 key={`n-${n.key}-${treeVers}`}
 transform={`translate(${n.px}, ${n.py})`}
 onClick={() => {
 if (playing) return;
 const has = inorderKeys(baseRef.current);
 if (has) runKey(n.key);
 }}
 className="cursor-pointer"
 >
 <circle
 r={isAimNode ? 24 : 20}
 fill={stepColor(n.key)}
 stroke={strokeColor(n.key)}
 strokeWidth={isAimNode ? 4 : 2}
 className="transition-all duration-300"
 />
 <text y={4} fill="rgba(0,0,0,0.1)" fontSize="13"
 >
 {n.key}
 </text>
 </g>
);
}}
</svg>

```

```

{/* Лог + механика */}
<div className="w-full flex flex-col lg:flex-row" >
 <div className="flex-1 bg-slate-950/80 p-4 rounded" >
 <p className="font-semibold text-amber-400" >
 {step?.kind === "aim"
 ? " Прицел:"
 : step?.kind === "cut"
 ? "∞ Отстёгиваем среднее поддерево:"
 : step?.kind === "rot"
 ? " Поворот:"
 : step?.kind === "search"
 ? " Спуск:"
 }
 </p>
 </div>
</div>

```

```

 </tr>
 </tbody>
 </table>
) : (
 <p className="text-xs text-slate-400">Запис
 })
 <p className="text-xs text-slate-500 mt-3">Од
 щений дешевлеет.</p>
</div>
<div className="w-full lg:w-1/2 bg-slate-900/60
 <h4 className="text-sm font-bold text-slate-2
 <div className="bg-slate-950/80 rounded-lg p-
 {CODE_LINES.map((l) => {
 const zig = step?.caseLabel === "Zig" &&
 const zz1 = step?.caseLabel === "Zig-Zig"
 const zz2 = step?.caseLabel === "Zig-Zig"
 const za1 = step?.caseLabel === "Zig-Zag"
 const za2 = step?.caseLabel === "Zig-Zag"
 const active = zig || zz1 || zz2 || za1 |
 return (
 <div key={l.line} className={`py-1 px-2
ber-400" : "text-slate-400"}`}>
 <span className="text-slate-600 selec
 {l.text}
 </div>
 </div>
);
 })}
 </div>
</div>
</div>
</div>
);
};

export default SplayWalkViz;

```

```

const runtime = useVizRuntime();
const liveStack = vizArray(runtime?.variables?.st) ??
const displayStack: Plate[] = liveStack
 ? liveStack.map((value, index) => ({
 id: `python-${index}`,
 name: String(value),
 emoji: " ",
 color: "from-indigo-500/20 to-indigo-600/20 border",
 isDirty: true,
 animState: "idle",
 }))
 : dirtyStack;

const [cleanRack, setCleanRack] = useState<Plate[]>([]);
const [isWashing, setIsWashing] = useState(false);
const [showSoap, setShowSoap] = useState(false);
const [shakeEmpty, setShake] = useState(false);
const [log, setLog] = useState<string[]>([]);

const addLog = (msg: string) => setLog(prev => [...prev, msg]);

// PUSH: Положить грязную тарелку сверху стопки
const pushPlate = useCallback(() => {
 if (isWashing) return;
 if (dirtyStack.length >= 7) {
 addLog('⚠ Стопка слишком высокая, тарелки могут упасть');
 setShake(true);
 setTimeout(() => setShake(false), 400);
 return;
 }
}, [isWashing]);

const preset = PLATE_PRESETS[counter % PLATE_PRESETS.length];
counter++;

const newPlate: Plate = {
 id: Date.now().toString(),
 ...preset,
 isDirty: true,

```

```

 setCleanRack(prev => [cleanPlate, ...prev].slice(0, cleanPlate.length - 1));
 setDirtyStack(prev => prev.slice(0, prev.length - 1));
 setShowSoap(false);
 setIsWashing(false);
 addLog(`    Готово! Чистая тарелка ${topPlate.emoji}`, 850);
 }, [dirtyStack, isWashing]);

```

```

const reset = () => {
 counter = 0;
 setDirtyStack([
 { id: 'r1', name: 'Тарелка из-под спагетти', emoji: '🍝', state: 'in' },
 { id: 'r2', name: 'Тарелка из-под пиццы', emoji: '🍕', state: 'in' },
 { id: 'r3', name: 'Блюдец от торта', emoji: '🍰', state: 'in' },
]);
 setCleanRack([]);
 setIsWashing(false);
 setShowSoap(false);
 setLog([' Стол сброшен. Посуда снова грязная!']);
};

```

```

const topIndex = displayStack.length - 1;

```

```

return (
 <div className="flex flex-col gap-6">
 {/* МНЕМОНИКА / ПРАВИЛО */}
 <div className="bg-blue-950/40 border border-blue-950/40 p-6">
 <div className="text-3xl"> </div>
 <div>
 <h3 className="font-black text-blue-400 text-2xl">Правило!</h3>
 <p className="text-xs text-slate-300 mt-1 leading-tight">
 Ты складываешь грязные тарелки друг на друга.
 А когда приходит время мыть, ты берёшь именно последнюю.
 Попробуй вытащить нижнюю – всё рухнет!
 </p>
 </div>
 </div>
 </div>
)

```

```

<div className="md:col-span-7 bg-slate-900 rounded-
 [380px]">
 <div className="absolute top-4 left-4 text-[10px] font-mono">
 Грязная стопка (Стек вызовов / LIFO)
 </div>

 {/* Визуализация раковины */}
 <div className="absolute top-4 right-4 flex items-center justify-center gap-2" style={{
 background: 'linear-gradient(to right, slate-800/80, slate-800/80) text-[10px] font-mono">

 РАКОВИНА
 </div>

 {/* Мыльные пузыри при мытье */}
 {showSoap && {
 <div className="absolute inset-0 pointer-events-none">
 <div className="absolute w-6 h-6 rounded-full border-1px solid white">
 </div>
 <div className="absolute w-4 h-4 rounded-full border-1px solid white">
 </div>
 <div className="absolute w-5 h-5 rounded-full border-1px solid white">
 </div>
 <div className="absolute text-4xl anim-crash">
 </div>
 </div>
 }}

 {displayStack.length === 0 ? {
 <div className="flex-1 flex flex-col items-center justify-center gap-2">

 <p className="text-white font-black text-2xl">
 <p className="text-slate-500 text-xs mt-1">
 </div>
 } : {
 <div className={`flex-1 flex flex-col justify-center gap-2`} style={{
 background: 'linear-gradient(to right, slate-800/80, slate-800/80) text-[10px] font-mono">
 {/* Указатель TOP */}
 <div
 className="absolute right-8 flex items-center justify-center gap-2" style={{

```



```

 { /* Столешница раковины */ }
 <div className="w-full h-4 bg-gradient-to-r from-slate-900 to-slate-800" />
 </div>

 { /* ПРАВАЯ КОЛОНКА: СУШИЛКА ДЛЯ ЧИСТЫХ ТАРЕЛОК */ }
 <div className="md:col-span-5 bg-slate-900 rounded-3xl p-4" />
 <div className="absolute top-4 left-4 text-[10px] font-mono" />
 Сушилка для чистой посуды
 </div>

 <div className="absolute top-4 right-4 flex items-center justify-center gap-2" />
 <div className="order-emerald-800/80 text-[9px] font-mono" />
 <Sparkles className="w-3.5 h-3.5 animate-pulse" />
 ЧИСТО
 </div>

 <div className="flex-1 flex flex-col justify-between p-4" />
 {cleanRack.length === 0 ? (
 <div className="flex-1 flex flex-col items-center justify-center gap-2" />

 <p className="text-xs font-bold font-mono" />
 <p className="text-[10px] mt-1" />Здесь 6 тарелок
 </p>
 </div>
) : (
 <div className="flex flex-col gap-2 w-full" />
 {cleanRack.map(plate => (
 <div
 key={plate.id}
 className="w-full h-8 rounded-full border-1" />
 <div className="flex items-center justify-center gap-2 opacity-0.5" />
 {plate.id}

 </div>
 </div>
))}
 </div>
)}
 </div>
 </div>
 </div>

```

```

const pi = new Array(n).fill(0);
const steps: any[] = [];

steps.push({ i: 0, j: 0, pi: [...pi], desc: `Начало` });

for (let i = 1; i < n; i++) {
 let j = pi[i - 1];

 steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });

 while (j > 0 && s[i] !== s[j]) {
 steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });
 j = pi[j - 1];
 }

 if (s[i] === s[j]) {
 steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });
 j++;
 } else {
 steps.push({ i, j, pi: [...pi], highlight: [i, j], desc: `Совпадение` });
 }

 pi[i] = j;
 steps.push({ i, j, pi: [...pi], desc: `Записываем` });

 if (j === pattern.length) {
 steps.push({ i, j, pi: [...pi], found: true });
 }
}

return { s, steps, patternLength: pattern.length };
}

function generateZSteps(pattern: string, text: string) {
 if (!pattern) pattern = " ";

```

```

 steps.push({ i, l, r, z: [...z], desc: ` 0
 }

 if (z[i] === pattern.length) {
 steps.push({ i, l, r, z: [...z], found: true
 }
}
return { s, steps, patternLength: pattern.length };
}

export function StringAlgorithmsViz({ defaultMode = "kmp"
const [mode, setMode] = useState<"kmp" | "z">(defaultMode);
const [pattern, setPattern] = useState("aba");
const [text, setText] = useState("abacaba");

// Ensure inputs are valid dynamically
const safePattern = pattern || " ";
const safeText = text || " ";

const { s: demoString, steps, patternLength: demoPatternLength } =
 if (mode === "kmp") return generateKmpSteps(safePattern, safeText);
 else return generateZSteps(safePattern, safeText);
}, [mode, safePattern, safeText]);
const runtime = useVizRuntime();
const runtimeString = vizString(runtime?.variables?.text);
const s = runtimeString ?? demoString;
const patternLength = runtimeString ? runtimeString.length : demoPatternLength;

const [autoPlay, setAutoPlay] = useState(false);
const [stepIdx, setStepIdx] = useState(0);
const timer = useRef<NodeJS.Timeout | null>(null);

// Reset when inputs or mode change
useEffect(() => {
 setStepIdx(0);
 setAutoPlay(false);
}, [s, mode]);

```

```

return (
 <div className="space-y-4">
 <div className="flex items-center justify-b
 <div className="flex bg-slate-900 borde
 <button onClick={() => setMode("kmp
 className={`px-3 py-1.5 text-xs
e" : "text-slate-400 hover:text-slate-300"}
 КМП (π-ФУНКЦИЯ)
 </button>
 <button onClick={() => setMode("z")
 className={`px-3 py-1.5 text-xs
" : "text-slate-400 hover:text-slate-300"}`
 Z-ФУНКЦИЯ
 </button>
 </div>

 <div className="flex items-center gap-2
 <div className="bg-slate-800 p-1 fl
 <span className="text-[10px] te
 <input value={pattern} onChange
xt-white text-xs font-bold font-mono px-2 p
 </div>
 <div className="bg-slate-800 p-1 fl
 <span className="text-[10px] te
 <input value={text} onChange={e
te text-xs font-bold font-mono px-2 py-1 ou
 </div>
 </div>
 </div>

 <div className="bg-slate-950 rounded-xl bor
px] flex items-center justify-start">
 <div className="flex gap-1.5 px-4 h-ful
 {s.split("").map((char, index) => {
 const isPattern = index < patte
 const isSeparator = index === p

```

```

 { /* Char Box */
 <div className={`w-8 h-
xtColor} font-mono text-lg transition-color
 {char}
 </div>

 { /* Value array box */
 <div className="text-[1
slate-700/50 font-mono font-bold">
 {(mode === "kmp" ?
 </div>

 { /* KMP fingers */
 {mode === "kmp" && step
 <div className="abs
z-20">
 <span classNam
 </div>
 })
 {mode === "kmp" && step
 <div className="abs
 <span classNam
 </div>
 })
 {mode === "kmp" && step
 <div className="abs
20">
 <span classNam
 </div>
 })

 { /* Z fingers */
 {mode === "z" && step.i
 <div className="abs
e z-20">
 <span classNam
 </div>

```

```

 <button onClick={nextStep} disabled={
 steps.length < 2 || !isStepComplete(p-2)}
 className="p-2 text-slate-400 hover:text-white hover:
 <SkipForward strokeWidth={3} size="lg" />
 </button>
 <button onClick={reset} aria-label="Reset"
 className="bg-slate-800 rounded-lg ml-2 border-l border-slate-400
 <RefreshCw strokeWidth={3} size="lg" />
 </button>
 </div>

 {/* Description Log */}
 <div className="flex-1 bg-slate-900/50 border border-slate-800
 overflow-hidden">
 <div className="absolute top-0 right-0 text-slate-400
 {steps.length}
 </div>
 <div className="text-[10px] text-slate-400">
 {steps.map((step, index) => (
 <div className="text-sm text-slate-400">
 {step.desc}
 </div>
))}
 </div>
 </div>

 {/* Why this matters card */}
 <div className="bg-slate-800 p-4 rounded-xl">
 {steps.length}
 <p className="text-xs text-slate-400">
 Обрати внимание на хитрость с решёткой.
 Мы пишем Шаблон + # + Текст.
 Когда значени {mode === 'kmp' ? "пробел" : "запятая"}
 было найдено!
 Пройди шаги вручную, посмотри, как работает алгоритм
 для пропуска работы (в Z).
 </p>
 </div>

 {/* Code Sneak Peek */}

```

```

 /** Элемент, на который наводят курсор / фокус. */
 children: React.ReactNode;
 /** С какой стороны показывать. По умолчанию – сверху
 side?: "top" | "bottom";
 className?: string;
 }

 /**
 * Всплывающая подсказка (как title="", только красивая
 * Появляется по наведению курсора и по фокусу с клавиатуры
 */
 export const Tooltip: React.FC<TooltipProps> = ({ content, ...props }) => {
 const [open, setOpen] = useState(false);

 const hiddenShift = side === "top" ? "translate-y-1" : "translate-y-0";

 return (
 <div>
 <span
 className={`relative inline-flex ${className ?? ""}
 onMouseEnter={() => setOpen(true)}
 onMouseLeave={() => setOpen(false)}
 onFocus={() => setOpen(true)}
 onBlur={() => setOpen(false)}
 >
 {children}

 <span
 role="tooltip"
 className={`pointer-events-none absolute left-1/2 top-1/2
 bg-slate-900 px-3 py-2 text-left text-[11px]
 duration-150 ${
 side === "top" ? "bottom-full mb-2" : "top-full mt-2"
 }
 ${open ? "visible translate-y-0 opacity-100" : "invisible"}
 >
 {content}

 </div>
 </div>
);
 };

```

```

 { id: 3, x: 460, y: 150, label: "3", name: "Алгоритмы"},
 { id: 4, x: 640, y: 150, label: "4", name: "Машинное"},
];
```

```
const dagEdges: TEdge[] = [
 { u: 0, v: 1 },
 { u: 0, v: 2 },
 { u: 1, v: 3 },
 { u: 2, v: 3 },
 { u: 3, v: 4 },
];
```

```
interface TStep {
 desc: string;
 visited: Set<number>;
 fullyProcessed: Set<number>;
 currentNode: number | null;
 topoList: number[];
 dfsStack: number[];
}
```

```
export const TopologicalSortViz: React.FC = () => {
 const [steps, setSteps] = useState<TStep[]>([]);
 const [currentStepIdx, setCurrentStepIdx] = useState(0);
 useVizStepSync(currentStepIdx, setCurrentStepIdx, steps);
 const v = vizNumber(vars.v);
 const orderLength = vizArray(vars.order)?.length;
 if (v === null && orderLength === undefined) return null;
 const candidates = steps
 .map((step, index) => ({ step, index }))
 .filter(({ step }) => (v === null || step.currentNode !== v));
 return candidates.at(-1)?.index ?? null;
});
const [isPlaying, setIsPlaying] = useState(false);

useEffect(() => {
 const listSteps: TStep[] = [];
```



```

 recordStep(
 `⚠ Внимание: Рёбра в серую вершину ${to} о
 u,
);
 }
}

fullyProcessed.add(u);
dfsStack.pop();
topoList.push(u); // prepend behavior: we write i
recordStep(
 `DFS: закончили обработку '${dagNodes[u].name}'
 ,
 u,
);
};

// Run DFS for all unvisited
for (let i = 0; i < 5; i++) {
 if (!visited.has(i)) {
 recordStep(
 `Запускаем новый обход DFS от нетронутой верши
 null,
);
 dfs(i);
 }
}

recordStep(
 "Топологическая сортировка завершена! Полученный
 null,
);
setSteps(listSteps);
setCurrentStepIdx(0);
}, []));

const step = steps[currentStepIdx] || {
 desc: "Запуск...",

```

```

</h3>

<div className="flex items-center gap-2 bg-slate-500" >
 <button
 onClick={togglePlay}
 className="flex items-center gap-2 px-3 py-5 500 text-white"
 >
 {isPlaying ? (
 <Pause className="w-4 h-4 ml-1" />
) : (
 <Play className="w-4 h-4 ml-1" />
)}
 {isPlaying ? "Пауза " : "Старт "}
 </button>

 <button
 onClick={reset}
 aria-label="Сбросить сортировку"
 title="Сбросить сортировку"
 className="flex items-center justify-center gap-2"
 >
 <RotateCcw className="w-4 h-4" />
 </button>
</div>
</div>

<div className="grid grid-cols-1 lg:grid-cols-12" >
 {/* Playfield SVG */}
 <div className="lg:col-span-8 bg-[#0B1120] relative" >
 <svg className="w-full h-full max-w-[650px]" >
 <defs>
 <marker
 id="dag-arrow"
 viewBox="0 0 10 10"
 refX="24"
 refY="5"

```

```

 className="transition-all duration-300"
 />
);
 }}}

```

```

{ /* Nodes */
dagNodes.map((node) => {
 const isCurrent = step.currentNode === node.id;
 const isVisited = step.visited.has(node.id);
 const isProcessed = step.fullyProcessed.has(node.id);

```

```

 let fill = "#1e293b";
 let stroke = "#334155";
 let textColor = "text-slate-400";
 void textColor;

```

```

 if (isCurrent) {
 fill = "#312e81";
 stroke = "#818cf8";
 textColor = "text-white font-bold";
 } else if (isProcessed) {
 fill = "#022c22";
 stroke = "#059669";
 textColor = "text-emerald-300";
 } else if (isVisited) {
 fill = "#3730a3";
 stroke = "#6366f1";
 textColor = "text-indigo-200";
 }

```

```

 return (
 <g key={node.id}>
 <rect
 x={node.x - 55}
 y={node.y - 22}
 width="110"
 height="44"
 rx="8"

```

```

<div className="bg-slate-900 border border-slate-800 p-2.5 rounded">
 <p className="text-xs text-indigo-300 min-h-[100px]">
 {step.desc}
 </p>
</div>

```

```

<div className="flex-1 space-y-4 overflow-y-auto">
 {/* Topological List output */}
 <div>

 РЕЗУЛЬТАТ (МАТРИЦА ОБУЧЕНИЯ):

 <div className="bg-slate-900/60 p-2.5 rounded">
 pr-1 text-slate-300">
 {step.topoList.length === 0 ? (

 ожидаем обработку узлов...

) : (
 getTopoListString()
)}
 </div>
 </div>
 </div>

```

```

{/* Color key */}
<div className="space-y-1">

 ВЕРШИНЫ В DFS:

 <div className="flex items-center gap-2">

 Серые (зашли)

 </div>
 <div className="flex items-center gap-2">


```

ФАЙЛ 78/111: src/components/TreapBuildViz.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

```
import React, { useContext, useEffect, useMemo, useState } from "react";
import { Terminal } from "lucide-react";
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizState } from "viz-react";
```

```
/**
```

```
 * Интерактивный Treap – шесть режимов:
```

```
 * build – сборка: ВИДИМАЯ сортировка точек (и чем сортировка, тем быстрее)
```

```
 * координатная плоскость НАВЕРХУ (ассоциативная)
```

```
 * чек-лист инвариантов (автопроверка, как у AVL)
```

```
 * split – разрез по x с двумя корзинами (грамотная сортировка)
```

```
 * merge – склейка двух деревьев по y.
```

```
 * erase – удаление ПРОИЗВОЛЬНОЙ точки: найти по x, удалить по y.
```

```
 * layers – миф «дети в массиве по $2^k/2^{k+1}$ »: работает
```

```
 * search – «отсортирую бинарным поиском»: поиск не сортировкой
```

```
 * сортировка платит $\geq n \cdot \log n$ сравнений, сортировка
```

```
 * быстрее
```

```
 * Канонический набор: все x и все y различны $\Rightarrow$ дерево
```

```
 * Узел идентифицируется парой $id = "x:y"$.
```

```
 */
```

```
export interface Pair {
```

```
 key: number;
```

```
 grade: number;
```

```
}
```

```
type NodeId = string;
```

```
const pairId = (key: number, grade: number): NodeId =>
```

```
interface TNode {
```

```
 id: NodeId;
```

```
 key: number;
```

```
 grade: number;
```

```
 left: TNode | null;
```

```
 right: TNode | null;
```

```
}
```

```

export function buildTree(ps: Pair[]): TNode | null {
 let root: TNode | null = null;
 for (const p of sortedByY(ps)) root = insert(root, p);
 return root;
}

export interface SplitResult {
 l: TNode | null;
 r: TNode | null;
 trace: { atId: NodeId; goLeft: boolean; note: string }
}

export function splitTree(root: TNode | null, x: number): SplitResult {
 const trace: SplitResult["trace"] = [];
 const go = (t: TNode | null): { l: TNode | null; r: TNode | null } {
 if (!t) return { l: null, r: null };
 if (t.key <= x) {
 const sub = go(t.right);
 trace.push({ atId: t.id, goLeft: true, note: `(${t.key}
 о правое поддерево.` });
 t.right = sub.l;
 return { l: t, r: sub.r };
 }
 const sub = go(t.left);
 trace.push({ atId: t.id, goLeft: false, note: `(${t.key}
 о левое поддерево.` });
 t.left = sub.r;
 return { l: sub.l, r: t };
 };
 const res = go(root);
 return { l: res.l, r: res.r, trace };
}

export interface MergeResult {
 root: TNode | null;
 trace: { aId: NodeId | null; bId: NodeId | null; chosen: boolean }
}

export function mergeTree(a: TNode | null, b: TNode | null): MergeResult {
 const trace: MergeResult["trace"] = [];

```

```

 };
 return { root: go(root), trace };
}

```

```

const cloneTree = (t: TNode | null): TNode | null =>
 t ? { id: t.id, key: t.key, grade: t.grade, left: cloneTree(t.left), right: cloneTree(t.right) } : null

```

/\* ----- валидация treap (для чек-листа и стресс-теста) ----- \*/

```

export function checkTreap(root: TNode | null): { bst: boolean; heap: boolean } {
 let bst = true;
 let heap = true;
 let prev: number | null = null;
 const walk = (n: TNode | null, min: number, max: number, parentGrade: number | null) => {
 if (!n) return;
 if (n.key <= min || n.key > max) bst = false;
 if (parentGrade !== null && n.grade > parentGrade) heap = false;
 walk(n.left, min, n.key, n.grade);
 if (prev !== null && n.key < prev) bst = false;
 prev = n.key;
 walk(n.right, n.key, max, n.grade);
 };
 walk(root, -Infinity, Infinity, null);
 return { bst, heap };
}

```

/\* ----- раскладка сцены дерева ----- \*/

```

const layoutTree = (root: TNode | null) => {
 const pos = new Map<NodeId, { px: number; py: number }>();
 let slot = 0;
 const walk = (n: TNode | null, depth: number) => {
 if (!n) return;
 walk(n.left, depth + 1);
 pos.set(n.id, { px: 64 + slot * 92, py: 44 + depth * 100 });
 slot += 1;
 walk(n.right, depth + 1);
 };
 walk(root, 0);
 return pos;
}

```

```

<div>
 <svg viewBox={`0 0 ${w} ${h}`} className="w-full h-full" >
 {edges.map((e) => {
 const a = byId(e.from);
 const b = byId(e.to);
 if (!a || !b) return null;
 return (
 <line
 key={`te-${e.from}-${e.to}`}
 x1={a.px} y1={a.py} x2={b.px} y2={b.py}
 stroke={edgeColor ? edgeColor(e.from, e.to) : "#ccc"}
 strokeWidth={2.5}
 strokeDasharray={edgeDash && edgeDash(e.from, e.to)}
 className="transition-all duration-500"
 />
);
 })}
 {nodes.map((n) => {
 const p = pairParts(n.id);
 const pulse = pulseId === n.id;
 return (
 <g key={`tn-${n.id}`} transform={`translate(${n.px}, ${n.py})`} >
 <circle r={pulse ? 24 : 20} fill={nodeColor(n.id)} stroke="#ccc"
 strokeWidth={pulse ? 4 : 2} className="transition-all duration-500" />
 <text y={-2} fill="#fff" fontSize="13" fontWeight="bold">{n.id}</text>
 <text y={13} fill="#cbd5e1" fontSize="10">{p[0]} - {p[1]}</text>
 </g>
);
 })}
 </svg>
 {subtitle && <p className="text-xs text-slate-400">{subtitle}</p>}
</div>
);
};

const pairParts = (id: NodeId): [number, number] => {
 const [a, b] = id.split(":").map(Number);
 return [a, b];
};

```



```

<line x1={PX(1) - 8} y1="16" x2={PX(1) - 8} y2={P
{[0, 10, 20, 30, 40].map((k) => {
 <text key={`tx-${k}`} x={PX(k)} y={PY(0) + 16}
)}}
{[-4, 0, 4, 8].map((g) => {
 <text key={`ty-${g}`} x={PX(1) - 14} y={PY(g) +
}})
<text x={PL_W - 14} y={PY(0) - 8} fill="#94a3b8"
<text x={PX(1) - 26} y="14" fill="#94a3b8" fontSi

{drawnEdges.map((e, i) => {
 const a = byPair(e.a);
 const b = byPair(e.b);
 const isBad = badEdge && ((badEdge.a.key === e.a
 ade === e.b.grade) || (badEdge.a.key === e.b.
 e === e.a.grade));
 return <line key={`pe-${i}`} x1={a.x} y1={a.y}
 2} strokeDasharray={isBad ? "4,3" : "none"}
}})

{/* временная линия игры */}
{interactive && selected && cursor && {
 <line
 x1={PX(selected.key)} y1={PY(selected.grade)}
 stroke="#818cf8" strokeWidth={2.5} strokeDash
 />
}}

{points.map((p) => {
 const done = drawnEdges.some((e) => (e.a.key ===
 const isSel = selected && selected.key === p.key
 return (
 <g
 key={`pp-${p.key}:${p.grade}`}
 className="transition-all duration-500"
 onPointerDown={interactive && onPointDown ?
 onPointerUp={interactive && onPointUp ? () :
 style={interactive ? { cursor: "pointer" }

```

```

const gpid = (p: Pair) => `${p.key}:${p.grade}`;
```

```
/** Цепочка родителей от точки вверх. */
```

```
const parentOf = (edges: GEdge[], id: string): GEdge | undefined =>
 edges.find((e) => gpid(e.b) === id);
const reaches = (edges: GEdge[], from: Pair, target: Pair): boolean => {
 let cur: Pair | undefined = from;
 const seen = new Set<string>();
 while (cur) {
 if (sameP(cur, target)) return true;
 if (seen.has(gpid(cur))) return false;
 seen.add(gpid(cur));
 cur = parentOf(edges, gpid(cur))?.a;
 }
 return false;
};
```

```
/** Проверка BST леса: у узла ≤1 ребёнка слева и ≤1 справа.
```

```
export function forestIsBst(edges: GEdge[], all: Pair[]): boolean {
 const kids = new Map<string, Pair[]>();
 for (const e of edges) kids.set(gpid(e.a), [...(kids.get(gpid(e.b)) || [])]);
 const hasParent = new Set(edges.map((e) => gpid(e.b)));
 const roots = all.filter((p) => !hasParent.has(gpid(p)));
 const ok = (n: Pair | undefined, lo: number, hi: number): boolean => {
 if (!n) return true;
 if (n.key <= lo || n.key > hi) return false;
 const ch = (kids.get(gpid(n)) ?? []).slice().sort((a, b) => a.key - b.key);
 const left = ch.filter((c) => c.key < n.key);
 const right = ch.filter((c) => c.key > n.key);
 if (left.length > 1 || right.length > 1) return false;
 return ok(left[0], lo, n.key) && ok(right[0], n.key, hi);
 };
 return roots.every((r) => ok(r, -Infinity, Infinity));
}
```

```
/** Полный вердикт по текущим линиям. */
```

```
export function gameVerdict(edges: GEdge[], all: Pair[]): boolean {
 const heap = edges.every((e) => e.a.grade > e.b.grade);
 return forestIsBst(edges, all);
}
```



```

const comp = new Map<string, string>(points.map((p) => {
const find = (x: string): string => {
 while (comp.get(x) !== x) {
 comp.set(x, comp.get(x)!);
 x = comp.get(x)!;
 }
 return x;
});
for (const e of edges) {
 const ra = find(pid(e.a));
 const rb = find(pid(e.b));
 if (ra !== rb) comp.set(ra, rb);
}
const comps = new Set(points.map((p) => find(pid(p))))
const connected = comps.size === 1;
const countOk = edges.length === n - 1;
// BST: дети по x, inorder не убывает; корень один; э
const children = new Map<string, Pair[]>();
for (const e of edges) children.set(pid(e.a), [...(ch
const roots = points.filter((p) => !parentCount.has(p
let bstOk = roots.length === 1;
if (bstOk) {
 const seen = new Set<string>();
 const inorder: number[] = [];
 const walk = (p: Pair): void => {
 if (seen.has(pid(p))) {
 bstOk = false; // цикл
 return;
 }
 seen.add(pid(p));
 const kids = children.get(pid(p)) ?? [];
 if (kids.length > 2) {
 bstOk = false;
 return;
 }
 const sorted = [...kids].sort((u, v) => u.key - v
 let left: Pair | null = null;
 let right: Pair | null = null;

```

```

const rootPair = roots.length ? roots[0].a : edges.l
if (!rootPair) return null;
const build = (p: Pair): TNode => {
 const node = make(p);
 nodes.set(pid(p), node);
 const kids = edges.filter((e) => pid(e.a) === pid(p)
 if (kids.length > 2) return node; // невалидно – по
 kids.sort((u, v) => u.key - v.key);
 const left = kids.filter((k) => k.key <= p.key); //
 const right = kids.filter((k) => k.key > p.key);
 if (left.length > 1 || right.length > 1) return node
 if (left[0]) node.left = build(left[0]);
 if (right[0]) node.right = build(right[0]);
 return node;
};
return build(rootPair);
}

const BuildMode: React.FC = () => {
 const [raw, setRaw] = useState(DEMO_RAW);
 const live = useTreapLive();

 // набор точек из Python-кода пересобирает плоскость
 useEffect(() => {
 if (!live.pairsText || live.pairsText === raw) return
 setRaw(live.pairsText);
 setPhase("connect");
 setOrder("y-desc");
 setSortIdx(3);
 setConnIdx(10000); // все рёбра проведены – ср
 setPlaying(false);
 // eslint-disable-next-line react-hooks/exhaustive-
 }, [live.pairsText]);
 const [phase, setPhase] = useState<BuildPhase>("sort"
 const [algo, setAlgo] = useState<SortAlgo>("none");
 const [order, setOrder] = useState<ConnectOrder>("y-d
 const [sortIdx, setSortIdx] = useState(0);
 const [connIdx, setConnIdx] = useState(0);

```

```

const startConnect = (ord: ConnectOrder) => {
 setOrder(ord);
 setPhase("connect");
 setConnIdx(0);
 setPlaying(true);
};

const comparisons = algo === "quicksort" ? (n > 1 ? M
const connEdgesAll = order === "y-desc" ? canonEdges
const connEdges = connEdgesAll.slice(0, connIdx);
const connDone = phase === "connect" && n >= 1 && conn

// --- игра «соедини сам» ---
const [gameEdges, setGameEdges] = useState<GameEdge[]>();
const [sel, setSel] = useState<Pair | null>(null);
const [cursor, setCursor] = useState<{ x: number; y: number}>();
const [gameMsg, setGameMsg] = useState<string | null>();
const [badEdge, setBadEdge] = useState<GameEdge | null>();
const [hintUsed, setHintUsed] = useState(false);

const resetGame = () => {
 setGameEdges([]);
 setSel(null);
 setCursor(null);
 setGameMsg(null);
 setBadEdge(null);
 setHintUsed(false);
};

const startGame = () => {
 resetGame();
 setPhase("game");
 setPlaying(false);
};

const verdict = useMemo(() => checkGameState(userPair
const gameWon = phase === "game" && verdict.won;
const gameTree = useMemo(() => (gameWon ? edgesToTree

```

```

const undoEdge = () => {
 setGameEdges((prev) => prev.slice(0, -1));
 setGameMsg(null);
};
const showSolution = () => {
 setGameEdges(canonEdges.map((e) => ({ a: e.a, b: e.b })));
 setHintUsed(true);
 setGameMsg("Решение показано – собери сам в следующий раз");
};

return (
 <div className="space-y-5">
 {/* Произвольный список */}
 <div className="bg-slate-900/60 rounded-xl p-4 border border-slate-700">
 <div className="flex flex-wrap items-center justify-between">
 <h4 className="text-sm font-bold text-slate-300">Задача
 <button onClick={() => { setRaw(DEMO_RAW); setRaw(DEMO_RAW); }}>
 demo-набор
 </button>
 </div>
 <textarea
 value={raw}
 onChange={(e) => { setRaw(e.target.value); setRaw(e.target.value); }}
 rows={2}
 className="w-full bg-slate-950 border border-slate-800 border-emerald-500"
 placeholder="3 5, 45 8, 1 -4 ..."
 />
 {parsed.errors.length > 0 && (
 <div className="mt-2 text-xs text-rose-300 bg-slate-900/60 rounded p-2">
 {parsed.errors.map((e, i) => <div key={i}>{e}</div>)}
 </div>
)}
 {parsed.errors.length === 0 && (parsed.equalX || parsed.notEqualX ? (
 <div className="mt-2 text-xs text-amber-300 bg-slate-900/60 rounded p-2">
 {parsed.equalX ? "Решение верно" : "Решение неверно"}
 </div>
) : null)}
 </div>
)
);

```

```

 </button>
 <button
 onClick={() => startConnect("left-right")}
 className={`px-3 py-1.5 rounded-lg text-white
 "bg-emerald-600 text-white" : "bg-slate-700 text-white"
 }>
 слева направо
 </button>
 <button
 onClick={startGame}
 disabled={n < 2}
 className={`px-3 py-1.5 rounded-lg text-white
 "go-600 text-white" : "bg-indigo-500/80 text-white"
 }>
 соединить сам
 </button>
 </>
)}
 </div>
</div>
<Plane
 points={userPairs}
 drawnEdges={phase === "game" ? gameEdges : phaseEdges}
 pending={null}
 interactive={phase === "game"}
 selected={sel}
 cursor={cursor}
 badEdge={badEdge}
 onPointDown={pointDown}
 onPointUp={pointUp}
 onMove={planeMove}
/>
<div className="mt-2 text-xs text-slate-300 bg-slate-800/80
 {phase === "sort" ? (
 algo === "none" ? (
 <>Ввод приходит как попало ({n} пар)
 </>
) : sortIdx < 3 ? (

```



```

 {verdict.connected ? " " : " "} все т
 {verdict.countOk ? " " : " "} линий р
 {verdict.bstOk ? " " : " "} обход сле
 </>
) : (
 <>
 {connDone ? " " : " "} каждое ребро в
 {connDone ? " " : " "} обход слева-на
 {connDone ? " " : " "} линий ровно <b
 </>
 })

{phase === "game" && (
 <div className="flex flex-wrap gap-2 mt-3">
 <button onClick={undoEdge} disabled={!gameE
 white disabled:opacity-30">↶ Убрать линию</
 <button onClick={resetGame} disabled={!gameE
 ate-700 text-white disabled:opacity-30"> С
 <button onClick={showSolution} disabled={ga
 te disabled:opacity-30"> Показать решение<
 </div>
)}
</div>

{/* Победное дерево в игре */}
{phase === "game" && gameTree && (
 <div className="bg-emerald-950/20 rounded-xl p-
 <h4 className="text-sm font-bold text-emerald
 <TreeView root={gameTree} subtitle="Слева-нап
 ществует." />
 </div>
)}

{/* Дерево */}
{order === "y-desc" && userTree && (
 <div className="bg-slate-900/40 rounded-xl p-4
 <h4 className="text-sm font-bold text-slate-3
 <TreeView root={userTree} subtitle="Слева-нап

```

```

const rTree = split.r;

return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center gap-3">
 Разрез
 <select value={x0} onChange={(e) => { setX0(Num
 -700 rounded-lg px-3 py-1.5 text-sm font-mono
 {[2, 3, 6, 7, 9, 12, 45].map((k) => <option k
 </select>
 <button onClick={() => setStepIdx(Math.max(0, i
 bg-slate-700 text-white disabled:opacity-30">
 <button onClick={() => setStepIdx(Math.min(step
 font-bold bg-indigo-600 text-white disabled:op
 шаг {i
 </div>

 <div className="bg-slate-900/40 rounded-xl p-4 bo
 <h4 className="text-sm font-bold text-slate-300
 , розовые → R
 <TreeView
 root={fullTree}
 nodeColor={(id) => (inL.has(id) ? "#1e3a8a" :
 nodeStroke={(id) => (inL.has(id) ? "#60a5fa"
 subtitle={done ? "Каждый узел покрашен ровно
 />
 </div>

 <div className="text-xs text-slate-300 bg-slate-9
 {steps[idx]?.note}
 </div>

 {done && (
 <div className="grid grid-cols-1 md:grid-cols-2
 <div className="bg-sky-950/20 rounded-xl p-3 l
 <h5 className="text-sm font-bold text-sky-3
 <TreeView root={lTree} width={480} height={
 </div>

```

```

 bg-slate-700 text-white disabled:opacity-30">
 <button onClick={() => setStepIdx(Math.min(step
 font-bold bg-indigo-600 text-white disabled:op
 шаг {id
 </div>

 <div className="grid grid-cols-1 md:grid-cols-2 g
 <div className="bg-sky-950/20 rounded-xl p-3 bo
 <h5 className="text-sm font-bold text-sky-300
 <TreeView root={l} width={480} height={220} n
 </div>
 <div className="bg-rose-950/20 rounded-xl p-3 b
 <h5 className="text-sm font-bold text-rose-300
 <TreeView root={r} width={480} height={220} n
 </div>
 </div>

 <div className="text-xs text-slate-300 bg-slate-9
 {cur?.chosenId ? {
 <>⊗ {cur.note}</>
 } : {
 <>Готово: {done ? "деревья склеены обратно в
 />
 }}
 </div>

 {done && (
 <div className="bg-emerald-950/20 rounded-xl p-3
 <h5 className="text-sm font-bold text-emerald
 <TreeView root={merged.root} subtitle="merge(
 </div>
)}

 <div className="text-xs text-slate-400 bg-slate-9
 Грамотно про «слияние»: у корней сравни
 в (то, что сохраняет порядок x) отправляется ,
 </div>
</div>

```

```

 Удалить
 <select value={key} onChange={(e) => { setKey(N
 te-700 rounded-lg px-3 py-1.5 text-sm font-mo
 {POINTS.map((p) => <option key={p.key} value=
 </select>
 <button onClick={() => setStepIdx(Math.max(0, i
 bg-slate-700 text-white disabled:opacity-30">
 <button onClick={() => setStepIdx(Math.min(step
 font-bold bg-indigo-600 text-white disabled:op
 шаг {i
 </div>

 {!done ? (
 <div className="bg-slate-900/40 rounded-xl p-4
 <TreeView
 root={fullTree}
 nodeColor={(id) => (pathSet.has(id) ? "#783
 nodeStroke={(id) => (pathSet.has(id) ? "#f5
 subtitle="Куча сама умеет удалять только вер
 />
 </div>
) : (
 <div className="bg-slate-900/40 rounded-xl p-4
 <h4 className="text-sm font-bold text-slate-3
 <TreeView root={res.root} subtitle="Валидность
 <ul className="mt-2 space-y-1 text-sm">
 обход слева-направо по-прежнему сорти
 каждое ребро вниз по y (куча целая) –

 </div>
))

 <div className="text-xs text-slate-300 bg-slate-9
 {steps[idx]?.note}
 </div>

 <div className="text-xs text-slate-400 bg-slate-9
 Ответ на «убрать можно только самый верхний

```

```

 <h4 className="text-sm font-bold text-slate-300">
 4>
 <div className="flex gap-2 overflow-x-auto mb-4">
 {heapArr.slice(1).map((v, i) => {
 <div key={i} className="px-3 py-2 rounded">
 <div className="text-slate-500 text-[10px]">{v}</div>
 <div className="text-white font-bold">{i}</div>
 <div className="text-emerald-400 text-[10px]">{i}</div>
 </div>
 })}
 </div>
 <p className="text-xs text-slate-400">Полное $2i/2i+1$ попадает в узел.</p>
 </div>
) : (
 <div className="bg-slate-900/40 rounded-xl p-4">
 <h4 className="text-sm font-bold text-rose-300">
 >
 <TreeView root={full} subtitle="Попробуй «посмотреть»" />
 <div className="flex gap-2 overflow-x-auto mt-2">
 {[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13].map((v, i) => {
 const id = slots[i];
 return (
 <div key={i} className={`px-2.5 py-2 rounded border-slate-700`} : "bg-rose-950/40 border-rose-950">
 <div className="text-slate-500 text-[10px]">{v}</div>
 {id ? <div className="text-white font-[10px]">{id}</div> : null}
 </div>
);
 })}
 </div>
 <p className="text-xs text-rose-300 mt-2">Смотри, что левая ветка ушла на 4 уровня и заняла два места в индексах указателей (left/right), а не арифметикой.</p>
 </div>
)}

```

```

 else { steps.push({ lo, hi, mid, found: false, no
 ` }); hi = mid - 1; }
 }
 return steps;
 }, [sortedByKey]);

```

```

const wrongMid = Math.floor((shuffled.length - 1) / 2)
const comparisons = stage === 2 ? bidx + 1 : 0;

```

```

return (
 <div className="space-y-4">
 <div className="flex gap-2">
 {stage < 2 && (
 <button onClick={() => { setStage(stage === 0
 digo-600 text-white">
 {stage === 0 ? "Найти y для x=7 бинарным по
 </button>
)}
 {stage === 2 && (
 <button onClick={() => setBidx(Math.min(bsStep
 -1.5 rounded-lg text-xs font-bold bg-indigo
 Шаг поиска →
 </button>
)}
 <button onClick={() => { setStage(0); setBidx(0
 Сброс</button>
 </div>

```

```

<div className="flex gap-1.5 overflow-x-auto">
 {(stage === 2 ? sortedByKey : shuffled).map((p, i) => {
 const highlight = stage === 1 && i === wrongMid
 const inBs = stage === 2 && (() => { const s = sortedByKey[
 const isMid = stage === 2 && (() => { const s = sortedByKey[
 const isFound = stage === 2 && bidx >= bsStep
 return (
 <div key={` ${p.key}:${p.grade}-${i}`} className="border-1
 nsition-all ${highlight ? "border-rose-500 border-rose-500
 r-amber-500 bg-amber-950/30" : inBs ? "border-amber-500

```

```

const v = gameVerdict(edges, POINTS);

const click = (p: Pair) => {
 if (!sel) {
 setSel(p);
 setMsg(`Выбрана (${p.key}; ${p.grade}). Теперь кл
 return;
 }
 if (sameP(sel, p)) {
 setSel(null);
 return;
 }
 const res = tryConnect(edges, sel, p);
 if (res.ok) {
 const parent = sel.grade > p.grade ? sel : p;
 const child = sel.grade > p.grade ? p : sel;
 setEdges([...edges, { a: parent, b: child }]);
 setMsg(` Линия (${parent.key}; ${parent.grade})
 setSel(null);
 } else {
 setMsg("x " + res.reason);
 setSel(null);
 }
};

const win = v.win;

return (
 <div className="space-y-4">
 <div className="flex flex-wrap items-center gap-2">
 <button onClick={() => { setEdges(edges.slice(0
 rounded-lg text-xs font-bold bg-slate-700 tex
 <button onClick={() => { setEdges([]); setSel(n
 3 py-1.5 rounded-lg text-xs font-bold bg-slate
 <button onClick={() => { setEdges(CANON_GAME.ma
); }} className="px-3 py-1.5 rounded-lg text-
 линий:
 </div>

```

```

 });
 }}}}
</svg>
</div>

{/* Судья – условия, как у Ахо–Корасика */}
<div className={`rounded-xl p-4 border transition
ate-900/40`} `>
 <h4 className="text-sm font-bold text-slate-300">
 <ul className="space-y-1.5 text-sm">
 {v.heap ? " " : " "} каждое ребро – вниз
 {v.oneParent ? " " : " "} у каждой точки
 {v.acyclic ? " " : " "} циклов нет
 {v.bst ? " " : " "} обход слева-направо –
 {v.complete ? " " : " "} линий $n-1 = \{POI$

 {win && {
 <p className="mt-3 text-sm font-bold text-eme
 ения просто не существует.</p>
 }}
</div>

 <div className="text-xs text-slate-300 bg-slate-9
</div>
 });
};

/* ===== СВЯЗЬ С КОМПИЛЯТОРОМ =====

/**
 * Живые переменные Python-кода страницы treap:
 * points / pairs – набор пар (x; y): по нему ПЕРЕСТРАИ
 * mid – разрез для вкладки Split;
 * key – удаляемая точка для вкладки Erase;
 * step / i – номер шага (листаёт кадры Split / E
 * height, n – высота дерева и число узлов (показ
 */
const useTreapLive = () => {

```



```

 { id: "layers", label: "Миф: 2k/2k+1" },
 { id: "search", label: "Поиск ≠ сортировка" },
 { id: "game", label: "Игра" },
];

export const TreapBuildViz = () => {
 const [tab, setTab] = useState<Tab>("build");
 const chapterId = useContext(VizChapterContext);
 const live = useTreapLive();

 // вкладка = подрежим страницы: панель Python подстав.
 useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, tab);
 }, [chapterId, tab]);

 return (
 <div className="bg-slate-800/90 p-6 rounded-2xl border">
 <div className="flex flex-wrap items-center justify-between">
 <div className="flex items-center gap-3">
 <div className="p-2.5 bg-emerald-600 text-white">

 </div>
 </div>
 <div>
 <h3 className="text-2xl font-bold text-white">
 <p className="text-sm text-emerald-300">Пар.
 </p>
 </div>
 </div>
 </div>

 <div className="flex flex-wrap gap-2 mb-5">
 {TABS.map((t) => (
 <button
 key={t.id}
 onClick={() => setTab(t.id)}
 className={`px-4 py-2 rounded-lg text-xs font-medium
 900/60 text-slate-400 hover:text-white border
 >
 {t.label}

```

```
 {tab === "search" && <SearchMode />}
 {tab === "game" && <GameMode />}
 </div>
);
};
```

---

ФАЙЛ 79/111: src/components/VizLiveStatus.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { liveMatrix, pickLiveVar, useVizRuntime } from
import { VizSourceBadge } from "../VizSourceBadge";

interface VizLiveStatusProps {
 /** Название демонстрации – подписывается в режиме вс
 demoTitle?: string;
 className?: string;
}

/** Имена, по которым визуализации обычно читают матрицу
const MATRIX_NAMES = ["d", "dist", "dp", "D", "S", "P",

/**
 * Строка состояния связи «компилятор → демонстрация» д
 *
 * Раньше компонент молча проигрывал собственный встроен
 * редакторе Python был пустой код, код с другими именам
 * размерности, – выглядело так, будто компилятор исполн
 * картинка живёт отдельной жизнью. Теперь источник данн
 * «данные из компилятора (i, j, dist 5x5)», «компилятор
 * пока нет» (отладчик стоит до первой строки) или «встр
 */
export function VizLiveStatus({ demoTitle, className =
 const runtime = useVizRuntime();
 const vars = runtime?.variables;
 const names = vars ? Object.keys(vars).filter((name) =>
```

```

import FloydViz from "./FloydViz";
import { GraphComponentsViz } from "./GraphComponentsViz";
import { GraphTraversalViz } from "./GraphTraversalViz";
import { ComplexityViz } from "./ComplexityViz";
import { HeapViz } from "./HeapViz";
import { JohnsonViz } from "./JohnsonViz";
import { KosarajuViz } from "./KosarajuViz";
import { KruskalSimulator } from "./KruskalSimulator";
import { PlanarColoringViz } from "./PlanarColoringViz";
import { QueueViz } from "./QueueViz";
import { SalmonAutomatonWidget } from "./SalmonAutomatonWidget";
import { SegmentTreeVisualizer } from "./SegmentTreeVisualizer";
import SplayPlayground from "./SplayPlayground";
import { StackViz } from "./StackViz";
import { StringAlgorithmsViz } from "./StringAlgorithmsViz";
import { TopologicalSortViz } from "./TopologicalSortViz";
import { WaterfallAnimationWidget } from "./WaterfallAnimationWidget";
import { Simulator } from "./Simulator";
import { PrefixSumViz } from "./visualizers/PrefixSumViz";
import { SparseTableViz } from "./visualizers/SparseTableViz";
import { TreapBuildViz } from "./TreapBuildViz";

export interface VizEntry {
 /** Заголовок панели/модалки. */
 title: string;
 /** Короткое пояснение, что здесь можно потыкать. */
 hint?: string;
 Component: React.FC;
}

/**
 * Единый реестр интерактивных демонстраций.
 * Используется и для панели под главой, и для всплывающей подсказки
 * и для модального окна «Открыть симулятор».
 */
export const VIZ_REGISTRY: Record<string, VizEntry> = {
 "stack-dfs": {
 title: "Стек и обход в глубину",
 },

```

```

 },
 "dynamic-programming": {
 title: "Динамическое программирование",
 hint: "Таблица подзадач заполняется на глазах.",
 Component: DPVisualizer,
 },
 "splay-tree": {
 title: "Splay: механика → большое дерево → собери с...",
 hint: "Три вкладки: покадровая механика поворотов в
 песочница «подними узел сам» с проверкой порядка.
 Component: SplayPlayground,
 },
 "graph-dfs-bfs": { title: "DFS и BFS на графе", Component: ... },
 "top-sort": { title: "Топологическая сортировка", Component: ... },
 "scc-kosaraju": { title: "Компоненты сильной связности", Component: ... },
 "graph-articulation": { title: "Точки сочленения", Component: ... },
 "bridges-code": { title: "Мосты: DFS + tin/low", Component: ... },
 "euler-path-vs-cycle": { title: "Эйлеров путь и цикл", Component: ... },
 "planarity-euler-formula": {
 title: "Планарность и покраска: $K_5/K_{3,3}$, $\chi \leq \Delta+1$, д...
 hint: "Четыре вкладки: непланарные K_5 и $K_{3,3}$, жадна...
 ашего кода красит вершины напрямую.",
 Component: PlanarColoringViz,
 },
 dijkstra: {
 title: "Дейкстра",
 hint: "Жадно забираем ближайшую вершину и релаксируем...",
 Component: DijkstraViz,
 },
 "bellman-ford": {
 title: "Форд–Беллман",
 Component: BellmanFordViz,
 },
 floyd: {
 title: "Флойд–Уоршелл",
 Component: FloydViz,
 },
 "johnson-algo": { title: "Алгоритм Джонсона", Component: ... }

```

```
 },
 };

export const getViz = (id?: string): VizEntry | undefined
```

---

ФАЙЛ 81/111: src/components/VizSourceBadge.tsx

КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОКА 1

---

```
import { Cpu, Film } from "lucide-react";
import { Tooltip } from "../Tooltip";
```

```
interface VizSourceBadgeProps {
```

```
 /**
```

```
 * true — картинку рисуют переменные из выполненного кода
```

```
 * false — компонент показывает собственную встроенную картинку
```

```
 */
```

```
 linked: boolean;
```

```
 /**
```

```
 * Компилятор уже выполнил код и прислал шаг, но переменные ещё не обновлены
```

```
 * (отладчик стоит ДО первой строки или в коде другие переменные не изменились)
```

```
 * подпись должна объяснять, почему картинка ещё не инициализирована
```

```
 */
```

```
 awaiting?: boolean;
```

```
 /** Строка пользовательского кода, на которой стоит отладчик
```

```
 line?: number;
```

```
 /** Короткая подпись данных компилятора: «матрица 5×5»
```

```
 detail?: string;
```

```
 /** Как называется встроенная демонстрация (для подписи)
```

```
 demoTitle?: string;
```

```
 /** Размеры демо-сетки и данных пользователя не совпадают
```

```
 mismatch?: string | null;
```

```
 className?: string;
```

```
}
```

```
/**
```

```

 сценарий: {demoTitle}

) : null}

 {mismatch ? (
 <Tooltip side="bottom" content={mismatch}>
 <span
 tabIndex={0}
 className="cursor-help rounded-full border border-slate-500 p-2"
 >
 размерность не совпадает

 </Tooltip>
) : null}
 </div>
);
}
```

---

ФАЙЛ 82/111: `src/components/WaterfallAnimationWidget.tsx`  
КАТЕГОРИЯ: Интерактивные компоненты и симуляторы | СТРОИТЕЛЬСТВО

---

```
import React, { useContext, useEffect, useMemo, useState } from 'react';
import { ArrowRight, CheckCircle2, AlertCircle, Lightbulb } from 'lucide-react';
import { VizChapterContext, emitVizDemo, useVizRuntime, useVizState } from './VizChapterContext';

// Структура заранее подготовленного красивого бора для визуализации
// Словарь: ["HE", "SHE", "HIS", "HERS"]
interface WNode {
 id: number;
 label: string; // путь от корня
 char: string;
 x: number; // Координаты для SVG водопада
 y: number;
```

```

export const WaterfallAnimationWidget: React.FC = () =>
 // Переключатель режимов: 'training' (Полный BFS обхо
 const [activeMode, setActiveMode] = useState<'trainin

 /* — связь с Python-компилятором (PAGE_SYNC["aho-cor
 text — сканируемый текст, state — вершина автомата
 matches — найденные вхождения (индекс конца, слово
 const chapterId = useContext(VizChapterContext);
 const runtime = useVizRuntime();
 const vars = runtime?.variables;

 useEffect(() => {
 if (chapterId) emitVizDemo(chapterId, "aho");
 }, [chapterId]);

 const liveText = vars ? vizString(vars.text) ?? vizSt
 const liveState = vars ? vizNumber(vars.state) ?? viz
 const liveI = vars ? vizNumber(vars.i) ?? vizNumber(v
 const liveMode = vars ? vizString(vars.mode) : null;
 const liveMatchesRaw = vars ? vizArray(vars.matches)
 const liveMatchesSig = useMemo(() => (liveMatchesRaw
 const liveFail = vars ? vars.fail : undefined;

 useEffect(() => {
 if (!vars) return;
 if (liveMode && /search|поиск/i.test(liveMode)) set
 else if (liveMode && /train|обуч|bfs/i.test(liveMod
 if (liveText) {
 const clean = liveText.toUpperCase().replace(/[^A
 if (clean && clean !== textToScan) {
 setTextToScan(clean);
 setCurrentIndex(0);
 setCurrentNode(0);
 setFoundMatches([]);
 setIsSearchDone(false);
 setJumpPath(null);
 setActiveMode("search");
 }
 }
 });

```

```

// === СОСТОЯНИЯ РЕЖИМА ОБУЧЕНИЯ (TRAINING) ===
// Шаги обучения от 0 до 8 (по числу вершин в боре, и
const [trainStep, setTrainStep] = useState<number>(0);

const handleTextChange = (e: React.ChangeEvent<HTMLInp
 const clean = e.target.value.toUpperCase().replace(
 setTextToScan(clean);
 resetSearchSimulation();
};

const resetSearchSimulation = () => {
 setCurrentIndex(0);
 setCurrentNode(0);
 setIsSearchDone(false);
 setSearchLog('Симуляция сброшена. Лосось вернулся н
 setJumpPath(null);
 setFoundMatches([]);
 setActiveSearchCodeLine(1);
};

const stepSearchSimulation = () => {
 if (currentIndex >= textToScan.length) {
 setIsSearchDone(true);
 setSearchLog('Мы проплыли весь текст! Лосось дово
 setActiveSearchCodeLine(4);
 return;
 }

 const char = textToScan[currentIndex];
 let curr = currentNode;
 let logMsg = `[Символ '${char}']`;

 let jumpType: 'normal' | 'fail' | null = null;
 void jumpType;
 let originalCurr = curr;

 if (TRIE_TRANSITIONS[curr][char] !== undefined) {
 const nextNode = TRIE_TRANSITIONS[curr][char];

```



```

 if (matchedWordsForLog.length > 0) {
 logMsg += ` БИНГО! Найдено слово: ${matchedWords}
 setActiveSearchCodeLine(4);
 }

 setCurrentNode(curr);
 setCurrentIndex(currentIndex + 1);
 setSearchLog(logMsg);
 if (currentMatches.length > 0) {
 setFoundMatches((prev) => [...prev, ...currentMat
 }
};

// ПОЛНАЯ ПОШАГОВАЯ ИНФОРМАЦИЯ О ПОСТРОЕНИИ ДЕРЕВА (В
const TRAINING_STEPS_INFO = [
 {
 targetNodeId: 0,
 title: 'Шаг 0 (Depth 0): Вершина #0 (ROOT)',
 desc: 'Начало алгоритма BFS. Корень (ROOT) не обр
 рние вершины сразу инициализируются с fail = RO
 codeLine: 1,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'ø',
 checkingBranchesFrom: null,
 },
 {
 targetNodeId: 1,
 title: 'Шаг 1 (Depth 1): Вершина #1 (H)',
 desc: 'Мы обрабатываем первую вершину из очереди
 просто не существует. fail[H] автоматически нап
 codeLine: 5,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'H',
 checkingBranchesFrom: null,
 },

```

```

 суффикс "H"!',
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'H',
 checkingBranchesFrom: 0,
 },
 {
 targetNodeId: 3,
 title: 'Шаг 6 (Depth 3): Вершина #3 (HER)',
 desc: 'Переходим на Depth 3 к #3 (HER). Родитель :
 е подходят. fail[HER] = ROOT.',
 codeLine: 3,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'R',
 checkingBranchesFrom: 0,
 },
 {
 targetNodeId: 6,
 title: 'Шаг 7 (Depth 3): Вершина #6 (HIS)',
 desc: 'Обрабатываем #6 (HIS) на Depth 3. Родитель :
 но ветка "S" → #7 (S) совпадает! fail[HIS] = #7',
 codeLine: 4,
 scoutPos: 0,
 inspectedParentFail: 0,
 checkTargetChar: 'S',
 checkingBranchesFrom: 0,
 },
 {
 targetNodeId: 9,
 title: 'Шаг 8 (Depth 3): Вершина #9 (SHE) — КАК С
 desc: ' МАГИЯ АХО-КОРАСИК! Обрабатываем #9 (SHE)
 ба плывет в #1 (H) и ищет ветку "E". У вершины :
 codeLine: 4,
 scoutPos: 1,
 inspectedParentFail: 1,
 checkTargetChar: 'E',
 },

```

```

 onClick={() => setActiveMode('training')}
 className={`flex items-center space-x-1.5 p-2
 activeMode === 'training'
 ? 'bg-indigo-600 text-white shadow-lg
 : 'text-slate-400 hover:text-white'
 `}
 }
 >
 <GitBranch className="w-4 h-4" />
 Режим 1: Полное обучение (BFS от корня)
</button>
<button
 onClick={() => setActiveMode('search')}
 className={`flex items-center space-x-1.5 p-2
 activeMode === 'search'
 ? 'bg-blue-600 text-white shadow-lg
 : 'text-slate-400 hover:text-white'
 `}
 >
 <Play className="w-4 h-4" />
 Режим 2: Поиск в тексте
</button>
</div>
</div>

{/* ПАНЕЛЬ УПРАВЛЕНИЯ В ЗАВИСИМОСТИ ОТ РЕЖИМА */}
{activeMode === 'training' ? (
 /* ПАНЕЛЬ РЕЖИМА ОБУЧЕНИЯ */
 <div className="grid grid-cols-1 lg:grid-cols-3
 {/* Управление шагами обучения */}
 <div className="bg-slate-900/90 p-5 rounded-x
 <div>
 <h5 className="text-indigo-300 font-bold
 Полный обход в ширину (В
 </h5>
 <p className="text-xs text-slate-300 mb-3
 Показаны все шаги очереди BFS (начиная с
 ар 8 (SHE → HE), где наглядно виден
 </p>

```

```

{ /* Блок с кодом построения fail-ссылки и опи
<div className="lg:col-span-2 bg-slate-900/90
>
<div>
 <h5 className="text-white font-bold mb-3
 <span className="flex items-center gap-
 { Код BFS построения fail

 <span className="text-xs bg-indigo-950
 Вершина: #{targetTrainingNode.id} ({t

 </h5>
 <div className="bg-slate-950 p-4 rounded-
 <div className={`p-1.5 rounded flex ite
 -l-4 border-indigo-500 text-indigo-300 font
 1. let u = trie[r][char]; // Те
 {currentTrainInfo.codeLine === 1 && <
 тут}
 </div>
 <div className={`p-1.5 rounded flex ite
 -l-4 border-indigo-400 text-indigo-200 font
 2. let v = fail[r]; // Подъем к
 ainInfo.inspectedParentFail].label}}
 {currentTrainInfo.codeLine === 2 && <
 к родителю}
 </div>
 <div className={`p-1.5 rounded flex ite
 l-4 border-amber-500 text-amber-300 font-bo
 3. while (v !== root && trie[v]
 {currentTrainInfo.codeLine === 3 && <
 т, возврат в корень}
 </div>
 <div className={`p-1.5 rounded flex ite
 r-l-4 border-emerald-500 text-emerald-300 f
 4. if (trie[v][char] !== undefin
 /span>
 {currentTrainInfo.codeLine === 4 && <

```

```

 placeholder="ВВЕДИТЕ ТЕКСТ..."
 className="w-full bg-slate-800 border border-blue-500 focus:outline-none focus:border-blue-500"
 />
 </div>

 <div>
 <div className="mb-4 flex flex-wrap gap-1">

 {textToScan.split('').map((char, idx) =>
 <span
 key={idx}
 className={`w-7 h-8 flex items-center justify-center
 ${idx === currentIndex ? 'bg-blue-600 text-white shadow' :
 idx < currentIndex ? 'bg-emerald-950/60 text-emerald-500' :
 'bg-slate-800 text-slate-500'}
 >
 {char}

)}
 </div>

 ГОТОВО ✓

 </div>

 <button
 onClick={stepSearchSimulation}
 disabled={isSearchDone || textToScan.length === 0}
 className={`w-full py-2.5 rounded-lg font-weight-medium
 ${isSearchDone || textToScan.length === 0 ? 'bg-slate-700 text-slate-500 cursor-not-allowed' :
 'bg-gradient-to-r from-blue-600 to-blue-900/40 active:scale-[0.98]'}
 >
 Поиск
 </button>
 </div>
 </div>
</div>

```

```

 </div>
 </div>

 <div>
 <h5 className="text-slate-300 font-bold m-0">
 <AlertCircle className="w-3.5 h-3.5 text-slate-300"/>
 </h5>
 <div className="bg-slate-800 p-3.5 rounded-lg items-center">
 {searchLog}
 </div>
 </div>
 </div>
 </div>
})

{/* SVG Анимация водопада */}
<div className="bg-gradient-to-b from-blue-950 via-blue-900 to-transparent pointer-events-none hidden shadow-2xl">
 {/* Водопадные фоновые эффекты */}
 <div className="absolute inset-0 opacity-10 bg-gradient-to-b from-blue-900 to-transparent pointer-events-none"></div>
 <div className="absolute top-0 left-1/2 -translate-x-1/2">
 <div className="w-8 bg-blue-400 h-full animate-pulse"></div>
 <div className="w-12 bg-blue-300 h-full animate-pulse"></div>
 <div className="w-6 bg-blue-500 h-full animate-pulse"></div>
 </div>

 <div className="flex items-center justify-between p-4">
 <h5 className="text-white font-bold text-base">
 Ветвящийся синий водопад (Борис)
 </h5>
 <div className="flex flex-wrap gap-4 text-xs">

 </div>
 </div>
</div>

```

```

return Object.entries(children).map(([char, children]) => {
 const toNode = WATERFALL_NODES[toId];

 // Подсветка в режиме поиска
 let isHighlighted = activeMode === 'search' ||
 type === 'normal';

 // Подсветка в режиме обучения (когда и
 let isTrainingExamined = activeMode === 'train' ||
 let isMatchBranch = isTrainingExamined & isMatchBranch;

 return (
 <g key={`edge-${fromNode.id}-${toNode.id}`}>
 {/* Линия течения воды */}
 <line
 x1={fromNode.x}
 y1={fromNode.y}
 x2={toNode.x}
 y2={toNode.y}
 stroke={isHighlighted ? '#3b82f6' : '#ccc'}
 strokeWidth={isHighlighted || isTrainingExamined ? 2 : 1}
 className={isHighlighted || isTrainingExamined ? 'highlighted' : ''}
 />
 {/* Символ перехода */}
 <circle cx={{fromNode.x + toNode.x} / 2} cy={{fromNode.y + toNode.y} / 2}
 r={10} fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5a') : 'white'}
 stroke={isHighlighted ? '#3b82f6' : '#ccc'}
 />
 <text
 x={{(fromNode.x + toNode.x) / 2}
 y={{(fromNode.y + toNode.y) / 2} + 15}
 fill={isTrainingExamined ? (isMatchBranch ? '#10b981' : '#f43f5a') : 'white'}
 fontSize="12"
 fontWeight="bold"
 textAnchor="middle"
 />
 {char}
 </text>
);
});

```

```

{/* ВИЗУАЛЬНЫЕ МЕТКИ ПРОВЕРКИ IF В
```

```

 <path
 d={`M ${node.x} ${node.y} Q ${cx} $
 fill="none"
 stroke={isHighlighted ? '#f43f5e' :
 strokeWidth={isHighlighted || isJustE
 strokeDasharray="6,6"
 className={isHighlighted || isJustE
 opacity={isHighlighted || isJustEst
 />
 {isJustEstablished && (
 <text x={cx} y={cy - 10} fill="#10b
 >
 fail[{node.label}] = {targetNode.
 </text>
)}
 </g>
);
 }}}

```

```

{ /* Рендер вершин (бассейнов водопада) */ }
Object.values(WATERFALL_NODES).map((node) => {
 const isCurrent = activeFishPosNode.id === node.id;
 const isTargetChild = activeMode === 'target' && node.id === targetNode.id;
 const hasWords = node.words.length > 0;

```

```

 return (
 <g key={`node-${node.id}`} className="node" >
 { /* Внешнее свечение для активной вершины */ }
 {isCurrent && (
 <circle cx={node.x} cy={node.y} r={node.radius * 1.5} fill="none" stroke={isHighlighted ? '#f43f5e' : '#10b981'} strokeDasharray="6,6" />
)}
 {isTargetChild && (
 <circle cx={node.x} cy={node.y} r={node.radius * 1.5} fill="none" stroke={isHighlighted ? '#f43f5e' : '#10b981'} strokeDasharray="6,6" />
)}

```

```

 { /* Основной круг вершины */ }
 <circle
 cx={node.x}

```



```

 }}
 </g>
);
 }}}

 { /* ОДИН ПЛАВНЫЙ АНИМИРОВАННЫЙ ЭХО-ЛОСОСЬ */
 <g>
 { /* Круг-подложка под рыбу */
 <circle
 cx={activeFishPosNode.x + 15}
 cy={activeFishPosNode.y - 25}
 r="18"
 fill={activeMode === 'training' ? '#8b5'
 stroke="#ffffff"
 strokeWidth="2"
 style={{ transition: 'cx 0.8s cubic-bez
 className="shadow-lg"
 />
 { /* Сама рыба */
 <text
 x={activeFishPosNode.x + 15}
 y={activeFishPosNode.y - 19}
 fontSize="18"
 textAnchor="middle"
 style={{ transition: 'x 0.8s cubic-bezi
 className="animate-float select-none"
 >
 {activeMode === 'training' ? ' ♂ ' :
 </text>
 </g>

 </svg>
</div>
</div>

```

```

 { /* Список найденных совпадений (только в режиме
 {activeMode === 'search' && {
 <div className="mt-6 bg-slate-900/50 p-5 rounded

```

```

 {liveState !== null && {
 <span className="text-[11px] font-mono bg-
 state = {liveState}{WATERFALL_NODES[liveState]}

 }}
 {liveMatchesSig && <span className="text-[11px] font-mono bg-
 d px-2 py-1">matches: {foundMatches.map((m) => {
 {liveFail !== undefined && <span className="text-[11px] font-mono bg-
 ed-md px-2 py-1">fail = {JSON.stringify(liveState)}

 }) : {

 открой панель Python и запусти Ахо–Корасик:
 "font-mono text-cyan-300">state</code>, <span className="text-[11px] font-mono bg-
 an-300">matches</code> поведут лосося по этому пути

 }}
 </div>
 </div>
);
 };
};

```

## РАЗДЕЛ: ВИЗУАЛИЗАТОРЫ (ВЛОЖЕННЫЕ)

ФАЙЛ 83/111: src/components/visualizers/PrefixSumViz.tsx  
 КАТЕГОРИЯ: Визуализаторы (вложенные) | СТРОК: 466 | РАЗМЕР: 1.5 КБ

```

import React, { useContext, useEffect, useMemo, useState } from 'react';
import {
 emitVizDemo,
 liveMatrix,
 pickLiveVar,
 useVizRuntime,
} from '@visuallib/core';

```

```

<div className="w-full bg-slate-950 rounded-2xl border-2 border-slate-800" >
 <div className="flex gap-2 mb-4 overflow-x-auto" >
 {(
 [
 ["1d", "1. Одномерные суммы"],
 ["2d", "2. Двумерные суммы"],
] as const
).map(({key, label}) => {
 <button
 key={key}
 onClick={() => setMode(key)}
 className={`px-3 py-2 rounded-lg text-xs sm:text-sm
 mode === key ? "bg-indigo-600 text-white" : "text-slate-400"
 }` >
 {label}
 </button>
))}
 </div>
 {mode === "1d" ? <Prefix1D /> : <Prefix2D />}
</div>
);
};

```

```

/* ----- 1D -----
const Prefix1D: React.FC = () => {
 const pref = useMemo(
 () => A1.reduce<number[]>((acc, v) => [...acc, acc[v]], []),
 []
);
};

```

```

/** Что сейчас под курсором: префикс P[i] или элемент A[i]
const [hover, setHover] = useState<{ kind: "pref" | "elem"; i: number }>({ kind: "pref", i: 0 });
const [range, setRange] = useState<{ l: number; r: number }>({ l: 0, r: 0 });
/** Снимок i/P из автоматически выполненного Python-кода
const runtime = useVizRuntime();
const vars = runtime?.variables;
const liveI = vizNumber(vars?.i);

```

```

const isDrivenAdd = shownDriven !== null &&
return (
 <div
 key={i}
 onMouseEnter={() => setHover({ kind:
 onMouseLeave={() => setHover(null)}
 onClick={() => setRange({r} => (i < r
 className={`${cellBase} cursor-pointe
 isDrivenAdd
 ? "bg-amber-500 text-black ring-2
 : inCover
 ? "bg-indigo-500 text-white rin
 : inRange
 ? "bg-indigo-900/70 text-indi
 : "bg-slate-800 text-slate-300
 }`}
 title={`a[${i}] = ${v}`}
 >
 {v}
 </div>
);
}}
</div>

```

```

{/* массив P */}
<div className="flex gap-1.5 items-center">
 <div className="w-[46px] shrink-0 text-right">
 {pref.map((v, i) => {
 const active = hover?.kind === "pref" &&
 const isL = i === range.l;
 const isR = i === range.r + 1;
 const liveValue = liveP?.[i];
 const blank = hasLiveP && (liveValue ===
 const masked = !hasLiveP && shownDriven !
 const isDrivenCur = shownDriven !== null &
 const shownValue = hasLiveP ? liveValue :
 return (
 <div

```

```

) : hover?.kind === "pref" ? (
 hover.i === 0 ? (
 <p className="text-slate-300">
 <b className="text-emerald-400">P[0] = 0
 отрезка, начинающегося с нуля.
 </p>
) : (
 <p className="text-slate-300">
 <b className="text-emerald-400">
 P[{hover.i}] = {pref[hover.i]}
 {" "}
 – это сумма всего, что набежало от начала

 a[0..{hover.i - 1}] = {A1.slice(0, hove

 </p>
)
) : hover?.kind === "a" ? (
 <p className="text-slate-300">
 <b className="text-indigo-400">
 a[{hover.i}] = {A1[hover.i]}
 {" "}
 входит во все префиксы, начиная с{" "}

 подвинуть границу запроса.

 </p>
) : (
 <p className="text-slate-500">
 Наведите курсор на любое число: у <b classN
 у <b className="text-indigo-400">a – ку
 </p>
)
 })
</div>

{/* Запрос */}
<div className="bg-slate-900 border border-indigo
 <div className="text-xs text-slate-400 mb-2">
 Запрос суммы на отрезке (кликайте по массиву

```

---

```

* Матрицы из кода пользователя рисуются В СВОЕЙ разме
*
* Раньше значения S[i][j] протискивались в демо-сетку
* размере матрицы часть ячеек становилась «пустыми»,
* шаги компилятора не совпадали с картинкой.
*/

```

```

const livePrefSource = pickLiveVar(vars, ["S", "P", "g"]);
const liveS = liveMatrix(livePrefSource?.value);
const hasLiveS = liveS !== null;

```

```

const liveArrSource = pickLiveVar(vars, ["a", "A", "g"]);
const liveA = liveMatrix(liveArrSource?.value);

```

```

const prefRows = liveS?.rows ?? P.length;
const prefCols = liveS?.cols ?? (P[0]?.length ?? 0);
const grid: (number | null)[][] = liveS
 ? Array.from({ length: prefRows }, (_, i) =>
 Array.from({ length: prefCols }, (_, j) => liveS[i][j]
)
 : P;

```

```

const srcRows = liveA?.rows ?? n;
const srcCols = liveA?.cols ?? m;
const srcGrid: (number | null)[][] = liveA
 ? Array.from({ length: srcRows }, (_, i) => Array.from(
 { length: srcCols }, (_, j) => liveA[i][j]
)
 : A2;

```

```

const cellAt = (i: number, j: number): number | null {
 if (i >= 0 && j >= 0 && i < prefRows && j < prefCols) return liveS[i][j];
 return null;
}

```

```

const sizeMismatch =
 hasLiveS && liveS && (liveS.rows !== P.length || liveS.cols !== P[0].length)
 ? `Демо построено на матрице ${n}x${m} (сетка пре
 мет имеет размер ${liveS.sizeLabel}. Ниже нарисована
 : null;

```

```

useEffect(() => {
 const r1 = vizNumber(vars?.r1);

```

```

 </p>
) : null}
 <div className="grid gap-5 lg:grid-cols-2">
 { /* Исходная матрица */ }
 <div className="overflow-x-auto">
 <div className="text-xs text-slate-400 mb-2">
 Исходная матрица {liveA ? `${liveArrSource?`
 чтобы задать прямоугольник
 </div>
 <div className="inline-block">
 {srcGrid.map((row, i) => {
 <div key={i} className="flex gap-1.5 mb-1">
 {row.map((v, j) => {
 const inRect = i >= r1 && i <= r2 &&
 return {
 <div
 key={j}
 onClick={() =>
 setRect((r) =>
 i < r.r1 || j < r.c1
 ? { r1: i, c1: j, r2: Math.r
 : { ...r, r2: i, c2: j }
)
 }
 className={` ${cellBase} cursor-po
 inRect ? "bg-indigo-500 text-wh
 }`}
 >
 {v === null ? "." : v}
 </div>
)};
 }}}
 </div>
)}}
 </div>
 { /* Матрица префиксных сумм */ }

```

```

 {corner && !isHover && {
 <span className="absolute -top-
ht">
 {corner}

 }}
 </div>
);
 }}}
 </div>
 }}}
 </div>
 </div>
 </div>

<div className="min-h-[54px] bg-slate-900 border
 {hover ? {
 <p className="text-slate-300">
 <b className="text-amber-400">
 {liveS ? livePrefSource?.name ?? "S" : "P
 {" "}
 – сумма всего прямоугольника от левого верх
 </p>
 } : {
 <p className="text-slate-500">Наведите курсор
 }}
 </div>

<div className="bg-slate-900 border border-indigo
 <p className="font-mono text-sm sm:text-base te
 сумма = A<
 C + <s
 ={" "}
 {
 </p>
 <p className="font-mono text-[11px] text-slate-
 прямоугольник ({r1}, {c1}) ... ({r2}, {c2}) · c
 </p>

```



```
const st1D: number[][] = Array.from({ length: N }, () =>
for (let i = 0; i < N; i++) st1D[i][0] = arr1D[i];
for (let j = 1; j < LOG; j++) {
 for (let i = 0; i + (1 << j) <= N; i++) {
 st1D[i][j] = Math.min(st1D[i][j - 1], st1D[i + (1 << j)][j - 1]);
 }
}
```

// Data for 2D Matrix (Sparse Table 2D)

```
const val2D = [
 [45, 12, 88, 34, 11, 76, 23, 90],
 [67, 19, 44, 55, 33, 21, 65, 87],
 [14, 51, 99, 13, 22, 64, 43, 76],
 [89, 32, 54, 71, 15, 88, 29, 60],
 [25, 41, 16, 92, 9, 17, 56, 31],
 [59, 18, 77, 24, 61, 82, 35, 12],
 [73, 8, 38, 85, 47, 95, 19, 58],
 [39, 81, 62, 28, 51, 42, 85, 14]
];
```

```
const ST2D: number[][][][] = Array.from({length: 8}, () =>
 Array.from({length: 8}, () =>
 Array.from({length: 4}, () =>
 Array(4).fill(0)
)
)
);
```

```
for (let r = 0; r < 8; r++) {
 for (let c = 0; c < 8; c++) {
 ST2D[r][c][0][0] = val2D[r][c];
 }
}
```

```
for (let kx = 0; kx <= 3; kx++) {
 for (let ky = 0; ky <= 3; ky++) {
 if (kx === 0 && ky === 0) continue;
```

```

 </button>
 <button
 onClick={() => setTab("2d_build")}
 className={`px-3 sm:px-4 py-2 rounded-lg text-
 digo-600 text-white" : "bg-slate-800 text-s
 >
 2. Сворачивание 2D (Построение)
 </button>
 <button
 onClick={() => setTab("2d_query")}
 className={`px-3 sm:px-4 py-2 rounded-lg text-
 se-600 text-white" : "bg-slate-800 text-sla
 >
 3. Запрос 2D (Формула)
 </button>
 </div>

 <AnimatePresence mode="wait">
 {tab === "1d" ? (
 <Viz1D key="1d" />
) : tab === "2d_build" ? (
 <Viz2DBuild key="2d_build" />
) : (
 <Viz2DQuery key="2d_query" />
)}
 </AnimatePresence>
 </div>
);
};

```

```

const Viz1D: React.FC = () => {
 const [step, setStep] = useState(0);
 const [hover, setHover] = useState<{ i: number; j: number}>({ i: 0, j: 0 });
 const runtime = useVizRuntime();
 const runtimeVars = runtime?.variables;
 const liveI = vizNumber(runtimeVars?.i);
 const liveJ = vizNumber(runtimeVars?.j);
 const liveSource = vizArray(runtimeVars?.a);

```

```

const pointedCell = hover ?? liveTarget;
const covered = pointedCell ? { from: pointedCell.i,

return (
 <div className="flex flex-col gap-5">
 <div className="flex items-center justify-between">
 <div>
 <h3 className="text-lg font-bold text-indigo-600">
 Построение Sparse Table (Min)
 </h3>
 <p className="text-sm text-slate-400">
 Формула: $ST[i][j] = \min(ST[i][j-1], ST[i + 1][j])$
 </p>
 {dimMismatch ? (
 <p className="mt-2 max-w-[560px] rounded-lg bg-slate-800 text-white">
 {dimMismatch}
 </p>
) : null}
 </div>
 <div className="flex gap-2">
 <button
 onClick={() => setStep(Math.max(0, step - 1))}
 className="p-2 bg-slate-800 hover:bg-slate-700 disabled:{step === 0}"
 >
 <ChevronLeft size={20} />
 </button>
 <button
 onClick={() => setStep(Math.min(maxStep, step + 1))}
 className="p-2 bg-indigo-600 hover:bg-indigo-500 disabled:{step === maxStep}"
 >
 <ChevronRight size={20} />
 </button>
 <button
 onClick={() => setStep(0)}
 className="p-2 bg-slate-800 hover:bg-slate-700 disabled:{step === 0}"
 >

```

```


 [{covered.from}, {covered.to}]
 {" "}
 длины $2^{\text{hover.j}}$ = {1 << hover.j}:{ " "}

 min({arr1D.slice(covered.from, covered.to)}

 </p>
) : (
 <p className="text-slate-500">
 Наведите курсор (или тапните) на любое число
 </p>
)}
 </div>
</div>

<div className="overflow-x-auto pb-2">
 <div className="inline-block min-w-full bg-slate-50">
 <div className="grid grid-cols-[auto_repeat(4,1fr)]">
 <div className="font-mono text-slate-500">
 /* Headers */
 i \ j
 </div>
 <div className="text-center font-bold text-slate-500">
 2^0 (L=1)
 </div>
 <div className="text-center font-bold text-slate-500">
 2^1 (L=2)
 </div>
 <div className="text-center font-bold text-slate-500">
 2^2 (L=4)
 </div>
 <div className="text-center font-bold text-slate-500">
 2^3 (L=8)
 </div>
 </div>
 </div>
</div>

```

```

// Готовые вычисленные уровни демо не
// уровень j=0 всегда берём из уже сущ
const shownValue = compilerLinked ? r
const displayValue =
 shownValue === undefined || shownVa
 ? "."
 : typeof shownValue === "object"
 ? JSON.stringify(shownValue)
 : String(shownValue);

return (
 <div key={j} className="relative fl
 {!isValid ? (
 <div className="w-[clamp(30px,9
) : (
 <motion.div
 onMouseEnter={() => (compiler
 onMouseLeave={() => setHover(
 onClick={() => (compilerLinker
)}}

 initial={{ opacity: 0, scale:
 animate={{
 // При st = [] каркас уже с
 opacity: compilerLinked ? 1
 scale: compilerLinked || is
 }}
 className={`w-[clamp(30px,9vw
clamp(11px,3vw,17px)] font-bold transition-
 hover && hover.i === i && h
 ? "bg-sky-500 text-white
 : isActive
 ? "bg-indigo-500 text-whi
 : isSrc1
 ? "bg-emerald-500 text-
 : isSrc2
 ? "bg-amber-500 text-
 : isVisible
 ? "bg-slate-800 tex

```

```

 </div>
 </div>

 { /* Formula helper text */
 <div className="bg-slate-900 border border-slate-800" >
 {compilerLinked && liveTarget ? (
 <p className="text-base text-slate-300">

 i =
 – активна ячейка <span className="font-mono"
 {(hasLiveTable || hasLiveSource) && <> = <span
 ?.[liveTarget.j] ?? (liveTarget.j === 0 ? l
 </p>
) : step > 0 && step <= maxStep ? (
 (() => {
 const c = computeSteps[step - 1];
 const half = 1 << (c.j - 1);
 return (
 <p className="text-lg text-slate-300">

 ST[{c.i}][{c.j}]
 {" "}
 = min(
 <span className="text-emerald-400 font-l"
 ST[{c.i}][{c.j - 1}]

 ,
 <span className="text-amber-400 font-bo"
 ST[{c.i + half}][{c.j - 1}]

) → min(
 {st1D

 {st1D[c.i + half][c.j - 1]}

) ={" "}
 <span className="text-indigo-400 font-b"
 {st1D[c.i][c.j]}

```

```

const hasLiveTable = !!vars && Object.prototype.hasOwnProperty.call(vars, 'liveTable');
const liveTableEmpty = hasLiveTable && Object.keys(liveTable).length === 0;
const codeCell = liveR !== null && liveC !== null ? {
 r: liveR, c: liveC, k: liveK,
 compilerLinked: codeCell !== null || liveK !== null,
 shownK: pinnedK !== null
 ? pinnedK
 : liveK !== null
 ? Math.max(0, Math.min(3, Math.trunc(liveK)))
 : hasLiveSource
 ? 0
 : k;
} : null;

/** 2D-демонстрация нарисована на фиксированной сетке */
const liveSource2D = liveMatrix(vars?.A);
const dimMismatch2D = liveSource2D
 ? liveSource2D.rows !== 8 || liveSource2D.cols !== 8
 ? `Демо 2D построено на матрице 8x8, а в вашем коде матрица 8x8
 не её не отрисовываются, значения вашей матрицы`
 : null
: null;

// Уровень и клетка берутся прямо из k/r/c; номер строки и столбца
useEffect(() => {
 setHover(null);
 setLocked(null);
}, [k]);

const L = 1 << shownK;
const activeCell = codeCell || locked || hover;

return (
 <div className="flex flex-col xl:flex-row gap-6">
 <div className="flex-1 min-w-0 bg-slate-900 p-3 shadow">
 <h3 className="text-xl font-bold text-indigo-400">Матрица
 <p className="text-sm text-slate-400 mb-6 text-slate-400">
 Для наглядности показываем только <b className="text-slate-400">


```

```

<h4 className={`font-bold mb-3 flex
 }>
 Матрица блоков {stepL}x{step
 <span className={`text-[10px] font
rder-indigo-800/40 text-indigo-300' : 'bg-s
</h4>
<div className="max-w-full overflow-
 <div className={`grid gap-[2px] p-
go-900/40 shadow-[0_0_25px_rgba(99,102,241,
eColumns: `repeat(${stepSize}, max-content)
 {Array.from({length: stepSize * st
 const r = Math.floor(idx / step
 const c = idx % stepSize;

 const liveValue = liveTable?.[
 // Базовый слой (блоки 1x1) – э
 // одного элемента равен ему сам
 // показываем из A сразу, как A
 // цикла построения st2. Уровни
 const sourceValue = step === 0
 const shownValue = compilerLink
 const isBlank = shownValue ===
 let highlightClass = isBlank
 ? 'bg-slate-950 text-slate-600'
 : isCurr
 ? 'bg-slate-800/80 text-slate-300'
 : 'bg-slate-800/50 text-slate-300'
 let zIndex = 'z-0';

 const isActive = !!activeCell &&
ll.c + L) && ((r - activeCell.r) % (1 << st

 if (isActive) {
 if (isCurr) {
 // Значения в ячейке ещё не
 // контур «сюда смотрит код
 // залитая пустая клетка вы
 highlightClass = isBlank

```





```

<p className="text-slate-300 mb-3 t
 Пример для ячейки:
 {locked && <span className="text
ded border border-rose-500/20"><Lock size={
</p>
<p className="text-indigo-300 borde
 <span className="font-bold text-w
}}[{{shownK}}][{{shownK}}]

</p>

<div className="space-y-2 pt-3 pl-2
 {{{}} => {
 const prevL = 1 << (shownK -
 return (
 <>
 <div className="flex ite
 <span className="flex
shadow-[0_0_8px_#f43f5e]"></div> ST[{{activeC
 <span className="text
tiveCell.r}[activeCell.c][shownK-1][shownK-
 </div>
 <div className="flex ite
 <span className="flex
shadow-[0_0_8px_#3b82f6]"></div> ST[{{activeC
 <span className="text
tiveCell.r}[activeCell.c + prevL][shownK-1]
 </div>
 <div className="flex ite
 <span className="flex
-500 shadow-[0_0_8px_#10b981]"></div> ST[{{a
 <span className="text
>{ST2D[activeCell.r + prevL][activeCell.c][
 </div>
 <div className="flex ite
 <span className="flex
shadow-[0_0_8px_#f59e0b]"></div> ST[{{activ
 <span className="text

```

```

const nr1 = vizNumber(vars?.r1);
const nc1 = vizNumber(vars?.c1);
const nr2 = vizNumber(vars?.r2);
const nc2 = vizNumber(vars?.c2);
if (nr1 !== null) setR1(Math.max(0, Math.min(7, M
if (nc1 !== null) setC1(Math.max(0, Math.min(7, M
if (nr2 !== null) setR2(Math.max(0, Math.min(7, M
if (nc2 !== null) setC2(Math.max(0, Math.min(7, M
}, [runtime]));

const queryVars = runtime?.variables;
const liveCoords = ["r1", "c1", "r2", "c2"].map((na
const queryMismatch = liveCoords.some((v) => v !== n
 ? `Координаты запроса в вашем коде выходят за дем
] ?? "-"}, r2=${liveCoords[2] ?? "-"}, c2=${live
 : null;

const H = Math.max(1, r2 - r1 + 1);
const W = Math.max(1, c2 - c1 + 1);
const kx = Math.floor(Math.log2(H));
const ky = Math.floor(Math.log2(W));
const Lx = 1 << kx;
const Ly = 1 << ky;

const matRows = 8 - Lx + 1;
const matCols = 8 - Ly + 1;

return (
 <div className="flex flex-col xl:flex-row gap-6
 <div className="flex-1 min-w-0 bg-slate-900
 <h3 className="text-xl font-bold text-ro
 <p className="text-slate-400 text-[13px]
 еpx, r2, c2 - правый низ).</p>
 {queryMismatch ? (
 <p className="mb-3 rounded-lg border b
 {queryMismatch}
 </p>
) : null}

```

```

 <div className="absolute pointer-e
ld-400 shadow-[0_0_10px_#10b981]" style={{
8px)` , width: `calc(${(Ly / 8) * 100}% - 16
 <div className="absolute pointer-e
00 shadow-[0_0_10px_#f59e0b]" style={{ top:
00}% + 8px)` , width: `calc(${(Ly / 8) * 100
 </div>

 <div className="bg-slate-950 p-4 round
er-slate-800 shadow-[inset_0_2px_10px_rgba(
 <label className="flex items-cen
 <span className="text-slate-
 <div className="flex gap-2">
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
 </div>
 </label>
 <label className="flex items-cen
 <span className="text-slate-
 <div className="flex gap-2">
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
 <input type="number" min={
14 bg-slate-900 border border-slate-700 py-
 </div>
 </label>
 </div>
</div>
</div>

<div className="flex-1 bg-slate-900 p-6 rou
 <h3 className="text-xl font-bold text-em
 <p className="text-sm font-sans text-sla
 Мы знаем, что максимальная степень дв
rounded text-white">{Lx}. Аналогично
te">{Ly}.

```

```

и влево от c2)

 >r2 - Lx + 1][
 {'}'}));
 </div>

 <div className="w-full flex justify-center">
 <div className="flex flex-col items-center">
 <h4 className="text-slate-400 font-bold">
 Откуда берутся эти 4 числа: матрица
 </h4>
 <div className="grid gap-[2px] p-2.5">
 Columns: `repeat(${matCols}, 1fr)`}}>
 {Array.from({length: matRows * matCols}, (v, idx) => {
 const r = Math.floor(idx / matCols);
 const c = idx % matCols;
 let isTL = r === r1 && c === c1;
 let isTR = r === r1 && c === c2;
 let isBL = r === r2 - Lx + 1 && c === c1;
 let isBR = r === r2 - Lx + 1 && c === c2;

 let color = "bg-slate-800 text-slate-200";
 let txt = ST2D[r][c][kx][ky];

 let badge = null;
 if (isTL) { color = "bg-rose-500 text-rose-100 border-2"; badge = "TL"; }
 else if (isTR) { color = "bg-lime-300 text-lime-100 border-2"; badge = "TR"; }
 else if (isBL) { color = "bg-emerald-300 text-emerald-100 border-2"; badge = "BL"; }
 else if (isBR) { color = "bg-amber-300 text-amber-100 border-2"; badge = "BR"; }

 return (
 <div key={idx} className={color} style={{padding: 5px, text-align: center}}>
 {txt}
 {badge}
 </div>
);
 })}
 </div>
 </div>
 </div>
 </div>
);
}

```

ФАЙЛ 85/111: src/layout/footer.html

КАТЕГОРИЯ: HTML-разметка (layout) | СТРOK: 137 | РАЗМЕР

```
 </div>
 </main>
</div>

<script>
 function setMode(mode) {
 document.getElementById('btn-mode-normal').className =
 ? 'px-3 py-1 rounded text-sm font-bold'
 : 'px-3 py-1 rounded text-sm font-bold';

 document.getElementById('btn-mode-wtf').className =
 ? 'px-3 py-1 rounded text-sm font-bold'
 : 'px-3 py-1 rounded text-sm font-bold';

 const aside = document.querySelector('aside');
 const headerMain = document.querySelector('header main');
 const questionsContainer = document.getElementById('questions-container');
 const wtfContainer = document.getElementById('wtf-container');

 if (mode === 'wtf') {
 document.body.classList.add('wtf-mode');
 if (aside) aside.style.display = 'none';
 if (headerMain) headerMain.style.display = 'none';
 if (questionsContainer) questionsContainer.style.display = 'none';
 if (wtfContainer) wtfContainer.classList.remove('hidden');

 // Remove the URL hash to avoid scrolling
 history.pushState("", document.title, window.location.pathname);
 window.scrollTo(0, 0);
 } else {
 document.body.classList.remove('wtf-mode');
 if (aside) aside.style.display = '';
 if (headerMain) headerMain.style.display = '';
 if (questionsContainer) questionsContainer.style.display = '';
 if (wtfContainer) wtfContainer.classList.add('hidden');
 }
 }

```

```

const closeDrawerBtn = document.getElementById("close-drawer");
const drawerOverlay = document.getElementById("drawer-overlay");

function toggleDrawer() {
 const isClosed = mobileDrawer.classList.contains("closed");
 if (isClosed) {
 mobileDrawer.classList.remove("closed");
 if (drawerOverlay) {
 drawerOverlay.classList.remove("hidden");
 // Timeout allows display:block to take effect
 setTimeout(() => drawerOverlay.classList.remove("hidden"), 100);
 }
 document.body.style.overflow = "hidden";
 } else {
 mobileDrawer.classList.add("closed");
 if (drawerOverlay) {
 drawerOverlay.classList.add("hidden");
 setTimeout(() => drawerOverlay.classList.add("hidden"), 100);
 }
 document.body.style.overflow = "";
 }
}

if (mobileMenuBtn && mobileDrawer) {
 mobileMenuBtn.addEventListener("click", () => toggleDrawer());
}
if (closeDrawerBtn && mobileDrawer) {
 closeDrawerBtn.addEventListener("click", () => toggleDrawer());
}
if (drawerOverlay) {
 drawerOverlay.addEventListener("click", () => toggleDrawer());
}

// Close on link click
document.querySelectorAll("#mobile-drawer a").forEach(link => {
 link.addEventListener("click", () => {
 if (mobileDrawer && !mobileDrawer.classList.contains("closed")) {
 toggleDrawer();
 }
 });
});

```

```
<!DOCTYPE html>
<html lang="ru" class="scroll-smooth">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width,
 <title>Пособие по алгебре: От пиццы к предикатам</t
 <script src="https://cdn.tailwindcss.com"></script>
 <script src="https://polyfill.io/v3/polyfill.min.js
 <script id="MathJax-script" async src="https://cdn.
 <script>
 window.MathJax = {
 tex: {
 inlineMath: [['$', '$'], ['\\(', '\\)']]
 displayMath: [['$$', '$$'], ['\\[', '\\
 }
 };
 </script>
 <style>
 @import url('https://fonts.googleapis.com/css2?
 @reference "./src/index.css";
 .math-font { font-family: 'Fira Code', monospac
 .uno-card {
 width: 70px; height: 105px; border-radius:
 box-shadow: 0 4px 8px rgba(0,0,0,0.5); posi
 align-items: center; justify-content: cente
 color: white; overflow: hidden; flex-shrink
 text-shadow: 2px 2px 0 #000, -1px -1px 0 #0
 font-family: 'Arial Black', Impact, sans-se
 }
 .uno-card::before, .uno-card::after {
 content: attr(data-val); position: absolute
 text-shadow: 1px 1px 0 #000, -1px -1px 0 #0
 }
 .uno-card::before { top: 2px; left: 4px; }
 .uno-card::after { bottom: 2px; right: 4px; tra
 .uno-card.mini { width: 45px !important; height
```



```

 us: 4px; }
 .uno-card.mini::before, .uno-card.mini::after {
 width: 100%; height: 100%;
 }

 .uno-equation .text-2xl { font-size: 1.2rem; }
 .uno-equation.gap-4 { gap: 0.5rem !important; }

 /* Allow horizontal scroll for equations in
 .formula-scroll { flex-wrap: nowrap !important; }
 px; }
 }
 .uno-inner {
 position: absolute; width: 110%; height: 75%;
 border-radius: 50%; transform: rotate(-30deg);
 box-shadow: inset 0 0 5px rgba(0,0,0,0.3);
 }
 .uno-val { position: relative; z-index: 2; }
 .card-red { background-color: #e52521; } .card-
 .card-blue { background-color: #004dcf; } .card-
 .card-green { background-color: #339e35; } .card-
 .card-yellow { background-color: #ffc400; } .ca
 .card-black { background-color: #222; } .card-b
 .card-black::before, .card-black::after { text-
 .info-block { @apply bg-slate-800/80 border-l-4
 .info-title { @apply font-bold mb-2 flex items-

/* Визуализация (Аналогии) */
.analogy-block { @apply bg-slate-900 border-2 b
 ems-center gap-6; }
.analogy-badge { @apply absolute -top-3 left-4 l
 king-widest border border-slate-500 shadow-md
.img-placeholder { @apply flex flex-col items-c
 r shrink-0 shadow-lg w-full md:w-auto; }
.img-caption { @apply font-mono text-sm mt-4 fo

/* THEME OVERRIDES */
body.theme-light {
 background-color: #f8fafc !important;
 color: #0f172a !important;

```

```

 .creepy-eye {
 transform-origin: center;
 transform-box: fill-box;
 animation: creepy-blink 4s infinite;
 }
 .creepy-eye-alt {
 transform-origin: center;
 transform-box: fill-box;
 animation: creepy-blink 5.2s infinite;
 animation-delay: 1.5s;
 }
 .creepy-emoji {
 display: inline-block;
 transform-origin: center;
 animation: creepy-blink 3.8s infinite;
 }
 </style>
</head>
<body class="bg-slate-900 text-slate-200 font-sans lead">

 <div class="sticky top-0 z-50 w-full bg-slate-950/90
 center items-center gap-4 sm:gap-8 transition-colors">
 <div class="flex items-center gap-3">
 Режим
 <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2">
 <button onclick="setMode('normal')" id="btn-normal" type="button"
 transition-colors">Список тем</button>
 <button onclick="setMode('wtf')" id="btn-wtf" type="button"
 transition-colors">WTF</button>
 </div>
 </div>
 <div class="flex items-center gap-3">
 Тема
 <div class="bg-slate-800 p-1 rounded-lg flex items-center gap-2">
 <button onclick="setTheme('dark')" id="btn-dark" type="button"
 transition-colors">Темная</button>
 <button onclick="setTheme('light')" id="btn-light" type="button"
 transition-colors">Светлая</button>
 </div>
 </div>
 </div>

```

```

<g id="lego-red">
 <rect x="0" y="0" width="40" height="20"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<g id="lego-rem">
 <rect x="0" y="0" width="40" height="10"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="2" y1="2" x2="38" y2="2" stro
</g>
<!-- МЯСОРУБКА -->
<g id="meat-grinder">
 <path d="М 10 40 L 15 20 Q 25 15 35 20
 <rect x="5" y="40" width="40" height="20"
 <!-- Ручка -->
 <path d="М 45 50 Q 60 50 60 30 L 75 30"
 <circle cx="75" cy="30" r="4" fill="#333"
 <!-- Выход -->
 <path d="М 5 45 L -5 45 L -5 55 L 5 55"
 <circle cx="25" cy="50" r="15" fill="#f
</g>
<g id="lego-green">
 <rect x="0" y="10" width="20" height="10"
 <rect x="3" y="5" width="14" height="5"
</g>
<g id="lego-white">
 <rect x="0" y="0" width="40" height="20"
 <rect x="6" y="-5" width="8" height="6"
 <rect x="26" y="-5" width="8" height="6"
 <line x1="0" y1="2" x2="40" y2="2" stro
 <text x="20" y="14" fill="#475569" font
</g>
</defs>
</svg>

<!-- Мобильная кнопка меню (Гамбургер) -->

```

```
<div class="pl-6 space-y-2 text
 <a href="#section-1-1" class="b
 <a href="#section-1-2" class="b
 <a href="#section-1-3" class="b
 <a href="#section-1-4" class="b
</div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-2" class=
er-indigo-500 pl-3 font-bold text-indigo-300">
 2. Комплексные числа

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#section-2-1" class="b
 <a href="#section-2-2" class="b
 <a href="#section-2-3" class="b
 </div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-3" class=
er-teal-500 pl-3 font-bold text-teal-300">
 3. Тригонометрическая форма

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#section-3-1" class="b
 <a href="#section-3-2" class="b
 <a href="#section-3-3" class="b
 <a href="#section-3-4" class="b
 </div>
</details>

<details class="group">
```

```
<div class="pl-6 space-y-2 text
 <a href="#q6-1" class="block ho
 <a href="#q6-2" class="block ho
 <a href="#q6-3" class="block ho
 <a href="#q6-4" class="block ho
 <a href="#q6-5" class="block ho
 <a href="#q6-6" class="block ho
</div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-7" class:
er-pink-500 pl-3 font-bold text-pink-300">
 7. Взаимно простые числ

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q7-1" class="bloc
 <a href="#q7-2" class="bloc
 <a href="#q7-3" class="bloc
 <a href="#q7-4" class="bloc
 <a href="#q7-5" class="bloc
 </div>
</details>

<details class="group">
 <summary class="list-none curso
 <a href="#question-8" class:
er-violet-500 pl-3 font-bold text-violet-300">
 8. Сравнимость и Поля В

 </summary>
 <div class="pl-6 space-y-2 text
 <a href="#q8-1" class="bloc
 <a href="#q8-2" class="bloc
 <a href="#q8-3" class="bloc
 <a href="#q8-4" class="bloc
```

```
<details class="group">
 <summary class="list-none cursor-pointer text-cyan-500 pl-3 font-bold text-cyan-400">
 11. НОД и НОК многочленов

</summary>
<div class="pl-6 space-y-2 text-cyan-400">

 Единственность НОД

 НОК

 ПИРАЛЬ)

 вращается!)
</div>
</details>
```

```
<details class="group">
 <summary class="list-none cursor-pointer text-blue-500 pl-3 font-bold text-blue-400">
 12. Взаимно простые многочлены

</summary>
<div class="pl-6 space-y-2 text-blue-400">

 НОД

 НОК

 деление)
</div>
</details>
```

```
<details class="group">
 <summary class="list-none cursor-pointer text-emerald-500 pl-3 font-bold text-emerald-400">
 13. Корни многочленов.

</summary>
```

```
 </div>
 </details>

 <details class="group">
 <summary class="list-none cursor-pointer text-red-500 pl-3 font-bold text-red-400">
 16. Неприводимые многоч.

 </summary>
 <div class="pl-6 mt-2 space-y-2">

 </div>
 </details>

 <div class="mt-4 mb-2 text-xs font-light text-slate-500 pl-3 text-slate-400">


```

```

const observer = new IntersectionObserver(
 if (isScrolling) return;

 let visibleId = null;
 entries.forEach(entry => {
 if (entry.isIntersecting) {
 visibleId = entry.target.id;

 // Highlight active section
 document.querySelectorAll('a').forEach(a => {
 a.style.fontWeight = 'normal';
 if (a.href.endsWith('#' + visibleId)) {
 a.style.fontWeight = 'bold';
 }
 });

 // Open the parent details
 const correspondingId = 'details-' + visibleId;
 if (correspondingLink) {
 const details = document.querySelector(correspondingId);
 if (details && !details.open) {
 details.open = true;
 document.querySelector('a[href="#' + correspondingId + '"]').focus();
 if (otherDetails) {
 otherDetails.close();
 }
 }
 }
 }
 });
}, {
 root: null,
 rootMargin: '-10% 0px -70% 0px',
 threshold: [0, 0.5]
});

sections.forEach(section => {

```



```
const textarea = document.createElement('div');
textarea.value = contentRaw;
textarea.style.flex = '1';
textarea.style.width = '100%';
textarea.style.color = '#000';
textarea.style.padding = '10px';
textarea.style.fontFamily = 'monospace';
```

```
const btnContainer = document.createElement('div');
btnContainer.style.display = 'flex';
btnContainer.style.gap = '10px';
btnContainer.style.marginTop = '10px';
```

```
const copyBtn = document.createElement('button');
copyBtn.innerText = 'Копировать';
copyBtn.style.padding = '10px';
copyBtn.style.backgroundColor = '#000';
copyBtn.style.color = '#fff';
copyBtn.style.border = 'none';
copyBtn.style.borderRadius = '5px';
copyBtn.style.cursor = 'pointer';
copyBtn.onclick = () => {
 if (navigator.clipboard) {
 navigator.clipboard.writeText(contentRaw);
 copyBtn.innerText = 'Скопировано';
 } else {
 textarea.select();
 document.execCommand('copy');
 copyBtn.innerText = 'Скопировано';
 }
 setTimeout(() => copyBtn.innerText = 'Копировать', 2000);
};
```

```
const closeBtn = document.createElement('button');
closeBtn.innerText = 'Закрыть';
closeBtn.style.padding = '10px';
closeBtn.style.backgroundColor = '#000';
```

```

 text = text.replace(/\n{3,}/g,
 '\n\n\n');

 if (window !== window.top) {
 fallbackModal(text, 'Markdown');
 return;
 }
 const blob = new Blob([text], {
 type: 'text/markdown'
 });
 const url = URL.createObjectURL(blob);
 const link = document.createElement('a');
 link.href = url;
 link.download = 'vyisshaya_algebra.md';
 document.body.appendChild(link);
 link.click();
 document.body.removeChild(link);
 }

 function setTheme(theme) {

 const body = document.body;
 body.classList.remove('bg-slate-900');

 document.getElementById('theme-dark').checked = theme === 'dark';
 document.getElementById('theme-light').checked = theme === 'light';
 document.getElementById('theme-auto').checked = theme === 'auto';

 let oldStyle = document.getElementById('theme-style');
 if (oldStyle) oldStyle.remove();
 const style = document.createElement('style');
 style.id = 'theme-style';

 if (theme === 'dark') {
 body.classList.add('bg-slate-900');
 document.getElementById('theme-dark').checked = true;
 } else if (theme === 'light') {
 body.classList.add('bg-white');
 document.getElementById('theme-light').checked = true;
 style.innerHTML = `

```

```

body.bg-white aside a.t
body.bg-white aside a:h

body.bg-white table tr.
body.bg-white table tr.
body.bg-white .text-tea
body.bg-white .text-ros
body.bg-white .text-eme
body.bg-white .text-cya
body.bg-white .text-amb
body.bg-white .text-fuch
body.bg-white .text-lim
body.bg-white .text-blur
body.bg-white .text-gre
body.bg-white .text-ind

`;
} else if (theme === 'terminal')
body.classList.add('bg-black')
document.getElementById('theme')
style.innerHTML = `
body.bg-black .bg-slate-950 {
background-color: transparent;
}
body.bg-black section header,
, body.bg-black button {
color: #4ade80 !important;
border-color: #22c55e !important;
font-family: monospace;
}
body.bg-black svg path,
stroke: #22c55e !important;
fill: transparent !important;
}
`;
}

```

```

 ">* </button>
 <button id="theme-terminal"
- center justify-center text-green-400 hover:
терминала"> </button>
 </div>
 </div>
 </div>
 </aside>

<!-- ОСНОВНОЙ КОНТЕНТ -->
<main class="flex-1 bg-slate-800 p-4 sm:p-8 md:p-12 lg:p-16
-base min-w-0">
 <header class="text-center mb-12 border-b border-slate-700">
 <h1 class="text-4xl md:text-5xl font-bl
4">
 ВЫСШАЯ АЛГЕБРА
 </h1>
 <p class="text-slate-400 italic text-lg">

 <!-- Print Table of Contents -->
 <div class="hidden print:block mt-8 text-left">
 <h2 class="font-bold text-xl mb-4">Print Table of Contents
 <ul class="text-base space-y-2">
 1. Алгебра
 2. Коммутативная алгебра
 3. Топология
 4. Статистика
 5. Группы
 6. Дифференциальное исчисление
 7. Анализ
 8. Квантовая механика
 9. Компьютерная графика
 10. Математический анализ
 11. Теория вероятностей
 12. Теория чисел

```

```

<a href="#question-2" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
 <div class="text-xs text-cyan-500"
 <div class="font-bold text-slate-700"

<!-- Row 2: Базовые операции / Евклид
<a href="#question-6" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow-500"
 <div class="font-bold text-slate-700"

<a href="#question-10" onclick="setM
-l-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange-500"
 <div class="font-bold text-slate-700"

<a href="#question-3" onclick="setM
-4 border-cyan-500 hover:bg-slate-700 trans
 <div class="text-xs text-cyan-500"
 <div class="font-bold text-slate-700"

<!-- Row 3: НОД и НОК / Алгоритм Ев
<a href="#question-7" onclick="setM
-4 border-yellow-500 hover:bg-slate-700 tra
 <div class="text-xs text-yellow-500"
 <div class="font-bold text-slate-700"

<a href="#question-12" onclick="setM
-l-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange-500"
 <div class="font-bold text-slate-700"

<!-- Empty 3rd row cell -->
<div class="hidden md:block"></div>

<!-- Row 4: Факторизация / Структур

```

```

 <!-- Row 7: Производная -->
 <div class="hidden md:block"></div>
 <a href="#question-15" onclick="setl
-l-4 border-orange-500 hover:bg-slate-700 t
 <div class="text-xs text-orange
 <div class="font-bold text-slate

 <div class="hidden md:block"></div>

 <!-- КЛАСТЕР 3: НЕПРИВОДИМОСТЬ -->
 <div class="col-span-1 md:col-span-1
b-2 border-b border-emerald-500/30 pb-2">Кл

 <a href="#question-17" onclick="setl
-l-4 border-teal-500 hover:bg-slate-700 tra
 <div class="text-xs text-teal-5
 <div class="font-bold text-slate

 <a href="#question-18" onclick="setl
-l-4 border-blue-500 hover:bg-slate-700 tra
 <div class="text-xs text-blue-5
 <div class="font-bold text-slate

 <a href="#question-19" onclick="setl
-l-4 border-emerald-500 hover:bg-slate-700
 <div class="text-xs text-emeral
 <div class="font-bold text-slate

 <a href="#question-20" onclick="setl
-l-4 border-rose-500 hover:bg-slate-700 tra
 <div class="text-xs text-rose-5
 <div class="font-bold text-slate

 <a href="#question-21" onclick="setl
-l-4 border-indigo-500 hover:bg-slate-700 t
 <div class="text-xs text-indigo
 <div class="font-bold text-slate

```

```
<h2 class="text-2xl font-bold b
(Меметика)</h2>
```

```
<!-- Прямое -->
<div class="bg-slate-800/80 p-6
 <h3 class="text-xl text-blue
 <p class="text-sm text-slate
 <ul class="list-disc pl-5 sp
 <span class="text-w
k$).
 <span class="text-w
 <span class="text-w
ово.

</div>
```

```
<!-- От противного -->
<div class="bg-slate-800/80 p-6
 <h3 class="text-xl text-fuch
 <p class="text-sm text-slate
 <ul class="list-disc pl-5 sp
 <span class="text-w
i>
 <span class="text-w
т.д.).
 <span class="text-w
что так и задумано: "Ой, противоречие, теор

</div>
```

```
<!-- Единственность -->
<div class="bg-slate-800/80 p-6
 <h3 class="text-xl text-eme
 <p class="text-sm text-slate
 <ul class="list-disc pl-5 sp
 <span class="text-w
, r_2$).
 <span class="text-w
```

```
<div class="bg-slate-800 p-4">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-medium">
 со следующим. Если в конце 0 – пациент мертв.
 </p>
 </h4>
 </div>
```

```
<div class="bg-slate-800 p-4">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-medium">
 c. Пока получается 0 – корень жив. Как то.
 корня минус один.</p>
 </h4>
 </div>
```

```
<div class="bg-slate-800 p-4">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-medium">
 ициентов (они делятся на p), кроме старше
 зложить.</p>
 </h4>
 </div>
```

```
<div class="bg-slate-800 p-4">
 <h4 class="text-red-400">
 <p class="text-sm text-slate-400">
 <p class="text-xs font-medium">
 . Чаще всего после этого Фейс-контроль p .
 </p>
 </h4>
 </div>
```

```
<h2 class="text-2xl font-bold bg-white p-4">
 (Правила экстрасенса)</h2>
 <p class="text-sm text-slate-400">
 тношения), а ты их не помнишь. Юзай базовые
 нейность) для делимости.</p>
```



```


 </div>
 </section>
</div>
```

```
<div id="questions-container">
<!-- ===== ВОПРОС 1 =====
```

---

## РАЗДЕЛ: УТИЛИТЫ

---

---

ФАЙЛ 87/111: src/utils/cn.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 7 | РАЗМЕР: 169 В

---

```
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
 return twMerge(clsx(inputs));
}
```

---

ФАЙЛ 88/111: src/utils/exportHtml.ts

КАТЕГОРИЯ: Утилиты | СТРОК: 341 | РАЗМЕР: 15.5 KB

---

```
import type { Chapter } from "../types";
import { GLOSSARY_BY_ID } from "../data/glossary";
import { annotateTerms } from "../hooks/useTermHints";
import { annotateTicketLinks } from "../ticketLinks";
```

```

.export-head { border-bottom: 1px solid #1e293b; padding-bottom: 1rem; }
.export-note { background: #0f172a; border: 1px solid #0f172a; border-radius: .5rem; padding: .85rem 1rem; font-size: .95rem; }
.export-note a { color: #a5b4fc; }
.export-gloss { margin-top: 3rem; border-top: 1px solid #0f172a; padding-top: 1rem; }
.export-gloss h2 { color: #fff; font-size: 1.25rem; font-weight: bold; }
.export-term { background: #0f172a; border: 1px solid #0f172a; padding: .5rem; }
.export-term h3 { color: #c7d2fe; font-size: .95rem; font-weight: bold; }
.export-term p { margin: 0 0 .3rem; font-size: .85rem; }
.export-term .formal { color: #94a3b8; font-size: .8rem; }
.export-term .cx { color: #6ee7b7; font-size: .78rem; font-weight: bold; }
.export-foot { margin-top: 3rem; border-top: 1px solid #0f172a; padding-top: 1rem; }
body { font-size: 15px; line-height: 1.6; }
.export-shell p, .export-shell li { line-height: 1.65; }
.export-shell pre { font-size: 12px; line-height: 1.5; }
.export-figs { margin: 0 0 1.6rem; }
.export-figs-title { color: #e2e8f0; font-size: .95rem; }
.export-fig { margin: 0 0 .9rem; }
.export-fig img { width: 100%; border: 1px solid #1e293b; }
.export-fig figcaption { color: #94a3b8; font-size: .75rem; }
@media print {
 body { background: #fff; color: #0f172a; }
 .export-note, details { break-inside: avoid; }
 details { display: block; }
 details > div, details > p { display: block !important; }
 pre { white-space: pre-wrap; }
}
`
;

```

```

const escapeHtml = (s: string) =>
 s.replace(/&/g, "&").replace(/</g, "<").replace(/>/g, ">");

/**
 * Размечает термины и превращает их в якорные ссылки на
 * Возвращает готовый HTML главы и список найденных терминов
 */
function prepareBody(html: string, opts: { selfId?: string;
 body: string;

```

```

 el.replaceWith(link);
 });

 // <details> в файле лучше раскрыть: иначе код и разб
 host.querySelectorAll("details").forEach((d) => d.set

 return { body: host.innerHTML, termIds: ids };
}

function glossarySection(termIds: string[], book = false) {
 if (!termIds.length) return "";
 const items = termIds
 .map((id) => {
 const e = GLOSSARY_BY_ID.get(id);
 if (!e) return "";
 return `<div class="export-term" id="term-${escape
 <h3>${escapeHtml(e.title)}</h3>
 <p>${escapeHtml(e.short)}</p>
 ${e.formal ? `<p class="formal">${escapeHtml(e.formal
 ${e.complexity ? `<p class="cx">Сложность: ${escapeHt
 </div>`;
 })
 .join("\n");

 return `<section class="export-gloss">
 <h2>${book ? "Словарик терминов пособия" : "Словарик
 <p style="font-size:.8rem;color:#94a3b8;margin:0 0 1r
 ны сюда.</p>
 ${items}
 </section>`;
}

function wrap(opts: { title: string; subtitle: string;
 const stamp = new Date().toLocaleString("ru-RU");
 return `<!doctype html>
 <html lang="ru">
 <head>
 <meta charset="utf-8">

```

```
const imgs = Array.from(doc.querySelectorAll("img[src
await Promise.all(
 imgs.map(async (img) => {
 const src = img.getAttribute("src") ?? "";
 if (!src || src.startsWith("data:")) return;
 try {
 const res = await fetch(src);
 if (!res.ok) return;
 const blob = await res.blob();
 if (blob.size > 4_000_000) return;
 const data = await new Promise<string>((resolve
 const fr = new FileReader();
 fr.onload = () => resolve(String(fr.result));
 fr.onerror = () => reject(fr.error);
 fr.readAsDataURL(blob);
));
 img.setAttribute("src", data);
 } catch {
 /* картинка недоступна – глава останется со ссы
 }
 })
);
return doc.body.innerHTML;
}
```

/\*\*\*

```
* Кадры визуализаций: книга встраивает снимки демонстр.
* файл показывал, что крутилось в приложении. Снимки д
* (capture-viz-shots.mjs) и публикует в /export/viz-sh
* без них (офлайн-сборка, локальный дев) книга остаётся
* нормально и молча пропускается.
*/
```

```

async function vizFigures(chapterId: string): Promise<string> {
 try {
 const base = import.meta.env.BASE_URL || "/";
 const manifestRes = await fetch(`${base}export/viz-figures/${chapterId}`);
 if (!manifestRes.ok) return "";
 const manifest = (await manifestRes.json()) as { figures: string[] };
 return manifest.figures.join("\n");
 } catch {
 return "";
 }
}

```

## Универсальное пособие • полный исходный код

```

/** Имя файла: «04-splay-derevo.html» – без кириллицы и
function safeName(title: string, id: string): string {
 const num = title.match(/^\\s*(\\d+)/)?.[1];
 const base = id.replace(/[^a-z0-9-]+/gi, "-").toLowerCase();
 return `${num ? String(num).padStart(2, "0") + "-": ""}${base}`;
}
```

```
/** Собрать автономный HTML одной главы. */
export async function buildChapterHtml(chapter: Chapter) {
 const css = await collectCss();
 const { body: rawBody, termIds } = prepareBody(chapter);
 const body = (await vizFigures(chapter.id)) + (await vizText(chapter.id));
 return wrap({
 title: chapter.title,
 subtitle: chapter.category || "Универсальное пособие",
 css,
 body,
 gloss: glossarySection(termIds),
 origin: window.location.origin,
 });
}
```

```
/** Выгрузить одну главу. */
export async function downloadChapterHtml(chapter: Chapter) {
 triggerDownload(await buildChapterHtml(chapter), safeName(chapter.title, chapter.id));
}
```

```
/** Собрать книгу: всё пособие одним файлом со сквозным индексом.
export async function buildBookHtml(chapters: Chapter[]) {
 const css = await collectCss();
 const allTerms: string[] = [];
 const bodies: string[] = [];

 const toc = chapters
 .map((c) => `<li style="margin:.2rem 0"><a href="#${c.id}">${c.title}</a>`)
 .join("\\n");

 for (const c of chapters) {
```

```

import { useEffect } from "react";
import { chapterTopics } from "../data/content";

/**
 * Перекрёстные ссылки между билетами.
 *
 * В тексте страниц десятки упоминаний вида «...берите Фор
 * или «сердце Краскала (билет 18)». Раньше это был про
 * номер, но попасть на него не мог — ссылка была «битой
 * в никуда. Здесь такие упоминания превращаются в настоя
 *
 * • в приложении — `

```

```

 const closing = part.startsWith("</");
 const tag = part.slice(closing ? 2 : 1).split(/
 if (tag === "pre" || tag === "code" || tag ===
 inCode = Math.max(0, inCode + (closing ? -1 :
 } else if (tag === "a") {
 inLink = Math.max(0, inLink + (closing ? -1 :
 }
 return part;
}
if (inCode > 0 || inLink > 0) return part;
return part.replace(TICKET_RE, (whole, a: string,
 const from = Number(a);
 const to = b ? Number(b) : from;
 const id = chapterForRange(from, to);
 if (!id || id === opts.selfId) return whole;
 return `<a class="ticket-link" data-goto="${id}
 });
});
.join("");
}

/**
 * DOM-версия — для интерактивной страницы. Проходит то
 * (как `annotateTerms`), поэтому атрибуты и разметка не
 * Возвращает число созданных ссылок — его проверяют DOM
 */
export function annotateTicketLinks(
 root: HTMLElement,
 opts: { selfId?: string; lite?: boolean } = {}
): number {
 if (typeof document === "undefined") return 0;
 const walker = document.createTreeWalker(root, SHOW_TEXT,
 acceptNode(node) {
 const parent = (node as Text).parentElement;
 if (!parent) return FILTER_REJECT;
 if (parent.closest(SKIP_SELECTOR)) return FILTER_REJECT;
 const text = node.nodeValue ?? "";
 if (/билет/i.test(text)) return FILTER_REJECT;

```

```
}

/** Размечает ссылки при смене главы и подстраховывается
export function useTicketLinks(
 ref: React.RefObject<HTMLElement | null>,
 deps: unknown[],
 opts: { selfId?: string; lite?: boolean } = {}
) {
 useEffect(() => {
 const el = ref.current;
 if (el) annotateTicketLinks(el, opts);
 // eslint-disable-next-line react-hooks/exhaustive-
 }, deps);
}
```

---

## РАЗДЕЛ: СКРИПТЫ СБОРКИ И ЭКСПОРТА

---

---

ФАЙЛ 90/111: scripts/audit-coverage.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 83 | РАЗМ

---

```
/**
 * Аудит покрытия глав пособия: у каких тем нет «объясн
 * (блок «Аналогия») и/или 2D-визуализации (интерактивн
 *
 * node scripts/audit-coverage.mjs # отчёт
 * node scripts/audit-coverage.mjs --md # markd
 */
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { build } from "esbuild";

const ROOT = path.resolve(path.dirname(fileURLToPath(im
```



```

const gapBoth = rows.filter((r) => !r.hasAnalogy && !ha
const gapAnalogy = rows.filter((r) => !r.hasAnalogy && !
const gapViz = rows.filter((r) => r.hasAnalogy && !has2
const orphanViz = [...vizIds].filter((id) => !chapters.

const nums = rows.map((r) => r.num).filter(Boolean);
const missingNums = Array.from({ length: 24 }, (_, i) =>

const flag = (b) => (b ? " " : "-");

if (process.argv.includes("--md")) {
 console.log("| # | Глава | Аналогия | 2D-виз. | SVG |
 console.log("| --- | --- | --- | --- | --- | --- |");
 for (const r of rows) {
 console.log(
 `| ${r.num || "-"} | ${r.title} | ${r.hasAnalogy
 ge)} |`
);
 }
} else {
 console.log(`Глав в пособии: ${rows.length}\n`);
 const dump = (title, list) => {
 console.log(`${title} (${list.length}):`);
 list.forEach((r) => console.log(` * ${r.title} [$
 console.log();
 };
 dump("НЕТ НИ АНАЛОГИИ, НИ 2D", gapBoth);
 dump("НЕТ АНАЛОГИИ (2D есть)", gapAnalogy);
 dump("НЕТ 2D (аналогия есть)", gapViz);
 console.log(`Пропущенные номера билетов 1-24: ${missi
 console.log(`Визуализаторы без главы: ${orphanViz.join
}

```

```

 .replace(/<[^>]+>/g, " ");

 const perTerm = new Map();
 const perSurface = new Map();
 let highlights = 0;
 const ids = new Set();

 for (const m of text.matchAll(re)) {
 const surface = m[1].toLowerCase();
 const id = map.get(surface);
 if (!id) continue;
 const ut = perTerm.get(id) ?? 0;
 const us = perSurface.get(surface) ?? 0;
 if (ut >= MAX_PER_TERM || us >= MAX_PER_SURFACE) continue;
 perTerm.set(id, ut + 1);
 perSurface.set(surface, us + 1);
 highlights += 1;
 ids.add(id);
 used.add(id);
 }

 if (ids.size === 0) bad.push(c.title);
 console.log(
 String(ids.size).padStart(3), "|", String(highlights).padStart(3),
 c.title.slice(0, 46).padEnd(47), [...ids].slice(0, 47).join(", ")
);
 }
 console.log("\nГлав без единой подсказки:", bad.length,);
 console.log("Терминов в глоссарии:", GLOSSARY.length,);
 console.log("Не встретились:", GLOSSARY.filter((g) => !bad.includes(g)).length,);

```

```

const BOTTOM = PAGE.height - PAGE.margin - 16;
```

```
/* ----- шрифты -----
```

```
const CANDIDATES = {
 regular: [
 "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf",
 "/usr/share/fonts/dejavu/DejaVuSans.ttf",
 "/usr/local/share/fonts/DejaVuSans.ttf",
 "/System/Library/Fonts/Supplemental/Arial Unicode.ttf",
 "C:/Windows/Fonts/arial.ttf",
],
 bold: [
 "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf",
 "/usr/share/fonts/dejavu/DejaVuSans-Bold.ttf",
 "/usr/local/share/fonts/DejaVuSans-Bold.ttf",
 "C:/Windows/Fonts/arialbd.ttf",
],
 oblique: [
 "/usr/share/fonts/truetype/dejavu/DejaVuSans-Oblique.ttf",
 "/usr/share/fonts/dejavu/DejaVuSans-Oblique.ttf",
],
 mono: [
 "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf",
 "/usr/share/fonts/dejavu/DejaVuSansMono.ttf",
 "/usr/local/share/fonts/DejaVuSansMono.ttf",
 "C:/Windows/Fonts/consola.ttf",
],
};
```

```
const pick = (list) => list.find((f) => fs.existsSync(f))
```

```
/**
```

```
 * Убираем символы, которых нет в DejaVu (эмодзи и пиктограммы)
 * на их месте появляется «пустой» глиф. Стрелки, математические
 * геометрические фигуры DejaVu знает – их оставляем.
 */
```

```
const UNSUPPORTED = /[\u{1F000}-\u{1FAFF}\u{2700}-\u{270F}]/
```

```

 loader: { ".ts": "ts", ".tsx": "tsx", ".jpg": "data:
 ss": "empty" },
 });
 return import("data:text/javascript;base64," + Buffer
}

```

/\* ————— снимки визуализаций ————— \*/

```

function shotsFor(chapterId) {
 const dirs = [SHOTS_DIR, path.join(ROOT, "public", "e
 for (const dir of dirs) {
 if (fs.existsSync(dir)) {
 const files = fs
 .readdirSync(dir)
 .filter((f) => /\.(png|jpe?g)$/i.test(f))
 .filter((f) => f === `${chapterId}.png` || f.st
 .sort();
 if (files.length > 0) {
 return files.map((f) => ({
 file: path.join(dir, f),
 name: f.replace(/\.(png|jpe?g)$/i, "").split(
 }));
 }
 }
 }
 return [];
}

```

/\* ————— отрисовка PDF ————— \*/

```

class Renderer {
 constructor(doc, fonts) {
 this.doc = doc;
 this.fonts = fonts;
 this.footerTitle = "";
 this.pages = 0;
 this.cy = TOP;
 }
}

```

```

 if (footerTitle !== undefined) this.footerTitle = footerTitle;
 this.doc.addPage({ size: [PAGE.width, PAGE.height],
 this.pages += 1;
 this.cy = TOP;
 }

 /** Сколько места нужно, чтобы элемент не оторвался от
 ensure(h, extra = 0) {
 if (this.cy + h + extra > BOTTOM) this.newPage();
 }

 height(text, { font, size, width, lineGap = 0 }) {
 return this.doc.font(font).fontSize(size).heightOfText(text, {
 }

 /**
 * Режет текст на фрагменты не выше max пунктов: сначала по
 * предложений, слишком длинное предложение – по словам.
 *
 * Без этого pdftk сам переносит остаток на новую строку, что
 * создаётся в обход рендерера – без нижнего колонтитула.
 * из-за чего в файле оказывалось на одну страницу больше.
 */
 splitToFit(text, opts, max) {
 const measure = (t) => this.height(t, opts);
 const source = String(text);
 if (measure(source) <= max) return [source];
 const chunks = [];
 let current = "";
 const flush = () => {
 if (current.trim()) chunks.push(current.trim());
 current = "";
 };
 for (const sentence of source.split(/(?<=[.!?…;:])\s/)) {
 const withSentence = current ? `${current} ${sentence}` : sentence;
 if (measure(withSentence) <= max) {
 current = withSentence;
 continue;
 }
 }
 }

```

```

 if (!text) return;
 const h = this.height(text, { font: this.fonts.bold });
 this.ensure(h + 6, 20);
 this.cy += 6;
 this.doc.font(this.fonts.bold).fontSize(size).fillColor(color);
 this.cy = Math.max(this.cy, this.doc.y) + 3;
 }

 h3(rawText, { size = 11.2, color = "#334155" } = {}) {
 const text = plain(rawText);
 if (!text) return;
 const h = this.height(text, { font: this.fonts.bold });
 this.ensure(h + 4, 16);
 this.cy += 4;
 this.doc.font(this.fonts.bold).fontSize(size).fillColor(color);
 this.cy = Math.max(this.cy, this.doc.y) + 2;
 }

 /** Абзац: режим по высоте страницы сами – автоперенос
 para(rawText, { size = 10.0, indent = 0, color = "#1f2f4f" } = {}) {
 const text = plain(rawText);
 if (!text) return;
 const font = mono ? this.fonts.mono : italic && this.fonts.sans;
 const width = CONTENT_W - indent;
 const opts = { font, size, width, lineGap };
 const chunks = this.splitToFit(text, opts, BOTTOM);
 chunks.forEach((chunk, index) => {
 const last = index === chunks.length - 1;
 const h = this.height(chunk, opts);
 this.ensure(h + (last ? gapAfter : 0), 6);
 const y0 = this.cy;
 this.doc.font(font).fontSize(size).fillColor(color);
 this.cy = Math.max(y0 + h, this.doc.y) + (last ? gapAfter : 0);
 if (this.cy > BOTTOM) this.newPage();
 });
 }

 bullet(rawText, { ordinal = null, depth = 0, size = 9

```

```

 const candidate = chunk.concat(lines[i]).join("\n");
 const ch = this.height(candidate, { font, size,
 if (chunk.length && ch > maxH) break;
 chunk.push(lines[i]);
 h = ch;
 i += 1;
 }
 const body = chunk.join("\n");
 const boxH = h + 14;
 this.ensure(boxH);
 const y0 = this.cy;
 this.doc.roundedRect(PAGE.margin, y0, CONTENT_W, boxH);
 this.doc.font(font).fontSize(size).fillColor("#1f77b4");
 this.cy = y0 + boxH + (i < lines.length ? 2 : 7);
 if (i < lines.length) this.newPage();
}
}

/** Таблица: строки режутся по страницам, заголовок по ширине
table(rawRows, { size = 8.5 } = {}) {
 const rows = rawRows.map((r) => r.map((c) => plain(c)));
 if (!rows.length) return;
 const cols = Math.max(...rows.map((r) => r.length));
 const widths = new Array(cols).fill(CONTENT_W / cols);
 const pad = 5;
 const header = rows[0];

 const rowHeight = (cells) => {
 let max = 0;
 for (let i = 0; i < cols; i++) {
 const h = this.height(cells[i] ?? "", { font: this.font, size });
 max = Math.max(max, h);
 }
 return max + pad * 2;
 };

 const drawRow = (cells, y, isHeader, zebra) => {
 const h = rowHeight(cells);

```

```

const chunks = this.splitToFit(text, opts, BOTTOM -
chunks.forEach((chunk) => {
 const h = this.height(chunk, opts);
 this.ensure(h + 10);
 const y0 = this.cy;
 this.doc.rect(PAGE.margin, y0, 3, h + 4).fill("#a
 this.doc.font(font).fontSize(9.5).fillColor("#374
 this.cy = y0 + h + 10;
 if (this.cy > BOTTOM) this.newPage();
}));
}

image(file, { maxWidth = CONTENT_W, maxHeight = 320,
 if (!fs.existsSync(file)) return false;
 let img;
 try {
 img = this.doc.openImage(file);
 } catch {
 return false;
 }
 const capH = caption ? 14 : 0;
 const scale = Math.min(maxWidth / img.width, (maxHe
 const w = img.width * scale;
 const h = img.height * scale;
 this.ensure(h + capH + 10, 20);
 const y0 = this.cy;
 const x0 = PAGE.margin + (CONTENT_W - w) / 2;
 this.doc.image(file, x0, y0, { width: w, height: h
 this.doc.rect(x0, y0, w, h).lineWidth(0.6).strokeCo
 this.cy = y0 + h + 4;
 if (caption) {
 this.doc.font(this.fonts.regular).fontSize(8).fil
 enter" });
 this.cy = Math.max(this.cy, this.doc.y) + 6;
 }
 return true;
}

```



```
async function main() {
 const fonts = {
 regular: pick(CANDIDATES.regular),
 bold: pick(CANDIDATES.bold),
 oblique: pick(CANDIDATES.oblique),
 mono: pick(CANDIDATES.mono),
 };
 if (!fonts.regular) throw new Error("Не найден TTF-шрифт");
 fonts.bold = fonts.bold ?? fonts.regular;
 fonts.mono = fonts.mono ?? fonts.regular;

 const { chapters, chapterTopics, PAGE_SYNC, VIZ_REGISTRY } = require('data');
 ensureDir(OUT_DIR);

 const doc = new PDFDocument({
 size: [PAGE.width, PAGE.height],
 margin: PAGE.margin,
 autoFirstPage: false,
 info: {
 Title: "Универсальное пособие – алгоритмы и структуры",
 Author: "algv0 · автоэкспорт",
 Subject: "Текст всех тем + визуализации, без кода",
 CreationDate: new Date(),
 },
 });
 const stream = fs.createWriteStream(GUIDE_PDF_PATH);
 doc.pipe(stream);

 const R = new Renderer(doc, fonts);
 const today = new Date().toISOString().slice(0, 10);
 const totalTopics = new Set(Object.values(chapterTopics));
 const vizCount = chapters.filter((c) => VIZ_REGISTRY[c]);

 /* — обложка — */
 R.firstPage("Обложка");
 R.cy = TOP + 90;
 R.h1("Универсальное пособие", { size: 26, rule: false
```

```

/* — темы — */
let shotsTotal = 0;
for (const chapter of chapters) {
 const topics = chapterTopics[chapter.id] ?? [];
 const shortTitle = chapter.title.replace(/^\\d+\\.--/);
 R.newPage(shortTitle);

 if (topics.length) {
 R.para(`Билет${topics.length > 1 ? "ы" : ""} ${topics[0]
 : 2 }`);
 }
 R.h1(chapter.title.replace(/^\\d+\\.--/?\\s*/, ""), {

 const chapterTitle = chapter.title.replace(/^\\d+\\.--/);
 for (const b of htmlToBlocks(chapter.content)) {
 if (b.type === "p" && /^Билет\\s/i.test(b.text)) c
 if (b.type === "heading" && b.level <= 2 && b.tex

 if (b.type === "heading") {
 if (b.level <= 2) R.h2(b.text);
 else if (b.level === 3) R.h3(b.text);
 else R.para(b.text, { size: 10.2, color: "#334155" });
 } else if (b.type === "summary") {
 R.para("► " + b.text, { size: 9.8, color: "#373737" });
 } else if (b.type === "li") {
 R.bullet(b.text, { ordinal: b.ordinal, depth: b
 } else if (b.type === "code") {
 R.code(b.text);
 } else if (b.type === "table") {
 R.table(b.rows);
 } else if (b.type === "quote") {
 R.quote(b.text);
 } else if (b.type === "caption") {
 R.para(b.text, { size: 8.3, color: "#6b7280", i
 } else {
 R.para(b.text);
 }
 }
}

```

```

 sync.variables.forEach((v) => lines.push(`• ${v}`));
 lines.push("• Референсный код демонстрации в этом файле");
 if (lines.length) R.box(title, lines);
 }

 if (!shots.length) {
 R.para(
 "Снимки демонстрации добавляются автоматически в этот файл.

 без браузера здесь остаётся текстовое описание",
 { size: 8.5, color: "#6b7280", italic: true }
);
 }
}

/* блиц: те же вопросы, что в приложении, но сразу в файле
const quiz = (quizzes && quizzes[chapter.id]) || [];
if (quiz.length) {
 R.h2("Блиц: вопросы по теме");
 quiz.forEach((q, qi) => {
 R.para(`${qi + 1}. ${q.question}`, { size: 9.6, color: "#4b5962" });
 q.options.forEach((option, oi) => {
 R.bullet(oi === q.correctIndex ? `${option} — верный ответ` : `${option}`);
 });
 R.para(q.explanation, { size: 8.8, color: "#4b5962" });
 });
}
}

R.stampFooter();
doc.end();
await new Promise((resolve, reject) => {
 stream.on("finish", resolve);
 stream.on("error", reject);
});

const size = fs.statSync(GUIDE_PDF_PATH).size;
console.log("[guide] PDF пособия собран");
console.log(`Содержит ${chapters.length} глав, ${chapters.length} глав и ${shots.length} снимков`);

```

```
const A4 = { width: 595.28, height: 841.89 };

async function main() {
 if (!fs.existsSync(PAGES_DIR)) {
 throw new Error(`Нет ${path.relative(ROOT, PAGES_DIR)}`);
 }

 const pages = fs
 .readdirSync(PAGES_DIR)
 .filter((f) => f.endsWith(".png"))
 .sort();

 if (pages.length === 0) throw new Error("Не найдено ни одного файла");

 ensureDir(OUT_DIR);

 const doc = new PDFDocument({
 size: [A4.width, A4.height],
 margin: 0,
 autoFirstPage: false,
 info: {
 Title: "Универсальное пособие – полный исходный код",
 Author: "CI: rebuild-pdf",
 Subject: "Экспорт всего кода проекта (код -> txt)",
 CreationDate: new Date(),
 },
 });

 const stream = fs.createWriteStream(PDF_PATH);
 doc.pipe(stream);

 for (const file of pages) {
 doc.addPage({ size: [A4.width, A4.height], margin: 0 });
 doc.image(path.join(PAGES_DIR, file), 0, 0, { width: A4.width });
 }

 doc.end();
 await new Promise((resolve, reject) => {
```

```
import puppeteer from "puppeteer";
import { ROOT } from "../common.mjs";

const OUT_DIR = path.join(ROOT, "tmp", "viz-shots");
const BASE_URL = process.env.SHOTS_BASE_URL || "http://";

/** Вкладки, которые стоит снять отдельно: глава → подпункт */
const TABS = {
 "sparse-table": ["1D", "2D: построение", "2D: запрос"],
 mst: ["Краскал", "Прим", "Борувка"],
 treap: ["Собрать", "Split", "Merge"],
 "splay-tree": ["Обход", "Повороты", "Прогулка"],
};

const slug = (s) =>
 s
 .toLowerCase()
 .replace(/[^a-zA-яё0-9]+/gi, "-")
 .replace(/^-+|-+$/g, "")
 .slice(0, 24);

async function vizIds() {
 const bundle = await build({
 stdin: {
 contents: `export { activeVizIds } from "../../src/viz";`,
 resolveDir: path.join(ROOT, "scripts", "export"),
 loader: "ts",
 },
 bundle: true,
 write: false,
 format: "esm",
 platform: "node",
 logLevel: "warning",
 });
 const mod = await import("data:text/javascript;base64:" + btoa(bundle.code));
 return mod.activeVizIds;
}
```

```

 await new Promise((r) => setTimeout(r, 1200));
 const el = await page.$("#chapter-viz");
 await el.screenshot({ path: path.join(OUT_DIR,
 done += 1;
 } catch (error) {
 console.log(`[export:shots] вкладка «${tab}»
 }
}
console.log(`[export:shots] ok ${id}`);
} catch (error) {
 failed += 1;
 console.log(`[export:shots] FAIL ${id}: ${String(er
}
}

const manifest = fs
 .readdirSync(OUT_DIR)
 .filter((f) => /\.(png|jpe?g)$/i.test(f))
 .sort()
 .map((f) => ({ file: f, name: f.replace(/\.(png|jpe?g)
fs.writeFileSync(path.join(OUT_DIR, "manifest.json"), J

await browser.close();
console.log(`[export:shots] снимков: ${done}, пропусков
// Пропущенные страницы не повод ронять пайплайн: PDF д
// текстовое описание демонстрации.
process.exit(0);

```

ФАЙЛ 95/111: scripts/export/collect-code.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 196 | РАЗМ

/\*\*

\* ШАГ 1: КОД -> TXT

\*

\* Собирает ВСЕ исходный код репозитория в один текст

```

 "Конфигурация проекта",
 "Ядро приложения",
 "Контент и данные пособия",
 "Билеты (учебный контент)",
 "Интерактивные компоненты и симуляторы",
 "Визуализаторы (вложенные)",
 "HTML-разметка (layout)",
 "Утилиты",
 "Скрипты сборки и экспорта",
 "CI / автоматизация",
 "Прочее",
];

function listFiles() {
 let files;
 try {
 files = execFileSync("git", ["ls-files", "--cached"], {
 cwd: ROOT,
 encoding: "utf8",
 })
 .split("\n")
 .filter(Boolean);
 } catch {
 files = [];
 const walk = (dir) => {
 for (const e of fs.readdirSync(dir, { withFileTypes: true })) {
 const abs = path.join(dir, e.name);
 const rel = path.relative(ROOT, abs).split(path.sep);
 if (EXCLUDE_DIRS.some((d) => rel === d || rel.startsWith(d + path.sep))) continue;
 if (e.isDirectory()) walk(abs);
 else files.push(rel);
 }
 };
 walk(ROOT);
 }

 return files
 .filter((rel) => !EXCLUDE_DIRS.some((d) => rel.startsWith(d + path.sep)))

```

```

const push = (s = "") => out.push(s);

const stamp = new Date().toLocaleString("ru-RU", { timeZoned: true });
const totalBytes = files.reduce((s, f) => s + fs.statSync(f).size, 0);

push(HR);
push(" УНИВЕРСАЛЬНОЕ ПОСОБИЕ ПО АЛГОРИТМАМ И СТРУКТУРАМ ДАННЫХ");
push(" ПОЛНЫЙ ИСХОДНЫЙ КОД ПРОЕКТА (ВСЕ ФАЙЛЫ И КОММЕНТАРИИ)");
push(HR);
push();
push(`Дата генерации:                ${stamp}`);
push(`Файлов исходного кода:         ${files.length}`);
push(`Суммарный объём кода:           ${humanSize(totalBytes)}`);
push(`Глав/билетов в пособии:         ${chapters.length}`);
push();

push(HR);
push(" ОГЛАВЛЕНИЕ: ГЛАВЫ И БИЛЕТЫ");
push(HR);
chapters.forEach((c, i) => {
 push(`  ${String(i + 1).padStart(3, " ")}. ${c.title}`);
});
push();

push(HR);
push(" ОГЛАВЛЕНИЕ: ФАЙЛЫ ИСХОДНОГО КОДА");
push(HR);
let current = null;
files.forEach((rel, i) => {
 const cat = categoryOf(rel);
 if (cat !== current) {
 current = cat;
 push();
 push(`  # ${cat}`);
 }
 push(`  ${String(i + 1).padStart(3, " ")}. ${rel}`);
});
push();

```



```
main();
```

ФАЙЛ 96/111: scripts/export/common.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 66 | РАЗМ

```
/**
 * Общие настройки и утилиты пайплайна экспорта.
 *
 * код -> full_code_all_pages.txt (collect-code.m
 * txt -> page-XXXX.png (render-pages.m
 * png -> full_code_all_pages.pdf (build-pdf.mjs)
 */
import fs from "node:fs";
import path from "node:path";
import { fileURLToPath } from "node:url";
import { execFileSync } from "node:child_process";

export const ROOT = path.resolve(path.dirname(fileURLTo

/** Куда кладём финальные артефакты (эта папка отдаётся
export const OUT_DIR = path.join(ROOT, "public", "export
/** Промежуточные PNG-страницы (в .gitignore, в CI уезжа
export const PAGES_DIR = path.join(ROOT, "tmp", "export

export const TXT_PATH = path.join(OUT_DIR, "full_code_a
export const PDF_PATH = path.join(OUT_DIR, "full_code_a

/** Параметры вёрстки страницы (подобраны под A4 @150dp
export const PAGE = {
 /** DPI раstra. Можно переопределить: EXPORT_DENSITY=
 density: Number(process.env.EXPORT_DENSITY ?? 150),
 /** 1-bit ч/б страницы весят в 2-4 раза меньше. EXPORT
 monochrome: process.env.EXPORT_GRAYSCALE !== "1",
 size: "A4",
 font: "DejaVu-Sans-Mono",
```

```
/**
 * HTML страницы пособия → плоский список блоков для переноса
 *
 * Задача: вытащить из Tailwind-разметки САМ ТЕКСТ (как в ридере), сохранив структуру и ПОРЯДОК слов: заголовки,
 * таблицы, код, спойлеры <details>. Служебная разметка (svg, кнопки) отбрасывается, содержимое спойлеров — нет.
 *
 * Никаких внешних зависимостей: собственный мини-парсер.
 * куски хранятся отдельными узлами #text, поэтому инлайны (b, code, span) не «уезжают» в конец абзаца.
 */

/** Содержимое этих тегов не нужно в текстовой версии. */
const SKIP = new Set([
 "svg", "script", "style", "button", "iframe", "video",
 "path", "circle", "rect", "line", "polyline", "polygon",
 "defs", "marker", "tspan", "ellipse", "use", "clipPath"
]);

/** Пустые элементы (без закрывающего тега). */
const VOID = new Set(["br", "hr", "img", "input", "meta"]);

/** Блочные теги: их содержимое собирается в отдельный блок. */
const BLOCK = new Set([
 "h1", "h2", "h3", "h4", "h5", "h6", "p", "li", "pre",
 "summary", "figcaption", "table", "tr", "td", "th", "thead",
 "article", "ul", "ol", "details", "figure", "header",
]);

const HEADING = new Set(["h1", "h2", "h3", "h4", "h5", "h6"]);

/**
 * Разбор HTML в дерево: { tag, attrs, children }, текст
```

```

 }
 return root;
}

const decodeEntities = (s) =>
 s
 .replace(/</g, "<")
 .replace(/>/g, ">")
 .replace(/"/g, '"')
 .replace(/'/g, "'")
 .replace(/ /g, "\u00a0")
 .replace(/—/g, "—")
 .replace(/–/g, "—")
 .replace(/«/g, "«")
 .replace(/»/g, "»")
 .replace(/…/g, "...")
 .replace(/×/g, "×")
 .replace(/≤/g, "≤")
 .replace(/≥/g, "≥")
 .replace(/≠/g, "≠")
 .replace(/∞/g, "∞")
 .replace(/→/g, "→")
 .replace(/←/g, "←")
 .replace(/&/g, "&")
 .replace(/&#(\d+)/g, (_, d) => String.fromCharCode

const clean = (s) => decodeEntities(s).replace(/\s+/g,

const isEl = (n) => n.tag !== "#text";
const isBlockEl = (n) => isEl(n) && BLOCK.has(n.tag) &&

/** Есть ли у узла блочные потомки (тогда его текст «ра
const hasBlockChild = (node) => node.children.some(isBl

/** Текст инлайнового поддеревя в порядке документа. */
function inlineText(node) {
 let out = "";
 for (const c of node.children) {

```

```

 return;
 }
 n.children.forEach(visit);
};
node.children.forEach(visit);
return rows;
}

/**
 * Обход дерева: собираем блоки в порядке документа.
 * @returns {Array<{type:string,text?:string,rows?:string}>}
 */
export function htmlToBlocks(html) {
 const root = parseHtml(html);
 const out = [];

 const push = (block) => {
 const prev = out[out.length - 1];
 if (prev && prev.type === block.type && prev.text === block.text) {
 out.push(block);
 }
 };

 const emitListItem = (li, ordinal, depth) => {
 const text = collectText(li);
 if (text) push({ type: "li", text, ordinal: ordinal });
 // вложенные списки внутри li
 li.children.forEach((c) => {
 if (c.tag === "ul" || c.tag === "ol") emitList(c, depth + 1);
 });
 };

 const emitList = (list, depth) => {
 let i = 0;
 for (const li of list.children) {
 if (li.tag !== "li") continue;
 i += 1;
 emitListItem(li, list.tag === "ol" ? i : null, depth);
 }
 };
}

```

```

 }
 flush();
 walk(c, listDepth);
 }
 flush();
 return;
}

// листовой блок: один блок по типу тега
const text = inlineText(node);
if (!text) return;
if (HEADING.has(node.tag)) push({ type: "heading",
else if (node.tag === "li") emitListItem(node, null
else if (node.tag === "summary") push({ type: "summ
else if (node.tag === "blockquote") push({ type: "q
else if (node.tag === "figcaption") push({ type: "c
else push({ type: "p", text });
};

root.children.forEach((c) => walk(c, 0));
return out;
}

/** То же самое, но простым текстом (для .txt-дампа и в
export function htmlToPlainText(html) {
 const lines = [];
 for (const b of htmlToBlocks(html)) {
 if (b.type === "heading") lines.push("", "#".repeat
 else if (b.type === "summary") lines.push("", "▶ "
 else if (b.type === "li") lines.push(" ".repeat(b
 else if (b.type === "code") lines.push("", ...b.tex
 else if (b.type === "table") {
 lines.push("");
 b.rows.forEach((r) => lines.push("| " + r.join("
 lines.push("");
 } else if (b.type === "quote") lines.push("> " + b.
 else if (b.type === "caption") lines.push(" " + b
 else lines.push(b.text, "");
 }
}

```

```

 const width = first ? PAGE.cols : PAGE.cols - indent;
 parts.push((first ? "" : indent) + rest.slice(0, width));
 rest = rest.slice(width);
 first = false;
 }
 return parts;
}

function paginate(txt) {
 const source = txt.replace(/\r\n/g, "\n").split("\n");
 const pages = [];
 for (let i = 0; i < source.length; i += PAGE.rows) {
 pages.push(source.slice(i, i + PAGE.rows));
 }
 return pages;
}

async function main() {
 if (!fs.existsSync(TXT_PATH)) {
 throw new Error(`Нет ${path.relative(ROOT, TXT_PATH)}
 }

 const im = imageMagickCmd();
 const pages = paginate(fs.readFileSync(TXT_PATH, "utf8"));

 fs.rmSync(PAGES_DIR, { recursive: true, force: true });
 ensureDir(PAGES_DIR);

 const total = pages.length;
 const jobs = pages.map((lines, idx) => {
 const num = String(idx + 1).padStart(4, "0");
 const header = `Универсальное пособие • полный исходный код
 Math.max(1, PAGE.cols - 44 - `стр. ${idx + 1} / ${total}`);
 const body = [header, "-".repeat(PAGE.cols), ...lines];
 const txtFile = path.join(PAGES_DIR, `page-${num}.txt`);
 const pngFile = path.join(PAGES_DIR, `page-${num}.png`);
 fs.writeFileSync(txtFile, body + "\n", "utf8");
 im.convert(txtFile, pngFile);
 });
}

```

```

 if (missing.length) throw new Error(`Не отрендерены с

 const bytes = jobs.reduce((s, j) => s + fs.statSync(j
 console.log("[2/3] txt -> png");
 console.log(`        страниц: ${total}`);
 console.log(` выход: ${path.relative(ROOT, PAG
}

main().catch((err) => {
 console.error("Ошибка рендера страниц:", err.message)
 process.exit(1);
});
```

-----  
ФАЙЛ 99/111: scripts/verify/browser-checks.tsx

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 1065 | РА

```

/**
 * DOM-харнес: настоящее приложение в jsdom + настоящий
 * Монтирует App, кликает кнопки, печатает в редакторе
 * визуализациям матрицы чужих размерностей и ловит пад
 */
import { act } from "react";
import { createRoot } from "react-dom/client";
import type { Root } from "react-dom/client";
import App from "../../src/App";
import { chapters, chapterTopics, SECTIONS, sectionOf }
import { quizzes } from "../../src/data/quizzes";
import { VIZ_REGISTRY } from "../../src/components/vizR
import { VizLiveStatus } from "../../src/components/Viz
import { PAGE_SYNC, buildInitTemplate } from "../../src
import { VizChapterContext, emitVizState, clearVizState
import { buildDebugRunner, type DebugResult } from "../../

type Py = { runPythonAsync: (src: string) => Promise<un
```

```

 .filter((b) => {
 const name = [b.getAttribute("aria-label"), b.get
 .filter(Boolean)
 .join(" ")
 .replace(/\s+/g, " ")
 .trim();
 return name.length === 0;
 })
 .map((b) => `<button class="${(b.className || "").s
 }

```

```

function findButton(match: string | RegExp): HTMLButton
 const list = Array.from(document.querySelectorAll("bu
 return list.find((b) => {
 const hay = [b.getAttribute("aria-label"), b.getAtt
 return typeof match === "string" ? hay.includes(mat
 }));
}

```

```

async function click(el: Element | undefined | null, lab
 if (!el) throw new Error(`не найдена ${label}`);
 await act(async () => {
 el.dispatchEvent(new window.MouseEvent("click", { b
 await sleep(20);
 });
}

```

```

/** Панель компилятора – все поиски внутри неё, чтобы н
function panel(): HTMLElement | null {
 return document.querySelector<HTMLElement>('aside[ari
}

```

```

function panelText(): string {
 return (panel()?.textContent ?? "").replace(/\s+/g, "
}

```

```

function textarea(): HTMLTextAreaElement | undefined {
 return panel()?.querySelector<HTMLTextAreaElement>("t

```



```
let root: Root | null = null;
let host: HTMLElement | null = null;

function freshHost(): HTMLElement {
 const div = document.createElement("div");
 document.body.appendChild(div);
 return div;
}

async function mountApp(topic: string) {
 if (root) {
 await act(async () => {
 root?.unmount();
 });
 root = null;
 }
 if (host) {
 host.remove();
 }
 host = freshHost();
 window.history.replaceState({}, "", `/?topic=${encodeURIComponent(topic)}`);
 root = createRoot(host);
 await act(async () => {
 root!.render(<App />);
 });
 await settle(80);
 return host;
}

/** Смонтировать одну визуализацию так, как это делает
 *
 */
async function mountViz(vizId: string) {
 if (root) {
 await act(async () => {
 root?.unmount();
 });
 root = null;
 }
 if (host) host.remove();
}
```

```

 }
 }
 node = walker.nextNode() as Text | null;
}
return [...new Set(found)];
}

async function test(group: string, name: string, fn: ()
 if (skipGroups.has(group[0])) return true;
 consoleErrors = [];
 const t0 = Date.now();
 let ok = true;
 let detail = "";
 try {
 const d = await fn();
 detail = d ?? "";
 } catch (error) {
 ok = false;
 detail = error instanceof Error ? `${error.message}
 }
 const ms = Date.now() - t0;
 const errs = consoleErrors.filter((e) => !/not wrapped/
 if (errs.length > 0) {
 ok = false;
 detail = `${detail}\nconsole.error (${errs.length})
 }
 results.push({ group, name, ok, ms, detail, consoleEr
 const mark = ok ? "PASS" : "FAIL";
 realConsoleError === console.error;
 process.stdout.write(`${mark} [${group}] ${name}${ok
 if (!ok) process.stdout.write(detail.split("\n").map(
 return ok;
}

/* -----
/* сценарии
/* -----

```

```

 if (!eye) throw new Error("нет кнопки-«глаза» (вс
const run = findButton(/Запустить код|автопроход/);
if (!run) throw new Error("нет кнопки запуска ►");
return `кнопок в панели: ${allButtons().length}`;
});

```

```

await test(group, `${chapter.id}: автозапуск инициа
await mountApp(chapter.id);
await click(findButton("Боковая панель Python")),
const done = await waitFor(() => RUN_DONE.some((m) =>
if (!done) throw new Error(`запуск не завершился`);
const msg = RUN_DONE.find((m) => bodyText().includes(m));
const sync = PAGE_SYNC[chapter.id];
if (!sync?.code) return `кода синхронизации нет: ${msg}`;
if (/Ошибка|Не удалось/.test(msg)) throw new Error(msg);
// после настоящего прогона визуализация обязана быть
const linked = await waitFor(() => /данные из комм/);
if (!linked) return `${msg}; визуализация осталась прежней`;
return `${msg} → визуализация переключилась на данные`;
});
}

```

```

/* — 2. Флойд: матрицы ЛЮБОЙ размерности из кода полн
const floydCases: { name: string; vars: Record<string, any> } = [
 { name: "5×5 (меньше демо-7×7)", vars: { n: 5, k: 1, i: 0, j: 0,
 }, [9, 6, 9, 9, 0]] }, expect: "5×5" },
 { name: "10×10 (больше демо – старый краш nodeNames)", vars: { n: 10, k: 1, i: 0, j: 0,
 from({ length: 10 }, (_, c) => (c === 0 ? 0 : (c - 1) * 10 + c)) }, expect: "10×10" },
 { name: "3×3 с бесконечностями", vars: { n: 3, k: 0, i: 0, j: 0,
 "3×3" }, expect: "3×3" },
 { name: "не квадратная 4×6", vars: { n: 4, k: 1, i: 0, j: 0,
 , 17, 18, 19, 0, 20]] }, expect: "Демо-схема графа" },
 { name: "словарь словарей d[0][1]", vars: { n: 4, k: 1, i: 0, j: 0,
 2: { 0: 9, 1: 9, 2: 0, 3: 1 }, 3: { 0: 3, 1: 9, 2: 0, 3: 1 } }, expect: "словарь словарей" },
 { name: "ключи-пары \"(i, j)\", vars: { n: 3, k: 0, i: 0, j: 0,
 "(1, 2)": 2, "(2, 0)": 1, "(2, 1)": 9, "(2, 2)": 9 }, expect: "ключи-пары" },
 { name: "плоский список – это НЕ матрица", vars: { n: 80, k: 0, i: 0, j: 0,
 }, expect: "плоский список" },
 { name: "80×80 (обрезка до 64)", vars: { n: 80, k: 0, i: 0, j: 0,
 }, expect: "80×80" },

```

```

 });
 for (const c of gridCases) {
 await test("С. матричные страницы", `${c.viz}: ${c.n}`);
 await mountViz(c.viz);
 await act(async () => {
 emitVizState(c.viz, { line: 2, func: "<module>" });
 await sleep(30);
 });
 const txt = (host?.textContent ?? "").replace(/\s/g, "");
 if (!c.expect.test(txt)) throw new Error(`ожидали ${c.expect}`);
 const defects = domDefects(host);
 if (defects.length) throw new Error(`артефакты в ${c.viz}`);
 return /размерность не совпадает/.test(txt) ? "проблема" : "окей";
 });
}

await test("С. матричные страницы", "2D-построение: оригинал");
await mountViz("sparse-table");
const A8 = Array.from({ length: 8 }, (_, i) => Array.from({ length: 8 }, (_, j) => i + j));
await act(async () => {
 emitVizState("sparse-table", { line: 7, func: "<module>" });
 await sleep(40);
});
const grids = () => Array.from((host ?? document).querySelectorAll("table"), (t) => {
 let g0 = grids()[0];
 if (!g0) throw new Error("нет ни одной матрицы блоков");
 const firstCell = () => (g0.children[0] as HTMLTableSectionElement).children[0].textContent;
 // «Оригинал (1x1)» = сама матрица A: значения видны
 if (firstCell() !== "1") throw new Error(`оригинал не 1x1`);
 if ((g0.children[7] as HTMLTableSectionElement).children[7].textContent ?? "").trim() !== "7"
 // шаги кликаются даже под live-k из компилятора
 await click(findButton("Шаг 2 (4x4)"), "вкладка шаг 2");
 const txt = (host?.textContent ?? "").replace(/\s+/g, "");
 if (!/Матрица блоков 4x4/.test(txt)) throw new Error(`не 4x4`);
 g0 = grids()[0];
 const cls = (g0.children[0] as HTMLTableSectionElement).children[0].className;
 // пустая активная клетка (код смотрит на (0,0), стили на (1,1))
 if (cls !== "active") throw new Error(`не активна`);
});

```

```

for (const key of syncKeys) {
 const sync = PAGE_SYNC[key];
 const chapterId = key.split("#")[0];
 await test("Е. эталонный код", `${key}: выполняется
 const raw = await py.runPythonAsync(buildDebugRun
 const result = JSON.parse(String(raw)) as DebugRe
 if (result.error) throw new Error(`Python-ошибка:
 if (result.steps.length === 0) throw new Error("н
 // объявленные переменные синхронизации реально п
 const seen = new Set<string>();
 for (const s of result.steps) for (const n of Obj
 for (const n of Object.keys(result.finalValues ??
 // в документации имя может быть видом «хк / х»,
 // проверяем, что хотя бы один python-идентификат
 const declared = sync.variables.map({v} => v.name
 const ident = (t: string) => /^[A-Za-z_][A-Za-z0-
 const missing = declared
 .map({raw} => raw.split(/[\/[\],]|\s+или\s+\/).m
 .filter({alts} => alts.some(ident))
 .filter({alts} => !alts.some((a) => ident(a) &&
 .map({alts} => alts.join(" / "));
 if (missing.length) throw new Error(`код не созда
);

 if (!VIZ_REGISTRY[chapterId]) return `шагов ${res
 await mountViz(chapterId);
 const finalVars = result.finalValues ?? result.st
 await act(async () => {
 emitVizState(chapterId, { line: result.steps[re
 ed: Object.keys(finalVars) });
 await sleep(40);
 });
 const txt = (host?.textContent ?? "").replace(/\\s
 if (!/данные из компилятора/.test(txt)) throw new
 const defects = domDefects(host);
 if (defects.length) throw new Error(`артефакты в
 return `шагов ${result.steps.length}${result.trun
 }.join(", ")`};

```

```

const ready = await waitFor(() => bodyText().includes('готов'))
if (!ready) throw new Error(`программа с огромным р
const block = document.querySelector('[data-testid=
if (!block) throw new Error("блок «Результат» не от
const out = block.textContent ?? "";
if (!/вывод обрезан/.test(out)) throw new Error(`па
if (out.includes("ХВОСТ_ВЫВОДА")) throw new Error("
if (out.length > 40000) throw new Error(`в панели $
if (!textarea()) throw new Error("редактор исчез по
if (domDefects(host).length) throw new Error(`артеф
return `вывод обрезан до ${out.length} знаков, хвос
});

```

```

await test("F. редактор", "полная очистка: ничего не
 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "п
 const ta = textarea();
 if (!ta) throw new Error("нет редактора");
 typeInto(ta, "");
 const done = await waitFor(() => bodyText().include
 if (!done) throw new Error(`на пустой редактор пане
 if (!textarea()) throw new Error("редактор исчез по
 if (!/встроенное демо/.test(bodyText())) throw new
 if (allButtons().length < 5) throw new Error(`кноп
 return `пустой код: кнопок ${allButtons().length},
});

```

```

await test("F. редактор", "глаз: эталон попадает в ре
 await mountApp(ch);
 await click(findButton("Боковая панель Python"), "п
 const ta = textarea();
 if (!ta) throw new Error("нет редактора");
 const init = ta.value;
 const editorButtons = allButtons();
 const ref = PAGE_SYNC[ch]?.code ?? "";
 if (!ref) throw new Error("у страницы нет эталонног
 const done0 = await waitFor(() => RUN_DONE.some((m)
 if (!done0) throw new Error(`инициализация не выпол

```

```

if (!/встроенное демо/.test(bodyText())) throw new Error("не встроенное демо");
if (/данные из компилятора/.test(bodyText())) throw new Error("данные из компилятора");
const afterButtons = allButtons();
const lost = editorButtons.filter((b) => b && !afterButtons.includes(b));
if (lost.length > 2) throw new Error(`после снятия с панели осталось ${lost.length} кнопки`);

// и возвращает инициализацию страницы
await click(findButton("Сбросить к шаблону страницы"));
const back = await waitFor(() => textarea()?.value === "1");
if (!back) throw new Error(`и не вернул инициализацию`);
return "эталон вставляется в редактор, ведёт демонстрацию";
});

```

```

await test("F. редактор", "код не перетекает между страницами");
await mountApp(ch);
await click(findButton("Боковая панель Python"), "панель Python");
const ta = textarea();
if (!ta) throw new Error("нет редактора");
typeInto(ta, "# MY-MARKER-14\nx = 41\nx += 1\nprint x\n");
await waitFor(() => RUN_DONE.some((m) => bodyText().includes(m)));
if (!bodyText().includes("42")) throw new Error("свой код не перетекает");
// переходим на другую страницу, не закрывая панель Python
await mountApp("floyd");
await click(findButton("Боковая панель Python"), "панель Python");
const ta2 = textarea();
if (!ta2) throw new Error("на новой странице нет редактора");
if (ta2.value.includes("MY-MARKER-14")) throw new Error("код перетекает");
if (!ta2.value.includes("# «")) throw new Error("на новой странице нет редактора");
await mountApp(ch);
await click(findButton("Боковая панель Python"), "панель Python");
const ta3 = textarea();
if (!ta3?.value.includes("MY-MARKER-14")) throw new Error("код перетекает");
return "у каждой страницы свой код, возврат сохраняется";
});

```

```

await test("F. редактор", "редактор живёт и во время загрузки");
await mountApp(ch);
await click(findButton("Боковая панель Python"), "панель Python");

```

```

 if (!/встроенное демо/.test(bodyText())) throw new Error("демо не работает");
 return "панель закрыта во время выполнения — демонстрация";
 });

```

```

await test("Н. доступность", "панель: все кнопки подпаны");
await mountApp(ch);
await click(findButton("Боковая панель Python"), "панель Python");
const namelessEditor = namelessButtons(panel() ?? document.querySelector("#editor"));
const editorBtns = allButtons().length;
const listing = await waitFor(() => !!findButton(/Остановить прогон трассы/));
if (!listing) throw new Error("панель не показала трассу");
const namelessListing = namelessButtons(panel() ?? document.querySelector("#listing"));
const listingBtns = allButtons().length;
if (namelessEditor.length) throw new Error(`в редакторе не найдены кнопки: ${namelessEditor.map(b => b.textContent).join(", ")}`);
if (namelessListing.length) throw new Error(`в листинге не найдены кнопки: ${namelessListing.map(b => b.textContent).join(", ")}`);
const required = [/Остановить прогон трассы/, /Свернуть панель Python/, /В начало трассы/, /В конец трассы/, /автопроход/i];
const btns = allButtons().join(" | ");
const missing = required.filter(re => !re.test(btns));
if (missing.length) throw new Error(`в листинге не найдены кнопки: ${missing.join(", ")}`);
return `редактор: ${editorBtns} кнопок, листинг: ${listingBtns} кнопок`;
});

```

```

await test("Н. доступность", "бейдж честно различает");
await mountApp(ch);
// 1) демо: компилятор ещё ничего не присылал
if (!/встроенное демо/.test(bodyText())) throw new Error("демо не работает");
if (/переменных пока нет/.test(bodyText())) throw new Error("переменные пока не прислали");
await click(findButton("Боковая панель Python"), "панель Python");
const listing = await waitFor(() => !!findButton(/Остановить прогон трассы/));
if (!listing) throw new Error("панель не показала трассу");
// 2) код выполнен, отладчик на первой строке: переменные уже прислали
const awaiting = await waitFor(() => /переменных пока нет/);
if (!awaiting) throw new Error(`после запуска бейдж не показывает: ${bodyText().match(/встроенное демо|данные из кода/)[0]}`);
// 3) дошли до конца трассы — картинка обязана жить

```



```

const menu = document.querySelector("#download-menu");
if (!menu) throw new Error("нет меню загрузки в шапке");
const want = [
 { id: "download-guide-pdf-btn", file: "export/guide.pdf", href: "https://www.pythontips.ru/export/guide.pdf" },
 { id: "download-pdf-btn", file: "export/full_code.pdf", href: "https://www.pythontips.ru/export/full_code.pdf" },
 { id: "download-txt-btn", file: "export/full_code.txt", href: "https://www.pythontips.ru/export/full_code.txt" },
];
for (const w of want) {
 const el = menu.querySelector<HTMLAnchorElement>(`a[id="${w.id}"]`);
 if (!el) throw new Error(`в шапке нет кнопки «${w.name}»`);
 const href = el.getAttribute("href") ?? "";
 if (!href.endsWith(w.file)) throw new Error(`ссылка «${w.name}» не ведет на файл`);
 const name = [el.getAttribute("aria-label"), el.getAttribute("title")].filter(Boolean).join(" ");
 if (name.trim().length < 5) throw new Error(`у кнопки «${w.name}» нет названия`);
}
if (!menu.querySelector("#download-html-btn")) throw new Error("нет кнопки загрузки HTML-версии");
const py = document.querySelector<HTMLButtonElement>("#download-python-btn");
if (!py) throw new Error("нет кнопки панели Python");
if (!py.getAttribute("aria-label")) throw new Error("у кнопки панели Python нет aria-label");
if (!py.getAttribute("title")) throw new Error("у кнопки панели Python нет title");
return "4 загрузки на месте, href ведут на файлы, кнопки работают";
});

```

```

await test("I. ссылки", "упоминание «билет 5» кликабельно");
await mountApp("sparse-table");
const links = Array.from(document.querySelectorAll("a"));
if (links.length === 0) throw new Error("на странице нет ссылок");
const toPrefix = links.find((a) => a.dataset.goto === "5");
if (!toPrefix) throw new Error("нет ссылки на билет 5");
if (!/билет 5/i.test(toPrefix.textContent ?? "")) throw new Error("ссылка не содержит «билет 5»");
const href = toPrefix.getAttribute("href") ?? "";
if (!href.includes("topic=prefix-sums-2d")) throw new Error("ссылка не ведет на страницу с суммами");
await click(toPrefix, "ссылка на билет 5");
await settle(120);
const txt = bodyText();
if (!/Двумерная матрица префиксных сумм/.test(txt)) throw new Error("клик по «билет 5» не открыл страницу с суммами");

```

```

 for (let i = 1; i < nums.length; i += 1) {
 if (nums[i] < nums[i - 1]) throw new Error(`порядок`)
 }
 if (!sidebar.textContent?.includes("Порядок чтения"))
 throw new Error("в содержании нет подсказки, в ка")
 }
 return `разделов ${headers.length}, 11+12 соседству
});

```

```

await test("I. ссылки", "введение: базовая база, обзор")
await mountApp("intro");
const txt = bodyText();
for (const must of ["Дерево как массив", "Куча", "Обзор"])
 if (!txt.includes(must)) throw new Error(`во введе`)
}
if (!txt.includes("2n") || !txt.includes("2n+1")) throw new Error("2n")
if (!txt.includes("≤ 1")) throw new Error("во введе")
if (/Обзор: Кратчайшие пути/.test(txt)) throw new Error("Обзор")
const goto = Array.from(document.querySelectorAll("a"));
if (!goto.includes("dijkstra")) throw new Error(`об`)
if (domDefects(document.querySelector("#chapter-viz")))
 throw new Error("артефакты в DOM введения");
}
return "введение = куча/очередь/бинпоиск/сортировка";
});

```

```

await test("I. блиц", "вопросы отвечают, объясняют и")
await mountApp("sparse-table");
const list = quizzes["sparse-table"] ?? [];
if (list.length === 0) throw new Error("у билета 2")
const section = Array.from(document.querySelectorAll("div"))
 .find(el => el.getAttribute("aria-label")?.includes("Блиц"));
if (!section) throw new Error("блиц не отрисован на")
const ask = (label: string | RegExp) =>
 Array.from(section.querySelectorAll("button")).find(b =>
 const hay = [b.getAttribute("aria-label"), b.textContent];
 return typeof label === "string" ? hay.includes(label) :

```

```

const prefixCell = sparse.find((c) => !/log/.test(c)
if (!prefixCell) throw new Error(`в сравнении билета
// билет 5: зеркально – у префиксной без логарифмов
if (!/log/.test(prefix.join(" "))) throw new Error(
;
if (/log/.test(prefix[1] ?? "")) throw new Error(`в
const txt = bodyText();
if (/префиксн[^\.]{0,40}(жирн|больше памяти)/i.test(
 throw new Error("на странице билета 5 префиксная
}
await mountApp("sparse-table");
const full = bodyText();
if (!/100 (коржей|копий|матриц)|сто (коржей|матриц)/.test(
 throw new Error("на странице билета 2 не объяснено
}
if (!/не конкуренты|разные задачи/i.test(full)) {
 throw new Error("на странице билета 2 не сказано,
}
// аналогии: торт стоит РЯДОМ с вафельными трубочками
for (const topic of ["sparse-table", "prefix-sums-2"])
 await mountApp(topic);
const full = bodyText();
if (!/Аналогия 1: Сетка из вафельных трубочек/.test(full))
if (!/Аналогия 2: Торт-пирамидка/.test(full)) throw new Error(
if (!/крем обратно/.test(full)) throw new Error(`на
if (!/трубочки можно не только докладывать, но и вынимать/.test(full))
 throw new Error(`на «${topic}» не объяснено, что
}
if (!/время: построение $O(N \cdot M \cdot \log N \cdot \log M)$, запрос суммы $O(N \cdot M)$ /.test(full))
 throw new Error(`на «${topic}» у карточки торта
}
if (!/память: $N \times M \times \log N \times \log M$ /.test(full)) throw new Error(
if (!/время: построение $O(N \cdot M)$, запрос суммы $O(N \cdot M)$ /.test(full))
 throw new Error(`на «${topic}» у карточки трубочки
}
if (!/память: $N \times M$ – как оригинал, тонкая/.test(full))
 throw new Error(`на «${topic}» у карточки трубочки
}

```

```

 if (!/4x4/.test(page)) throw new Error(`демонстрация не работает`);
 const defects = domDefects(document.querySelector("#defects"));
 if (defects.length) throw new Error(`артефакты в демонстрации`);
 return "набрал 4x4 с INF=999 → d[0] = [0, 3, 5, 6]";
 });

```

```

await test("К. свой код", "своя разреженная таблица не строится");
await mountApp("sparse-table");
await click(findButton("Боковая панель Python"), "поиск");
const ta = textarea();
if (!ta) throw new Error("нет редактора");
typeInto(
 ta,
 [
 "a = [5, 1, 9, 3, 7, 2]",
 "n = len(a)",
 "LOG = 3",
 "i, j = 0, 0",
 "st = []",
 "for row in a:",
 " st.append([row] + [None] * (LOG - 1))",
 "for j in range(1, LOG):",
 " for i in range(n - (1 << j) + 1):",
 " st[i][j] = min(st[i][j - 1], st[i + (1 << j)][j - 1])",
 "print(n, st[0][2], st[1][1])",
].join("\n")
);
const ready = await waitFor(() => bodyText().includes("готово"));
if (!ready) throw new Error(`своя таблица не построилась`);
if (!/6 1 1/.test(panelText())) throw new Error(`отсутствует панель`);
await click(findButton("В конец трассы"), "в конец трассы");
const linked = await waitFor(() => /данные из компилятора/);
if (!linked) throw new Error("демонстрация не приняла данные");
const defects = domDefects(document.querySelector("#defects"));
if (defects.length) throw new Error(`артефакты в демонстрации`);
return "свой массив из 6 чисел: st[0][2] = 1, st[1][1] = 1";
});

```

```

 if (/main\.py|Pyodide/.test(txt)) throw new Error("в lite-режиме");
 if (!/разрежен/i.test(txt)) throw new Error("в lite-режиме");
 if (findButton("Боковая панель Python")) throw new Error("в lite-режиме");
 // текстовая версия: ссылки на другие билеты ведут к документам
 // а блиц напечатан списком с ответами
 const liteLinks = Array.from(document.querySelectorAll('a'));
 const bad = liteLinks.filter((a) => !a.getAttribute('href'));
 if (liteLinks.length === 0) throw new Error("в lite-режиме");
 if (bad.length) throw new Error(`в lite-версии ${bad.length} ссылок не найдено`);
 const unknown = liteLinks.filter((a) => !chapters.some(c => c.href === a.href));
 if (unknown.length) throw new Error(`в lite-версии ${unknown.length} ссылок не найдено`);
 if (!/Блиц: вопросы по теме/.test(txt)) throw new Error("в lite-режиме");
 if (!/Верно:/.test(txt)) throw new Error("в lite-режиме");
 return `${txt.length} знаков, ссылок в lite ${liteLinks.length}`;
 });

 await test("G. lite-режим", "HTML-книга: оглавление, словарь терминов");
 const { buildBookHtml } = await import("../src/buildBookHtml.js");
 const html = await buildBookHtml(chapters);
 if (!html.includes('<nav id="toc"')) throw new Error("в lite-режиме");
 const missing = chapters.filter((c) => !html.includes(c.title));
 if (missing.length) throw new Error(`в книге потеряно ${missing.length} глав`);
 const tocLinks = (html.match(/href="#ch-/g) ?? []).length;
 if (tocLinks < chapters.length) throw new Error(`в книге потеряно ${chapters.length - tocLinks} глав`);
 if (!/Словарик терминов пособия/.test(html)) throw new Error("в lite-режиме");
 if (!/к оглавлению/.test(html)) throw new Error("в lite-режиме");
 if (html.length < 200_000) throw new Error(`книга слишком маленькая: ${html.length} байт`);
 return `${(html.length / 1024).toFixed(0)} КБ, глав ${chapters.length}`;
});

console.error = realConsoleError;
return results;
}

/** Сводка для оркестратора. */
export function summary() {
 const byGroup = new Map<string, { ok: number; fail: number; }>();
 for (const r of results) {
 const { group, ok, fail } = r;
 const current = byGroup.get(group) || { ok: 0, fail: 0 };
 current.ok += ok;
 current.fail += fail;
 byGroup.set(group, current);
 }
 const totalOk = byGroup.values().reduce((sum, { ok }) => sum + ok, 0);
 const totalFail = byGroup.values().reduce((sum, { fail }) => sum + fail, 0);
 return { totalOk, totalFail };
}

```

```

const OUT = "tmp/verify/links-entry.mjs";
const FAILS = [];
const NOTES = [];
const ok = (cond, message) => (cond ? true : (FAILS.push(
 message), false));

mkdirSync("tmp/verify", { recursive: true });
writeFileSync(
 "tmp/verify/links-entry.ts",
 [
 'export { chapters, chapterTopics, SECTIONS, sectionOf, GLOSSARY } from "../src/data/glossary";'
 'export { VIZ_REGISTRY } from "../src/components/vizRegistry";'
 'export { PAGE_SYNC } from "../src/data/vizSync";'
 'export { quizzes } from "../src/data/quizzes";'
 'export { MINI_DEMO_KINDS } from "../src/components/minidemoKinds";'
].join("\n")
);
await esbuild.build({
 entryPoints: ["tmp/verify/links-entry.ts"],
 bundle: true,
 outfile: OUT,
 format: "esm",
 platform: "browser",
 target: "es2022",
 jsx: "automatic",
 logLevel: "error",
 define: {
 "process.env.NODE_ENV": '"development"',
 "import.meta.env.BASE_URL": '"/'',
 "import.meta.env.MODE": '"development"',
 "import.meta.env.DEV": "true",
 "import.meta.env.PROD": "false",
 "import.meta.env.SSR": "false",
 },
 loader: { ".css": "empty", ".png": "dataurl", ".jpg": "dataurl" },
});
const { chapters, chapterTopics, SECTIONS, sectionOf, GLOSSARY } = await import(`${process.cwd()}/${OUT}?run=${Date.now()}`);

```

```

 for (let n = all[0]; n <= all[all.length - 1]; n += 1)
 ok(all.includes(n), `в содержании дыра: билет ${n}`);
 }
}

```

/\* 3–5. ссылки внутри текста страниц \*/

```

const TICKET_RE = /билет(?:ы|ов|ам|ах|ом|у|а|е)?\s*(\d{
let ticketMentions = 0;
let explicitLinks = 0;
const withoutLinks = [];
for (const chapter of chapters) {
 const html = chapter.content ?? "";
 let mentionsHere = 0;

 // 3. упоминания «билет N» – только внутри текста, не
 const textOnly = html
 .replace(/<pre[\s\S]*?<\/pre>/gi, " ")
 .replace(/<code[\s\S]*?<\/code>/gi, " ")
 .replace(/<[>]*>/g, " ");
 for (const m of textOnly.matchAll(TICKET_RE)) {
 const from = Number(m[1]);
 const to = m[2] ? Number(m[2]) : from;
 mentionsHere += 1;
 for (let n = from; n <= to; n += 1) {
 ok(n >= 1 && n <= 24 && covered.has(n), `«${chapter
 }
 }
 ticketMentions += mentionsHere;
}

```

// 4. явные переходы

```

for (const m of html.matchAll(/data-goto="(^[^"]+)"/g))
 explicitLinks += 1;
 ok(chapterIds.has(m[1]), `«${chapter.id}»: data-goto
}
for (const m of html.matchAll(/href="\?(?:lite=1&)?to
 explicitLinks += 1;
 ok(chapterIds.has(decodeURIComponent(m[1])), `«${ch
}

```

```

{
 const { existsSync, statSync } = await import("fs");
 for (const file of ["public/export/guide_lite.pdf", "public/export/guide_lite.pdf", "public/export/guide_lite.pdf"]) {
 ok(existsSync(file), `шапка ссылается на ${file}, но не существует`);
 if (existsSync(file) && statSync(file).size < 1000) {
 console.log(`файл ${file} слишком мал`);
 }
 }
}

```

```

/* 10. связанные темы + блиц */
for (const id of withoutLinks) NOTES.push(`нет перекрёска`);
const noQuiz = chapters.filter((c) => !(quizzes[c.id] ? true : false));

```

```

console.log("————— ссылки и структура содержания —————");
console.log(` страниц: ${chapters.length} · разделов: ${sections.length}`);
console.log(` упоминаний «билет N» в тексте: ${ticketMentions.length}`);
console.log(` визуализаций в реестре: ${registryIds.length} + ${GLOSSARY.length}`);
console.log(` блиц есть у ${Object.keys(quizzes).length} билетов`);
if (NOTES.length) console.log(` заметки:\n ${NOTES.join("\n")}`);
if (noQuiz.length) console.log(` без блица: ${noQuiz.join(", ")}`);

```

```

if (FAILS.length) {
 console.log(`\nПРОВАЛЕНО ${FAILS.length}:`);
 FAILS.slice(0, 40).forEach((f) => console.log(` x ${f}`));
 process.exit(1);
}
console.log("\nвсе ссылки целы: 0 битых переходов");

```

ФАЙЛ 101/111: scripts/verify/check-reference-code.mjs  
 КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 133 | РАЗМЕР: 1.5 КБ

```

/**
 * Контракт «эталонный код ↔ страница ↔ визуализация».
 *
 * npm run verify:code

```



```

 loader: { ".css": "empty", ".png": "dataurl", ".jpg":
 logLevel: "warning",
});
void built;
const mod = await import(`${process.cwd()}/${OUT_DIR}/c

const py = await loadPyodide({ indexURL: "node_modules/
const printed = [];
py.setStdout({ batched: (m) => printed.push(m) });
py.setStderr({ batched: (m) => printed.push("[stderr] "

const only = process.argv.slice(2);
const ident = (t) => /^[A-Za-z_][A-Za-z0-9_]*$/ .test(t)
/** «xk / x», «dist[v]», «(i-1)//2», «π» → набор python
const alts = (raw) => raw.split(/[/[\],]|\\s+или\\s+/) .m

const rows = [];
let problems = 0;
for (const [key, entry] of Object.entries(mod.sync)) {
 if (only.length && !only.some((o) => key.includes(o)))
 printed.length = 0;
 let result;
 try {
 result = JSON.parse(String(await py.runPythonAsync(
 } catch (error) {
 problems++;
 rows.push({ key, steps: 0, status: `СБОЙ ЗАПУСКА: $
 continue;
 }
 const seen = new Set();
 for (const step of result.steps) for (const name of 0
 for (const name of Object.keys(result.finalValues ??

const issues = [];
if (result.error) issues.push(`ошибка Python: ${result
if (result.truncated) issues.push("трасса оборвана по
if (result.steps.length === 0) issues.push("нет ни од
const missing = (mod.declared[key] ?? [])

```

```

rows.push({ key, steps: result.steps.length, vars: se
}

rows.sort((a, b) => (a.status === "ок") - (b.status ===
console.log("ЭТАЛОННЫЕ КОДЫ СТРАНИЦ (настоящий CPython
console.log("=".repeat(96));
for (const r of rows) {
 const mark = r.status === "ок" ? " ok " : " x ";
 console.log(`${mark}${r.key.padEnd(34)} шагов ${String
 === "ок" ? "" : r.status}`);
}
console.log("=".repeat(96));
console.log(`всего: ${rows.length} | с проблемами: ${
writeFileSync(`${OUT_DIR}/reference-code.json`, JSON.st
process.exit(problems > 0 ? 1 : 0);

```

-----  
ФАЙЛ 102/111: scripts/verify/dom-env.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 107 | РАЗМ

```

/**
 * Браузерное окружение для проверок: jsdom + те API, к
 * Создаётся ПОСЛЕ загрузки Pyodide (иначе pyodide реши
 */
import { JSDOM, VirtualConsole } from "jsdom";

const OBSERVER_STUB = class {
 observe() {}
 unobserve() {}
 disconnect() {}
 takeRecords() {
 return [];
 }
};

export function createDom({ url = "http://localhost/?to

```

```
Object.defineProperty(window, "innerHeight", { value:
Object.defineProperty(window, "devicePixelRatio", { v
window.Element.prototype.getBoundingClientRect = func
 return { x: 0, y: 0, top: 0, left: 0, right: 400, b
};
```

```
const copy = [
 "window",
 "document",
 "navigator",
 "HTMLElement",
 "HTMLInputElement",
 "HTMLTextAreaElement",
 "HTMLButtonElement",
 "HTMLAnchorElement",
 "HTMLDivElement",
 "HTMLSelectElement",
 "Element",
 "Node",
 "Event",
 "CustomEvent",
 "MouseEvent",
 "KeyboardEvent",
 "InputEvent",
 "getComputedStyle",
 "requestAnimationFrame",
 "cancelAnimationFrame",
 "DOMParser",
 "SVGElement",
 "Text",
 "DocumentFragment",
 "NodeList",
 "HTMLCollection",
 "Range",
 "MutationObserver",
 "localStorage",
 "sessionStorage",
 "Blob",
```

```

});
await esbuild.build({
 entryPoints: [`${OUT_DIR}/unit-entry.ts`],
 bundle: true,
 outfile: `${OUT_DIR}/unit-bundle.mjs`,
 format: "esm",
 platform: "node",
 jsx: "automatic",
 loader: { ".css": "empty" },
 logLevel: "warning",
});
const { liveMatrix, pickLiveVar, clampIndex } = await import("./unit-bundle.mjs");

let passed = 0;
const failures = [];
function check(name, actual, expected) {
 const a = JSON.stringify(actual);
 const e = JSON.stringify(expected);
 if (a === e) passed++;
 else failures.push(`${name}\n          получено: ${a}\n          ожидалось: ${e}`);
}
const m = (v) => liveMatrix(v);
const shape = (v) => {
 const r = m(v);
 return r ? { rows: r.rows, cols: r.cols, label: r.size } : null;
};
const cells = (v) => {
 const r = m(v);
 if (!r) return null;
 return Array.from({ length: r.rows }, (_, i) => Array.from({ length: r.cols }, (_, j) => r.get(i, j)));
};

/* — liveMatrix: формы данных —————
check("квадратная 3x3 (список списков)", shape([[1, 2, 3], [4, 5, 6], [7, 8, 9]]), 3);
check("не квадратная 2x4", shape([[1, 2, 3, 4], [5, 6, 7, 8]]), 2);
check("значения ячеек 2x2", cells([[0, 1], [2, 3]]), [[0, 1], [2, 3]]);
check("рванные строки: cols = максимум", shape([[1, 2, 3], [4, 5, 6], [7, 8, 9]]), 3);
check("рванные строки: дырка = null", cells([[1, 2, 3], [4, 5, 6], [7, 8, 9]]), [[1, 2, 3], [4, 5, 6], [7, 8, 9]]);

```

```

check("обрезка снизу", clampIndex(-7, 5), 0);
check("дробный индекс усекается", clampIndex(2.9, 5), 2);
check("NaN → null", clampIndex(Number.NaN, 5), null);
check("Infinity → null", clampIndex(Number.POSITIVE_INFINITY, 5), null);

console.log("=".repeat(72));
console.log(`liveMatrix / pickLiveVar / clampIndex:  проделано`);
for (const f of failures) console.log(`  × ${f}`);
console.log("=".repeat(72));
process.exit(failures.length ? 1 : 0);

```

ФАЙЛ 104/111: scripts/verify/probe-compiler.tsx

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 121 | РАЗМЕР: 1.5 КБ

```

/**
 * Зонд состояния панели Python во времени: что происходит
 * Запуск: node tmp/harness/run-probe.mjs
 */
import { act } from "react";
import { createRoot } from "react-dom/client";
import App from "../../src/App";

const sleep = (ms: number) => new Promise((r) => setTimeout(r, ms));

async function settle(ms = 40) {
 await act(async () => {
 await sleep(ms);
 });
}

function panel(): HTMLElement | null {
 return document.querySelector<HTMLElement>('aside[aria-label="panel"]');
}

function editor(): HTMLTextAreaElement | undefined {
 return panel()?.querySelector<HTMLTextAreaElement>('textarea');
}

function footer(): string {
 return document.querySelector<HTMLFootnoteElement>('hr[aria-label="footer"]')?.text || '';
}

```

```

document.body.appendChild(host);
window.history.replaceState({}, "", "/?topic=dijkstra");
const root = createRoot(host);
await act(async () => {
 root.render(<App />);
});
await settle(80);

process.stdout.write("\n=== 1. открыли панель, ждём а
await click(findButton(/Боковая панель Python/), "пан
state("t=0");
for (const t of [300, 1000, 3000, 8000]) {
 await settle(t - (t === 300 ? 0 : 0));
 state(`t+${t}`);
 if (/Трасса готова|Нет шагов|Пустой код|Ошибка/.tes
}

process.stdout.write("\n=== 2. вводим код ТОЛЬКО с ко
const ed = editor();
if (!ed) {
 process.stdout.write(" редактора нет – жмём Esc и
 await act(async () => {
 document.dispatchEvent(new window.KeyboardEvent("
 await sleep(30);
 });
}
const ed2 = editor();
if (!ed2) throw new Error("редактор так и не появился
typeInto(ed2, "# только комментарий\n# и ещё\n");
state("ввели");
for (const t of [200, 800, 2000, 6000]) {
 await settle(t);
 state(`t+${t}`);
}

process.stdout.write("\n=== 3. вводим рабочий код x=4
const ed3 = editor();
if (!ed3) throw new Error("редактор пропал после комм

```

-----  
```bash

```
npm run verify          # всё сразу: unit → эталонные к  
npm run verify:unit     # 38 проверок разбора живых дан  
npm run verify:code     # 46 эталонных кодов страниц по  
npm run verify:dom      # 139 сценариев: страницы, пане  
npm run verify:probe    # диагностика панели Python во  
```\n
```

Отчёты кладутся в `tmp/verify/` (каталог в .gitignore):  
помарочно по DOM-сценариям, `reference-code.json` — по

## Что именно проверяется

### `verify:unit` — `live-matrix.test.mjs`

Разбор данных, которые присылает компилятор, в матрицу  
размерности: список списков, рванные строки, словарь слов  
`(0, 3)` (так снапшотятся tuple-ключи Python), `inf`  
числа-строки, обрезка до 64x64. Отдельно — что матрицей  
список и словарь по именам (`dist`, `prev`): иначе визу  
чужие данные за сетку. Плюс `pickLiveVar` (приоритет им  
(индексы пользователя вне диапазона демо).

### `verify:code` — `check-reference-code.mjs`

Каждый референсный код из `PAGE\_SYNC` выполняется насто  
`sys.settrace`-трейсером, что и панель компилятора. Прог  
трасса не оборвана по лимиту шагов, шагов больше нуля (и  
пустой листинг), и \*\*каждая переменная, обещанная стран  
кодом\*\* — иначе визуализация физически не может за ним  
ловятся предупреждения интерпретатора, которые иначе по  
«Результат».

### `verify:dom` — `browser-checks.tsx`

Настоящее приложение монтируется в jsdom (`App` целиком  
глава, визуализация, панель Python), сценарии кликают к  
редакторе.

| Группа | Что делает |

| --- | --- |

```
* Фильтры:
* HAR_CHAPTERS=floyd,dijkstra только эти страницы
* HAR_SKIP=A,E пропустить группы (A
* С матрицы, D графы,
* F редактор, G lite,
* HAR_SYNC=floyd,mst#prim только эти записи etc
*
* Оркестратор: грузит CPython ДО создания jsdom (иначе
* в браузере, и лезет за wasm на CDN), кладёт runtime на
* window.__algv0PythonRuntime и собирает реальные компоненты
*/
import * as esbuild from "esbuild";
import { writeFileSync, mkdirSync } from "fs";
import { loadPyodide } from "pyodide";
import { createDom } from "./dom-env.mjs";

const OUT_DIR = "tmp/verify";
const t0 = Date.now();
const jsdomErrors = [];

log("гружаю CPython (pyodide из node_modules)...");
const py = await loadPyodide({ indexURL: "node_modules/pyodide/" });
log(` Python готов за ${Date.now() - t0} мс`);

log("поднимаю jsdom...");
const { window, document, consoleErrors } = createDom({
 window.__algv0PythonRuntime = py;
});

log("собираю приложение esbuild'ом..");
mkdirSync(OUT_DIR, { recursive: true });
await esbuild.build({
 entryPoints: ["scripts/verify/browser-checks.tsx"],
 bundle: true,
 outfile: `${OUT_DIR}/bundle.mjs`,
 format: "esm",
 platform: "browser",
 target: "es2022",
 jsx: "automatic",
```



```

 console.log("\nОШИБКИ ОКРУЖЕНИЯ (jsdom/неперехваченны
 for (const e of [...new Set(jsdomErrors)].slice(0, 10
 }
const stray = consoleErrors.filter((e) => !/not wrapped
if (stray.length) {
 console.log(`\nCONSOLE.ERROR вне тестов (${stray.leng
 for (const e of [...new Set(stray)].slice(0, 10)) con
 }
console.log(`\nотчёт: ${OUT_DIR}/report.json | время:
process.exit(sum.failed > 0 || jsdomErrors.length > 0 ?

function log(message) {
 process.stdout.write(`[verify:dom] ${message}\n`);
}

```

ФАЙЛ 107/111: scripts/verify/run-probe.mjs

КАТЕГОРИЯ: Скрипты сборки и экспорта | СТРОК: 45 | РАЗМ

```

/**
 * Зонд панели Python: печатает состояние редактора/тра
 * Не тест, а диагностика — когда нужно понять, что пан
 * ввода кода (автозапуск, переход в листинг, сообщения
 *
 * npm run verify:probe
 */
import * as esbuild from "esbuild";
import { mkdirSync } from "fs";
import { loadPyodide } from "pyodide";
import { createDom } from "../dom-env.mjs";

const OUT_DIR = "tmp/verify";
mkdirSync(OUT_DIR, { recursive: true });
const log = (m) => process.stdout.write(`[verify:probe]

log("гружаю CPython...");
```

name: Deploy app to GitHub Pages

```
ДВЕ настройки репозитория нужно сделать один раз ВРУЧНУЮ
(у бота-интеграции нет admin-прав на них – API отвечает)
#
1. Settings → Pages → Build and deployment → Source
Если там «Deploy from a branch» (legacy) – шаг D
2. Settings → Environments → github-pages → Deployment
добавьте паттерн «arena/**» и ветку «main» (или
иначе деплой из новой рабочей ветки отклоняется)
```

on:

push:

branches:

- main

# Позволяет проверить Pages прямо из рабочей ветки

- "arena/\*\*"

workflow\_dispatch:

permissions:

contents: read

pages: write

id-token: write

concurrency:

group: pages

cancel-in-progress: true

jobs:

build:

name: Build static site

runs-on: ubuntu-latest

steps:

- name: Checkout

uses: actions/checkout@v6

- name: Setup Node.js

uses: actions/setup-node@v7

```
 те этот запуск. См. комментарий в начале de
 exit 1
 fi
```

```
- name: Deploy
 id: deployment
 uses: actions/deploy-pages@v5
```

ФАЙЛ 109/111: `.github/workflows/rebuild-pdf.yml`  
КАТЕГОРИЯ: CI / автоматизация | СТРӨК: 155 | РАЗМЕР: 6.1

name: Rebuild code PDF

```
Пайплайн пересборки PDF при каждом коммите:

исходный код → full_code_all_pages.txt (scrip
|
└→ page-0001.png ... page-NNNN.png (
|
└→ full_code_all_page

Готовые TXT и PDF коммитаются обратно в ветку (их отда
промежуточные PNG сохраняются как artifacts запуска.
```

on:

push:

branches:

- "\*\*"

paths-ignore:

# чтобы коммит самого workflow не запускал бескон

- "public/export/\*\*"

- "\*\*/\*.md"

workflow\_dispatch:

inputs:

density:

```
 if [-f "$policy"]; then
 sudo sed -i 's/rights="none" pattern="\{P
 fi
 done
 convert -version
```

- name: Install dependencies  
run: npm ci --no-audit --no-fund
- name: Step 1 – код → txt  
run: npm run export:txt
- name: Step 2 – txt → png  
env:  
EXPORT\_DENSITY: \${github.event.inputs.density} || 1  
run: npm run export:png
- name: Step 3 – png → pdf  
run: npm run export:pdf
- name: Step 4 – собрать приложение и поднять pre  
run: |  
# headless-браузер для снимков: chrome ставит  
# этот вызов – страховка, если кэш оказался п  
npx puppeteer browsers install chrome || true  
npm run build  
mkdir -p tmp  
nohup npx vite preview --host 127.0.0.1 --por  
for i in \$(seq 1 40); do  
 if curl -sf http://127.0.0.1:4173/ > /dev/n  
 sleep 1  
done
- name: Step 5 – снимки визуализаций (headless-бр  
run: npm run export:shots  
continue-on-error: true
- name: Step 6 – PDF-конспект со снимками

```
 name: full-code-pages-png
 path: tmp/export-pages/*.png
 if-no-files-found: error
 retention-days: 7

- name: Commit rebuilt export back to the branch
 run: |
 git config user.name "github-actions[bot]"
 git config user.email "41898282+github-action
 git add -- public/export/full_code_all_pages.
 t/viz-shots
 if git diff --cached --quiet; then
 echo "Экспорт не изменился — коммит не нужен"
 exit 0
 fi
 git commit -m "chore(export): пересборка full
 git push origin "HEAD:${GITHUB_REF_NAME}"
```

---

ФАЙЛ 110/111: .github/workflows/verify.yml

КАТЕГОРИЯ: CI / автоматизация | СТРОК: 71 | РАЗМЕР: 2.3

---

name: Verify

# Проверки приложения на каждом пуше и PR:

```
#
verify:unit — разбор «живых» данных компилятора (l
verify:code — все эталонные Python-коды страниц вы
под трейсером панели и создают обеща
verify:dom — приложение монтируется в jsdom: стра
матрицы чужих размерностей, кнопки,
#
Headless-браузер не нужен: jsdom + pyodide из node_mo
```

on:

push:

```
npm run build
```

- name: Модульные проверки (liveMatrix)  
run: npm run verify:unit
- name: Эталонные коды страниц (настоящий CPython)  
run: npm run verify:code
- name: DOM-проверки приложения (jsdom)  
run: npm run verify:dom
- name: Отчёт проверок  
if: always()  
uses: actions/upload-artifact@v4  
with:  
 name: verify-report  
 path: tmp/verify/report.json  
 if-no-files-found: ignore

---

РАЗДЕЛ: ПРОЧЕЕ

---

---

ФАЙЛ 111/111: .ai\_skills/universal\_guide\_authoring/SKIL  
КАТЕГОРИЯ: Прочее | СТРОК: 252 | РАЗМЕР: 13.3 KB

---

# Universal Guide Authoring – шаблоны страниц пособия

Обязательный справочник для \*\*создания и правки контента репозитории. На него ссылаются `AGENTS.md` и `GEMINI.md` собираем \*\*только\*\* из блоков ниже – не придумываем новый «голый» markdown.

Контент живёт в `src/data/\*\*/\*ts` как массив объектов `src/data/content.ts` (при одинаковом `id` побеждает бо

```

<div class="flex items-center mb-6 flex-wrap gap-3">

 <h2 class="text-2xl sm:text-3xl font-bold text-white">
 </div>

<div class="space-y-8">
 <!-- смысловой блок (один или несколько) -->
 <div class="bg-slate-700/50 p-6 rounded-xl border-l">
 <h3 class="text-xl font-bold text-emerald-400 mb-4">
 ...определение, абзацы, стат-сетка, аналогии, спомогательные
 </div>
 </div>
</section>
```
```

Цвет левой полосы и заголовка блока – семантика раздела

| Полоса / цвет | Смысл раздела |
|---|----------------------------|
| --- | --- |
| `border-blue-500` + `text-blue-400` | определение, «определение» |
| `border-emerald-500` + `text-emerald-400` | алгоритм, «алгоритм» |
| `border-indigo-500` + `text-indigo-400` | теория, доктрина |
| `border-amber-500` + `text-amber-400` | ловушки, частные случаи |
| `border-rose-500` + `text-rose-400` | ограничения, «критерии» |

3. Библиотека блоков (точные классы)

3.1 Определение / строгое правило – всегда по центру

```
```html
<div class="bg-slate-900 p-4 rounded-lg mb-6 text-center">
 <p class="text-lg text-emerald-300 italic mb-2">Строгое
 <p class="text-xl font-mono text-white">if d[v] > d[u]
</div>
```
```

```

<p class="text-slate-300 text-sm mb-4">...</p>
</div>

<div class="bg-slate-900 p-6 rounded-lg border-2 border-slate-700">
  <div class="absolute -top-3 left-4 bg-rose-900 text-white shadow-md">
    Ограничения
  </div>
  <p class="text-slate-300 text-sm mb-4">...</p>
</div>
```

```

Второй карточкой может быть «Аналогия 2», «Ограничения» «Когда не работает» — макрип тот же (пунктирная rose-ра

### ### 3.5 Инфо-блок с таблицей

```

```html
<div class="bg-slate-800/70 p-5 rounded-lg border border-slate-700">
  <p class="text-sm font-bold text-white mb-2">Заголовок
  <p class="text-slate-300 text-sm mb-3">Пояснение пере...
  <table class="w-full text-xs text-left border-collapse">
    <thead>
      <tr class="text-slate-500 border-b border-slate-700">
        <th class="py-1.5 pr-2 font-bold">Колонка</th>
        <th class="py-1.5 pr-2 font-bold">Колонка</th>
      </tr>
    </thead>
    <tbody>
      <tr class="border-b border-slate-800 align-top">
        <td class="py-2 pr-3 font-bold">Значение</td>
        <td class="py-2 pr-3 font-mono">O(log n)</td>
      </tr>
    </tbody>
  </table>
</div>
```

```



Универсальное пособие • полный исходный код

- 2. «Одной фразой» / «Суть алгоритма» + центрированное о
- 3. 1–3 абзаца объяснения (3.2), затем сетка ключевых чи
- 4. Аналогии (3.4) – обязательный блок.
- 5. Механика: пошаговое описание, таблица состояний/срав
- 6. Спойлеры (3.6): референсный код, доказательство, форм
- 7. Ловушки (3.7) + границы применимости.
- 8. Связь с другими билетами («тот же find/union – сердце

Демонстрацию вставлять не нужно: `ChapterView` монтирует по `id` главы после текста.

---

## ## 5. Контракт «демонстрация ↔ компилятор»

Каждая страница обязана иметь и визуализацию, и синхрон

- `src/components/vizRegistry.tsx` – запись с ключом `**` (``title``, опциональный ``hint``, ``Component``).
- `src/data/vizSync.ts` – запись ``PAGE_SYNC[id]`` или ``P`, где ``demoId`` – второй аргумент ``emitVizDemo(demoId, t` (вкладки одной демонстрации).
- Поля записи: ``vizTitle``, ``stepNote`` (что считается од ``variables: [{ name, role, range }]`, ``code`` (референ
- Переменные, которые читает визуализация, должны быть (``int``, ``list``, ``dict``, ``str``): экземпляры классов сна до графики не доезжают.
- Стартовый шаблон в редакторе компилятора – ``buildInit`` инициализация переменных до строки ``# --- тело алгори`

Проверка перед коммитом: ``npm run dev`` → страница открыв подставляет код, шаг демонстрации подсвечивает нужные п

---

## ## 6. Экспорт в PDF (``npm run export:guide``)

Текст страниц парсится ``scripts/export/html-to-blocks.m`